

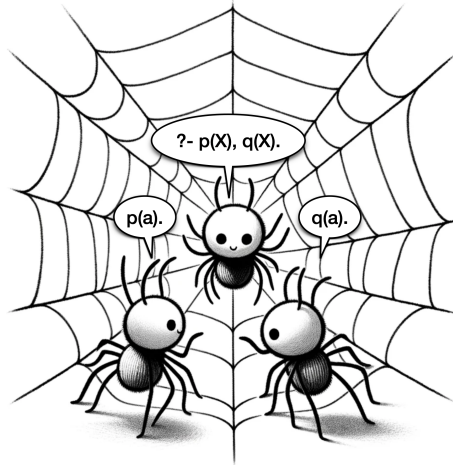
Torbjörn Lager

The Prolog Trinity ecosystem

Let's webize Prolog, prologize the Web, and
populate the Web with clever Prolog agents!

(Draft printed 2026-07-14 at 10:21:04)

Preface



In this book, we present a proposal for a profile of Prolog called *Web Prolog*. We like to think of it as a *web programming language*, or more specifically, as a *web logic programming language*, but also as a kind of simple web-based *agent programming language*. In addition, the architecture for an extension of the traditional Web that we think of as *the Prolog Web* is described, along with descriptions of some of the *Prolog agents* that might dwell there. Together, Web Prolog, Prolog agents and the Prolog Web form what we shall refer to as the *Prolog Trinity ecosystem* – a niche within the vast ecosystem of the World Wide Web.

As can be gleaned from its title, as well as from the previous paragraph, the Prolog programming language plays the lead role in the book, and Prolog programmers and Prolog system implementers are indeed among its intended readers. However, we do not think a reader has to be a very experienced Prolog programmer or implementer to get something out of it. In fact, we are presenting what we believe is a somewhat novel view of Prolog and Prolog programming, so it might even be good if your mind is a beginner's mind, still open to a somewhat different story.

Another programming language, namely Erlang, has an important role to play in the book, so Erlang developers and other actor-programming aficionados might be interested. Or at least we hope so, as we might have some work cut out for them, both as regards the future specification of the Web Prolog language, as well as for its implementation. Note, however, that although the book is written with an audience of Erlangers in mind, some basic knowledge of Prolog is still assumed.

Logic programming researchers and system developers aiming for logical purity might be interested too, but should be aware that the book is focusing on a non-declarative concurrency and distribution model for Prolog, and does not purport to contribute to its logic programming aspects. It is still only “good ol’ Prolog.” Logic

programming researchers might find *use* for it, however, as a way to present their own work to the world, and in particular to other Web Prolog programmers. Oh, and we almost forgot, we also intend to show how to wrap the whole globe in pure logic, and logic programming researchers may want to find out what that means.

The book is also written with an audience of Semantic Web researchers and practitioners in mind, in the hope that it will seem relevant to some of them. After all, the Prolog community and the Semantic Web community have something in common, namely *computational logic*, and probably also a deep conviction that, on some level at least, building software agents that are *really* clever requires logic, reasoning and a relational way of thinking. We hope to be able to show that Web Prolog might serve as a *semantic web logic programming language* – a language that fits in very well among the other web logic languages defined and standardized by the W3C, and a suitable language for building semantically aware web applications, such as intelligent web agents.

Readers interested in Artificial Intelligence (AI) might also be able to get something out of the book, at least if they are somewhat familiar with Prolog. In particular, readers who, along with many others, hold the view that a lot of what happens within the field of AI in the coming decades will take place on the Web, and that intelligent conversational agents are likely to serve as the *face* of AI. Note, however, that the book has almost nothing to say about artificial neural networks and other non-symbolic approaches to AI, but is primarily concerned with symbolic approaches, focusing on the use of logic and reasoning for building web agents. Having said that, it is of course impossible to ignore the recent success of so called *large language models* (LLMs), so the text briefly speculates about the relation between LLM technology and our own approach to symbolic AI on the Web. We can only scratch the surface, but can say already at this point that finding the proper balance between such technologies appears to be as challenging as it is important for the logic programming community going forward.

The categories of potential readers outlined above – Prolog programmers and system implementers, Erlang developers, logic programming researchers, and Semantic Web practitioners – are not merely an audience for this book but also (potentially) the main stakeholders in a broader endeavor: the realization of the Prolog Trinity ecosystem. Each of these groups brings essential expertise and perspectives that contribute to the development, refinement, and practical deployment of Web Prolog, Prolog agents, and the Prolog Web. Together, these stakeholders form a core community that might shape the Trinity, ensuring that it is not only an intellectually compelling idea but also a practical and viable extension of the Web.

It may all be a pipe dream, but it is not a castle in the air!¹

¹ <https://blogs.bmj.com/bmj/2017/01/27/neville-goodmans-metaphor-watch-dream-a-dream-of-ivory-castles-in-the-air/>

Acknowledgements

Contents

1	Introduction	1
1.1	The Prolog Trinity ecosystem	2
1.2	The Prolog Trinity ecosystem concretized	3
1.2.1	Prolog nodes and Prolog actors	3
1.2.2	Web Prolog programs and data	5
1.3	The current state of Prolog	8
1.4	Motivating the Prolog Trinity ecosystem	10
1.4.1	Why Web Prolog?	10
1.4.2	Why Prolog agents?	12
1.4.3	Why the Prolog Web?	14
1.4.4	Why the Prolog Trinity ecosystem as a whole?	15
1.5	Webizing Prolog and prologizing the Web	16
1.5.1	Webizing Prolog	17
1.5.2	Prologizing the Web	18
1.6	Walking in the footsteps of inventors	19
1.7	Thirty years of Prolog on the Web	20
1.8	The Trinity demonstrator	22
1.9	The structure of the book	24

Part I The conceptual and architectural core

2	Web Prolog	29
2.1	The essence of Prolog	29
2.2	Web Prolog in a nutshell	32
2.2.1	Web Prolog is inspired by Erlang	35
2.2.2	Web Prolog is a multi-paradigm programming language	37
2.2.3	Web Prolog as a scripting language for the Web	40
2.3	Erlang-style message-passing concurrency in Web Prolog	44
2.3.1	Programmer talking to actor, actor talking to itself	44
2.3.2	Two utility tools for programming in the shell	46
2.3.3	Actors talking to other actors	47

2.3.4	A closer look at receive/1-2	54
2.4	Erlang-style programming examples in Web Prolog	60
2.4.1	A count server	61
2.4.2	A bigger, tastier example	62
2.4.3	Hiding the details of protocols	64
2.4.4	Using delayed sending	64
2.4.5	Prolog actors playing ping-pong	65
2.4.6	Event-driven state machines	67
2.4.7	Getting answers through backtracking	68
2.4.8	The toplevel behavior: a preliminary sketch	70
2.5	Summary	72
3	Prolog agents	75
3.1	Prolog agents in a nutshell	75
3.2	More about Prolog actor agents	77
3.2.1	An actor agent is equipped with a private Prolog database . .	77
3.2.2	Private beliefs and shared knowledge – shadowing	78
3.2.3	The life-cycle of a typical Prolog actor agent	80
3.2.4	The dynamic (and still private) Prolog database	82
3.3	Prolog shells and other toplevel actors	84
3.3.1	A Prolog toplevel is an actor with a built-in protocol	84
3.3.2	The Prolog Toplevel Communication Protocol	86
3.3.3	Signatures and options for the toplevel API	87
3.3.4	Signatures and options for <code>toplevel_*</code> predicates	90
3.3.5	Shell talking to a Prolog toplevel	91
3.3.6	Programming with toplevel actors	98
3.4	Prolog nodes are also agents	101
3.4.1	The node controller	101
3.4.2	The shared Prolog database	102
3.4.3	Deploying a node	103
3.4.4	Settings	104
3.4.5	Two transports – three web APIs	105
3.5	Node profiles	107
3.5.1	A profile in practice: the Discovery Hub	110
3.5.2	Browser talking to a RELATION node	111
3.5.3	Browser talking to an ISOBASE node	114
3.5.4	Browser talking to an ISOTOPE node	115
3.5.5	Browser talking to an ACTOR node	117
3.5.6	A terminal talking to a browser node	119
3.5.7	Social robot talking to an ACTOR node	120
3.5.8	Could there be other profiles?	122
3.6	Notes on node implementation	122
3.6.1	Statelessness as a protocol property	123
3.6.2	Semi-stateful HTTP sessions and response retrieval	124
3.6.3	Stateful WebSockets and push-driven interaction	125

3.7	Summary	126
4	The Prolog Web	129
4.1	The concurrent Prolog Web	130
4.1.1	Actor talking to remote actor	131
4.1.2	Actors playing ping-pong while on different nodes	131
4.1.3	The node option works for toplevels too!	132
4.1.4	Node-resident actor processes	133
4.1.5	Uniform messaging across node boundaries	137
4.2	The sequential Prolog Web	139
4.2.1	Nondeterministic remote procedure calls	139
4.2.2	Implementing <code>rpc/2-3</code> on top of a toplevel actor	141
4.2.3	Implementing <code>rpc/2-3</code> on top of the stateless HTTP API ..	143
4.2.4	Browser talking to a node talking to a node	145
4.2.5	Backtracking as pagination	147
4.2.6	Programming with <code>rpc/2-3</code>	151
4.2.7	Promise and yield over HTTP	153
4.3	The programmable Prolog Web	157
4.3.1	Injecting code with the <code>load_*</code> options	157
4.3.2	Code shipping and data locality	158
4.3.3	A small design space	159
4.4	Timing and failure boundaries on the Prolog Web	160
4.4.1	Three blocking surfaces	160
4.4.2	A comparative example	161
4.4.3	Late replies	161
4.4.4	Design principle	161
4.5	Security on the Prolog Web	162
4.5.1	Inexpressibility: the sandbox	162
4.5.2	Invisibility: name discipline	163
4.5.3	Containment: lifetime discipline	164
4.5.4	Deployment concerns	165
4.6	Summary	166

Part II Behaviors and other generics

5	Implementing Web Prolog generics	169
5.1	The server behaviour	170
5.1.1	The client interface	171
5.1.2	Hot code replacement	172
5.1.3	Monitoring and fail-fast calls	173
5.1.4	Graceful shutdown	174
5.1.5	From relations to concurrent services	174
5.2	The supervisor behaviour	175
5.2.1	A minimal supervisor loop	175
5.2.2	An example	176

5.2.3	From sketch to implemented behaviour	177
5.3	The toplevel behaviour	177
5.3.1	Computing slices and answer terms	178
5.3.2	The three inner states	179
5.3.3	Aborting a running goal	182
5.3.4	The client interface	183
5.4	The parallel/1 meta-predicate	183
5.4.1	From two goals to a list	184
5.4.2	Detecting failure and errors	184
5.4.3	Race-free cleanup	185
5.4.4	Behaviour and performance	186
5.4.5	Relation to <code>concurrent/3</code>	187
5.5	The <code>first_solution/2-3</code> meta-predicate	187
5.5.1	Racing alternative solvers	188
5.5.2	Failure and error policies	189
5.5.3	An example	190
5.6	Summary	190
6	The statechart behaviour	193
6.1	STATECHARTS	193
6.2	Statechart actors	194
6.2.1	A playful intro to statecharts	195
6.2.2	A statechart process is an actor in disguise	196
6.3	Hierarchy and history	197
6.4	Parallelism and broadcast communication	198
6.4.1	A slightly richer orthogonality example: an NPC controller	199
6.4.2	A cautionary broadcast example: an unbounded internal cascade	200
6.4.3	What “broadcast” means here	201
6.4.4	Why not model each region as a separate actor?	201
6.5	The Web Prolog data model	202
6.6	Process invocation and communication	203
6.7	The execution semantics of SXML, precisely	204
6.8	A flat profile of SXML	206
6.9	Discussion	206
6.9.1	Previous work	207
6.9.2	Comparison with an ordinary message loop	207
6.9.3	Comparison with Erlang’s <code>gen_statem</code> behaviour	208
6.9.4	Relationship to SCXML	210
6.9.5	Toward an actor-oriented successor to SCXML	211
6.10	Summary	211

Part III The broader technological contexts

7	The Prolog Web $\approx$ the Semantic Web?	215
7.1	The Semantic Web tower	216
7.2	RDF as <code>rdf/3</code>	217
7.3	SPARQL-style querying in Prolog	218
7.4	OWL as an external capability	221
7.5	Positioning the Prolog Web alongside the Semantic Web	222
7.6	Cooperation patterns	223
7.6.1	A: Web Prolog as a SPARQL client and post-processor	223
7.6.2	B: RDF as <code>rdf/3</code> , Prolog rules as the API	224
7.6.3	C: OWL reasoning as a service, Web Prolog for orchestration	225
7.6.4	Summary	226
7.7	Proof and trust	226
7.8	User interfaces and applications	226
7.9	Related work: towers of logic programming	227
7.10	Outlook: Prolog Trinity towers	228
7.11	Summary and discussion	230
8	Prolog AI agents on the Agentic Web	233
8.1	Why Prolog agents now?	234
8.2	Symbolic competence in one engine	236
8.2.1	Knowledge representation and inference	236
8.2.2	Planning as search in a state-transition model	237
8.2.3	Meta-interpreters as a control and explanation lever	237
8.2.4	An expert system with a query-the-user facility	238
8.2.5	Parsing as deduction: DCGs	239
8.2.6	Summary	239
8.3	Prolog AI agents on the Agentic Prolog Web	239
8.3.1	An expert-system agent and a simulation	240
8.3.2	Explainable AI across nodes: distributed proof trees	242
8.3.3	Meta-interpreters as generic behaviours	243
8.3.4	Parsing as remote deduction	244
8.3.5	Implementing agent frameworks in Web Prolog	245
8.3.6	Scalability and interoperability on the Web	246
8.4	Neuro-symbolic AI agents	247
8.4.1	Why hybridize?	248
8.4.2	What is neuro-symbolic AI?	250
8.4.3	Architectural patterns for hybrid agents	250
8.4.4	Web Prolog as symbolic backend and agent layer	252
8.4.5	Interaction patterns between LLMs and Prolog	253
8.4.6	Statechart-wrapped medical triage agent	256
8.4.7	Challenges and research directions	256
8.5	User-interface agents	257
8.5.1	UI agents versus MAS agents	258
8.5.2	Turn-taking and repair as control structure	258
8.5.3	Dialogue as proof: the missing-axioms view	258

8.5.4	Statecharts as dialogue managers	259
8.5.5	Dialogue as a governed interface	261
8.6	Summary	261

Part IV From architecture to paradigm

9	The Trinity as a web-native symbolic paradigm	265
9.1	The two partitions as a foundational principle	266
9.1.1	Two levels of abstraction	267
9.1.2	Unification as the universal data contract	269
9.1.3	Backtracking as pagination	270
9.2	From compatibility to interaction portability	270
9.2.1	Node profiles as interaction contracts	272
9.3	Agents as the primary abstraction	273
9.3.1	Mutual constraint	274
9.3.2	Capability-oriented security	274
9.3.3	Five complementary views	275
9.4	The logical foundation: purity, belief, and scoped inference	276
9.4.1	Pure Web Prolog and the pure Prolog Web	276
9.4.2	Epistemic reading and scoped inference	277
9.4.3	The less pure Prolog Web	278
9.5	The distribution-first model	278
9.5.1	The browser as a Web Prolog runtime	279
9.6	Control as a first-class dimension	280
9.6.1	Governance of interaction	280
9.6.2	A platform for the Agentic Web	281
9.7	Executable logic on the Web	281
9.7.1	Positioning relative to the Semantic Web	282
9.8	Comparison to other approaches	283
9.8.1	Linda and tuple-space coordination	283
9.8.2	Concurrent Prolog and the committed-choice languages	283
9.8.3	Oz/Mozart and the multiparadigm kernel language	283
9.9	Why the Trinity fits Prolog	284
9.10	The dual process revisited	285
9.11	Summary	287

Part V What's in it for you and your community?

10	Communities, interoperability, and adoption	291
10.1	A middle-way approach to systems fragmentation	293
10.1.1	Reducing systems fragmentation one bite at a time	293
10.1.2	One Prolog to rule them all	294
10.1.3	The middle way: interoperability as a federating force	295
10.2	How to deal with community fragmentation	298
10.2.1	Coming together through interaction	299

10.2.2	Coming together through collaboration and sharing	299
10.2.3	Coming together through learning	300
10.3	Extending Prolog's user base	302
10.3.1	The imperative for growth	302
10.3.2	Key factors for popularity	303
10.4	Opening the network to neighbouring communities	304
10.4.1	For the broader logic programming community?	306
10.4.2	For the Semantic Web community?	307
10.4.3	For the AI community?	308
10.4.4	For web programmers?	309
10.4.5	For the Erlang community?	310
10.5	Risk analysis	311
10.6	Summary	314

Part VI Final words

11	From vision to execution	317
11.1	A research and engineering agenda	318
11.2	Two routes from prototype to platform	320
11.2.1	A production-oriented SWI-Prolog platform	320
11.2.2	A federated and standardised Prolog Web	321
11.2.3	A decision that need not be made all at once	321
11.3	Standardisation and shared ownership	322
11.3.1	What should be standardised, and when?	322
11.3.2	Institutional coordinates	322
11.3.3	Who should do it?	323
11.3.4	The statechart language and SCXML	324
11.4	The first specification targets	324
11.4.1	The actor substrate as the distinctive target	325
11.4.2	Actor identity, messaging, and lifecycle	325
11.4.3	The semantics of <code>spawn/3</code>	325
11.4.4	The resumable toplevel actor	325
11.4.5	Solution cursors and the ISO liaison	326
11.4.6	Layering the first standards	326
11.5	Implementation and interoperability	326
11.5.1	Why implementation comes early	326
11.5.2	Independent implementations and test suites	327
11.6	AI-assisted implementation and standardisation	327
11.6.1	Implementation support	327
11.6.2	Test generation and adversarial experimentation	328
11.6.3	Specification drafting and editorial assistance	328
11.7	Community and adoption	328
11.8	Closure	329

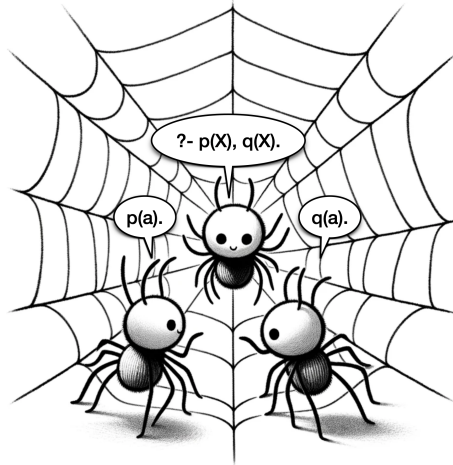
Part VII Appendices

A	Manual	333
A.1	Predicates for programming with core actors	333
A.2	Predicates for programming with toplevel actors	339
A.3	Predicates for programming with server actors	341
A.4	Predicates for programming with statechart actors	343
A.5	Predicates for programming with supervisor actors	344
A.6	Actor-based predicate generics	346
A.7	Predicates for remote Prolog-style calls	347
A.8	ISO predicates and related constructs	349
A.9	Node profile APIs	350
A.9.1	RELATION node API	350
A.9.2	ISOBASE node API	351
A.9.3	ISOTOPE node API	351
A.9.4	ACTOR node API	351
A.9.5	Summary comparison of node profiles	352
A.10	The stateful WebSocket API	352
A.10.1	Connection lifecycle	352
A.10.2	Toplevel session commands	353
A.10.3	Bare actor commands	353
A.10.4	Server-push event types	354
A.10.5	Summary of WebSocket commands	354
A.11	The semi-stateful HTTP API	354
A.12	The stateless HTTP API	356
B	Glossary	357
C	How to implement a Prolog node	363
C.1	Wrapping a node around an existing Prolog system	363
C.1.1	Implementing an ISOBASE node	364
C.1.2	Implementing rpc/2-3 on top of the stateless HTTP API	367
C.1.3	Fixing a problem due to spurious recomputation	368
C.1.4	Implementing the Erlang-style concurrency predicates	373
C.1.5	Implementing output/1, input/2 and respond/2	377
C.1.6	What is missing from the sketches?	378
C.2	Sandboxing, isolation, and capability control	380
C.2.1	Whitelist and blacklist strategies	380
C.2.2	Language sandboxing versus host isolation	381
C.2.3	WASM and WASI	381
C.2.4	Capability control inside the node	382
C.2.5	Resource limits are not sandboxing	382
C.2.6	A realistic implementation strategy	382
C.3	Alternative implementation approaches	383
C.3.1	Implementation for browsers	383
C.3.2	Implementing a browser node	384
C.3.3	Implementations on top of other programming systems	393

Contents	xvii
C.3.4 Prospects for a dedicated Web Prolog VM	393
C.3.5 Less capable profiles need less work	394
References	397

Chapter 1

Introduction



This book articulates a *vision* – an ambitious and forward-looking proposal – for a reconceptualization of Prolog in the context of the modern Web. At the heart of this vision lies *Web Prolog*, a carefully crafted *profile* of the Prolog language – more precisely, an extension of a well-chosen subset – reimagined as a logic-based programming language for the Web. More than just a variant of Prolog adapted for networked environments, Web Prolog aspires to function as a foundational language for a new generation of *web-native logic programming* and *web-based agent systems*.

We propose to think of Web Prolog not only as a tool for building programs that interact over the Web, but as a language for defining logic-driven agents¹ – *Prolog agents* – capable of reasoning, communicating, and coordinating their behavior across distributed contexts. These agents inhabit a conceptual space we call the *Prolog Web*: an architectural extension of the traditional Web, designed to accommodate structured logical interactions, intelligent agent communication, and declarative knowledge sharing.

¹ A word on terminology. The term “agent” is used throughout this book in a deliberately modest, architectural sense: an agent is a computational process with a persistent identity, private state, and the ability to communicate with other such processes by exchanging messages. No commitment is implied to agency in any stronger sense – neither the beliefs, desires, and intentions of the classical agent-theoretic literature, nor autonomy, goals, or anything approaching a mental life. Nor should the term be read through the lens of its most recent usage, in which an “agent” is a large language model equipped with tools and a control loop. Prolog agents as defined here are far simpler creatures. That said, the modest sense is designed to be a substrate for the stronger ones: an entity that persists, remembers, reasons over a private knowledge base, and converses is precisely the kind of entity out of which more ambitious agents can be built, and Chapter 8 makes contact with both the classical and the LLM-flavoured notions deliberately.

Taken together, these three components form the *Prolog Trinity ecosystem*. This ecosystem is envisioned as a niche within the broader World Wide Web, offering a unique paradigm for symbolic AI, declarative communication, and logical coordination on the Web. It represents not merely a technical enhancement, but a philosophical stance: a reassertion of the relevance of symbolic reasoning and declarative knowledge in an era increasingly dominated by opaque statistical models.

This opening chapter serves as a prologue to that larger vision. It prepares the ground for the conceptual foundations, technical proposals, and speculative possibilities explored in the chapters ahead, and it introduces the *what*, *why* and *how* of a project that aims to bring it to life.

1.1 The Prolog Trinity ecosystem

Employed as a language for communicating with computers, logic is higher-level and more human-oriented than other formalisms specifically developed for computers.

Robert Kowalski

As suggested by the simple diagram in Figure 1.1, we think of Web Prolog, Prolog agents and the Prolog Web as three different kinds of entities, playing three distinct roles in the comprehensive Prolog-based ecosystem that we think of as the Trinity.

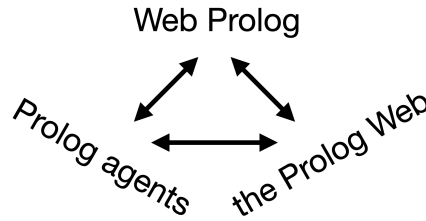


Fig. 1.1 The Prolog Trinity ecosystem: Web Prolog (a language), Prolog agents (an execution model), and the Prolog Web (an infrastructure), and their mutual dependencies.

The diagram illustrates the main idea on the highest level of abstraction we shall care about. The arrows are meant to indicate the many interdependencies and symbiotic relationships that exist between these entities, and to remind ourselves that we are looking at one and the same ecosystem from three different angles.

Put simply, on this level of abstraction the Prolog Web is a network of Prolog agents being programmable in Web Prolog and talking Web Prolog to each other. Expressed more elaborately, the Prolog Web is populated, and indeed also propped up, by Prolog agents, and provides, in the form of distributed Web Prolog databases, some of the knowledge that these agents need to do their thing. Prolog agents in the Prolog Web environment interact with other Prolog agents, using Web Prolog as their language of communication. They are also able to talk with (other kinds of) agents outside this environment such as humans, maybe through a conventional graphical user interface (a GUI), or perhaps using spoken natural language in a voice

user interface (a VUI), and maybe even in a dialog with an *embodied conversational agent* (an ECA).

It is important to understand that the Prolog Trinity ecosystem is not an island. The Prolog Web does not exist in complete isolation, cut off from the rest of the Web. Rather, it is meant to serve as an *extension* of the traditional Web, along with other extensions such as the Semantic Web and the Web of Things. Just like these, the Prolog Trinity can be regarded as a *sub-ecosystem* of the Web ecosystem in its entirety. As we shall see, it can also be thought of as a sub-ecosystem of the larger ecosystem surrounding Prolog – the already existing Prolog systems and the communities of logic programming researchers, system implementers, and other users of Prolog.

1.2 The Prolog Trinity ecosystem concretized

A deeper understanding of what the Prolog Trinity is all about can hopefully be gained by considering the tiny fragment of the Prolog Web suggested in Figure 1.2. On this fairly low level of abstraction, the Prolog Trinity ecosystem can be understood as a concrete network of concrete *Prolog nodes*. In Figure 1.2, nodes are shown as boxes labeled by a *domain name* at the top. The arrows symbolize connections between different nodes, and between end-user clients and nodes. As suggested by the diagram, each node is divided into two “containers,” one container dedicated to the management of actor processes, the other to the storage of Web Prolog programs and data.

1.2.1 Prolog nodes and Prolog actors

Inspired by Erlang’s notion of an actor, the *Prolog actor* is the fundamental unit of computation – and the simplest type of agent – in the Prolog Trinity ecosystem. The smiley emojis in Figure 1.2 represent Prolog actors of various kinds that live and work at nodes. When a node is computing something for a client, it is often one or more of these actors that are responsible, even if this is not evident from the outside.

Hiding an actor behind an emoji makes it look simpler than it actually is, and in fact, an actor has an interesting internal structure. The diagram in Figure 1.3 shows three actors, one of which has just been spawned and one that has been opened up so that we can look inside (where, at this level of abstraction, they all look the same).

A Prolog actor is organised around a *process* – a thread of control capable of communicating with the world around it through messaging. An actor has a unique address – a *process identifier*, or *pid* for short. If we have the pid of an actor, we can message it, and since pids can themselves be part of a message, it allows other actor processes to communicate back. Furthermore, an actor has a *mailbox* that stores incoming messages. There is a *receive* operation that allows the actor to select and

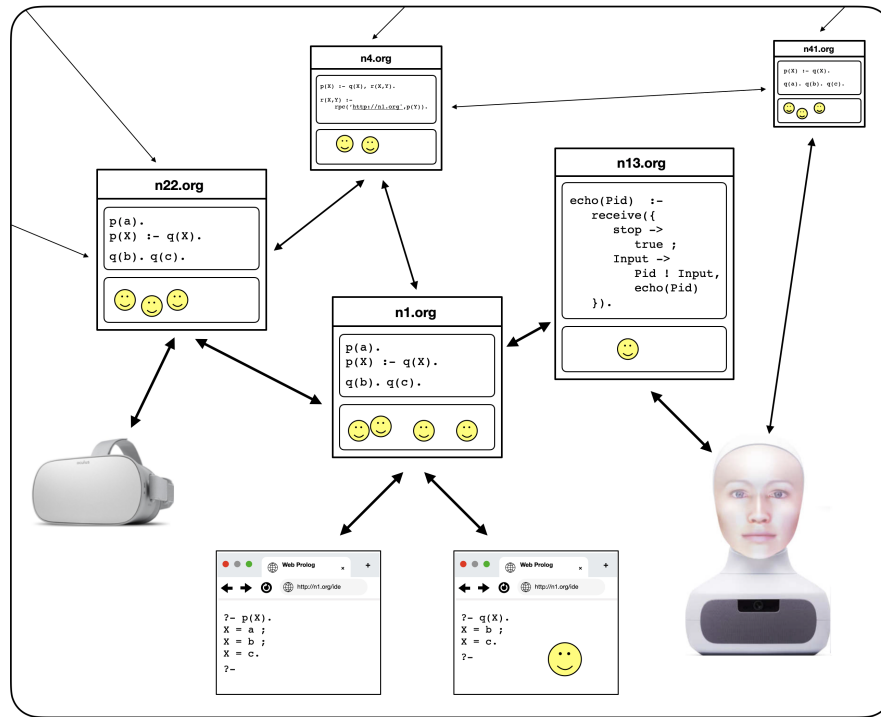


Fig. 1.2 A small fragment of the Prolog Web. Nodes (boxes) store Web Prolog programs and data (upper container) and host actor processes (lower container, shown as smileys). Clients – here two browser-based terminals, a VR headset, and a social robot – connect to nodes, and nodes connect to one another; the second clause at `n4.org` shows a node acting as a client of another node. Chapter 10 revisits this network twice: first with heterogeneous Prolog implementations revealed, and then opened to neighbouring ecosystems.

process messages from the mailbox, and a *send* operation that can be used to send messages to any other actor. Note that, as the figure shows, incoming messages are delivered to the mailbox, not to the process itself: sending is asynchronous, and the actor retrieves messages with *receive* at a time of its own choosing. There is a *spawn* operation that allows an actor to create other actors – child actors as it were. Finally, every actor is equipped with a private Prolog clause database, a dedicated space where programs and data exclusive to that actor are stored.

Authorized end-user clients can connect and interact with nodes. These clients may range from web browsers running simple JavaScript-based Prolog terminals or intelligent web applications, to more sophisticated devices like VR headsets or social robots. (Note that not all clients are end-user clients because nodes can be clients too.) Human users interacting with end-user clients typically trigger activity in the Prolog Web, where agents as well as connections between agents are more or less frequently being created, interacted with, and then destroyed. Some agents and some

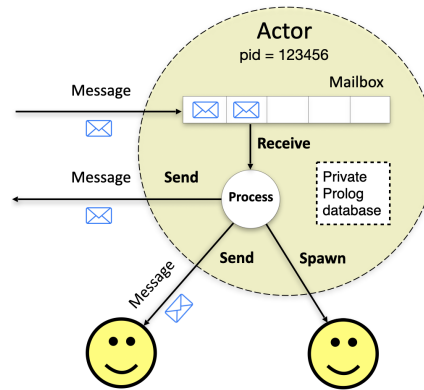


Fig. 1.3 The anatomy of a Prolog actor.

connections live for a long time, while others may only last for fractions of a second. This is obviously a form of dynamics that is not easy to capture in a static diagram such as Figure 1.2.

A node can be seen as a special-purpose web server serving data in the form of Prolog terms (possibly encoded in JSON) to clients as well as to other nodes. Just like a web server, a node has a domain name, makes use of HTTP and/or WebSockets, and is usually capable of serving more than one client at the same time.

We use the term *node* rather than *server*, partly because this is what Erlang uses, but a more important reason is that a node is much *more* than just a web server. A node is also a *runtime system* for Web Prolog capable of executing Web Prolog programs stored by the node and/or passed to it by the client over the web APIs. The Prolog Web is a network of such code, ready to be executed by actors, ready to be queried by clients.

1.2.2 Web Prolog programs and data

Data and programs stored at a node come in two flavors, characterized by two kinds of Web Prolog clauses. One flavor might have clauses looking like this:

```
mortal(Who) :- human(Who).
```

Clauses of this flavor have logical interpretations, and this particular clause translates into the formula $\forall x[human(x) \rightarrow mortal(x)]$ in first-order predicate logic.

As suggested in Figure 1.2, a node can be queried from a terminal running in a web browser. Given our definition of `mortal/1` and a set of Prolog *facts* such as

```
human(socrates).
human(plato).
```

```
human(aristotle).
```

we can ask for the solutions to a query `?-mortal(Who)`. When we run the query from a terminal we enter into a kind of interaction with a Prolog process that most readers are probably familiar with – in which the system only offers one solution at a time and the user may request further solutions by typing a semicolon:

```
?- mortal(Who).
Who = socrates ;
Who = plato ;
Who = aristotle.
?-
```

Such queries can also be given logical interpretations, and this particular one translates into the formula $\exists x[mortal(x)]$. The interaction above not only shows us that there exists an x such that x is mortal, but also provides us with examples. The point here is to remind ourselves that Prolog is a *logic programming language* eminently suitable for *knowledge representation*, or for other uses that require logic. The interaction also demonstrates that Prolog is a *nondeterministic* programming language, meaning that a query may have several solutions, which can be lazily iterated over using Prolog’s backtracking mechanism.

In comparison with the clause defining the predicate `mortal/1`, the following clause has a very different flavor and looks a lot more like a program, and indeed something from the Erlang world – a clause in the *style* of Erlang:

```
echo_actor :-
    receive({
        echo(From, Msg) ->
            From ! echo(Msg),
            echo_actor
    }).
```

As its name suggests, it implements the core of an echo server: an actor capable of receiving messages and returning them to the sender as an “echo.” In contrast to the definition of `mortal/1`, this clause cannot be given a logical interpretation; both `receive/1` and `!/2` are built-in predicates with side effects, so the clause must be understood as a procedural construct rather than a declarative one – a procedure that, when executed by an actor process, determines that actor’s *behavior*.

The predicate `spawn/2` creates a concurrent process running in parallel with the caller, binding the variable in its second argument to a process identifier:

```
?- spawn(echo_actor, Pid).
Pid = 10783471.
?-
```

An interaction exercising the echo actor might look like this:

```
?- self(Self).
Self = 81902375.
?- 10783471 ! echo(81902375, hello).
true.
?- receive({Message -> true}).
Message = echo(hello).
?-
```

Readers familiar with Erlang will recognise the pattern. Readers less familiar with Erlang only need to know, at this point, that the predicate `self/1` retrieves the pid of the calling process – in this case the *shell*, which is itself an actor – that `!/2` sends a message to another process, and that `receive/1` waits for an incoming message. Even this tiny example already conveys the essential properties of an actor. It has its own mailbox, reacts asynchronously to messages, can run indefinitely, and maintains identity through its process identifier.

Beyond local concurrency, Web Prolog also supports two complementary modes of distributed interaction across node boundaries. The simpler of the two is sequential: a remote procedure call that lets one node query another as naturally as it would query its own database.

The built-in predicate `rpc/2-3` makes this possible. Given a URI identifying a remote node, it evaluates a goal on that node and returns solutions to the caller through backtracking, just as if the remote clauses were local:

```
?- rpc('http://n2.org', human(Who)).
Who = socrates ;
Who = plato ;
Who = aristotle.
?-
```

From the caller's perspective, this looks and behaves like an ordinary Prolog query. The distribution is transparent: the goal is sent to the remote node over HTTP, evaluated against the node's knowledge base, and the solutions are returned one at a time through the familiar backtracking interface. No actors, no mailboxes, no WebSocket connections – just a logic-driven query answered by a remote node.

The second mode of distribution is concurrent, and it builds on the actor model introduced above. The `spawn/3` predicate accepts a `node` option that takes a URI, allowing an actor to be created on a remote node:

```
?- spawn(echo_actor, Pid, [
    node('http://n4.org')
]).
Pid = 41983256@'http://n4.org'.
?-
```

The returned pid is a compound term pairing a process identifier with a URI, and it can be used for messaging exactly as a local pid would be. This mode supports persistent,

stateful, bidirectional interaction: long-lived services, ongoing conversations, and coordinated multi-agent behaviour across the network.

Other options to `spawn/3` inject source code into the spawned actor's private database, or specify what should happen when the actor dies, i.e. what its parent (if it has one) might learn about the reason for its death, and how its children (if it has any) will be treated when it terminates. These aspects of actor lifecycle management will be explored in greater depth later in the book.

Although the echo actor is intentionally simple – a specialized process whose only task is to reflect incoming messages back to the sender – it represents only the most basic form of actor behaviour. Through the combined use of options and meta-programming techniques, Web Prolog, like Erlang, can support a wide range of generic behavioural patterns. These include actors that implement a generic client-server model, actors capable of supervising other actors, and actors that function as event-driven, hierarchical state machines. We shall return to these behavioural patterns when we discuss server templates, supervision trees, and statechart actors in later chapters – including several powerful patterns that cannot easily be expressed in Erlang but arise naturally from properties that Prolog, but not Erlang, has.

Together, these two modes define the “Web” in “Web Prolog.” The sequential mode – stateless querying over HTTP, with backtracking mapped onto paginated retrieval – is the simpler and often sufficient choice, and the one closest to how the traditional Web already works. The concurrent mode – actor creation and message passing over WebSocket connections – extends the model to applications that require persistent state and ongoing interaction. Both modes use the same language, the same term-based data model, and the same unification-based semantics; they differ only in their transport assumptions and in the kinds of interaction they support. Chapter 4 develops each mode in detail.

Importantly, the web APIs that nodes expose are not restricted to Prolog clients. Because nodes communicate over standard HTTP and WebSocket transports and can exchange data in JSON, any programming language capable of making an HTTP request or opening a WebSocket connection can interact with a Prolog node. A Python script can query a node and receive structured answers. A JavaScript frontend can open a WebSocket to a node and engage in a real-time conversation with a Prolog agent. The predicate `rpc/2-3` is a convenience for Prolog-to-Prolog interaction, but the underlying APIs are language-agnostic. This is essential to the vision: the Prolog Web is not a closed network of Prolog systems talking to each other, but an open layer of logic-driven services accessible to the Web at large.

1.3 The current state of Prolog

Created in 1972, Prolog is a rather old programming language. Any rumours of its imminent death can easily be debunked, however, as it is currently experiencing something of a revival. New systems are being implemented; features such as tabling and constraint-logic programming libraries are being added to both new and estab-

lished systems; new books are being written; the community is small but thriving, and the language is still taught at some universities.

Still, the current state of Prolog is far from ideal. The community is *too* small and lacks the cultural visibility that defines a healthy, growing language ecosystem. This is well known and was fairly recently documented in a series of papers.

In 2022, as part of the celebration of the Year of Prolog,² a number of logic-programming scholars published a survey paper titled *Fifty Years of Prolog and Beyond* (Körner et al., 2022) (hereafter referred to as FYPB), as well as a book of papers titled *Prolog: The Next 50 Years* (Warren et al., 2023) (henceforth PN50Y), which followed a year later. As they are written by a substantial portion of the *crème de la crème* of the Prolog community, it makes sense to take these papers as probably the best and most current descriptions of the history of Prolog, its current state, and what the authors wish for its future. While this book does not discuss the history of Prolog, its *current* state and its future are certainly relevant.

The picture these papers paint is candid. Among the weaknesses they document, four are of particular relevance here. First, portability between Prolog systems is limited: the ISO standard covers only a core of the language, and features that real applications depend on differ from system to system, so that non-trivial programs rarely run unchanged on more than one implementation. Second, and partly as a consequence, the ecosystem is fragmented: a dozen or so actively maintained systems, each with its own extensions, libraries, and user base, and no shared mechanism by which their strengths can be combined. Third, the tooling and library ecosystem lags well behind what developers of mainstream languages take for granted, raising the cost of entry for newcomers. Fourth, the computing landscape has shifted beneath the language: Prolog systems were shaped by the assumptions of the desktop era, and the authors identify a threat in what they call the “post-desktop world of JavaScript web applications” – a world of browsers, mobile devices, and networked services in which Prolog has so far been a marginal presence. These four concerns – portability, fragmentation, tooling, and the Web – will recur throughout this book, and the Trinity should be read, in part, as a considered response to each of them.

What makes FYPB and PN50Y suitable as background for the project described in this book – and the reason we shall refer to them at various points going forward – is that we share a common interest with their authors: to improve Prolog and its ecosystem as a whole. We ask ourselves the same questions they pose in the following passage from FYPB:

How might Prolog and its community evolve in the future? Can we better unify the new aspects that are offered by different implementations? How should efforts for increased portability be organized? Does it make sense to aim for a unified language? And what tools could be provided to ease development? Furthermore, we propose a plan for future steps that need to be taken to evolve Prolog as a language.

However, as will become apparent, we arrive at different answers and at different ideas for how things might be improved. The Prolog Trinity ecosystem should be seen as our overarching proposal for these various ideas – a scaffolding that helps

² <https://prologyear.logicprogramming.org>

ensure that they are mutually coherent and consistent, improves the cultural visibility of the current Prolog ecosystem, steadies its state, and helps heal what is broken.

1.4 Motivating the Prolog Trinity ecosystem

A vision may be coherent, a design elegant, and an implementation technically feasible, yet this alone does not suffice to justify its realization. What is also required is a clear motivation and a compelling argument for why the undertaking would be worthwhile.

The Prolog Trinity ecosystem brings together three components: an adaptation to the Web of a profile of the Prolog language (Web Prolog), a distributed execution model based on communicating processes (Prolog agents), and an architectural extension of the Web supporting logical interaction and knowledge exchange (the Prolog Web). Each of these components can be understood on its own. Taken together, however, they form a coherent response to a single underlying problem: how to make symbolic reasoning a first-class computational activity on the Web.

The motivation for the Trinity therefore does not arise from three independent ideas placed side by side. Rather, it arises from the observation that once Prolog programs become distributed across the Web, they must be executed by interacting processes rather than by isolated interpreters; and once they are executed by interacting processes, they must inhabit an infrastructure that supports persistent addressing, communication, and knowledge exchange. Web Prolog, Prolog agents, and the Prolog Web thus form three aspects of one and the same architectural response to the problem of web-native symbolic computation.

In the following subsections we motivate each component separately before returning to the question of why the Trinity should be understood as a single ecosystem rather than as a collection of independent extensions.

1.4.1 Why Web Prolog?

Prolog has long served as a language for representing knowledge and performing symbolic reasoning. Its execution model integrates deduction, search, and unification in a single computational framework, making it unusually well suited for logic programming and for building systems that reason about structured information. Yet traditional Prolog systems were designed primarily as stand-alone environments in which programs execute locally within a single runtime context.

The Web, by contrast, is a distributed environment in which programs, data, and users interact across node boundaries. If symbolic reasoning is to become a first-class activity on the Web, it must be possible to express such reasoning in a form that is compatible with networked interaction, remote knowledge access, and distributed execution.

Web Prolog is intended as such a form. It preserves the essential relational character of Prolog while adapting the language to a setting in which goals may be evaluated remotely, knowledge may be distributed across nodes, and interaction with other agents is a normal part of execution.

It is important to appreciate that much of the fit between Prolog and the Web does not depend on concurrency or actors at all. Two properties that Prolog already possesses turn out to align remarkably well with the request–response architecture of HTTP, and together they define a complete and useful web programming model before any concurrent extensions are introduced.

Unification as the universal data contract

In a conventional web API, the shape of a request, the shape of a response, and the rule that determines whether the one satisfies the other are specified separately and enforced indirectly. In Prolog, unification collapses all three into the query itself. The goal `?- employee(Name, Dept, salary(Min, Max))` is at once a request for employees, a specification of the structure answers must have, and the test that decides whether a given answer qualifies: if the node holds a matching fact, the goal unifies and bindings flow back; if client and node have diverged, the mismatch surfaces at the point of invocation rather than downstream in deserialisation code. The query *is* the contract – and with it goes an entire category of glue code. And because a relation supports many instantiation patterns, one goal does the work of several endpoints. Section 9.1.2 develops this observation into one of the defining commitments of the Trinity as a paradigm, including a precise account of how the different kinds of divergence manifest.

Backtracking as pagination

Prolog’s second deep affinity with the Web is one the reader has already seen in action. The interaction in Section 1.2.2 – solutions delivered one at a time, the user typing a semicolon for the next – is, structurally, pagination: a paginated endpoint hands out results a slice at a time and waits to be asked for more; a Prolog toplevel hands out solutions one at a time and waits for the semicolon. The choice point and the next-page token do the same job. The language’s native mode of delivering answers and the Web’s native mode of delivering resources need no adaptation layer between them. Chapter 4 works out the operational details of this mapping, and Section 9.1.3 returns to it as a paradigmatic property.

A sequential web programming model

Together, unification and backtracking already define a self-sufficient model for logic-driven web interaction over stateless HTTP. A node stores a knowledge base,

exposes it through a simple web API, and answers queries by unification and search. Clients pose goals, receive structured answers, and paginate through solutions – all without WebSocket connections, actor lifecycle management, or any concurrent infrastructure. This is not a simplified preliminary to the “real” system; it is a complete and practically useful mode of web programming, and the one that is easiest to implement, deploy, and secure. Three of the four node profiles introduced later in the book operate entirely within this sequential model, and many applications need nothing more.

Extending the model with concurrency and distribution

Some applications, however, do need more. Persistent services, ongoing conversations, coordinated multi-agent interaction, and fault-tolerant supervision all require long-lived processes that maintain state between requests. For these, Web Prolog extends the sequential foundation with Erlang-style concurrency: lightweight actor processes with private state, communicating exclusively by asynchronous message passing. Distribution is treated as a natural extension of local communication – actors interact across nodes with the same simplicity and uniform semantics as within a single node.

The key architectural point is that concurrency *extends* the sequential model rather than replacing it. The two modes – stateless query–answer interaction over HTTP and stateful actor communication over WebSockets – are deliberately kept orthogonal. A system can be built primarily in one mode while selectively invoking the other. Sequential queries may trigger concurrent actors; actors may internally perform sequential reasoning. Yet the boundary remains conceptually intact, and each mode has its own transport assumptions, blocking surfaces, and failure semantics.

In this way, Web Prolog can be understood not merely as a variant of Prolog adapted for networked environments, but as a web logic programming language designed from the outset for participation in a web-scale symbolic infrastructure – a language that is intentionally bilingual at the transport level (supporting both stateless HTTP and stateful WebSocket interaction) while remaining monolingual at the language level. Conceptually, the sequential partition is the foundation and the concurrent partition the superstructure – even though, as we shall see in Chapter 4, the implementation inverts this, realising the sequential interface on top of the concurrent one.

1.4.2 Why Prolog agents?

The sequential web programming model described above – stateless query–answer interaction over HTTP – is already a useful and distinctive contribution. But it has a clear limitation: each request is self-contained. The node evaluates a goal, returns answers, and forgets the interaction. There is no persistent state between requests,

no ongoing conversation, and no way for a computation on one node to coordinate with a computation on another.

Many applications can live within this constraint, and the less capable profiles are designed precisely for them. But some cannot. An intelligent conversational agent must maintain dialogue state across turns. A supervision system must monitor the health of its charges and react to failures. A distributed search must coordinate partial results from multiple nodes. These are inherently stateful, long-lived, interactive activities, and they require a different kind of computational entity: one that persists, receives messages, and coordinates its behavior with other such entities over time.

This observation motivates the introduction of Prolog agents.

Inspired by the actor model and by Erlang-style concurrency, Prolog agents serve as autonomous computational entities that carry private knowledge, execute logic programs, and interact through messaging. They provide a uniform abstraction for representing interactive reasoning processes that may span multiple nodes and multiple sessions.

The significance of this move should not be understated. Without actors, Prolog on the Web is a batch query engine: it can answer questions, but it cannot maintain persistent services, conduct ongoing interactions, or coordinate with other processes. Concurrency is not a bolt-on performance optimization; it is the architectural move that transforms Prolog from a local tool into a web-native platform. With actors, a Prolog node becomes a persistent, multi-client service. With remote spawning and network-transparent messaging, Prolog nodes become participants in a distributed network. With supervision and fault handling, that network becomes resilient. And because the sequential core of each actor is still Prolog – with unification, backtracking, and a clause database – every actor retains the full reasoning power of the language.

This is also what distinguishes a Prolog agent from a generic actor in a framework such as Erlang: its sequential core is a reasoning engine. It can represent knowledge as clauses, search for solutions nondeterministically, parse structured input with DCGs, and explain its conclusions through proof trees. The language gives the agents their intelligence; the agents give the language its presence on the Web.

Prolog agents come in specialized varieties. Toplevel actors expose a conversational query-answer interface governed by an explicit protocol (the PTCP). Statechart actors, inspired by Harel's Statecharts, provide a principled and highly expressive model for event-driven control, integrating declarative reasoning with structured reactive behavior. Prolog nodes are themselves agents, equipped with web-facing APIs, shared knowledge bases, and policy enforcement. These forms are developed in detail in Chapters 2–6.

Seen from this perspective, agents are not an optional addition to logic programming on the Web. They are the natural execution model for logic programs once reasoning becomes distributed and interactive.

1.4.3 Why the Prolog Web?

The previous subsections introduced two complementary modes of distributed logic programming: a sequential mode in which clients query nodes over stateless HTTP, and a concurrent mode in which actors interact through persistent message-passing connections. If these modes are to function across multiple independently operated nodes – each potentially running a different Prolog system – then an infrastructure is required that defines how nodes discover each other, what capabilities they expose, and what interaction contracts clients may rely upon.

The traditional Web already provides a powerful substrate for addressing, communication, and information exchange, but it does not by itself provide a uniform framework for executing logic programs or for coordinating reasoning processes across distributed knowledge sources.

The Prolog Web is intended as an architectural extension of the traditional Web in which logic programs, agents, and knowledge bases can interact in a structured and interoperable manner. It can be understood as a network of Prolog nodes capable of storing Web Prolog programs, hosting Prolog agents, and supporting logical interaction through well-defined web-facing interfaces.

Several properties of this architecture deserve emphasis:

1. **Nondeterministic RPC as a web-programming capability.** The Prolog Web supports forms of distributed interaction that are difficult to realise cleanly in other web programming models. Its notion of nondeterministic remote procedure calls (rpc/2–3) allows a client to request multiple answers to a query from a remote node, mirroring Prolog’s local search behaviour across the network. This preserves the declarative reading of logic programs while extending it seamlessly to distributed settings, enabling agents to cooperate in solving search problems, generating alternative plans, or exploring hypotheses.
2. **Federation through profiles.** The Prolog Web is designed as a loosely coupled federation: nodes are free to specialise internally, but must expose small, well-specified interfaces at the boundary. The profile hierarchy makes explicit which capabilities a node offers to its clients, from simple relational querying to full actor-based interaction. Profiles are what turn a collection of independent servers into a coherent network: they specify which predicates, transports, and behavioural contracts a node must support, while leaving internal implementation choices free. This means that nodes built on top of different Prolog systems can interoperate at runtime without requiring internal uniformity.
3. **An executable logic layer for the Web.** Importantly, the Prolog Web is not conceived as a replacement for the existing Web, but as a symbolic layer within it. Just as the Semantic Web introduced a layer of machine-interpretable data on top of the document Web, the Prolog Web introduces a layer of executable logic on top of the interaction Web. The Semantic Web foregrounds representation and shared meaning; the Prolog Web foregrounds turning knowledge into running, interactive computation. The two are better understood as complementary rather than competitive, as we argue in Chapter 7.

1.4.4 Why the Prolog Trinity ecosystem as a whole?

Each of the three components introduced above addresses a different aspect of the same underlying challenge. Web Prolog provides a language for expressing distributed symbolic computation. Prolog agents provide an execution abstraction for interactive reasoning processes. The Prolog Web provides an infrastructure in which such processes can cooperate across node boundaries.

None of these components is sufficient on its own.

A logic programming language adapted to the Web but lacking an agent-based execution model can serve as a useful stateless query endpoint – and indeed, that is exactly what the least capable profiles provide. But it cannot sustain persistent services, manage lifecycles, or recover from failures; the range of applications it supports remains limited to request–response interaction. An agent model without a relational reasoning substrate is just another actor framework: powerful for concurrency, but lacking a uniform language for representing knowledge and inference, and unable to exploit the structural fit between unification and web interaction that makes the sequential partition distinctive. An infrastructure for distributed interaction without a logic programming layer cannot support executable symbolic knowledge as a first-class web resource: it is a network of web servers, not a web of reasoning.

The Trinity arises from the observation that these three layers must co-evolve if symbolic reasoning is to become a native activity on the Web. A Prolog query endpoint without actors is useful but limited. Actors without a logic substrate are just Erlang. Profiles without a shared language are API specifications. The Prolog Trinity is the *intersection* of these three contributions, and it is the intersection – the mutual constraint between language, execution model, and network substrate – that constitutes the paradigmatic claim developed in Chapter 9. In particular, the deliberate orthogonality of the sequential and concurrent partitions is itself a paradigmatic property: each is complete in its own domain, and they compose rather than compete.

Figure 1.3 does not reveal anything about the implementation of the nodes it depicts. They could all be implemented on top of a single Prolog system, or several systems may be involved, perhaps one for each node, reflecting the diversity of the Prolog landscape. Building the Prolog Web on top of one system is probably the fastest path to a public demo, early feedback, and something that runs on today’s Web and demonstrates value. In a research-oriented community such as Prolog’s, stopping at that stage would not be unusual. Our proposal, however, is to turn the endeavour into a collective, community-driven project.

The Prolog Trinity ecosystem is therefore motivated not only by the technical observations developed above, but also by the need for a common scaffolding that supports, rather than suppresses, the existing diversity of Prolog systems. Different implementations excel in different domains: some prioritise performance, others constraint solving, tabling, or integration with foreign languages. Rather than seeking unification through a large monolithic standard or a single system, we treat this diversity as a resource. Web Prolog is defined as a small, implementable, web-aligned fragment that acts as a *lingua franca* for cross-system interoperability. By

wrapping each system in a node that speaks Web Prolog, heterogeneous engines can collaborate in real time without surrendering their strengths.

This is federation in a precise sense: independent parties agree on how to communicate without agreeing on how to think. The agreement is operational – we will exchange messages in this format, over these transports, with these semantics – rather than ontological. This mirrors how the Web itself grew: loosely coupled, strictly specified at the boundaries, and free to specialise behind them. Modest systems can implement just the essential subset and still participate fully.

To move from the narrow view of a single implementation to the broader perspective of the Prolog ecosystem, we must also relate our aims to concerns already articulated by the community. As we learned above, FYPB and PN50Y ask how Prolog might evolve, how portability might be improved, whether a unified language is sensible, and what tools would best support development. These questions form a shared point of departure, yet our proposed answers differ in important ways. Where FYPB sees weakness in “limited portability”, we concentrate on a portable, carefully defined fragment of Prolog as a basis for interoperability. Where they see a problem of fragmentation, we see an opportunity for diversification, tied together by a clear Web Prolog substrate. Where they identify a threat in the “post-desktop world of JavaScript web applications”, we see an opportunity to design a logic-based, actor-oriented, federated programming environment capable of thriving on the Web itself.

Our programme complements the forward-looking agendas in FYPB and PN50Y: where they outline desirable directions for the future of Prolog, we propose a web-native and agent-centric realisation path, complete with concrete artefacts that can be implemented, tested, compared, and incrementally standardised. For these reasons, we argue that the realisation of the Prolog Trinity ecosystem is not merely feasible but worthwhile: it offers simplicity for developers, coherence for the community, and a principled platform for the next generation of AI on the Web.

Finally, a note on scope. The Trinity is not proposed as a replacement for existing Prolog systems, but as a way to make them cooperate. It is not a demand for a monolithic unified Prolog, but an attempt to define small boundaries that many systems can implement. And it is not a promise of a frictionless open network; rather, it is an argument that openness becomes manageable once capabilities and interfaces are explicit.

1.5 Webizing Prolog and prologizing the Web

The essential process in webizing is to take a system which is designed as a closed world, and then ask what happens when it is considered as part of an open world. Practically, this effect on a computer language is to replace the names/tokens/identifiers for URIs. Thus, where before reference could only be made to something in the same document/program/module one can with equal ease make reference to something in a different one somewhere in that abstract space which is the Web.

Tim Berners-Lee, 1998

Some thirty years in the making, the World Wide Web is a formidable success and the biggest distributed programming system ever constructed, and has, over the years, become a key delivery platform for a variety of sophisticated interactive applications in just about any conceivable domain.

The traditional Web, already in its most primitive form as a Web of Documents (also known as Web 1.0), is *distributed* (running on more than one machine), *decentralized* (running at different geographical locations and/or having different owners) and *open* (anyone can use and contribute). These are some of the traits the design of the Prolog Web seeks to inherit from the conventional Web. Whereas distribution comes with the territory, decentralization and openness have more to do with *web culture*.

The process of adapting Prolog for the Web can be described from two complementary perspectives. From Prolog's side, the task is *webizing*: taking a system designed as a closed world and asking what happens when it is reconceived as part of an open one. From the Web's side, the task is *prologizing*: extending the Web with the capabilities that Prolog uniquely provides. These are two views of one and the same design process, and neither is complete without the other.

1.5.1 Webizing Prolog

Webizing Prolog means equipping the language with everything it needs to function as a native participant on the Web. The requirements fall into four areas: addressing, infrastructure, security, and culture.

Addressing and identification

Uniform Resource Identifiers (URIs) are fundamental to the Web. They uniquely identify resources and make it possible to refer to something on a different machine as easily as to something local. Webizing Prolog means integrating URIs into the language so that nodes, actors, knowledge bases, and individual predicates can all be addressed and linked across the network.

Web infrastructure and transports

The Web's infrastructure includes protocols such as HTTP and WebSocket as well as data formats such as JSON. Web Prolog should exploit these seamlessly and, as much as possible, behind the scenes. In particular, the two modes of distributed interaction introduced in Section 1.2.2 – stateless querying over HTTP and stateful actor communication over WebSocket – should be realized on top of the transports that every web-capable platform already supports. A Prolog node must be reachable and usable by any HTTP client, not only by other Prolog systems.

Security and scalability

Web security is a multifaceted domain encompassing authentication, authorization, data integrity, and confidentiality. Web Prolog must integrate with existing security mechanisms – TLS for encrypted communications, OAuth or similar schemes for authorization, and the structural security measures (sandboxing, name discipline, containment) discussed in Chapter 4. Scalability is likewise essential: the architecture must handle large numbers of simultaneous connections and manage extensive datasets, which requires efficient concurrency, an architecture that supports caching, and the ability to distribute load across nodes.

Standards and web culture

For Prolog to be web-friendly, it must enter into fruitful relations with existing W3C standards. This means handling standard web protocols and document formats, working efficiently with web APIs, and following relevant guidelines such as those for internationalization. Prolog’s strength in dealing with ontologies and rule-based systems positions it well for integration with Semantic Web standards. We believe we may also be able to connect to the W3C’s SCXML, though this requires further work; more on this in Chapter 6.

Beyond technical compatibility, webizing Prolog also means embracing the culture of the Web: open-source development, clear and liberal licensing, excellent documentation, and a healthy community. Finally, to gain widespread adoption and to ensure interoperability between different implementations, Web Prolog must strive for standardization – a theme we return to at length in Chapter 12.

1.5.2 Prologizing the Web

Prologizing the Web means extending the traditional Web with capabilities that Prolog uniquely provides. Where the previous subsection asked what Prolog must do to fit the Web, this one asks what the Web gains from the addition of Prolog. We do this because we believe that a prologized Web is better than a Web not prologized.

1. **Bring logic-driven querying to the Web.** A node stores a knowledge base, exposes it through a stateless HTTP API, and answers queries by unification and search. Clients pose goals, receive structured answers, and paginate through solutions – all using standard web transports. Unification serves as the data contract; backtracking maps onto paginated retrieval. This sequential interaction model requires no concurrent infrastructure and is usable by any HTTP client in any programming language.
2. **Exploit Prolog’s broader symbolic capabilities.** Beyond basic querying, Prolog offers knowledge representation, rule-based reasoning, meta-interpretation, planning, and parsing through definite clause grammars. Prologizing the Web

means ensuring that these capabilities can be put to productive use on the Prolog Web – not only within isolated Prolog programs, but as networked services that other agents and applications can invoke.

3. **Leverage Erlang-style concurrency for persistent interaction.** For applications that need persistent services, ongoing conversations, or coordinated multi-agent behaviour, Web Prolog extends the sequential foundation with actor-based concurrency and distribution over WebSocket connections – an extension of, not a replacement for, the sequential model.
4. **Ensure that Web Prolog can run in web browsers.** A fully prologized Web should not be restricted to server-side execution. The language design should avoid constructs that would make it impossible to implement Web Prolog in browsers, whether through WebAssembly, JavaScript-based implementations, or other client-side technologies.

In Chapter 9 we return to these two movements at a more conceptual level and show how they jointly shape the Trinity as a web-native symbolic paradigm. The final chapter then turns from paradigm to execution: standardization, implementation, and the practical work required to make the proposal shareable.

1.6 Walking in the footsteps of inventors

Before we continue, let us first pay tribute to five people who invented technologies that form the basis for the work presented in the book and laid the groundwork for the Prolog Trinity ecosystem. We have put them side by side in Figure 1.4.

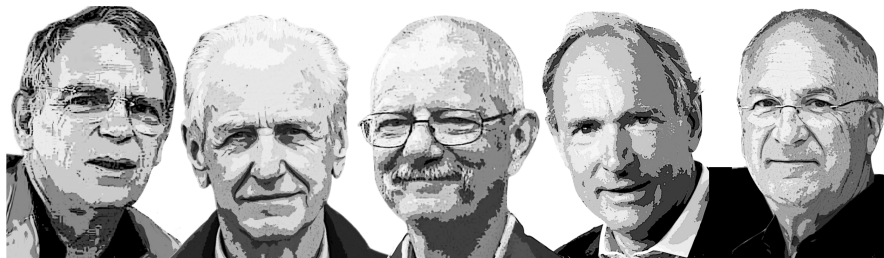


Fig. 1.4 Five inventors whose ideas underpin the Trinity: Colmerauer, Kowalski, Armstrong, Berners-Lee, and Harel (photo montage).

On the left we find Alain Colmerauer. He invented Prolog in 1972 and built the first implementation. Sadly, he passed away in 2017. To the right of him is Robert Kowalski – who came up with a lot of the theory behind Prolog which inspired Colmerauer. He is still around, and is still doing a lot of logic programming research and development. Third from the left, we find Joe Armstrong, who invented another programming language called Erlang. He liked Prolog, and to some extent, as we

shall see, Erlang was inspired by Prolog. Regretfully, Joe Armstrong died in 2019. To the right of him, we find Tim Berners-Lee. He invented the Web, the first web browser, and the protocols and algorithms allowing the Web to scale. He is still very much alive and active trying to build a better Web. Finally, on the right, we find David Harel, the inventor of Statecharts, a rather successful visual state-machine programming language. Why his work has a role to play in the Prolog Trinity ecosystem will be revealed later in the book.

These five inventors are (or were, for those that are no longer with us) scientists and engineers who have made significant contributions to the field of computer science. We do not mean to suggest that they were alone in inventing these technologies.³ The work presented in this book is indebted also to the many, many scholars who evolved the original ideas into the shape they have today. This book tries to build on their achievements, combining chunks of their work into what we hope will emerge as a coherent whole, and preferably a whole greater than the simple sum of its parts.

1.7 Thirty years of Prolog on the Web

We do not want to give the impression that the suitability of Prolog for programming the Web has not been investigated before. On the contrary, in the mid-1990s the advent of the Web sparked a great deal of interest in the logic programming community, reflected in two workshops devoted to the subject and in the development of libraries that mostly relied on creating so-called CGI programs in Prolog. Here is a snippet of text lifted from the call for papers for the second workshop:⁴

This workshop is the second in a series intended to explore the elective affinities between Logic Programming and Internet technologies with emphasis on enhancing the World Wide Web with knowledge, deductive abilities and superior forms of interactive behavior. With the paradigm shift to highly inter-connected computers and programming tools, logic programming languages have a unique opportunity to contribute to practical Internet application development. Simplicity, remote executability, robustness, automatic memory management, are among the features some LP languages share with emerging tools like Java. Superior meta-programming and high-level distributed programming facilities, built-in grammars and dynamic databases, declarative semantics are among their competitive advantages.

Since then the Web has developed at an amazingly fast pace, and web applications have become increasingly speedier and more interactive. The Web of today is *very* different from what it was almost thirty years ago – faster, more reliable, more mobile, and with a fantastic selection of browser APIs that lets a web developer add features such as spoken interaction, video or 3D graphics to an application. There is a reason we nowadays talk about, not only the Semantic Web, but also

³ For example, the Association for Logic Programming has given the official title “Founders of Logic Programming” to fifteen pioneering researchers who laid the groundwork for the field (see https://en.wikipedia.org/wiki/Association_for_Logic_Programming), and Robert Virding and Mike Williams were heavily involved in the design and development of Erlang.

⁴ <http://www.cliplab.org/lpnet/proceedings97/preface.html>

the Conversational Web, the Mobile Web, the Web of Things, and (more recently) the Agentic Web.⁵ Furthermore, many (potentially) disruptive technologies such as automation, artificial intelligence, blockchain technology, wearable technology, augmented reality, virtual reality, robotics and quantum computing are likely to more or less depend on and be integrated into the Web. In this book we focus on artificial intelligence and its integration into the Web.

With the Web (technically) so much better and so different from what it used to be, there is, in our opinion, every reason to make a new attempt at “enhancing the World Wide Web with knowledge, deductive abilities and superior forms of interactive behavior.” In particular, “superior forms of interactive behavior” is likely to depend on the WebSocket protocol and/or the implementation of Web Prolog in browsers or (mobile) apps. The WebSocket standard did not exist twenty years ago, which meant that every approach to web logic programming had to work solely with HTTP. While HTTP remains the most important protocol (and as we shall see, two out of the three APIs driving the Prolog Trinity ecosystem rely on it), important aspects of some of the inventions described in this book rely on the WebSocket protocol, a so called server-push technology which opens a whole slew of opportunities for developers of web applications based on Prolog.

Regarding the history of Prolog, we do not suggest that nothing has happened between then and now. From the 1990s onward, Prolog has repeatedly adapted to the web’s changing landscape. Early work such as the CLIP group’s PiLLoW library showed how HTML and HTTP could be handled declaratively by treating web structures as Prolog terms. With the rise of the Semantic Web in the 2000s, Prolog gained renewed relevance: SWI-Prolog’s RDF store, web libraries, and the ClioPatria platform demonstrated that a Prolog system could serve simultaneously as a high-performance knowledge base and a full HTTP server.

In the 2010s, more interactive and distributed styles of web programming led to further innovations. Pengines (Lager & Wielemaker) made server-side Prolog engines accessible from JavaScript through a simple RPC model, enabling incremental solution streaming and smooth integration with modern web applications. At the same time, projects such as Tau Prolog brought ISO-style Prolog directly into the browser, allowing entirely client-side logic applications written in JavaScript.

More recently, WebAssembly has made it possible to run full Prolog systems inside the browser. SWI-Prolog can now be compiled to WASM via Emscripten, and lightweight systems such as Scryer and Trealla have been ported as well. These provide fast, sandboxed Prolog engines that interoperate smoothly with JavaScript, opening the way for high-performance logic programming in modern web applications.

Among Prolog systems, SWI-Prolog in particular has all the right tools for building server-side support for web applications, in the form of mature libraries for protocols such as HTTP and WebSocket, and excellent support for formats such as HTML, XML and JSON. Therefore, it might be argued that since Prolog can and has been used for building server-side support for web applications, it should already

⁵ See (Lager and Myrendal, 2012) for a description of the many facets of the current Web.

be counted as a web programming language. But since this is true also for Python, or Ruby, or Java, or just about any other general-purpose programming language in existence, it would make the notion of a web programming language more or less empty.

We could of course argue that SWI-Prolog is much better at providing what is needed, but we shall go further than that. We believe that Prolog should claim an even more prominent position in this space. Web Prolog should be used not only for building server-side support for web applications, but should be considered as *part* of the Web, in much the same way as HTML, CSS, JavaScript, RDF and OWL are parts of the Web. In other words, as we think it deserves it, we would like it to be recognized as *a web technology in its own right*.

1.8 The Trinity demonstrator

In theory, there is no difference between theory and practice. In practice, there is.

Yogi Berra

The concepts described in the chapters ahead are accompanied by a running implementation: the *Trinity demonstrator*, a browser-based demonstrator that puts the Prolog Trinity ecosystem in working form.

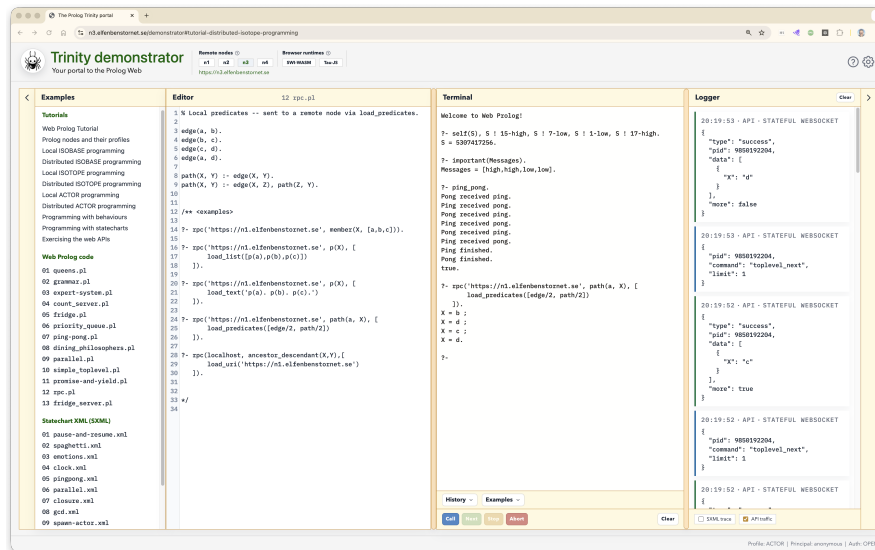


Fig. 1.5 The Trinity demonstrator.

It is not a finished product. It is an attempt to make the essential ideas observable, reproducible, and interactable, rather than merely readable. The demonstrator is accessible from the Trinity homepage at

<https://trinity.elfenbenstornet.se>

and its companion source code is available in the Git repository linked from that page. Readers who prefer to read the book without stopping to experiment may do so; the text is self-contained. But readers who wish to try things out – to inspect which demonstrator nodes are currently available, to run a query against a live node, to spawn an actor, to load and exercise a statechart, or simply to watch the system respond in real time – will find the demonstrator a useful companion.

The demonstrator presents a workspace on the left and a terminal pane on the right. The workspace has three modes – a *Prolog editor*, a *Statechart editor*, and an embedded *Tutorial* – while the *Terminal* executes queries against the currently selected node using whichever transport mode the node allows. An *Examples* drawer provides one-click loading of the actor and statechart examples resident on each node. A *Logger* panel records all transport-level activity, and its trace filter exposes the step-by-step execution of a statechart as an inspectable audit trail. The implementation is written in SWI-Prolog and JavaScript, and the work has been done in cooperation with several AIs.

A second browser-facing component of the demonstrator is the *Discovery Hub*, itself implemented as the node `n0` and accessible at

<https://n0.elfenbenstornet.se/discovery-hub>.

The hub maintains a live directory of the demonstrator nodes, periodically probing them and presenting their advertised profiles, authorisation modes, database portability, and provided predicates and services. Its browser interface is backed by ordinary relations exposed through `n0`'s stateless API: in this sense, the demonstrator uses the Prolog Web to describe the Prolog Web. The hub is shown in Figure 3.9 and discussed more fully in Chapter 3.

Two caveats deserve mention. First, this is a proof-of-concept, not a production system. Security hardening, browser-side execution, and robust multi-node federation are acknowledged as open work. The demonstrator is hosted with best-effort availability, and readers should not be surprised if it is occasionally unreachable. Second, the implementation covers SWI-Prolog specifically. The book's proposals are intended to be implementable on other Prolog systems as well, and one of the long-term goals of the Trinity project is to encourage independent implementations that can be compared and tested for interoperability. The SWI-Prolog proof of concept is a starting point, not a reference implementation to be frozen.

1.9 The structure of the book

In the hope that it might help the reader navigate the book's most important ideas and their connections, the mindmap in Figure 1.6 expands the exposition of the three most important aspects of the Prolog Trinity ecosystem.

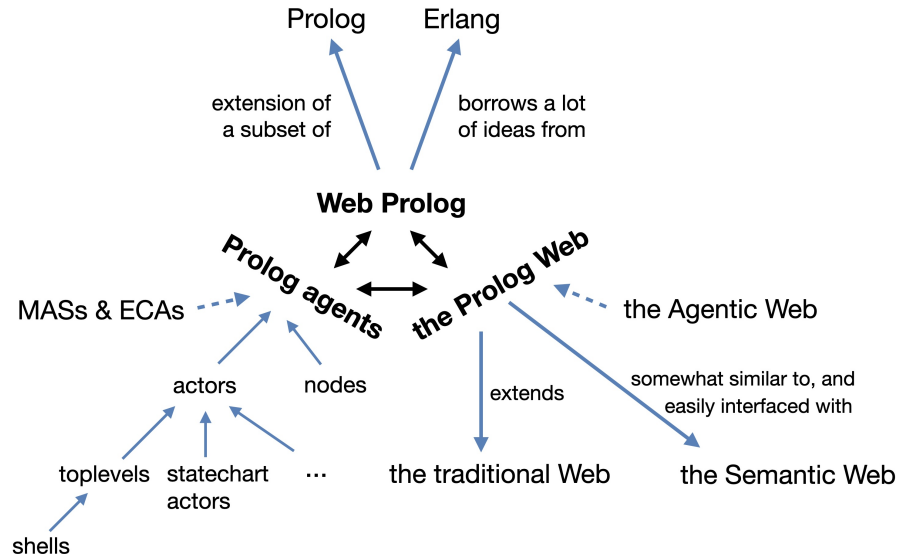


Fig. 1.6 Mind map of the Prolog Trinity ecosystem and its main conceptual threads. MASs means multi-agent systems and ECAs means embodied conversational agents. Chapter 11 redraws the map at the journey's end, distinguishing established relationships from proposed standardisation and adoption paths.

The mindmap reflects the structure of the rest of this book. Broadly speaking, the exposition moves from foundations to consequences. It begins with the conceptual and architectural core of the Trinity, then turns to reusable behavioral abstractions built on top of that core, then places the proposal in relation to larger technological contexts, and finally steps back to ask what sort of paradigm may be taking shape, who it might matter to, and what would be required to move from vision to execution.

Part I: The conceptual and architectural core

Chapter 2 introduces Web Prolog as a multi-paradigm language for concurrent and distributed programming. It explains what is inherited from standard Prolog, what is added to support actor-style concurrency, and why Erlang is a particularly useful

reference point. A range of Erlang programs are translated into Web Prolog to clarify the intended operational behavior.

Chapter 3 develops the agent strand. It focuses on the two agent forms that matter most for the rest of the book – the Prolog toplevel actor and the Prolog node – and describes the built-in interaction protocol, node architecture, and client–node interaction patterns through a series of scenarios.

Chapter 4 presents the Prolog Web: how multiple nodes compose into an open, decentralized network. It introduces `rpc/2-3` for nondeterministic remote procedure calls, delineates the sequential and concurrent partitions, and addresses timing, failure, programmability, and security as first-class architectural concerns.

Part II: Behaviors and other generics

Chapter 5 shifts from individual actors to reusable behavioral patterns – servers, supervisors, and toplevels – showing how common coordination concerns can be packaged as generic, configurable building blocks. It also introduces higher-level constructs such as `parallel/1` and `first_solution/2`.

Chapter 6 introduces the statechart actor: an event-driven state machine implemented in an SCXML-inspired notation with Web Prolog as its data model and scripting language. The chapter explores hierarchy, history, orthogonality, and process invocation, and situates the result relative to W3C’s SCXML and Erlang’s `gen_statem`.

Part III: The broader technological contexts

Chapter 7 compares the Prolog Web with the Semantic Web and argues that the relationship is cooperative rather than competitive. It moves from conceptual comparison to concrete cooperation patterns involving RDF access, SPARQL-style querying, and OWL-based reasoning.

Chapter 8 turns to AI applications, exploring how statechart-style control and symbolic scripting can be combined for building interactive agents. It revisits classical symbolic AI strengths, examines neuro-symbolic combinations involving large language models, and discusses the governance of user-interface agents.

Part IV: From architecture to paradigm

Chapter 9 asks what conceptual pattern emerges when the Trinity’s components are considered together, and argues that they constitute a candidate paradigm for

web-native symbolic computation – characterized by the orthogonality of the two partitions, interaction portability, the agent as the primary unit of structure, a logical foundation that survives distribution, a distribution-first programming model, and the treatment of interaction governance as a first-class dimension.

Part V: What's in it for you and your community?

Chapter 10 asks what the Trinity offers both to the Prolog community and to neighbouring communities. It frames the inward-facing contribution in terms of interoperability, ecosystem health, and responses to systems and community fragmentation, then opens the same network to logic programming, the Semantic Web, AI, web development, and Erlang. The chapter closes by considering the risks and conditions of adoption.

Part VI: Final words

Chapter 11 moves from vision to execution: standardization, implementation paths, shared ownership, and the practical work required to turn the proposal into a testable reality.

Part I

The conceptual and architectural core

Chapter 2

Web Prolog

Imagine a dialect of Prolog with actors and mailboxes and send and receive – all the means necessary for powerful concurrent and distributed programming. Alternatively, think of it as a dialect of Erlang with logic variables, backtracking search and a built-in database of facts and rules – the means for logic programming, knowledge representation and reasoning. Also, think of it as a web programming language, and as a *lingua franca* for logic-based programming systems, standardised by the W3C. This is what **Web Prolog** is all about.

Web Prolog – the elevator pitch

2.1 The essence of Prolog

Based on formal logic, a subject dating all the way back to antiquity and tried and tested by generations of logicians and philosophers, logic programming forms a paradigm of its own, very different from the imperative or functional programming paradigms. Prolog is generally regarded as the first logic programming language, and is arguably the most important one. Consider the following program:

```
husband(Wife, Husband) :- wife(Husband, Wife).  
wife(socrates, xantippa).  
wife(aristotle, pythias).
```

We have here a *rule* that translates into the following formula in predicate logic:

$$\forall x \forall y [wife(x, y) \rightarrow husband(y, x)]$$

The rule in combination with the two *facts* of the predicate `wife/2` (that need no translation) allow us to query the predicate `husband/2`. We may for example ask if it is true that Aristotle is the husband of Pythias, ask who is the husband of Pythias, ask who is the wife of Socrates, or enumerate the married couples one by one:

```

?- husband(pythias, aristotle).
true.
?- husband(pythias, Husband).
Husband = aristotle.
?- husband(Wife, socrates).
Wife = xantippa.
?- husband(Wife, Husband).
Wife = xantippa, Husband = socrates ;
Wife = pythias, Husband = aristotle.
?-

```

This simple example serves as a reminder that Prolog is not only a logic language, but also a *relational* language that can be used to check if a sentence is true, to look up the value of one argument given the value of another, or to enumerate all pairs of values. When querying a binary relation, these are the *modes* that exist.

Because Prolog can pattern-match on structured terms in clause heads, `append/3` defines list concatenation with a base case and a recursive case that peels off the head `H` of the first list and rebuilds it on the result:

```

append([], L, L).
append([H|L1], L2, [H|L]) :- append(L1, L2, L).

```

All three arguments are relational – any of them may be given or left as variables – so `?-append(Xs, Ys, Zs)` holds precisely when `Zs` is `Xs` followed by `Ys`.

Complex terms in the arguments of clauses ensure that Prolog is a *Turing complete* programming language. This is what makes it different from a language such as Datalog, which is similar in many ways, but is not Turing complete.

In order to be not only Turing complete but also a practical and efficient programming language, serious Prolog systems need to offer a lot more, and they normally do. In Section 2.1 of *Fifty Years of Prolog and Beyond*, the authors provide a conceptual and minimalist definition of the important features of Prolog, and are thus able to draw a line between what can be considered a Prolog implementation and what can not. Actually, they draw *two* lines, since they distinguish the *essential* features of Prolog from *important* (yet non-essential) features. Below, we reproduce their list of features, where 1-6 are considered essential features and 7-12 less essential. As it turns out, all the essential features except for 2) and 6) are involved when querying the binary relation between the married couples, or the ternary relation between the lists, in the examples given above.

1. Horn clauses with variables in the terms and arbitrarily nested function symbols as the basic knowledge representation means for both programs (a.k.a. knowledge bases) and queries;
2. the ability to manipulate predicates and clauses as terms, so that meta-predicates can be written as ordinary predicates;
3. SLD-resolution (Kowalski, 1974) based on Robinson's principle (1965) and Kowalski's procedural semantics (Kowalski, 1974) as the basic execution mechanism;

4. unification of arbitrary terms which may contain logic variables at any position, both during SLD-resolution steps and as an explicit mechanism (e.g., via the built-in `=/2`);
5. the automatic depth-first exploration of the proof tree for each logic query;
6. some control mechanism aimed at letting programmers manage the aforementioned exploration;
7. negation as failure (Clark, 1978), and other logic aspects such as disjunction or implication;
8. the possibility to alter the execution context during resolution, via ad-hoc primitives;
9. an efficient way of indexing clauses in the knowledge base, for both the read-only and read-write use cases;
10. the possibility to express definite clause grammars (DCG) and parse strings using them;
11. constraint logic programming (Jaffar and Lassez, 1987) via ad-hoc predicates or specialized rules (Frühwirth, 2009);
12. the possibility to define custom infix, prefix, or postfix operators, with arbitrary priority and associativity.

One cannot avoid noticing that, as far as we know, no other community in support of a programming language has found itself in the position of having to determine what should be considered a *real* language of this kind. But as we found in Chapter 1, the problem with Prolog is that there are so many systems around that can rightly claim to implement it but still are incompatible with each other in important ways. This is what prompted the development of a standard. For almost thirty years now, the core features of Prolog have been standardized by ISO, documented in a report known as ISO/IEC 13211-1:1995.¹

The logic programming landscape

Prolog is the most widely known logic programming language, but it is far from the only one. Over the past five decades, researchers have extended, restricted, or reinterpreted the Horn clause core in many directions. The following survey is necessarily selective, but it covers the formalisms and systems that will reappear in later chapters when we consider how the Prolog Web can serve as an integrating substrate for the logic programming family as a whole.

Prolog’s native mode of reasoning is deduction – deriving consequences from given rules and facts – but logic programming as a field has long explored other modes of inference as well. Abductive logic programming (ALP) reverses the direction: given an observation, it searches for hypotheses that, together with background knowledge, would explain it (Kakas et al., 1992). Inductive logic programming (ILP) goes further still, learning new rules from examples and background theory (Muggleton and De Raedt, 1994; Cropper et al., 2022). Both traditions extend the reach

¹ <https://www.iso.org/obp/ui/#iso:std:iso-iec:13211:-1:ed-1:v1:en>

of logic-based reasoning well beyond what a standard Prolog system provides out of the box.

Nor is deductive logic programming itself a monolith. Datalog restricts Horn clause logic to function-free programs, sacrificing Turing completeness in exchange for guaranteed termination and well-understood complexity bounds (Ceri et al., 1989; Green et al., 2013). Well-founded semantics (WFS) addresses a different limitation, providing a three-valued account of negation in which every normal logic program has a unique minimal model assigning each ground atom the value *true*, *false*, or *undefined* (Van Gelder et al., 1991); systems such as XSB and SWI-Prolog implement WFS via tabling, giving them stronger semantic guarantees than standard Negation As Failure. Answer Set Programming (ASP) departs more radically, searching for the stable models of a logic program rather than computing answers by resolution; a recent variant, s(CASP) (Arias et al., 2018), combines ASP with constraint logic programming in a top-down, query-driven execution model with support for explanation generation. Probabilistic logic programming (PLP) equips logic programs with probabilistic reasoning: ProbLog (De Raedt et al., 2007) is a Python-based toolbox that defines distributions over possible worlds and supports inference, sampling, and parameter learning, while cplint (Riguzzi, 2018) is a complementary SWI-Prolog library for LPADs under the distribution semantics, accessible both locally and through “cplint on SWISH.” Constraint logic programming (CLP) extends Prolog’s unification with constraint solvers over domains such as finite integers, reals, and sets, and most major Prolog systems now ship with CLP libraries as standard. Finally, Logic Production Systems (LPS) (Kowalski and Sadri, 2015; Kowalski et al., 2023b) combines logic programming with reactive, event-driven computation.

The common thread is that logic programming is not a single language but a family of formalisms. This breadth will become relevant in later chapters, where we show how the Prolog Web can accommodate any system in this family that can accept a goal and return answers.

2.2 Web Prolog in a nutshell

Web Prolog is designed as a superset of a subset of the ISO Prolog core standard. The dialect defined by ISO is well understood, and is reasonably close to most of the dialects used in Prolog textbooks. The ISO Prolog syntax specification as well as a large subset of its built-in predicates form an excellent point of departure for our attempt to create a design and a standard for Web Prolog.

The syntax of Web Prolog is exactly as in ISO Prolog, except that three infix operators (`!/2`, `if/2` and `@/2`) that are not in the standard are defined. They can easily be added by means of `op/3`, the ISO Prolog predicate allowing a programmer to define prefix, postfix or infix operators and specify their precedences. This means that syntactically well-formed Web Prolog programs can always be *read* (but not necessarily *run*) by a conforming implementation of ISO/IEC 13211-1:1995. It is easy to imagine circumstances where this might come in handy.

The capabilities of a programming language are to a large extent determined by the built-in constructs it provides. Rather than to make a long list, Figure 2.1 shows a “cloud” of built-in control constructs and predicates specified by the ISO standard.

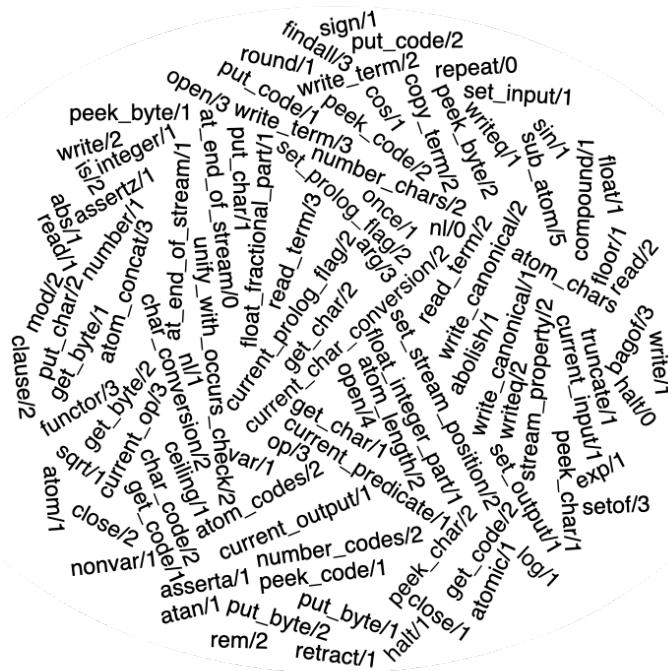


Fig. 2.1 A cloud of ISO Prolog built-in predicates.

Most of the predicates in this cloud *can* and *should* be supported by Web Prolog. (We will get to the exceptions a bit further ahead.) However, while most of them are necessary, they are not sufficient. Here is what else we think is needed:

- In addition to the built-in predicates offered by the ISO Prolog core, Web Prolog needs to support a set of *standard library predicates* that conforming nodes on the Prolog Web must normally offer. They should just be there – a programmer should not have to load them explicitly. The *Prologue for Prolog* lists `member/2`, `append/3`, `length/2`, `between/3`, `select/3`, `succ/2`, `maplist/2-8`, `nth0/3`, `nth1/3`, `nth0/4`, `nth1/4`, `call_nth/2`, `foldl/4-6` and `countall/2`.² See also the public-domain set of libraries developed by the Prolog Commons Working Group.³
- Item number 10 in the list of essential and important features of Prolog points to the possibility to express Definite Clause Grammars (DCGs) and parse strings

² <https://www.complang.tuwien.ac.at/ulrich/iso-prolog/prologue>

³ <http://prolog-commons.org>

using them. DCG notation and the associated parsing primitives are specified separately as an extension, in ISO/IEC 13211-3 (Part 3: Definite clause grammar rules).

- As we are aiming for a profile of Prolog suitable for programming the Web we need predicates such as `http_open/3` for accessing its resources. Also, JSON serialization/deserialization seems essential to have.
- Last but not least, we extend our subset of ISO Prolog with carefully crafted concurrency and distribution primitives heavily inspired by Erlang. When we write “heavily inspired,” we really mean it. We try to stay as close to Erlang as possible, we use the same *kind* of actor as Erlang, and we use the same names for our concurrency-oriented predicates that Erlang uses for the corresponding functions. This is not to say that we never deviate from the way Erlang does its thing, because as we shall see, we do when we have good reason to.

We would expect the community to reach broad agreement on the first three points. The fourth is the only one likely to pose a substantial technical challenge. Even so, the difficulty should not be overstated: Erlang already provides a clear indication of what is missing and of the direction in which development must proceed.

There are features, built-in predicates and libraries that may never make it into Web Prolog. The reason why may vary – they may be dangerous, or they may be superfluous. Other features may be deemed not developed enough to be included at this point in time.

- We only use a subset of ISO Prolog as it contains primitives that do not seem to “belong” on the Web and which may also compromise the security of a node, such as predicates that gives a program access to the operative system.
- Even though they are certainly relevant to the Web, there is definitely no need for predicates for building web servers, such as the ones available in SWI-Prolog’s library(`http/http_server`). After all, a Prolog node *is* a web server.
- Even though predicates for constraint logic programming is listed as an important Prolog feature, and unless they are already defined by ISO, we do not believe time is ripe to include them in Web Prolog. As they mature, this may of course change.
- We expect that it is possible to make only limited use of modules. After all, a node *is* a module, and an actor may be seen as a module too.

We have characterized Web Prolog as a profile of Prolog suitable for programming on the Web. As we shall see, Web Prolog can be cut up into subsets that are themselves profiles, or sub-profiles if you will. Some such profiles of Web Prolog do not support predicates for database updates such as `assert` and `retract`, or predicates for reading from and writing to a terminal. Other profiles do not support `op/3`.

2.2.1 Web Prolog is inspired by Erlang

I would prefer multi-threading in Prolog to look as much as possible like Erlang.

*Richard O’Keefe*⁴

Erlang is a general-purpose concurrent functional programming language developed by Joe Armstrong, Robert Virding and Mike Williams in 1986. It began as a proprietary language within Ericsson, but was released as open source in 1998. Originally designed for telephony applications, Erlang later proved highly effective for backend web systems and for a wide range of soft real-time control problems (Virding et al., 1996). Regarded by many as the first actor programming language to achieve broad practical success, it is especially notable for its built-in support for concurrency, distribution and fault tolerance.

A good overview of the language, its design intent and its underlying philosophy is given in Joe Armstrong’s Ph.D. thesis (Armstrong, 2003b). There, the role of concurrency is described as threefold:

1. In Erlang, the fundamental role of concurrency is architectural. Systems are structured as sets of communicating processes that interact exclusively by message passing, which yields a natural decomposition into well-defined components that can be independently implemented and tested.
2. A system designed in this way can also be executed efficiently on multiprocessor machines or distributed across a network. But concurrency was not introduced in Erlang merely as a mechanism for improving performance. Efficiency is only a potential benefit, and depends on whether the application can in fact be partitioned into genuinely independent tasks.
3. The main purpose of this architectural discipline is to limit the consequences of software errors. As Armstrong argues, large reliable systems are best built by partitioning them into independent concurrent processes that share no data and communicate only by message passing, combined with mechanisms for monitoring and restarting processes when they fail (Armstrong, 2003b, pp. 18–19).

Armstrong refers to this paradigm as *concurrency-oriented programming* (COP). This idea is central to the design of Web Prolog, which is itself conceived as a language for COP. The actor processes introduced in this book are not merely devices for exploiting parallelism. They are the primary units of organisation in a distributed system on the Web. In that sense, they are not just concurrency mechanisms but *agents* – a perspective that will be developed more fully in Chapter 9.

Inspired by the list of essential features of Prolog given in the previous section, we may also ask what it would mean for a language to count as Erlang-style. At a minimum, such a language would seem to require the following:

1. the ability to execute a large number of actor processes concurrently;

⁴ <https://groups.google.com/g/erlang-programming/c/1jdsnqZ4XfQ/m/ve9WfF12YBwJ>

2. the ability to enforce process isolation, so that actors share no mutable state and communicate exclusively by asynchronous message passing;
3. the ability to support network-transparent concurrent programming, allowing actors to create other actors on remote computers and communicate with them seamlessly.

Added to the essential features of ISO Prolog, these capabilities provide crucial parts of the foundation for Web Prolog, and indeed for the wider Prolog Trinity ecosystem. For this synthesis to work, however, the two sets of features must be compatible. Our current view is that they are, and part of the purpose of this book is to demonstrate that claim.

Why did we choose Erlang as a source of inspiration, rather than any alternative? After all, there are other actor programming languages – Akka, Pony and E, to name a few. The main reason for the choice of Erlang is that it is arguably the most *mature* concurrency-oriented language in existence, with a larger community than the alternatives, and a community that includes a fair number of industrial users. Many millions of lines of source code have been written in Erlang, many books teaching the language have been authored, and university courses teaching concurrent and distributed programming often use Erlang. In our opinion, since we are aiming for a standard it makes a lot of sense to borrow the required concurrency-oriented features from a language as mature and battle-tested as Erlang.

Another reason for our choice is that Erlang and Prolog are in many ways remarkably similar, both when it comes to syntax, and the reliance on dynamic typing, assign-once variables, pattern matching and recursion. The similarities can be explained by the fact that historically, Erlang evolved from Prolog and the first version of Erlang was actually implemented as an interpreter in Prolog (?). The following listings of Erlang code and Web Prolog code show some of the similarities as well as some of the differences between the two languages:

<code>append([], L) -> L ;</code>	<code>append([], L, L).</code>
<code>append([H L1], L2) -></code>	<code>append([H L1], L2, [H L]) :-</code>
<code> [H append(L1, L2)].</code>	<code> append(L1, L2, L).</code>

Note the use of capital letters for variables, the familiar notation for lists, and the reliance on pattern matching and recursion. A major difference is that Erlang is a functional programming language, whereas Web Prolog is relational. Since a function is just a special case of a relation this difference must not be exaggerated, but it does show in the way function calls may be nested, something that cannot be done in Prolog (or Web Prolog) since arguments are used to represent outputs as well as inputs. What *can* be done in Prolog, however, but not in Erlang, is to call `append/3` in more than one *mode* – not only to append one list to another, but also, for example, to nondeterministically split a list up into two parts. Furthermore,

thanks to logic variables, `append/3` is tail-recursive in Prolog, but not in Erlang's `append/2` (as it is written here).

Despite such differences, as long as we do deterministic computation only, and no search is involved, logic programming and functional programming are fairly similar in the way they work, and methods used to achieve success with one often transpose to the other. Again, features such as pattern matching, assign-once variables and recursion play important roles in both languages.

During its evolution, Erlang gradually shed several features that Prolog had – and still has – while at the same time developing exceptionally strong support for concurrent and distributed programming, an area where Prolog historically lacked, and still lacks, comparable built-in capability. Key here is the concept of an actor, and the ways actors can be scripted. Below, the echo actor written in Web Prolog is placed side-by-side with an implementation in Erlang – a function in Erlang and a predicate in Web Prolog – both making a recursive call that implements a loop:

<pre>echo_actor() -> receive {echo, From, Msg} -> From ! {echo,Msg}, echo_actor() end.</pre>	<pre>echo_actor :- receive({ echo(From, Msg) -> From ! echo(Msg), echo_actor }).</pre>
--	---

The Web Prolog code on the right demonstrates the use of two constructs foreign to traditional Prolog, implementing the sending and receiving of messages in the style of Erlang. The programs have a similar look, and this is intentional. We have *strived* to make Web Prolog look as similar as possible to Erlang within the constraints imposed by the syntax and semantics of ISO Prolog.

A function call such as in `Pid=spawn(fun() -> echo_actor() end)` (or, but only for zero-argument functions, `spawn(fun echo_actor/0)`) can be made in order to spawn our echo actor in Erlang, while doing it in Web Prolog would use something like `spawn(echo_actor, Pid)`. These calls look somewhat similar too, but while Erlang is a higher-order language in which the spawn function takes an anonymous function as its argument, Prolog (or Web Prolog) is not a higher-order language in this sense. In Web Prolog, `spawn/2` is a *meta predicate* which expects a callable term to be passed in the first argument, a term that when called will execute a procedure that determines how the actor will behave.

2.2.2 Web Prolog is a multi-paradigm programming language

Prolog is different, but not that different.

Richard O'Keefe

A common way to categorize programming languages is in terms of the programming paradigm(s) they support. In the very nicely organized and very well argued taxonomy of programming paradigms by Peter Van Roy in (Van Roy, 2009), depicted in Figure 2.2,⁵ it is indeed possible to pinpoint exactly where Web Prolog fits in. We have enclosed the relevant square boxes in boxes with rounded corners.

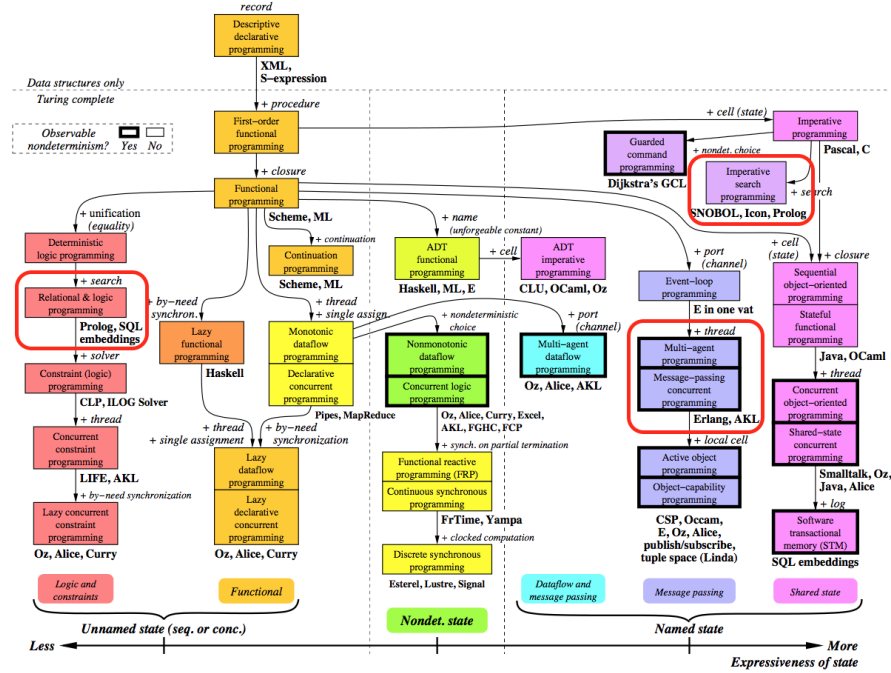


Fig. 2.2 A taxonomy of programming paradigms due to Peter Van Roy. (Image source and license to be confirmed.)

This suggests that we can regard Web Prolog as a language that supports three programming models: *relational logic programming* (as in Prolog), *imperative search programming* (also as in Prolog), and (as in Erlang) *message-passing concurrent programming*.

With access to predicates for asserting and retracting clauses in the dynamic database, traditional Prolog has always, albeit somewhat reluctantly, been able to support imperative programming (and later additions such as `nb_setval/2` have of course amplified this). Also, the built-in backtracking *search* that is so useful in Prolog has little to do with logic. Thus Prolog has never really been a single-paradigm programming language, and this is reflected in Van Roy's taxonomy. In (Van Roy, 2009) he suggests that this is not a bad thing:

⁵ The diagram is available at <https://www.info.ucl.ac.be/~pvr/paradigmsDIAGRAMeng108.pdf>

A good language for large programs must support several paradigms. One approach that works surprisingly well is the *dual-paradigm language*: a language that supports one paradigm for programming in the small and another for programming in the large.

With the addition of support for the actor programming model, Web Prolog becomes a much clearer as well as (in our view) a more interesting case of a multi-paradigm programming language. In particular, it can be argued that the support for programming in the large has been greatly improved.

Any unexpected interactions between paradigms?

It is probably wise to entertain a suspicion of unexpected interactions between language features and possible impedance mismatches between the two paradigms – between Prolog’s relational, nondeterministic programming model based on logic and Erlang’s functional and message passing model. How well do the Erlang-style constructs mix with Prolog – with backtracking for example, or with the features for imperative programming? What do we get if we combine them? A kludge, or something quite beautiful?

In theory, we should be on the safe side. Sequential Erlang is basically Erlang with its data types, one-way pattern matching, functions and control structures, but without spawn, send, receive, and other constructs used for concurrent programming. The idea behind Web Prolog can thus be described as an attempt to “plug out” the sequential part from Erlang and “plug in” sequential Prolog instead. There seems to be no principled reasons why we would not be able to replace the sequential functional language with a sequential relational language, e.g. a logic programming language such as Prolog. Indeed, Armstrong (2003a) writes:

Erlang is a concurrent programming language with a functional core. By this we mean that the most important property of the language is that it is concurrent and that secondly, the sequential part of the language is a functional programming language.

The sequential subset of the language expresses what happens from the point in time [sic] where a process receives a message to the point in time when it emits a message. From the point of view of an external observer two systems are indistinguishable if they obey the principle of observational equivalence. *From this point of view, it does not matter what family of programming language is used to perform sequential computation.* (Our emphasis.)

Alternatively – and this has been our choice – the approach can be described as an attempt to *extend* Prolog with constructs such as spawn, send and receive. The idea is to keep everything that core Prolog has to offer, and extend it with a number of those primitives that make Erlang such a great language for programming message-passing concurrency. The choice between extending Prolog with Erlang-style constructs and extending Erlang with Prolog-style constructs is easy to make, and a lot has to do with syntax. Provided we can accept using a syntax which is relational rather than functional, precluding the nesting of function calls, it may be clear already at this point that the surface syntax of Prolog can easily be adapted to express the needed Erlang-style primitives. It is arguably a lot harder to express Prolog rules and other constructs using the syntax of Erlang.

2.2.3 Web Prolog as a scripting language for the Web

Scripting languages are a lot like obscenity. I can't define it, but I'll know it when I see it.

Larry Wall

Over the years, the Web has become a key delivery platform for a variety of sophisticated interactive applications in just about any conceivable domain. As a consequence, JavaScript, more or less the only game in town for programming the front-end of a web application, has become among the most commonly used programming languages on Earth. Indeed, JavaScript must be regarded as *the* web programming language of our times.

JavaScript started out as a very small, highly domain-specific scripting language that was limited to running within a web browser to dynamically modify the web page being shown, but has over time evolved into a widely portable general-purpose programming language. Modern JavaScript is a high-level, dynamically typed, interpreted multi-paradigm programming language with several essential features that make it a versatile tool for web development. JavaScript's syntax supports various programming paradigms, including imperative, functional, and object-oriented programming styles. With features such as callbacks, promises, and `async/await`, JavaScript supports asynchronous programming, enabling non-blocking operations, especially useful in web applications. JavaScript's native format for data interchange, JSON, is lightweight and widely used for data transmission. JavaScript runs on almost all modern web browsers and platforms, making it universally applicable for web development. It easily interfaces with numerous web APIs for tasks like accessing the Document Object Model (DOM), making HTTP requests, and handling events. This makes it an excellent tool for connecting disparate systems in web applications.

With the advent of Node.js,⁶ JavaScript extended its reach to server-side development. This allowed for a unified language experience across both client and server sides, simplifying the development process by allowing the same language to be used throughout the entire stack. Not having to learn more than one language in order to become a so called “full-stack developer,” and not having to switch languages when changing from working on the server-side to working on the client-side are important advantages. Also, Node.js is frequently used to build and connect microservices. Its non-blocking I/O model and event-driven architecture make it suitable for building scalable and efficient network applications.

It is clear that as a relational logic programming language with the essential and important features listed above, Prolog is very different from JavaScript in terms of paradigms supported. The addition of Erlang-style concurrency and asynchronous communication does not really narrow the gap, as those are features that work differently in Erlang. What matters here is the *role* that JavaScript plays on the Web. We want Web Prolog to play a similar role in the Prolog Trinity ecosystem as JavaScript does in the JavaScript ecosystem, just as clear-cut.

⁶ <https://en.wikipedia.org/wiki/Node.js>

The connection between scripting languages and the Web has been noted before. For example, in his article “Scripting: higher level programming for the 21st Century,” John Ousterhout (1998) writes (about the Internet rather than the Web, but that makes little difference):

The growth of the Internet has also popularized scripting languages. The Internet is nothing more than a gluing tool. It does not create any new computations or data; it simply makes a huge number of existing things easily accessible. The ideal language for most Internet programming tasks is one that makes it possible for all the connected components to work together; that is, a scripting language.

So what about the suitability of Web Prolog for programming the Web? Can Prolog challenge JavaScript in this space, at least for some applications, such as web-based AI? For this to be possible, we must ensure that Web Prolog is

- viable as a scripting language,
- suitable for programming the Web,
- possible to implement in browsers, and
- secure.

Web Prolog as a scripting language

As regards the viability of Web Prolog as a scripting language we note that while hardly a definition, Ousterhout (1998) characterizes scripting languages as follows:

1. They are suitable for “programming in the large,”
2. they are designed for “glueing” applications together,
3. they tend to be typeless, thus making it easier to connect components,
4. they are usually interpreted, thus providing rapid turnaround during development by eliminating compile times,
5. they are higher level than a system programming language in the sense that a single statement does more work on average, and
6. they allow rapid prototyping.

JavaScript does indeed fit this characterization, but given the development described above, it should now probably be described as general-purpose languages which also happens to be suitable for scripting. By contrast, languages such as C++ and Java are not well suited to that purpose.

What about Prolog? Does it satisfy Ousterhout’s characterization? The points 3-6 in his list are probably true of the Prolog programming language in general. (With the possible exception of 4 – some systems compile, but do it very quickly.) Whether or not the points 1 and 2 applies varies between systems. For example, the people behind the commercial SICStus Prolog appears to regard Prolog more as a language for writing problem-solving modules to be embedded into other code, written in Java, C++, Python or what have you.⁷ It is only a guess, but we suspect that the

⁷ Mats Carlsson, main developer of SICStus Prolog, personal communication

people behind SICStus Prolog would not be willing to characterise it as a scripting language, although that may simply be a business decision more than anything else. In contrast, acting as a glue language and a language for programming in the large is the ambition of SWI-Prolog, witness the following quote from the online manual:⁸

SWI-Prolog positions itself primarily as a Prolog environment for ‘programming in the large’ and use cases where it plays a central role in an application, i.e., where it acts as ‘glue’ between components.

Interestingly, the people behind Erlang also think of it as a glue language:

We use Erlang as the glue to handle all orchestration, and then we use Python, C, Julia, ... It is actually a language *intended* to act as a hub towards other languages. The interfaces could be protocols, could be RESTful APIs, or other programming languages. It’s ideal for that.⁹

Erlang-style, network-transparent message-passing concurrency seems ideal for programming in the large on the Web, and for orchestrating both local and remote processes.

Erlang glue is likely a different kind of glue, with different properties, than Prolog glue. There is hope that combining Prolog and Erlang might yield an even stronger and more flexible glue of the required kind – a *superglue*, if you will.

Despite the good fit with Ousterhout’s characterization, neither Prolog nor Erlang are usually *advertised* as scripting languages, and they were hardly *designed* for the purpose of glueing applications together (or at least Prolog was not). They are both full-blown very flexible general-purpose programming languages with “batteries included.” Like JavaScript, they should probably be regarded as general-purpose languages which also *happen* to be suitable for scripting.

Web Prolog is a very *general* special-purpose scripting language. Similar to most other scripting languages, Web Prolog can be used for purposes for which “scripting” is not really the right word. Web Prolog comes with a focus on web *logic* programming as well as web *agent* programming, and in client-side browsers as well as server-side. Using Web Prolog, the owner of a node is for example able to build knowledge bases consisting of many millions of clauses – facts as well as rules, and/or web agents that can make use of such knowledge bases.

Is Web Prolog suitable for programming the Web?

To demonstrate that Web Prolog is suitable for programming the Web is what this book is about. But we are going further than that, as we are also aiming for a novel extension of the Web – the Prolog Web and even an Agentic Web – and a complete ecosystem, for which Web Prolog will serve as the default language.

⁸ <http://www.swi-prolog.org/pldoc/man?section=swiprolog>

⁹ See <https://www.youtube.com/watch?v=K8nxTSPHZhs5>, 8:58 into the discussion.

Web Prolog running in browsers

Regarding our third point, for Web Prolog to become a viable web programming complement to JavaScript it is vital that it can be implemented in browsers too, since this comes with a number of advantages:

1. When network connection is slow, it is best to perform the majority of computations in the browser.
2. This is where, in most scenarios, the *state* of an interaction between a user and an application should preferably be represented.
3. Client-side computation reduces the demands placed on nodes.
4. This is also where we find browser APIs that lets a web developer manipulate the DOM, store data locally, and add features such as spoken interaction, video or graphics to an application.

For Web Prolog in the browser to play the same role as JavaScript in the browser currently does, it must allow Web Prolog source code to be loaded from any server on the Web by means of the HTML `<script>` element, inline or with a link.

We do not, however, go all the way in our attempt to replicate Javascript's abilities as a scripting language. One thing that we do not at this time aim for is to make Web Prolog into a language with primitives for scripting the Document Object Model (DOM) in web browsers. With time it may well be interesting to develop such capabilities, but we believe it is simply too early to try to standardize them. In fact, we suspect that JavaScript will reign supreme in that role, and that if a browser is running Web Prolog locally, it will be as a *web worker*,¹⁰ with which the main JavaScript process is talking.

Web Prolog and security

As for security, similar to JavaScript, Web Prolog is a *sandboxed language*, open to the execution of untested or untrusted source code, possibly from unverified or untrusted clients without risking harm to the host machine or operating system. Therefore, Web Prolog does not include predicates for file I/O, socket programming or persistent storage, but must rely on the host environment in which it is embedded for such features.

Web Prolog does indeed share some of the properties that made JavaScript succeed on the Web. A visit to a web application server starts a JavaScript process on the client, running code that has been downloaded from the application's host to the user's client. Such a process must be allowed to run there (i.e. the owner of the client must allow it), and when it is (and most users allow it by default), it must execute in a way that does not harm the client. When a Web Prolog process is run on a node it must be allowed by its owner to do so, and it must run without harming the node. So one thing they share, Web Prolog and JavaScript, is the ability to run untested

¹⁰ https://en.wikipedia.org/wiki/Web_worker

and untrusted source code, authored by unverified and untrusted programmers, in a sandbox on someone else's computer.

In summary, while JavaScript's initial role was confined to client-side scripting in web browsers, its capabilities have expanded significantly. Today, it serves as a versatile glue language that connects various components in both client and server environments, as well as in the broader web development ecosystem. When designing Web Prolog and the rest of the Prolog Trinity ecosystem, we must of course try to learn from such a popular and versatile tool for web development, with a focus on the good parts in particular.

2.3 Erlang-style message-passing concurrency in Web Prolog

In a nutshell, if you were an actor in Erlang's world, you would be a lonely person, sitting in a dark room with no window, waiting by your mailbox to get a message. Once you get a message, you react to it in a specific way: you pay the bills when receiving them, you respond to Birthday cards with a "Thank you" letter and you ignore the letters you can't understand.

Fred Hébert

Corresponding to the send, receive, and spawn operations are built-in Web Prolog predicates `!/2`, `receive/1-2`, and `spawn/1-3`, among others. Their use is demonstrated through a series of simple examples, many drawn from Erlang tutorials and introductory texts. The examples are pitched at two kinds of reader. Prolog programmers taking their first steps into concurrent programming may wish to work through them carefully, perhaps alongside the Trinity demonstrator. Experienced Prolog teachers, by contrast, are invited to read as evaluators: does this material constitute a sound basis for teaching actor-style concurrency to Prolog students, and does Erlang-style concurrent programming sit naturally within a Prolog setting? We believe it does – that Web Prolog offers a promising vehicle for teaching both logic programming and actor programming within a single, web-based system, with minimal setup overhead for instructor and student alike.

2.3.1 Programmer talking to actor, actor talking to itself

If we need to hold a Prolog-style conversation with an actor, we may want to talk to a Prolog *shell*. By using a terminal attached to a shell, we can interact with it the way we normally interact with Prolog, by querying it, updating its dynamic database, and so on. For example, here is how we instruct the shell to split a list up into a prefix and a postfix using `append/3`:

```
?- append(Xs, Ys, [a,b,c]).
Xs = [], Ys = [a, b, c] ;
Xs = [a], Ys = [b, c] ;
```



```

Xs = [a, b], Ys = [c] ;
Xs = [a, b, c], Ys = [] ;
false.
?-

```

This book, as we have warned, is not a beginner's textbook, so how Prolog is able to come up with solutions to such queries is not something we will cover. Our focus is rather on the concurrency-oriented, message-passing features of Web Prolog, so we begin instead by introducing a couple of simple messaging primitives.

First, using `self/1`, we need to determine the identity of the shell we will be talking to:

```

?- self(Self).
Self = 85234512.
?-

```

What we got back here is a *pid*, an opaque identifier for an actor process. Conceptually, every pid is globally qualified and has the form `Id@Node`, where `Id` identifies the process within the node and `Node` is the node's URI. Thus, the full pid displayed above would be something like

```
85234512@'http://n7.org'
```

The shell has a presentation setting that may suppress the node component when the pid names a process on the shell's own node. With that setting enabled, the same pid is printed simply as `85234512`. This is only a display abbreviation: internally, and whenever the pid is serialised or transmitted to another node, the node component remains part of the pid. A pid embedded in a message therefore remains a globally meaningful return address even after the message crosses a node boundary.

The local identifier is chosen from a sufficiently large, unpredictable space to make guessing a live pid infeasible. Global identity comes from the complete `Id@Node` pair, rather than from the integer component alone.

As stated above, if we know the pid of an actor process we can send it a message. Just as any other kind of actor, the shell is equipped with a mailbox, and this is where the message will end up. The syntax and semantics of the send operator is easy to explain. With a call such as `Actor ! Message` the term `Message` is sent to the mailbox of the process identified as `Actor`, either by means of a pid or (as we shall see) using a registered *name*. For example, using `!/2` with the pid of our shell we can instruct it to send *itself* a message:

```

?- 85234512 ! hello.
true.
?-

```

Now we can use `receive/1` to make sure that the message we sent was received by the shell and is present in its mailbox. A single receive clause with a variable in the head and `true` in the body does the job:

```
?- receive({Message -> true}).
Message = hello.
?-
```

This is one of the simplest uses of the receive operation we can think of, but `receive/1` really is more complex than this, so expect more to come further ahead in this chapter. One more thing about the above call to `receive/1` is worth a mention already at this point though: had the mailbox been empty, `receive/1` would have *blocked* the execution of the process running as a shell until a message shows up.

There is an even simpler call to `receive/1` that one can make, namely this:

```
?- receive({}).
```

This call does not wait for any specific message, which just means that it will block forever, or until the process that runs it is killed. Control-C will work and send the user back to the `?-` prompt.

2.3.2 Two utility tools for programming in the shell

Copying and pasting pids is not very convenient. Instead, we can use a small shell feature (borrowed from SWI-Prolog): if a variable name is prefixed with a dollar sign, the shell substitutes it with the variable's most recent binding before running the query. For example:

```
?- $Self ! hello.
true.
?-
```

Here, the shell kept track of the most recent binding of `Self` to a value (85234512 in this case) and substituted the term `$Self` with that value before the query was run.

During interactive development, it is also important to inspect the shell's mailbox. Using `receive/1` is often awkward here: if the mailbox happens to be empty, `receive/1` will block, effectively freezing the session until the call is interrupted (e.g. with Control-C). Moreover, if we want to examine *all* pending messages, we would need to call `receive/1` repeatedly, without necessarily knowing how many messages there are – which again risks blocking, or that messages are left behind.

A more convenient tool is the utility predicate `flush/0`. It prints (and removes) all messages currently in the shell's mailbox and, crucially, it never blocks.

Below, we demonstrate both of these utilities by first sending yet another message to our shell and then use `flush/0` to inspect and empty the contents of its mailbox:

```
?- $Self ! goodbye.
true.
?- flush.
```

```
Shell got hello
Shell got goodbye
true.
?-
```

The `$Var` substitution mechanism (or “dollar notation”) in combination with `flush/0` comes in very handy during interactive programming in the shell and we shall rely on them extensively when running examples through all of the book.

2.3.3 Actors talking to other actors

It takes two to tango, and an actor talking to itself is not very interesting. Fortunately, as already mentioned, an actor is capable of spawning other actors and then start talking to them.

The built-in predicate `spawn/1-3` is used to create new actor processes. In a call such as `spawn(Goal, Pid)` it expects a callable goal to be passed in the first argument, and will bind the variable in the second argument to the pid of the actor created. Here is how this might look like:

```
?- spawn(echo_actor, Pid).
Pid = 72347585.
?-
```

The goal calls a predicate which determines the behavior of the actor. The actor process runs in parallel with the caller, the shell in this case. Here is an example script that implements a simple echo server:

```
echo_actor :-
    receive({
        echo(From, Msg) ->
            From ! echo(Msg),
            echo_actor
    }).
```

We have seen this piece of code before; it is now time to explain how it works. The predicate `echo_actor/0` implements a simple receive loop: each call to `receive/1` selects a message of the form `echo(From, Msg)` from the mailbox, sends `echo(Msg)` back to the actor identified by `From`, and then recurses.

To test the server interactively, we send it a message with `From` bound to the shell’s own pid, obtained via `self/1`:

```
?- self(Self), $Pid ! echo(Self, hi).
Self = 85234512.
?- flush.
Shell got echo(hi)
```

```
true.
?-
```

The sending is asynchronous, i.e. `!/2` does not block waiting for a response but continues immediately. Also, sending a message to a pid always succeeds and never throws an error, even if the pid points to a non-existing process.

In that case the message is simply discarded. This means that short of having the targeted actor return a confirmation message we will not always know if the message reached it. However, the above case did not leave us in the dark, as the actual echo served as a confirmation message.

Web Prolog guarantees per-sender FIFO delivery: for any fixed sender–receiver pair, messages are enqueued in the receiver’s mailbox in the order they were sent. There is no single total order across different senders; concurrent sends to the same receiver may be observed in either order by `receive/1-2`.

Monitoring

In a call such as `spawn(Goal, Pid, Options)` the optional third argument is a list of options used for configuring the actor. One such option is `monitor`, which if set to `true` requests that the parent process be notified when the spawned actor terminates.

In the following example, `monitor(true)` instructs the child actor to send a down message to its parent just before terminating:

```
?- spawn(2 > 1, Pid, [
    monitor(true)
]).
Pid = 60367387.
?- flush.
Shell got down(60367387,60367387,true)
true.
?-
```

A down message has the form `down(Ref, Pid, Reason)` where `Pid` is the process identifier of the terminated actor, `Reason` describes *why* it terminated, and `Ref` identifies the *monitor instance* that produced this notification. When monitoring is installed via `monitor(true)`, the monitor reference is chosen to be identical to the child’s pid, which is why the first two arguments coincide in the example.

In addition to the `monitor` option, the predicate `monitor(Pid, Ref)` is provided, which installs a monitor and returns a fresh reference in `Ref`. The reference matters whenever there may be more than one monitor, because each monitor produces its own down message and we must be able to tell them apart:

```
?- spawn(sleep(3), Pid),
    monitor(Pid, Ref1),
    monitor(Pid, Ref2).
Pid = 41544343,
```

```

Ref1 = 10924110,
Ref2 = 92041933.
?- flush.
Shell got down(10924110,41544343,true)
Shell got down(92041933,41544343,true)
true.
?-

```

Here the same parent process installs two monitors on the same child. When the child terminates after three seconds, two `down` messages are delivered, one per monitor. The distinct references (`Ref1` and `Ref2`) identify which monitor instance each message corresponds to.

Using the `monitor` option is the *safest* way to establish monitoring, because it is *atomic* with respect to process creation: either the child is spawned with monitoring already in place, or it is not spawned at all. In contrast, calling `monitor/2` after `spawn/2-3` is a separate step and therefore not atomic. This means that a very short-lived child may terminate in the gap between the `spawn` returning and the monitor being installed, in which case the parent will never receive a `down` message. For actors that may complete immediately (or fail immediately), `monitor(true)` avoids this race condition.

Monitoring can be stopped by passing the monitor ref to `demonitor/1-2`. When monitoring is installed using the `monitor` option, the pid of the monitored actor can also be used since it is identical to the ref.

The above example of the use of `monitor/2` was trivial, but in Chapter 5, both `monitor/2` and `demonitor/1-2` will be put to more serious use.

Registering

Calling `register(Name,Pid)` associates the atom `Name` with `Pid`. The name can be used instead of the pid when calling `!/2`. For example, we can register our shell under the name `shell`:

```

?- self(Self), register(shell, Self).
Self = 85234512.
?- shell ! hello.
true.
?- spawn(shell ! goodbye).
true.
?- flush.
Shell got hello
Shell got goodbye
true.
?-

```

Registering is useful when we want to offer a service that should always be available under a name that is easy to remember. Should a crash occur, all our system needs to do is to restart it and associate the same name with the pid of the new process.

Exiting

Web Prolog supports two predicates, `exit/1` and `exit/2`, that can be used to terminate an actor process.

If the process calls `exit(Reason)` it will terminate immediately, and if monitored by its parent process, the parent will be sent a `down` message, containing the term that `Reason` was bound to when the predicate was called. Here is a simple and somewhat silly example where a process is spawned and monitored by the shell. Since the process is told to exit immediately with the reason `my_reason`, the shell receives a `down` message with the atom `my_reason` in its third argument:

```
?- spawn(exit(my_reason), Pid, [
    monitor(true)
]).
Pid = 91325643.
?- flush.
Shell got down(91325643,91325643,my_reason)
true.
?-
```

It sometimes happens that we need to terminate an actor process by force. Our echo server, for example, can only be terminated in this way. The predicate `exit/2` can be used to terminate any actor process with a known pid. Calling `exit(Pid, Reason)` sends an exit *signal* (not a message) to the actor with that pid, killing it.

If we do not know the pid, but it has a name that we know, we can use that instead. To see how this works, let us spawn a new monitored echo server and register it:

```
?- spawn(echo_actor, Pid, [
    monitor(true)
]),
    register(echo_actor, Pid).
Pid = 21562390.
?-
```

Now, let us say we change our minds and want to get rid of it again:

```
?- whereis(echo_actor, Pid),
    exit(Pid, 'We changed our minds!').
Pid = 21562390.
?- flush.
Shell got down(21562390,21562390,'We changed our minds!')
true.
?-
```

By calling `exit/2` with the pid in the first argument and a term detailing the reason for exiting in the second, a signal was sent to the process that terminated it. Note that a call to `exit/2` will only accept a pid in its first argument, so if all we have is a name, the built-in predicate `whereis/2` must be used to locate it.

Note that the reason passed to `exit/1-2` can be any term, not just an atom. This means that it can be used to transfer an arbitrarily large chunk of information back to the parent. However, in most cases an atom is all that is needed, and it is worth noticing that using `true` as a reason can be used to suggest to the parent that the termination was *normal*, even though it was caused by a call to `exit/1-2`.

What might seem a bit odd is that even though 21562390 is now provably dead, trying to send it a message using a pid or calling `exit/2` still succeeds without an error, although no `down` message is being sent:

```
?- self(Self), $Pid ! echo(Self, bye).
Self = 85234512.
?- exit($Pid, 'We changed our minds!').
true.
?- flush.
true.
?-
```

Treating `!/2` and `exit/2` as no-ops when the pid points to a non-existent process is consistent with how it works in Erlang. However, trying to send a message using the *name* of a non-existent process generates an error:

```
?- echo_actor ! echo($Self, bye).
Error: The name 'echo_actor' is not associated with a pid.
?-
```

This too is consistent with how Erlang works.

Linking

A further spawn option, `link`, governs the lifetime of the spawned actor. If the option is set to `true`, the life of the new actor is tied to the life of its parent: should the parent terminate – for whatever reason – the child is forced to terminate too. In our proposal, the default value of this option is `true`.

Readers who know Erlang should pause here, because this is a point at which Web Prolog deliberately departs from its role model, in two ways. First, Erlang's links are bidirectional, whereas the dependence just described runs in one direction only, from parent to child. Second, Erlang's `spawn` does not link at all – linking requires `spawn_link` – whereas in Web Prolog the tie to the parent is the default and must be explicitly declined. Both departures are motivated by the client–owner separation of the open Web, and both are justified in Chapter 4; for now, we simply demonstrate the behaviour.

In the following example, the outer call to `spawn/2` creates an actor whose goal spawns a second, non-terminating actor and then completes. When its goal completes, the outer actor terminates – and because the inner actor was created with the default `link(true)`, it is automatically killed along with it:

```
?- spawn(spawn(receive({})), Pid).
Pid = 10782012.
?-
```

Calling the built-in predicate `actors/1` confirms that only one actor remains alive – the shell itself:

```
?- actors(Alive).
Alive = [91033267].
?-
```

The dependence does not run the other way. A parent whose child terminates – even abnormally – is unaffected:

```
?- spawn((spawn(exit(die)), receive({})), Pid).
Pid = 40276924.
?-
```

Even though the child killed itself immediately, calling `actors/1` again shows that its parent 40276924 is still alive:

```
?- actors(Alive).
Alive = [91033267, 40276924].
?-
```

The link, then, makes the life of a child dependent on the life of its parent, and never the reverse.

Since the option is on by default, declining it must be done explicitly, by passing `link(false)` *when the child is spawned*. Note carefully where the option goes: it belongs to the child's own spawn, and describes that child's relationship to its parent. In the following example, it is the *inner* actor that is spawned with `link(false)`, and it consequently survives the termination of the outer actor that created it:

```
?- spawn(spawn(receive({}), _, [link(false)]), Pid).
Pid = 10023812.
?- actors(Alive).
Alive = [91033267, 24917621].
?-
```

The outer actor (10023812) is gone, its goal having completed, but the inner actor (24917621) lives on – now without a living parent. Actors of this kind, deliberately detached from the hierarchy that created them, are useful for long-lived services, but they are also actors that no parent's termination will ever clean up. For a reason explained in Chapter 4, the distributed setting therefore restricts them: clients are typically required to spawn with `link(true)`, with `link(false)` reserved for node owners.

A hierarchy of actors

The lifetime dependence introduced earlier is *transitive*: because each child depends on its parent, terminating an actor tears down not just its immediate children but its entire descendant subtree. The following example makes this cascade visible.

The predicate `root/0` spawns a child and then blocks indefinitely. The child spawns a grandchild and does the same. Neither names anything or monitors anything; the only structure is the spawn hierarchy itself:

```
root :-
    spawn(child),
    receive({}).

child :-
    spawn(grandchild),
    receive({}).

grandchild :-
    receive({}).
```

We spawn the root from the shell and then confirm, using `actors/1`, that all three processes (plus the shell itself) are alive:

```
?- spawn(root, Pid).
Pid = 50318842.
?- actors(Alive).
Alive = [91033267, 50318842, 71200934, 33910576].
?-
```

The shell is 91033267; the other three are the root, the child, and the grandchild. Now we terminate the root, and *only* the root, by force:

```
?- exit($Pid, die).
true.
?- actors(Alive).
Alive = [91033267].
?-
```

The root is gone, as expected – but so are the child and the grandchild, even though we sent an exit signal to neither. This is the cascade: terminating the root made its child terminate, which in turn made the grandchild terminate. Lifetime dependence propagates down the whole hierarchy.

The reverse does not hold. Terminating a child never forces its parent to terminate, because the dependence is *unidirectional* – as we saw in the previous section and central to the containment discipline we rely on in Chapter 4. A client that spawns a subtree of workers therefore cannot leave orphans behind it: when the client's own process ends, everything it spawned, however deeply nested, is reclaimed with it.

As flagged above, Erlang behaves differently here. Its links are *bidirectional* and its exit signals propagate *symmetrically* through the link set, subject to trapping; the asymmetric, strictly downward propagation shown above is a deliberate Web Prolog design choice, motivated by the client–owner separation of the open Web and justified in Chapter 4.

2.3.4 A closer look at receive/1-2

As shown already, but only for a trivial case, an actor process uses the receive primitive to extract messages from its mailbox. Since the syntax and semantics of `receive/1-2` is fairly complex, a closer look and more examples are needed. Below we give several simple examples illustrating different ways to use the `receive/1-2` predicate in Web Prolog, demonstrating how to handle different *types* of messages, use *timeouts*, apply *guards*, and more. Other examples illustrate how messages are deferred and handled in subsequent `receive/1-2` calls, demonstrating the flexibility of passing and processing messages in Web Prolog.

Basic receive

In Web Prolog, just like in Erlang, the receive operation specifies an ordered sequence of *receive clauses* delimited by semicolons. A receive clause always has a *head* (a term) and a *body* of Prolog goals. Schematically, the basic form of a receive call looks like this:

```
receive({
    Head1 -> Body1 ;
    Head2 -> Body2 ;
    ...
    HeadN -> BodyN
})
```

Often, the head is just a single term serving as a *pattern*. Any term will do, ground or non-ground, and even a bare variable is fine.

As in Erlang, `receive/1` scans the mailbox looking for the first message (i.e. the oldest) that matches a pattern in any of the receive clauses, blocking if no such message is found. If a matching clause is found, the message is removed from the mailbox and the body of the clause is called. In Web Prolog, just like in Erlang, values of any variables bound by the matching of the pattern with a message are available in the body of the clause.

Often, a `receive/1` call waits for a specific message and executes the corresponding code in the body of the receive clause if a message that matches the pattern shows up. For example, the following call waits for a message in the form of `hello(Name)` and prints a greeting when it appears:

```
receive({
    hello(Name) ->
        format("Hello, ~s!~n", [Name])
})
```

We can specify multiple patterns in a single `receive/1` call. For example, this call handles messages of either the form `hello(Name)` or the form `goodbye(Name)`, but only one of them:

```
receive({
    hello(Name) ->
        format("Hello, ~s!~n", [Name]) ;
    goodbye(Name) ->
        format("Goodbye, ~s!~n", [Name])
})
```

We can use the `_` pattern to catch any message. In the following case, any message that does not match `hello(Name)` will be caught by the `_` pattern:

```
receive({
    hello(Name) ->
        format("Hello, ~s!~n", [Name]) ;
    _ ->
        format("Unknown message received.~n")
})
```

We can nest `receive/1-2` calls. Here, after having received the `start(Name)` message, the call waits for a `continue(Msg)` message:

```
receive({
    start(Name) ->
        format("Starting with ~s.~n", [Name]),
        receive({
            continue(Msg) ->
                format("Continuing with ~s~n", [Msg])
        })
})
```

Messages deferred

If no pattern matches a message in the mailbox, the message is *deferred*, which means that the message does not match any of the patterns in the current `receive/1-2` call and remains in the process's mailbox, possibly to be handled later in the control flow of the process. The `receive` is still running, waiting for more messages to arrive, and for one that will match. Some simple examples illustrating this behavior are shown below.

In the following example, the `goodbye("Bob")` message – which is the oldest and therefore first in line – does not match the clause in the first receive call and is deferred. The `hello("Alice")` message matches that clause, and then the clause in the second receive call handles the `goodbye("Bob")` message:

```
?- self(Self),
    Self ! goodbye("Bob"),
    Self ! hello("Alice"),
    receive({
        hello(Name1) ->
            format("Hello, ~s!~n", [Name1])
    }),
    receive({
        goodbye(Name2) ->
            format("Goodbye, ~s!~n", [Name2])
    }).
Hello, Alice!
Goodbye, Bob!
true.
?-
```

This behavior is particularly useful if we expect two messages but are not sure which one will arrive first.

Guards

The head of a receive clause can, in addition to the pattern, optionally specify a *guard* in the form of a Prolog goal. A head of the form `Pattern if Goal` makes pattern matching more expressive: the clause is selected only if the incoming message matches `Pattern` *and* `Goal` succeeds. Here is an example that distinguishes positive from non-positive numbers:

```
receive({
    number(N) if N > 0 ->
        format("Positive number: ~p~n", [N]) ;
    number(N) if N <= 0 ->
        format("Non-positive number: ~p~n", [N])
})
```

This is the familiar use of guards, and it works in Erlang too: the guard is a test, a yes-or-no question about values already bound by the pattern.

But in Web Prolog a guard can be far more than a test, and the next example is worth dwelling on, because it exposes one of the deepest points of contact between the actor model and logic programming. The guard is an ordinary Prolog goal, evaluated against the process's knowledge base – and like any Prolog goal, it can *bind variables that the pattern never mentions*:

```
?- self(Self),
   Self ! hello(xantippa),
   receive({
       hello(W) if husband(W, H) ->
           format("Hello, ~w, say hello to ~w!\n", [W,H])
   }).
Hello, xantippa, say hello to socrates!
true.
?-
```

Look closely at what happened. The message supplied `W=xantippa`. The variable `H` appears nowhere in the pattern `hello(W)`; it is introduced by the guard. When `husband(W, H)` was evaluated against the knowledge base, it did not merely *test* whether Xanthippe has a husband – it *looked him up*, binding `H` to `socrates`, and that binding then flowed into the body of the clause and did real work there.

This is something no Erlang guard can do. Erlang restricts guards to a small repertoire of side-effect-free built-in tests precisely so that guard evaluation cannot compute, query, or bind – a guard may inspect the message but never enrich it. In Web Prolog the guard is a goal, so message reception and knowledge-base query become a single act: the clause fires only when the relation holds, and *the witness to that relation is delivered to the body as a binding*. Pattern matching and deduction, two mechanisms that Erlang keeps strictly apart, here compose into one.

It is this difference – a guard that genuinely *reasons* rather than merely tests – that leads us to depart from Erlang’s vocabulary and write `if` where Erlang writes `when`. The choice is deliberate. `when` suggests a side-condition on matching, a gate that opens or stays shut; `if` suggests a logical antecedent, a relation whose satisfaction is itself part of what the clause computes. The name follows the semantics: in Web Prolog, a guard is a relational test, not a restricted one. We return to this capability in Section 2.4.6, where guards that consult the knowledge base become the basis for event-driven state machines whose transitions depend on what the process knows.

Timeouts

The optional second argument of `receive/1-2` expects a list of options. The `timeout` option takes an integer or float that specifies the number of seconds before the call will succeed anyway, even if no match has been found. The `on_timeout` option takes a goal that is called if timeout occurs. Here is an example of its use:

```
receive({
    hello(Name) ->
        format("Hello, ~s!\n", [Name])
}, [ timeout(5),
     on_timeout(format("No match received in 5 seconds.\n"))
])
```

If no message of the form `hello(Name)` is received within 5 seconds, the code in the `on_timeout` option is executed.

An implementation of `sleep/1`

Here is how a predicate `sleep/1` that suspends execution `Time` seconds can be defined:

```
sleep(Time) :-
    receive({},[
        timeout(Time)
    ]).
```

The call to `receive/2` does not wait for messages, but when `Time` seconds have passed, it still succeeds.

An implementation of `flush/0`

As we noted earlier, being able to inspect the contents of the shell's mailbox during interactive programming is important, and `flush/0` is a nice tool for doing just that. Its definition also serves as yet another example of the use of the `timeout` option:

```
flush :-
    receive({
        Message ->
            format("Shell got ~q~n",[Message]),
            flush
    },[
        timeout(0)
    ]).
```

The value `0` of the `timeout` option ensures that the loop terminates immediately if no messages remain in the mailbox. This is how the hanging of the `receive/1` call is avoided.

A simple priority queue

To demonstrate the use of the `if` operator and the use of two `receive/2` options that causes a goal to run on timeout, we show a priority queue example borrowed from Fred Hébert's textbook on Erlang (Hébert, 2013). The purpose is to build a list of messages with those with a priority above 10 coming first:

```
important(Messages) :-
    receive({
```

```

        Priority-Message if Priority > 10 ->
            Messages = [Message|MoreMessages],
            important(MoreMessages)
    },[ timeout(0),
        on_timeout(normal(Messages))
    ]).

normal(Messages) :-
    receive({
        _-Message ->
            Messages = [Message|MoreMessages],
            normal(MoreMessages)
    },[ timeout(0),
        on_timeout(Messages=[])
    ]).

```

The timeout set to 0 means that it will occur immediately, but the system tries all messages currently in the mailbox first.

Below, we test this program by first sending four messages to the shell process, and then calling `important/1`:

```

?- self(S),
   S ! 15-high, S ! 7-low, S ! 1-low, S ! 17-high.
S = 34871244.
?- important(Messages).
Messages = [high,high,low,low].
?-

```

Note that the implementation of the priority-queue example relies on the deferring behavior of `receive/1-2` and would not work without it.

The receive predicate is semi-deterministic

The predicate `receive/1-2` is *semi-deterministic*, i.e. it either fails, or succeeds exactly once. The only way it will fail is if the goal in the *body* of one of its receive clauses fails, or if timeout occurs and the goal passed in the `on_timeout` option fails.

To see how it pans out in a simple corner case, consider the following two calls:

```

receive({foo(X) -> true})      receive({foo(X) -> fail})

```

The call on the left will succeed if a message matching the pattern `foo(X)` appears in the mailbox of the actor process executing the call, a term such as `foo(314)` for example. The call on the right will fail (and possibly cause backtracking) once `foo(314)` appears. Only by the first call will the variable `X` be bound (to 314). Both calls will remove the matched message from the mailbox – and, as we now explain,

that removal stands even under backtracking. In both cases, if a message appears that does not match the pattern, it is deferred.

One point deserves emphasis, because it has no analogue in pure Prolog. The send and receive operations are effects on a mailbox, and like the database mutations of `assert/2` and `retract/1`, they are not undone on backtracking. A message sent by `!/2` remains in its target mailbox even if the sending goal is later backtracked into. A message matched by a receive is removed whether the clause body then succeeds or fails, and is not restored by backtracking past the call. Consequently, a goal re-entered through backtracking does not see the mailbox as it was on first entry – it sees the current mailbox, reflecting every send and consumption (including those of other actors) that has occurred in the meantime. Backtracking over message operations is therefore re-execution against the current state of a shared, time-varying store, not a return to the state that held on first entry. This is the same discipline Prolog programmers already accept for the dynamic database, now extended to the mailbox.

To implement a looping behaviour, Prolog programmers occasionally use a *failure-driven* loop that relies on backtracking rather than recursion. Using this technique, our echo server can be rewritten like so:

```
echo_actor  :-
    repeat,
    receive({
        echo(From, Msg) ->
            From ! echo(Msg),
            fail
    }).
```

For an Erlang programmer, this use of `receive/1` may come as a surprise and is not a technique that can be used in Erlang. In this particular case, a Web Prolog programmer would be advised to stick to the recursive version. However, there are cases when a failure-driven loop is the only way forward and we shall look at an important such case towards the end of this chapter.

2.4 Erlang-style programming examples in Web Prolog

Reading the code was fun – I had to do a double take – was I reading Erlang or Prolog – they often look pretty much the same.

Joe Armstrong (p.c. June 18, 2018)

Not only do Web Prolog programs *look* a lot like Erlang, they *behave* a lot like Erlang too. A good way to demonstrate this is to translate a fair number of different Erlang programming examples borrowed from tutorials and text books into Web Prolog and show that they run just like in Erlang. In this section we shall look at a count server, a fridge simulation, a technique for hiding message protocols behind predicate APIs, delayed sending, actors playing ping-pong, event-driven state machines whose

transitions consult the knowledge base, and – most importantly – a demonstration that Prolog’s backtracking search can be placed under external, message-driven control. They are all concurrent programs, but only locally so. (Distributed programming is dealt with in Chapter 4, and reusable behaviours such as generic servers and supervisors in Chapter 5.)

2.4.1 A count server

We have already looked at an example of a *stateless* server, namely the echo server presented earlier in this chapter. Let us now turn to *stateful* servers and show how state may be programmed. In the following example, we demonstrate how to script an actor that can keep a *count*:

```
count_actor(Count0) :-
    receive({
        count(From) ->
            Count is Count0 + 1,
            From ! count(Count),
            count_actor(Count) ;
        stop ->
            true
    }).
```

The predicate `count_actor/1` defines a loop where a call to the built-in predicate `receive/1` tries to select a message of the form `count(From)` from the mailbox, increment the counter, send the current count back to the actor referenced by the pid to which the variable `From` is bound, and continue looping by making a recursive call. Note how the state of the counter – the current count, that is – is kept in the argument of the predicate. Note also that we added a second receive clause that will allow us to terminate the server without using `exit/2`. We just have to send it a message of the form `stop`.

Here is how an actor following this script can be made to perform when spawning it:

```
?- spawn(count_actor(0), Pid, [
    monitor(true)
]).
Pid = 60367387.
?- 
```

In the next step the send operator `!/2` is used for sending a message to the spawned actor, instructing it to increment the count and to return the result in the form of a message:

```
?- self(Self), $Pid ! count(Self).
```

```
Self = 41167597.
```

```
?-
```

A call to `receive/1` can be made in order to collect the message arriving from the actor:

```
?- receive({Count -> true}).
```

```
Count = count(1).
```

```
?-
```

Here, `!/2` is used to send two messages to the actor that will end up in its mailbox, in the order they were sent:

```
?- $Pid ! count($Self), $Pid ! stop.
```

```
true.
```

```
?-
```

Finally, `flush/0` is used to inspect the contents of the mailbox belonging to the toplevel process:

```
?- flush.
```

```
Shell got count(2)
```

```
Shell got down(60367387,60367387,true)
```

```
true.
```

```
?-
```

Because the actor was monitored, a `down` message was found in addition to the current count.

2.4.2 A bigger, tastier example

As a tastier example of how a process can be made to hold an updatable state during a conversation we have adapted a fridge simulation example from Fred Hébert's introduction to Erlang (Hébert, 2013):¹¹

```
fridge(FoodList0) :-
    receive({
        store(From, Food) ->
            self(Self),
            From ! Self-ok,
            fridge([Food|FoodList0]);
        take(From, Food) ->
            self(Self),
            (    select(Food, FoodList0, FoodList)
```

¹¹ <http://learnyousomeerlang.com/more-on-multiprocessing#state-your-state>

```

-> From ! Self-ok(Food),
    fridge(FoodList)
;   From ! Self-not_found,
    fridge(FoodList0)
);
terminate ->
true
}).

```

The program creates a process allowing three operations: storing food in the fridge, taking food from the fridge, and terminating the fridge. It is only possible to take food that has been stored beforehand. Again, with the help of recursion the state of a process can be held entirely in the argument of the predicate. In this case we choose to store all the food as a list, and then look in that list when someone needs something.

Assuming the above program is already available, the following session creates the server process. We can then call `self/1`, `!/2` and `flush/0` from the shell in order to simulate the actions of a client:

```

?- spawn(fridge([]), Pid, [
    monitor(true)
]).
Pid = 77346122.
?- self(Me), $Pid ! store(Me, meat), $Pid ! store(Me, cheese).
Me = 97216744.
?- flush.
Shell got 77346122-ok
Shell got 77346122-ok
true.
?- self(Me), $Pid ! take(Me, cheese).
Me = 97216744.
?- flush.
Shell got 77346122-ok(cheese)
true.
?- $Pid ! terminate.
true.
?- flush.
Shell got down(77346122,77346122,true)
true.
?-

```

2.4.3 Hiding the details of protocols

In the previous example, we expected programmers to know the details of the protocol that must be followed when interacting with our fridge simulation, which forced them to make raw calls using the send operator in combination with the `flush/0` utility predicate. As stated by Fred Hébert in his book, that is often a useless burden, and good way around it is to abstract messages away with the help of predicates (or in Hébert's case, functions) dealing with receiving and sending them:

```
store(Pid, Food, Response) :-
    self(Self),
    Pid ! store(Self, Food),
    receive({
        Pid-Response -> true
    }).
```

```
take(Pid, Food, Response) :-
    self(Self),
    Pid ! take(Self, Food),
    receive({
        Pid-Response -> true
    }).
```

Calling `receive/1` immediately after having sent the message guarantees that the communication between client and server stays synchronous. Using `store/3` and `take/3`, the interaction with the fridge becomes somewhat easier:

```
?- spawn(fridge([]), Pid, [
    monitor(true)
]).
Pid = 55289322.
?- store($Pid, cheese, Response).
Response = ok.
?- take($Pid, cheese, Response).
Response = ok(cheese).
?-
```

When dealing with actors with more complex protocols, such abstractions turns out to be of considerable value.

2.4.4 Using delayed sending

Appendix A documents a built-in predicate `send/2-3` that allows a process to delay the sending of a message, but also to cancel the sending if needed, by calling `cancel/1`. This can be used to implement an alarm that can be canceled, like so:

```
alarm :-
    receive({
        ring ->
            writeln('Alarm ringing!'),
            alarm;
        stop ->
            true
    }).
```

Here we schedule the alarm to ring in 5 seconds, with an id set to alarm1:

```
?- spawn(alarm, Pid),
    send(Pid, ring, [
        delay(5),
        id(alarm1)
    ]).
Pid = 60034123.
?-
```

Then we cancel the alarm after a couple of seconds:

```
?- cancel(alarm1).
true.
?-
```

If the cancellation succeeds, the message is withdrawn and the alarm never rings. If `cancel/1` is called too late, the alarm message has already been delivered and “Alarm ringing!” will be printed.

2.4.5 Prolog actors playing ping-pong

Since the servers in the previous sections are running in parallel to the shell and are talking to it using asynchronous messaging, we have already demonstrated the use of concurrency. Below, in a probably more convincing example inspired by a user’s guide to Erlang,¹² two processes are first created and then start sending messages to each other a specified number of times:

```
ping(0, Pong_Pid) :-
    Pong_Pid ! finished,
    format('Ping finished.~n').
ping(N, Pong_Pid) :-
    self(Self),
    Pong_Pid ! ping(Self),
    receive({
```

¹² See http://erlang.org/doc/getting_started/conc_prog.html

```

    pong ->
        format('Ping received pong.~n')
    }),
    N1 is N - 1,
    ping(N1, Pong_Pid).

pong :-
    receive({
        finished ->
            format('Pong finished.~n');
        ping(Ping_Pid) ->
            format('Pong received ping.~n'),
            Ping_Pid ! pong,
            pong
    }).

ping_pong :-
    spawn(pong, Pong_Pid),
    spawn(ping(3, Pong_Pid)).

```

When `ping_pong/0` is called the behavior of this program exactly mirrors the behaviour of the original version in Erlang:

```

?- ping_pong.
true.
Pong received ping.
Ping received pong.
Pong received ping.
Ping received pong.
Pong received ping.
Ping received pong.
Ping finished.
Pong finished.
?-

```

This is a concurrent program, but its execution is not done in parallel, but rather as is illustrated in Figure 2.3.

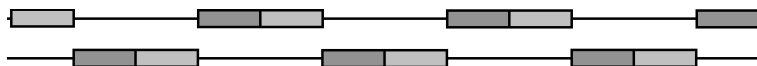


Fig. 2.3 A timeline where the upper line represent the “pinger” and the lower line the “ponger,” and where each gray bar represent a task consisting of the sending of a message (in light gray) and the reception of the response (in darker gray)

2.4.6 Event-driven state machines

Building on the guard mechanism introduced above, we now show a simple but representative application: an event-driven state machine whose transitions can consult the process's knowledge base. The key point is that clause selection can depend not only on the incoming message pattern, but also on arbitrary relational conditions, and any bindings produced by those conditions can be used by the transition action.

Web Prolog's `receive` primitive can implement such machines directly. Message patterns represent events, guards represent transition conditions, and clause bodies implement actions and state updates. Figure 2.4 depicts a machine with two states and four transitions.

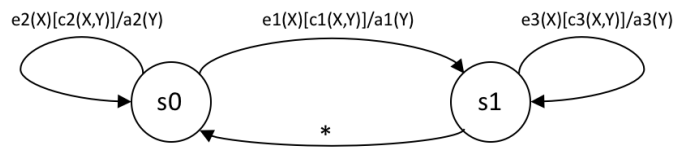


Fig. 2.4 A simple event-driven state machine. `s0` is the initial state.

Using predicate names to encode states, one possible implementation is:

```

s0 :-
    receive({
        e1(X) if c1(X,Y) ->
            a1(Y),
            s1 ;
        e2(X) if c2(X,Y) ->
            a2(Y),
            s0
    }).

s1 :-
    receive({
        e3(X) if c3(X,Y) ->
            a3(Y),
            s1 ;
        _AnyEvent ->
            s0
    }).
  
```

In state `s0` the machine waits for an event message of the form `e1(X)` or `e2(X)` in the current process's mailbox. If `e1(X)` matches and `c1(X,Y)` succeeds, the action `a1(Y)` is executed and the machine transitions to `s1`. If `e2(X)` matches and `c2(X,Y)` succeeds, the action `a2(Y)` is executed and the machine remains in `s0`. The other state is analogous: `e3(X)` triggers a guarded self-loop in `s1`, while any other event falls back to `s0`.¹³

This highlights a capability of Web Prolog that Erlang deliberately restricts. Erlang enforces purity and efficiency by restricting guards to a small set of built-in primitives and by disallowing calls to user-defined functions. For the reliability-oriented concurrent systems Erlang was designed to support, that trade-off made

¹³ Note that `s0` defers unknown events while `s1` discards them; the designer must choose deliberately, a matter we return to with SXML in Chapter 6.

good sense. In Web Prolog, by contrast, any goal may be used as a guard, though only its first solution is considered, and variables bound by that goal remain available in the body. This makes `receive` more expressive than in Erlang: clause selection may depend not merely on the shape of an incoming message, but on relational conditions evaluated against the process's (shared and private) knowledge bases. This can make knowledge-base-dependent transitions feel natural and concise.

The price of this expressiveness is that responsibility shifts to the programmer. Guards should be kept efficient, and they should remain free of side effects. In our view, however, the gain is substantial. For state-machine programming in particular, it is often useful to condition a transition on relational knowledge rather than on a fixed repertoire of primitive tests. This broader role is also one reason why we prefer the operator name `if` to Erlang's `when`: it suggests that guard evaluation in Web Prolog is not merely a restricted side-condition on pattern matching, but a genuine relational test.¹⁴

2.4.7 Getting answers through backtracking

We come now to the example that most directly tests the central claim of this chapter. Back in Section 2.2.2 we worried aloud about an impedance mismatch: Prolog's model is relational and nondeterministic, driven by backtracking search; Erlang's is functional and message-passing, with no backtracking at all. When the two are combined in one language, what happens at the seam – where a goal that can produce many solutions meets a `receive` that consumes messages one at a time? Does backtracking compose with messaging, or do the two models grind against each other?

The following example is the test, and it goes well beyond anything Erlang can express. We suggest the seam holds – that backtracking and messaging compose not awkwardly but naturally. Judge for yourself.

Consider a spawned goal that enumerates the solutions to `human(Who)`, but instead of collecting them all at once, sends the *first* solution back to its parent and then waits. It will produce the next solution only when told to, by a `next` message, and will stop on `stop`:

```
?- self(Self),
    spawn(( human(Who),
             Self ! Who,
             receive({
```

¹⁴ Event-driven state machines have historically been less prominent in the Prolog world, most likely because message-passing concurrency arrived relatively late and unevenly across Prolog systems, gaining wider traction only with facilities such as those in the ISO Prolog Threads draft. By contrast, Erlang/OTP treats this programming model as foundational, as exemplified by `gen_statem`. In Chapter 6 we build on this observation and propose Web Prolog as a scripting language for Statechart XML (SXML), a simplification of W3C's SCXML standard which is a better fit with Prolog.


```

        next -> fail ;
        stop -> true
    })
    ), Pid).
Pid = 76123351,
Self = 90054377.
?- flush.
Shell got socrates
true.
?- $Pid ! next.
true.
?- flush.
Shell got plato
true.
?- $Pid ! stop.
true.
?-

```

Follow the control flow carefully, because the mechanism is more subtle than its three lines suggest. The spawned goal solves `human(Who)`, binding `Who=socrates`, sends that solution to the parent's mailbox with `Self!Who`, and then blocks in `receive`. So far this is ordinary sequential Prolog with one message send. The interesting part is what `next` does. The `next` clause has body `fail`, so receiving `next` makes the `receive fail` – and a failure here propagates backwards into `human(Who)`, forcing it to backtrack and produce its next solution, `plato`. Execution then runs forward again through `Self!Who`, sending `plato`, and blocks once more. Each `next` thus drives one step of backtracking through the solution set; `stop`, whose body is `true`, lets the goal succeed and the actor terminate.

What makes this work is precisely the property established in Section 2.3.4: message operations are effects that backtracking does not undo. When the goal backtracks from `plato`'s `receive` back into `human(Who)`, the send that already delivered `socrates` is *not* retracted – `socrates` stays in the parent's mailbox, and `plato` is delivered as an *additional* message. If sends were undone on backtracking, the parent would end up seeing only the last solution; because they are not, backtracking through the solution set leaves behind a trail of delivered answers, one per `next`. The non-backtrackability of `!/2` is not an incidental detail here; it is the very thing that turns Prolog's search into an externally-driven answer stream.

Step back and notice what has been achieved. Prolog's backtracking – normally an internal, automatic mechanism the programmer cannot pause – has been placed *under external control*. A remote client, by sending messages, now drives the search one solution at a time, exactly as a user pressing semicolon drives an interactive toplevel. The actor is, in miniature, a query server: it holds a live nondeterministic computation and exposes its solutions on demand. This is something Erlang simply cannot do, because Erlang has no backtracking to expose – and it is the seed of one of the most important abstractions in the book.

For the actor above already adheres, in embryo, to a tiny communication protocol: it understands exactly two messages, `next` and `stop`, and answers with a stream of solutions. That is the kernel of what we will later develop into the *toplevel* – a full-fledged, reusable behaviour for driving nondeterministic goals across the network, and one of the load-bearing components of the Prolog Web.

It is, however, only a kernel, and an honest one should admit its limits. The goal `human(Who)` is hard-coded into the actor rather than supplied by the client; the protocol is bare, understanding only `next` and `stop`; and nothing is done about a goal that *fails* outright or *throws* an error. The next section takes a first step toward removing these limitations, and Chapter 3 removes them properly, replacing this hand-built sketch with built-in toplevel predicates adhering to a robust, standardised protocol.

2.4.8 The toplevel behavior: a preliminary sketch

In essence, a Prolog toplevel is a failure-driven loop running inside another loop – recursive or failure driven. The failure-driven inner loop allows a client to ask for one solution at a time to a goal, while the outer loop allows it to call several goals in a row and run each of them to completion.

Below, we show how we can build a simple Prolog toplevel by using a meta-predicate (such as `call_cleanup/2`) and by specifying a small set of custom messages carrying answers and/or the state of the process that needs to be returned to the calling process. The predicate `call_cleanup/2` is here used not only to call a goal, but also to check if any choice points remain after the goal has been called or backtracked into.¹⁵

```
simple_toplevel(Pid) :-
    simple_toplevel(Pid, []).

simple_toplevel(Pid, Options) :-
    self(Self),
    spawn(session(Pid, Self), Pid, Options).

session(Pid, Parent) :-
    receive({
        '$call'(Template, Goal) ->
            ( call_cleanup(Goal, Det=true),
              ( var(Det)
                -> Parent ! success(Pid, Template, true),
                  receive({
                      '$next' -> fail ;
                      '$stop' -> true
                  })
              )
            )
    })
```

¹⁵ http://www.swi-prolog.org/pldoc/man?predicate=call_cleanup/2

```

        }},
        !
        ;   Parent ! success(Pid, Template, false)
        )
        ;   Parent ! failure(Pid)
        )
    }},
    session(Pid, Parent).

```

A suitable predicate API hiding the details of the protocol from the programmer can be written like so:

```

simple_toplevel_call(Pid, Template, Goal) :-
    Pid ! '$call'(Template, Goal).

```

```

simple_toplevel_next(Pid) :-
    Pid ! '$next'.

```

```

simple_toplevel_stop(Pid) :-
    Pid ! '$stop'.

```

Here is a session that shows that these predicates work as intended:

```

?- simple_toplevel(Pid, [
    monitor(true)
]).
Pid = 23891373.
?- simple_toplevel_call($Pid, Who, human(Who)).
true.
?- flush.
Shell got success(23891373,socrates,true)
true.
?- simple_toplevel_next($Pid).
true.
?- flush.
Shell got success(23891373,plato,true)
true.
?- simple_toplevel_next($Pid).
true.
?- flush.
Shell got success(23891373,aristotle,false)
true.
?-

```

Note that because of the outer recursive loop, our simple toplevel actor is again poised to run another query:

```
?- simple_toplevel_call($Pid, X, q(X)).
true.
?- flush.
Shell got down(23891373,23891373,existence_error(proc,q/1))
true.
?-
```

Here the actor crashed, and the reason is that the code for `simple_toplevel/1-2` does not say anything about what should happen if an error is thrown, which is the case if the predicate called by the goal is not defined. Since the spawned process is monitored, however, the error message did eventually reach the mailbox of the spawning process anyway, in the form of a `down` message.

A similar problem is evident should we tell the actor to execute a query that does not terminate. Using `exit/2` works, but will once again kill the entire actor process.

Such problems can be fixed. As it turns out, however, and as will be demonstrated in Chapter 3, Web Prolog programmers do not need to define their own toplevel predicates. Instead, they can use a set of built-in predicates in order to create and control more powerful Prolog toplevels, which are actors adhering to a much improved protocol. In the terminology introduced above, these are Erlang-style behaviors implementing a standardized toplevel protocol.

2.5 Summary

This chapter introduced Web Prolog as a hybrid language that extends a substantial subset of ISO Prolog with Erlang-style constructs for programming concurrency. Web Prolog extends ISO Prolog with a small set of core additions – actors with private state, asynchronous message passing (`!/2`), process creation (`spawn/2-3`), and fault-handling primitives such as links, monitors, and exit signals. A program that never mentions these constructs behaves like a conventional Prolog program, while programs that do use them gain access to an Erlang-like world of concurrent processes.

We also argued that Erlang provides an unusually suitable point of departure for such an extension. Its actor model is mature, battle-tested, and close enough in syntax and spirit to Prolog that both languages can share patterns, idioms, and even didactic material. Many of the examples in this chapter are thin Prolog translations of familiar Erlang patterns. The combination of Prolog's backtracking search with Erlang-style concurrency is made workable by allowing receive to be semi-deterministic and by separating the nondeterminism of search from the determinism of message handling. This makes it possible to treat a Prolog toplevel as just another actor and to encapsulate interactive sessions as behaviors implemented on top of a common concurrency substrate.

Taken together, these choices define Web Prolog as a multi-paradigm language. Its logical expressivity remains that of ISO Prolog, but its operational repertoire is enlarged by an actor-based concurrency model that is both network-transparent and

compatible with the rest of the Prolog Trinity ecosystem. There are other possible ways of adding concurrency to Prolog, but the thesis of this chapter is simply that Erlang-style actors provide a coherent and practically manageable foundation on which the subsequent chapters can build.

Chapter 3

Prolog agents

Imagine a world of **Prolog agents**, some useful, others playful, bringing joy to games and virtual worlds; some short-lived, others long-lived, some simple, others complex – but maybe built from simpler ones. Written in Web Prolog, talking Web Prolog with other agents, using Web Prolog knowledge bases to guide their actions and conversations, making sure important capabilities of clever conversational agents, such as natural language understanding, knowledge representation, reasoning and real-time interaction, are accounted for.

Prolog agents – the elevator pitch

Chapter 1 introduced the notion of a Prolog agent, and Chapter 2 the more precise concept of a Prolog actor – the most elementary form of Prolog agent in the Prolog Trinity ecosystem. This chapter develops the concept further and introduces two more kinds of agent that matter for the rest of the book: Prolog toplevels and Prolog nodes.

3.1 Prolog agents in a nutshell

Chapter 2 introduced the Prolog actor as the elementary unit of concurrent computation. This chapter makes a wider claim: that the actor, the toplevel, and the node – entities that differ enormously in scale, lifetime, and role – are most productively understood as instances of a single abstraction, the *Prolog agent*. A one-line echo actor that lives for a fraction of a second and a node that serves thousands of clients for months sit at opposite ends of a spectrum, and yet a client addresses them, and reasons about them, in the same way. What unites them is not how they are built but how they are met: each is addressable, persists across interactions, and converses in Prolog terms, receiving goals or messages and responding with answers, messages, or effects in the same term-based form. Being a Prolog agent is therefore an interface

and behaviour property, not a requirement about implementation language – a node need not be written in Prolog, only able to talk it.¹

This single vocabulary is what lets us describe the whole inhabited layer of the Prolog Web at one level of abstraction. The rest of this section sketches the dimensions along which agents nevertheless differ; the taxonomy in Figure 3.1 arranges them accordingly, and the sections that follow develop each kind in turn – actor agents and their lifecycle, toplevel agents and their protocol, and finally nodes as agents in the Prolog Web.

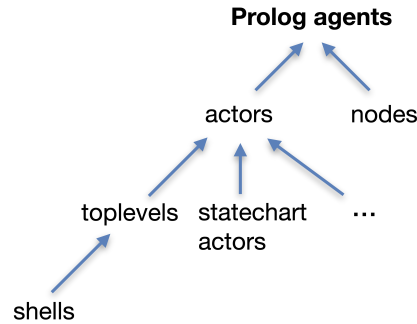


Fig. 3.1 A taxonomy of Prolog agents.

Actors, toplevels, and nodes. We treat all three uniformly as agents because they are alike in the ways that matter to a client: addressable, persistent, and interactive, with interactions mediated by Prolog terms. They differ in role. Actors are loci of concurrent computation. Toplevels expose a conversational, query-answer interface. Nodes host agents and knowledge, expose web APIs, and enforce policy; a node is typically long-lived, serves many clients at once, and can be programmed only by its owner.

State and interaction. Agents are persistent and interactive, but they need not be stateful. Some are effectively stateless, such as the echo actor; others maintain state in receive-loop parameters, in control structure, or in a private dynamic database. The private database is thus a capability an agent may have, not a feature that defines it.

Private beliefs and shared knowledge. Each agent may combine a private workspace – including an optional private dynamic database – with shared, node-provided knowledge, typically a read-only database maintained by the node owner. Private definitions can *shadow* shared ones, allowing local specialisation without modifying the shared base; the precise semantics of shadowing is worked out in Section 3.2.2.

¹ In treating “agent” as an analytical lens rather than an absolute classification, we follow Russell and Norvig (2016); the term carries no commitment to autonomy or intelligence in any fuller sense.

Protocols as contracts. Toplevel actors are distinctive in that their behaviour is governed by an explicit, observable protocol. The Prolog Toplevel Communication Protocol (PTCP) specifies how goals are submitted, how solutions are delivered incrementally, and how exceptional situations such as aborts and exits are handled. This turns informal REPL interaction into a first-class behavioural contract, and underpins the later discussions of timeouts and of stateless versus session-oriented interfaces.

3.2 More about Prolog actor agents

Actor agents are the workhorses of the concurrent strand of the Trinity: they execute Prolog code concurrently, interact by exchanging Prolog terms, and may maintain actor-local state across repeated interactions. This section makes that operational picture more concrete. We first introduce the actor's private Prolog database and the `load_*` options used to initialise it. We then sketch the life-cycle of a typical actor and explain how receive loops realise explicit communication protocols. Finally, we discuss the dynamic (still private) database as a non-logical but sometimes convenient mechanism for actor-local updates.

3.2.1 An actor agent is equipped with a private Prolog database

When running the examples in Chapter 2 we were somewhat vague about the origin of the code being executed. The only clue provided was that it might reside in the node's shared database, which defines read-only data and programs accessible to any actor living on that node. While we will delve deeper into the shared database later in this chapter, we first turn our attention to the other storage area – the one located *within* the actor itself and where programs and data exclusive to that actor are stored. While all actors have a private database, it starts out empty by default.

During the spawn operation, the (soon-to-be) parent process can initialize the private database of the (soon-to-be) child actor with clauses defining one or more predicates. Consider the following example where the `load_text` option is passed to `spawn/3`, specifying that the source code for our count server should be loaded into the actor's private database before calling the goal in the first argument:

```
?- spawn(count_actor(0), Pid, [
    load_text("
        count_actor(Count0) :-
            receive({
                count(From) ->
                    Count is Count0 + 1,
                    From ! count(Count),
                    count_actor(Count) ;
```

```

                                stop ->
                                true
                                }).
                                "),
                                monitor(true)
                                ]).
Pid = 45092311.
?-

```

In addition to the `load_text` option, three other options can be used to populate the private database of an actor. The `load_list` option loads a list of clauses or directives, the `load_uri` option loads the content specified by a URI, and `load_predicates` takes a list of predicate indicators and loads the clauses for the indicated predicates that are accessible by the caller. See Section A.1 for more information.

The clauses in an actor's private database represent its unique beliefs and skills, distinguishing them from the shared beliefs and skills accessible to other actor agents on the same node. In this sense, the private database can be seen as the core of the actor's individuality, encapsulating the knowledge and capabilities that are exclusive to it.

The `load_*` options make a kind of sharing of beliefs and skills between a parent process and a child process possible. The child is "born" with these beliefs and skills, they are "innate." It is up to the parent actor to determine exactly what should be passed on to the child actor.

At this point, and practically speaking, the actor's private database and the `load_*` options may not seem all that useful. After all, it seems we could just as well have stored the implementation of `count_actor/1` in the node's shared database. As we shall see, they are more useful in the distributed case, for shipping predicates from a client to a node, or from a node to a client, and execute them there.

3.2.2 Private beliefs and shared knowledge – shadowing

A predicate defined in an actor's private database takes precedence over a predicate with the same name and arity in the shared database. Thus, for an unqualified call made from the actor's private database, Prolog first looks for a private definition and otherwise continues to the shared database. We say that the private predicate *shadows* the shared predicate.

Suppose the shared database contains the following price list and default pricing rule:

```

list_price(widget, 100).
list_price(gadget, 250).
list_price(gizmo, 400).

```

```
price(Item, Price) :-
    list_price(Item, Price).
```

An actor without a private definition of `price/2` uses the shared rule:

```
?- price(widget, Price).
Price = 100.
?-
```

We can give a newly spawned actor a private pricing policy by passing it with the `load_text` option. The actor uses the private `price/2`, but its call to `list_price/2` still resolves to the shared facts:

```
?- self(Self),
    spawn(
        ( price(widget, Price),
          Self ! discounted_price(Price)
        ),
        -,
        [ load_text("
            price(Item, Price) :-
                list_price(Item, ListPrice),
                Price is ListPrice * 0.9.
          ")
        ]
    ),
    receive({discounted_price(Price) -> true}).
Self = 34001234,
Price = 90.0.
?-
```

The precise form of the process identifier bound to `Self` is implementation dependent.

The private pricing rule belongs only to the spawned actor. It neither modifies the shared rule nor becomes part of the parent's private database. Consequently, another actor spawned by the parent without that `load_text` option uses the shared definition:

```
?- self(Self),
    spawn(
        ( price(widget, Price),
          Self ! ordinary_price(Price)
        ),
        -,
        ),
    receive({ordinary_price(Price) -> true}).
Self = 34001234,
```

```
Price = 100.
?-
```

Shadowing applies to a complete predicate, not to individual clauses. The following actor receives a private `list_price/2` containing only one fact:

```
?- self(Self),
    spawn(
      ( findall(Item-Price,
                list_price(Item, Price),
                Prices),
        Self ! private_prices(Prices)
      ),
      -,
      [load_text("list_price(widget, 80).")],
    ),
    receive({private_prices(Prices) -> true}).
Self = 34001234,
Prices = [widget-80].
?-
```

The private fact does not combine with the shared facts. Once the actor has defined its own `list_price/2`, all shared clauses of `list_price/2` are hidden from unqualified calls made from that actor's private database. In particular, the actor does not obtain prices for `gadget` or `gizmo`.

The shared database itself remains unchanged. The parent still sees the complete shared price list:

```
?- findall(Item-Price,
          list_price(Item, Price),
          Prices).
Prices = [widget-100, gadget-250, gizmo-400].
?-
```

Shadowing is therefore local to an actor: it changes how that actor resolves a predicate without modifying the node-provided knowledge seen by other actors.

3.2.3 The life-cycle of a typical Prolog actor agent

We have the black boxes that do things, and we have the messages in between. What goes on inside the black boxes does not really matter; what takes place there is abstracted over.

Joe Armstrong

The diagram in Figure 3.2 illustrates the life-cycle of a typical Prolog actor.

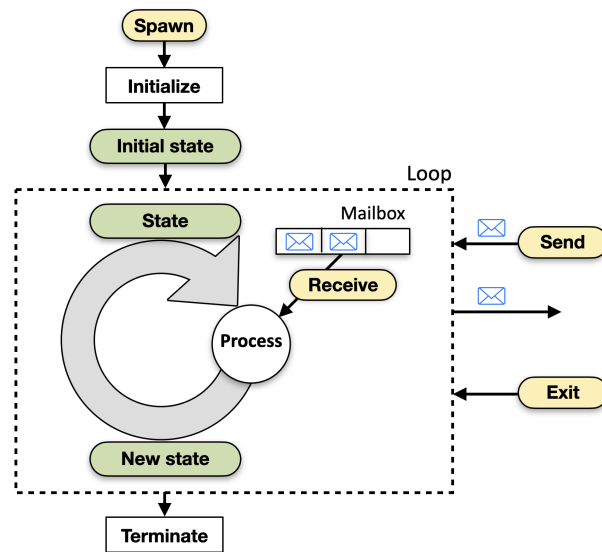


Fig. 3.2 Lifecycle of a Prolog actor: spawn, interact, terminate, and clean up

When an actor is spawned, it first passes through an initialization phase where it is configured in a way that will suit its intended purpose. This includes specifying values for options such as `monitor`, `link` as well as the `load_*` options.

For example, when the `count` actor in the previous sections was spawned, the source code specified by the `load_text` option was loaded into the actor's private Prolog database and the `monitor` options instructed the actor to send a `down` message to the shell just before terminating. The call to `spawn/3` succeeded, binding the variable in the second argument to a `pid`. By this, the initialization phase is completed, the actor is now in its *initial state*, and the loop that implements the actor's *communication protocol* is entered. Typically, processing stops at a call to `receive/1-2` where it blocks, waiting for a `count` message or a `stop` message to appear in its mailbox.

The notion of a communication protocol is central to Erlang-style concurrent programming. The loop implements the communication protocol that clients that want to interact with the actor must follow. The interaction is driven by coordinating send and receive operations, with the protocol defining the rules. The sending process transmits messages in the specified format, and the receiving process captures and interprets them. They must adhere to the syntax, semantics and timing constraints of the communication protocol:

- **Syntax:** This refers to the structure and format of the data exchanged between processes. It dictates how data is packaged, including the organization of various data elements.

- **Semantics:** This involves the meaning and interpretation of the exchanged data. It defines the rules for understanding the content of the message, what actions should be triggered upon receiving certain data, and how to handle errors or unexpected data.
- **Timing:** This concerns the timing aspects of the communication, such as when data is sent, the pace of data exchange, and managing the order and sequence of messages so that both processes can coordinate despite delay, reordering, and failure.

The Web Prolog receive operator, while blocking the process at the point of a `receive/1-2` call, allows for the implementation of very flexible communication protocols through mechanisms like selective pattern matching and timeouts. The selective pattern matching allows the process to wait for a particular message but can also defer out-of-order messages for later processing. By specifying a timeout to avoid the receive call being blocked indefinitely, alternative actions can be taken if an expected message does not arrive in time. This helps reduce the chance of breakdowns and allows for more adaptive communication.

Whenever the actor runs a `receive/1-2` call, a message is selected from the mailbox and processed sequentially. The result often depends on the state of the process, and as part of processing, a new state for the actor may be computed. This will be the current state for processing the next message selected.

Some actors are stateless (e.g. the echo actor), and those that are stateful represent their state and perform state transitions in a variety of ways: as values threaded through the arguments of a receive loop (for example an integer updated by calls to `is/2` in the count actor, or a list of atoms in the fridge simulation), as the explicit states and transitions of a state machine, or implicitly through the choice points created by a failure-driven loop. These techniques all encode state in the control flow or data flow of the program itself. In addition to these, Web Prolog offers a more direct – though non-logical – mechanism for maintaining actor-local state: a private dynamic Prolog database.

3.2.4 The dynamic (and still private) Prolog database

As long as the actor process is alive, it is allowed to update its private database using predicates such as `assert/1`, `retract/1` and `retractall/1`. Following the ISO standard, predicates that are modified this way need to be declared using the `dynamic/1` directive. Updates only affect that actor's private database: they are not visible to other actors running on the same node.

Because the dynamic database is strictly actor-local, it neither introduces shared-state races nor provides a mechanism for shared-state inter-process communication.

For example, although the following goal is permitted, it is of limited use: the clause `foo(a)` exists only inside the spawned actor and disappears when that actor terminates.

```
?- spawn(assert(foo(a)), Pid).
Pid = 32861299.
?-
```

The dynamic database provides another way to transfer from one state to the next. This is often a bad idea.

Throughout the Prolog Trinity ecosystem, private state is treated as volatile and local to an agent, whereas shared data is governed by explicit node-level policies and exposed under controlled conditions.

More about the dynamic database

The dynamic database is a non-logical extension to Prolog, and its use for managing state is often viewed with suspicion by Prolog programmers. The reason is well known: changes to the database are not reverted on backtracking, which makes dynamic predicates behave like mutable global variables and exposes programs to the usual problems associated with such entities.

At first glance, this raises the question of whether the dynamic database could lead to problems in Web Prolog, especially in a concurrent setting. The short answer is no. Although Erlang does not offer a Prolog-style dynamic database, it provides a closely related mechanism in the form of the *process dictionary*. Each Erlang process maintains a private key-value store that cannot be accessed or modified by other processes. As with the dynamic database, its use is generally discouraged, since it undermines referential transparency and makes programs harder to reason about and debug.

The similarity is hard to miss. This suggests that the dynamic database in a Web Prolog actor is not inherently more dangerous than the process dictionary in Erlang. At the same time, the dynamic database is arguably more expressive and convenient, which also means that programmers may be tempted to rely on it more often than is advisable.

While both the dynamic database in Web Prolog actors and the Erlang process dictionary are strictly private, Erlang also provides mechanisms for mutable *shared* state. These include Erlang Term Storage (ETS), which offers in-memory tables with configurable access rights, and Mnesia, a distributed database supporting transactions, persistence, and replication. More recently, Erlang has also introduced `persistent_term`, which provides efficient read-only access to immutable global data, at the cost of expensive updates.

This raises a broader design question for Web Prolog: should it offer forms of dynamic storage beyond what can be represented by predicate arguments or asserted in an actor's private database? Given Web Prolog's focus on the Web, a transactional RDF store would be a natural candidate, and such a solution is readily available in systems like SWI-Prolog. A Linda-style blackboard is another possible direction. For the time being, however, Web Prolog deliberately restricts dynamic state to the

private scope of individual agents, leaving shared mutable state to be addressed by explicit, higher-level mechanisms.

3.3 Prolog shells and other toplevel actors

As we saw already in Chapter 2, calling `self/1` in a terminal binds its argument to the pid of the currently running shell process:

```
?- self(Pid).
Pid = 72097632.
?-
```

Here, the actor with the pid 72097632 is a shell. Shells are toplevels, and therefore also agents. Shells handle all the things that toplevels handle, but also add a number of useful things for when we are talking to Prolog over a terminal. This includes I/O (read and write) and utilities such as `flush/0` and the dollar notation. Such features are supplied by the *node controller* rather than a toplevel alone. (We will say more about the node controller further ahead in this chapter.)

Here is how we instruct the shell to spawn a new toplevel:

```
?- toplevel_spawn(Pid).
Pid = 16226587.
?-
```

Note that 16226587 is a toplevel actor but not a shell. As with every other Prolog actor, it comes with a mailbox, a private Prolog database, the ability to send messages to other actors, as well as the ability to create other actors.

The feature that distinguishes a toplevel from other actors is that it comes with a standardized built-in special-purpose communication protocol.

3.3.1 A Prolog toplevel is an actor with a built-in protocol

A programmer firing up a traditional Prolog system is likely met with a query prompt. In the literature, this is usually referred to as the *toplevel*. The reason we refer to it as a *shell* is because we want to use the term *toplevel* to refer to toplevels that are not shells.

In traditional Prolog, a program cannot *internally* create a toplevel, pose queries and request solutions on demand, but this is something that Web Prolog allows. In traditional Prolog the shell is *lazy* in the sense that new solutions to a query are only computed on demand. As we shall see, this is how the Web Prolog toplevel actor works too.

In Web Prolog, a toplevel actor is a programming abstraction modelled on the interactive toplevel of Prolog. A toplevel actor is like a first-class interactive Prolog

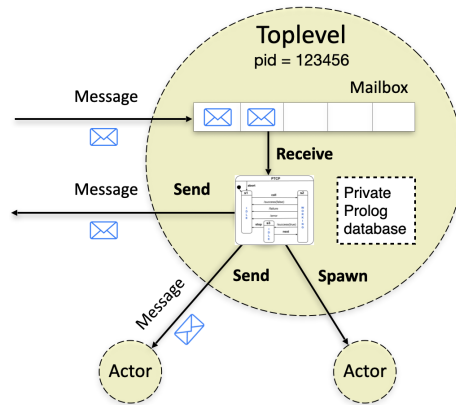


Fig. 3.3 The anatomy of a Prolog toplevel actor.

toplevel, accessible from Web Prolog as well as from other programming languages such as JavaScript. We can also think of it as an *encapsulated Prolog session*, an abstraction aiming at making Prolog programmers feel right at home.

A toplevel is a kind of actor, and what distinguishes it from other kinds of actors is the *protocol* it follows when it communicates, i.e. the kind of messages it listens for, the kind of messages it sends and in what order, and the behavior this gives rise to.

The protocol must not only allow a client to submit queries and a toplevel to respond with answers, it must also allow the toplevel to prompt for input or produce output at any time, in an order and with a content as dictated by the program that it runs. All toplevels follow this protocol. The terminal adheres to it as well, and even a human user of a terminal talking to a shell must adapt to it in order to have a successful interaction.

The design of the toplevel actor in Web Prolog is in fact very much inspired by the informal communication protocol that we as programmers adhere to when we invoke a Prolog shell from our OS prompt, load a program, submit a query, are presented with a solution (or a failure or an error), type a semicolon in order to ask for more solutions, or hit return to stop. These are “conversational moves” that Prolog understands. There are even more such moves, because after having run one query to completion, the programmer can choose to submit another one, and so on. The session does not end until the programmer decides to terminate it. There are only a few moves a client can successfully make when the protocol is in a particular state, and the possibilities can easily be described, by a state machine for example, as will be shown in the next section.

3.3.2 The Prolog Toplevel Communication Protocol

The diagram in Figure 3.4 is a statechart specifying the Prolog Toplevel Communication Protocol (PTCP) – a protocol for the communication between a client and a toplevel actor. The client can be any process (including another actor or (say) a JavaScript process) capable of sending the messages and signals in bold to the toplevel. The toplevel actor is responsible for returning the messages with a leading / back to the client. The use of a statechart allows us to show that no matter the current state of the protocol, **abort** will always take it to the state from which a new goal can be called and **exit** will always terminate the toplevel process.

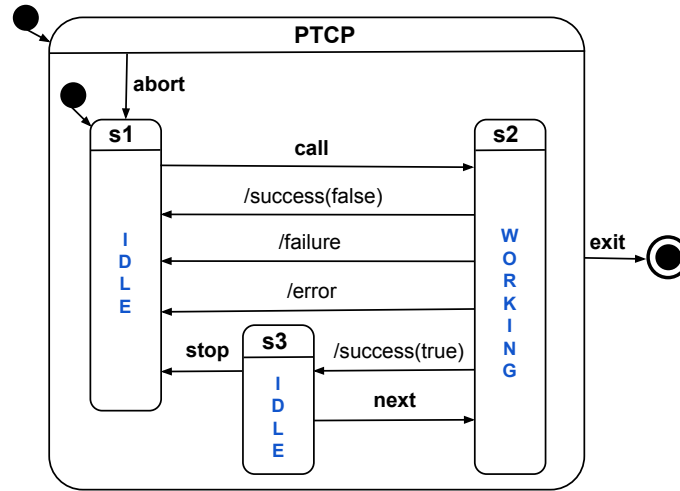


Fig. 3.4 Statechart for the Prolog Toplevel Communication Protocol (PTCP): client moves and toplevel responses.

A process is able to *spawn* a toplevel. After having done so, the process becomes the parent of the toplevel and can start communicating with it according to the protocol. Initially, the protocol is in state **s1**, where the toplevel rests idle, waiting for a **call** message containing a goal. When such a message arrives, the protocol transitions to state **s2**, where the toplevel is doing actual work. The protocol will remain in state **s2** until some work is done and the toplevel sends a message indicating either **/success**, **/failure**, **/error**, or (if the process is monitored) **/down** to the client. On the event of the toplevel sending **/failure** or **/error**, the protocol will transition back to state **s1**. This will be the case also for a **/success** message that indicates no more solutions to the query can be found (marked with false in the chart). However, if a **/success** message indicates more solutions may exist (marked with true in the chart), the protocol transitions to state **s3**, where it will wait for a message **next**, **stop**, **abort** or **exit** to arrive from the client. If the message is **stop**

or **abort** the protocol will transition back to state **s1**, if it is **next** it will transition to state **s2** and trigger the search for more solutions, and finally, if it is **exit**, it will (if the process is monitored) force the toplevel to send a message `/down` back to the client and then to terminate. Any other messages will be deferred to the code being executed by the toplevel.

Web Prolog comes with six built-in predicates which allow a client to spawn a toplevel actor and interact with it during its life-cycle:

```
toplevel_spawn/1-2    toplevel_call/2-3    toplevel_next/1-2
toplevel_stop/1       toplevel_abort/1     toplevel_halt/2
```

As witnessed by some of the examples in Chapter 2, such predicates can be defined in terms of the more primitive `spawn/1-3`, `!/2` and `receive/1-2` predicates. The predicates proposed here come with a number of important additional options, the use of which will be demonstrated in the sections that follow. Appendix A contains an excerpt from the manual which covers most of the details of our proposal.

Toplevel actors are stateful

Clearly, a Prolog toplevel is a kind of server (in the sense of Erlang – see Section 2.4.1). Also, note that a toplevel, even when not explicitly threading any state, nor using the dynamic database, must still be considered stateful. Rather than using an explicit data structure for holding the state, it is the underlying Prolog process *as such* that holds it, most clearly shown in the way the toplevel “remembers” its history and how its behavior is influenced by this, enabling it to react appropriately when a client requests the next solution to a query.

3.3.3 Signatures and options for the toplevel API

This section summarises the client-facing API; Appendix A gives the full specification. In signatures, `+` marks an input argument, `-` an output argument, and a trailing `?` marks an argument that may be omitted (accounting for the double arities such as `toplevel_spawn/1-2`).

`toplevel_spawn(-Pid, +Options?)`

Option	Values	Default
<code>session</code>	<code>true</code> , <code>false</code>	<code>false</code>
<code>target</code>	<code>pid</code>	caller’s <code>pid</code>
<code>node</code>	<code>URI</code>	the local node

In addition, `toplevel_spawn/2` accepts every option that `spawn/3` accepts – `monitor`, `link`, and the `load_*` family (Section 3.2.1).

toplevel_call(+Pid, +Goal, +Options?)

Option	Values	Default
template	term	Goal
offset	integer ≥ 0	0
limit	integer > 0	all
once	true, false	false
target	pid	from toplevel_spawn

toplevel_next(+Pid, +Options?)

Option	Values	Default
limit	integer > 0	from toplevel_call
target	pid	from toplevel_call

The remaining predicates

Predicate	Effect	Response
<code>toplevel_stop(+Pid)</code>	abandons remaining solutions; PTCP returns to s1	none
<code>toplevel_abort(+Pid)</code>	signal; aborts the running goal from any state	abort ack
<code>toplevel_halt(+Pid, -Reply)</code>	signal; terminates the toplevel (synchronous)	Reply, plus down if monitored
<code>output(+Term)</code>	called <i>by</i> the running goal; emits an event to the target	output term
<code>input(+Prompt, -Term)</code>	called <i>by</i> the running goal; requests input and blocks	prompt term
<code>respond(+Pid, +Term)</code>	called <i>by</i> the client; answers a pending prompt	none

Answer and event terms

A toplevel communicates with its target through a small vocabulary of terms, each with a JSON counterpart used when the client is not a Prolog process:

Term	JSON type	Sent when
success(Pid, Solutions, More)	success	a slice of solutions was produced; More is true iff further solutions may exist
failure(Pid)	failure	the goal failed
error(Pid, Error)	error	the goal raised Error
output(Pid, Term)	output	the goal called output/1
prompt(Pid, Prompt)	prompt	the goal called input/2
down(Ref, Pid, Reason)	down	a monitored toplevel terminated
—	spawned	controller-level: an actor was created (carries the pid)
—	abort	controller-level: a goal was aborted

The last two rows have no answer-term counterpart because, in the Prolog API, the corresponding information is delivered through argument bindings (`toplevel_spawn/2` binds the pid) or a plain true (`toplevel_abort/1`).

3.3.4 Signatures and options for `toplevel_*` predicates

`toplevel_spawn(-Pid, +Options?)`

Option	Values	Default
session	true, false	false
target	Pid	caller's pid

`toplevel_call(+Pid, +Goal, +Options?)`

Option	Values	Default
template	term	Goal
offset	integer ≥ 0	0
limit	integer > 0	all
once	true, false	false
target	Pid	from <code>toplevel_spawn</code>

`toplevel_next(+Pid, +Options?)`

Option	Values	Default
limit	integer > 0	from <code>toplevel_call</code>
target	Pid	from <code>toplevel_call</code>

3.3.5 Shell talking to a Prolog toplevel

Below, we show an example of how to create and interact with a toplevel process from a shell. We start by spawning a new toplevel, using `toplevel_spawn/2` with the `monitor` option to instruct the toplevel to send us a `down` message when the process eventually terminates. The `load_list` option is used to populate the private Prolog database belonging to the toplevel actor with two simple unit clauses `p(a)` and `p(b)`:

```
?- toplevel_spawn(Pid, [
    session(true),
    monitor(true),
    load_list([p(a),p(b)])
]).
Pid = 74122981.
?-
```

With this, the toplevel actor is initiated, and with the PTCP now in state `s1`, it is ready to accept messages and signals sent by the other `toplevel_*` predicates.

The `monitor` and `load_list` options are inherited from `spawn/1-3`. However, `toplevel_spawn/2` offers a couple of additional options that can be used to specify the behavior of the toplevel actor at creation time. In the above example the `session` option is used to configure the toplevel to run a multi-goal session rather than exit after having run one goal to completion.

Making the toplevel call a goal

Let us see what happens if we call `toplevel_call/2` with the default values for options:

```
?- toplevel_call($Pid, p(X)).
true.
?- flush.
Shell got success(74122981,[p(a),p(b)],false)
true.
?-
```

The answer is returned to the mailbox of the calling process in the form of an *answer term* with three arguments. The functor of the term represents its *type*. In this case, it shows that the goal succeeded. (In other cases it might indicate a failure or an error.) The first argument of the success term is the pid, and the list in the second argument represents the two solutions that were computed. The value `false` in the third argument of the term indicates that no more solutions exist.

Since the `session` option was set to true, the PTCP is now, after a brief visit to state `s2` for the execution of the goal, back in state `s1` and is prepared to accept another use of `toplevel_call/2-3`.

Using the template option

Below, `toplevel_call/3` is called with the goal `p(X)` and with the `template` option set to the variable `X`:

```
?- toplevel_call($Pid, p(X), [
    template(X)
]).
true.
?- flush.
Shell got success(74122981,[a,b],false)
true.
?-
```

Note how the value of the `template` option determined the form of the list of solutions in the second argument of the answer term. The relation to the Prolog built-in standard predicate `findall/3` is evident.

Using the offset and limit options

When querying a relational database using SQL or an RDF dataset using SPARQL, the use of the `OFFSET` and `LIMIT` keywords serves to control the subset of results that are returned by a query. In Web Prolog they can be used as options for the configuration of calls to `toplevel_call/3`. The `limit` option specifies the maximum number of solutions to return, while the `offset` option specifies the number of solutions to skip before starting to return solutions. Here is what happens if `limit` is set to 1:

```
?- toplevel_call($Pid, p(X), [
    template(X),
    limit(1)
]).
true.
?- flush.
Shell got success(74122981,[a],true)
true.
?-
```

The value `true` in the `success` term means that the state machine is now in state `s3`, waiting for a message `next` or `stop` to appear. Calling `toplevel_next/1` produces the next solution:

```
?- toplevel_next($Pid).
true.
?- flush.
Shell got success(74122981,[b],false)
```



```
true.
?-
```

`toplevel_call/3` supports the `offset` option. Together with the `limit` option it allows the call to pick out an arbitrary slice of solutions to a goal, even when the number of solutions is infinite:²

```
?- toplevel_call($Pid, between(1,infinite,I), [
    offset(100),
    template(I),
    limit(3)
]).
true.
?- flush.
Shell got success(74122981,[101,102,103],true)
true.
?-
```

Calling `toplevel_next/1` produces three more solutions:

```
?- toplevel_next($Pid).
true.
?- flush.
Shell got success(74122981,[104,105,106],true)
true.
?-
```

Note that the value of the `limit` option set in the call to `toplevel_call/3` was still in effect. However, `toplevel_next/2` accepts the `limit` option too, which allows us to increase or decrease the limit on the number of solutions returned:

```
?- toplevel_next($Pid, [
    limit(5)
]).
true.
?- flush.
Shell got success(74122981,[107,108,109,110,111],true)
true.
?-
```

Calling `toplevel_stop/1` will stop the execution of the goal:

```
?- toplevel_stop($Pid).
true.
?-
```

² It may seem odd and not all that useful for `toplevel_call/2-3` to support the `offset` option, but it turns out to be important when implementing the stateless HTTP protocol.

These options are particularly useful in several scenarios: pagination of results in web applications, where it is impractical to retrieve and display all records at once; implementing infinite scrolling interfaces that load data dynamically as the user scrolls; handling goals with very many or infinitely many solutions without overwhelming the client; and reducing the number of network roundtrips on the Prolog Web by fetching solutions in batches. A special case is when `limit` is set to 1, which makes it straightforward to implement a Prolog-style REPL in JavaScript, where each press of `;` triggers a new request for the next solution.

One assumption underlies the use of `offset` and `limit`: they identify a slice by position, which is well-defined only when the goal enumerates its solutions in a stable order across the separate requests that page through it. Stably enumerable goals over the shared database satisfy this, since clients cannot modify that database and the order in which a goal's solutions are produced is then fixed. If the owner mutates the shared database midway through a paging interaction, a client may observe duplicated or skipped solutions, or an incorrect report of whether further solutions remain; detecting and recovering from such drift is left to the application. This is the same contract offered by `OFFSET` in SQL, and it is no accident that the profiles which expose stateless paging also withhold client-side database modification – a client that cannot mutate cannot, by paging alone, violate the stability its own slices depend on.

Using the `once` option

The `once` option only makes sense in combination with the `limit` option; its default is `false`. Here it is set to `true`:

```
?- toplevel_call($Pid, between(1,infinite,I), [
    offset(100),
    template(I),
    limit(3),
    once(true)
]).
true.
?- flush.
Shell got success(23091243,[101,102,103],false)
true.
?-
```

The `once` option requests *committed* evaluation: the node returns the first slice (of up to `limit` solutions) and then discards any remaining choice points rather than retaining them for a possible `toplevel_next`. This is reflected in the answer term, whose third argument is `false` even when further solutions could in principle have been found – the call reports completion because the node has undertaken to produce nothing more. It differs from simply declining to call `toplevel_next/1`: that would leave the session suspended in state `s3`, holding resume state until the next request

or an idle timeout, whereas `once(true)` releases that state at once. The option is therefore a resource directive as much as a control one, useful when a client knows it wants at most `limit` solutions and wants the node to be free of any obligation to retain the computation.

A toplevel can do a kind of I/O

Whenever a toplevel is in state `s2` and doing some real work, it is able to send messages to its parent (or elsewhere) using the ordinary send operator. However, a better idea can often be to use `output/1`:

```
?- toplevel_call($Pid, output(hello)).
true.
?- flush.
Shell got output(74122981,hello)
Shell got success(74122981,[output(hello)],false)
true.
?-
```

Input can be collected by calling `input/2`, which sends a `prompt` message to the client, which in turn can respond by calling `respond/2`:

```
?- toplevel_call($Pid, input('Input', X)),
   receive({Answer -> true}).
Answer = prompt(74122981,'Input').
?- respond($Pid, hello),
   receive({Answer -> true}).
Answer = success(74122981,[input('Input',hello)],false).
?-
```

Aborting a nonterminating goal

Our toplevel actor is still not dead so let us see what happens when we ask the toplevel to first update its private Prolog database with a silly recursive clause for a predicate `p/0` and then call it:

```
?- toplevel_call($Pid, assert((p :- p))),
   receive({Answer -> true}).
Answer = success(74122981,[assert((p:-p))],false).
?- toplevel_call($Pid, p).
true.
?-
```

Although nothing is shown in the terminal, it is clear that the toplevel is now just wasting CPU cycles to no avail. Fortunately, a nonterminating goal can be aborted by calling `toplevel_abort/1`:

```
?- toplevel_abort($Pid).
true.
?-
```

With this, the PTCP is back in state **s1**.

Halting the toplevel

When we are done talking to the toplevel we can halt it:

```
?- toplevel_halt($Pid, Reply).
Reply = true.
?-
```

Toplevels and the message deferring mechanism

In the following example, a toplevel is spawned, and then `toplevel_next/1` is called. The protocol is in state **s1**, which is obviously not in a state where it can react to the **next** message (see Figure 3.4). The message is therefore deferred and it is not until `toplevel_call/3` is called and the protocol changes states that it has an effect.

```
?- toplevel_spawn(Pid).
Pid = 78340943.
?- toplevel_next($Pid).
true.
?- flush.
true.
?- toplevel_call($Pid, member(X,[a,b,c]), [
    limit(1),
    template(X)
]).
true.
?- flush.
Shell got success(78340943, [a], true)
Shell got success(78340943, [b], true)
true.
?-
```

Note that messages sent to a toplevel will often be handled in the right order even if they arrive in the “wrong” order (e.g. **next** before **call**). This is due to the selective receive which defers their handling until the PTCP permits it. The messages **abort** and **exit**, however, will never be deferred. This is guaranteed by the fact that **abort** and **exit** are valid transitions from any state in the protocol (see Figure 3.4). (They are actually signals rather than messages).

One way to think of this is in terms of the so called *robustness principle*: “Be conservative in what you send, be liberal in what you accept.”³ Due to the deferring behavior a toplevel is liberal in this way, but, as implied by the principle, clients are advised not to rely on this behavior.

Redirecting answers

The `target` option can be used to redirect the answer terms produced by the toplevel to any actor of choice. By default, it is set to the pid of the parent but allows us to instruct a toplevel actor to send answer terms to a different destination. To demonstrate how this works, we first create a simple actor that can serve as a target:

```
?- spawn(( repeat,
            receive({
                Msg ->
                    format("Received ~p~n", [Msg]),
                    fail
            })
        ), Pid0).
Pid0 = 98380209.
?-
```

The pid of that actor can now be used as the value of the `target` option:

```
?- toplevel_spawn(Pid, [
    target($Pid0)
]).
Pid = 97919106.
?-
```

When `toplevel_call/3` is called, the success answer term is printed by the designated target actor:

```
?- toplevel_call($Pid, between(1,1000,N), [
    template(N),
    limit(5)
]).
true.
Received success(97919106,[1,2,3,4,5],true)
?-
```

So, 98380209 rather than the shell received the message from 97919106.

Calling `flush/0` shows that the mailbox belonging to the shell is empty:

³ https://en.wikipedia.org/wiki/Robustness_principle

```
?- flush.
true.
?-
```

The `target` option is supported by `toplevel_spawn/2`, `toplevel_call/3` and `toplevel_next/2`. It is not supported by the other `toplevel_*` predicates as they do not produce answer terms. It can be used in calls to `output/2`. See Appendix A for more details.

3.3.6 Programming with toplevel actors

The toplevel actor is an important kind of agent in the Prolog Trinity ecosystem. However, the reasons for its importance will have to be deferred to later chapters, and this section only offers a few exercises to build familiarity with it.

A synchronous predicate API to toplevels

The communication between a client and a toplevel illustrated above is asynchronous, but if we want, it is quite easy to define a synchronous alternative using a technique we have already seen. A predicate `wait_for_answer/3` can be defined like so:

```
wait_for_answer(Pid, Answer, Options) :-
    receive({
        Answer if arg(1, Answer, Pid) ->
            true
    }, Options).
```

Then `toplevel_call_synch/3-4` can be implemented as follows:

```
toplevel_call_synch(Pid, Goal, Answer) :-
    toplevel_call_synch(Pid, Goal, Answer, []).

toplevel_call_synch(Pid, Goal, Answer, Options) :-
    toplevel_call(Pid, Goal, Options),
    wait_for_answer(Pid, Answer, Options).
```

and `toplevel_next_synch/2-3` like so:

```
toplevel_next_synch(Pid, Answer) :-
    toplevel_next_synch(Pid, Answer, []).

toplevel_next_synch(Pid, Answer, Options) :-
    toplevel_next(Pid, Options),
    wait_for_answer(Pid, Answer, Options).
```

Note that since `toplevel_stop/1` does not produce a response message, no `toplevel_stop_synch/2-3` is defined, and `toplevel_stop/1` can be used as it is.

Here is a simple test that shows how it works:

```
?- toplevel_spawn(Pid).
Pid = 67590967.
?- toplevel_call_synch($Pid, mortal(Who), Answer, [
    template(Who),
    limit(1)
]).
Answer = success(67590967, [socrates], true).
?- toplevel_next_synch($Pid, Answer).
Answer = success(67590967, [plato], true).
?- toplevel_next_synch($Pid, Answer).
Answer = success(67590967, [aristotle], false).
?-
```

Reconstructing `findall/3` and `findnsols/4`

In order to specify the semantics of a new Prolog predicate API it can be a good idea to implement built-in Prolog predicates with a well-known semantics using the API. For example, if we did not already have a built-in predicate `findall/3`, using a toplevel actor we could have implemented it like so:

```
findall(Template, Goal, List) :-
    toplevel_spawn(Pid, [
        session(false)
    ]),
    toplevel_call(Pid, Goal, [
        template(Template)
    ]),
    receive({
        success(Pid, List, false) ->
            true ;
        failure(Pid) ->
            List = [] ;
        error(Pid, Error) ->
            throw(Error)
    }).
```

Of course, implementing these predicates in separate processes makes absolutely no sense. Examples such as these only exercises some of the capabilities of the

`toplevel_*` API and may seem somewhat pointless. But they do say something about the relation between `findall/3` and the `toplevel_*` predicates.⁴

We can take this one step further by also implementing `findnsols/4`, which works like `findall/3`, but generates at most *N* solutions to the specified goal.

This predicate is especially useful if the goal may have an infinite number of solutions.

```
findnsols(N, Template, Goal, List) :-
    toplevel_spawn(Pid, [
        session(false)
    ]),
    toplevel_call(Pid, Goal, [
        template(Template),
        limit(N)
    ]),
    wait_solutions(Pid, List).

wait_solutions(Pid, List) :-
    receive({
        success(Pid, List, false) ->
            true ;
        success(Pid, List0, true) ->
            ( List = List0
            ; toplevel_next(Pid),
              wait_solutions(Pid, List)
            ) ;
        failure(Pid) ->
            List = [] ;
        error(Pid, Error) ->
            throw(Error)
    }).
```

Ciao Prolog provides this predicate, but only in a deterministic version. The version provided by SWI-Prolog is nondeterministic, and so is our reconstruction. We borrow an example of its use from the SWI-Prolog manual:⁵

```
?- findnsols(5, I, between(1, 12, I), L).
L = [1, 2, 3, 4, 5] ;
L = [6, 7, 8, 9, 10] ;
L = [11, 12].
?-
```

Interestingly, it turns out that the version provided by SWI-Prolog serves as an important building block in our proof-of-concept implementation of the `toplevel_*`

⁴ Here, we were inspired by (Tarau, 2000).

⁵ https://www.swi-prolog.org/pldoc/doc_for?object=findnsols/4

predicates. (It was implemented in SWI-Prolog by Jan Wielemaker in order to support library(pengines), the library on top of which SWISH is built.)

3.4 Prolog nodes are also agents

As is evident from our agent taxonomy in Figure 3.1, we regard a Prolog node as a kind of Prolog agent. We consider it as such because, as we shall see, a node incorporates a significant subset of the capabilities of a toplevel actor. In particular, it enables clients to submit queries and to receive solutions lazily – one at a time (or, if required, more than one at a time), in the order that Prolog finds them.

The diagram in Figure 3.5 illustrates the anatomy of an executing Prolog node, which is accessed by a user-client that knows its URI and is authorized to utilize its capabilities.

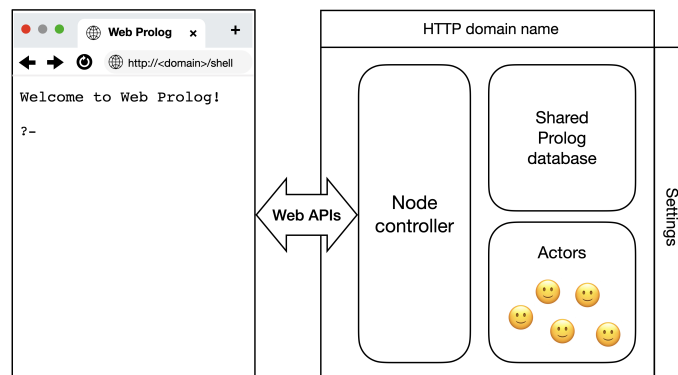


Fig. 3.5 The anatomy of a Prolog node. A terminal running in a web browser is attached to it.

As suggested by the diagram, a node is comprised of components such as a node controller, web APIs, a shared database containing predicates defined by the owner, a container serving as the “home” of actors such as echo actors, supervisors and toplevels, and various settings for the owner to configure the node. As we shall see, not all nodes include all these components.

3.4.1 The node controller

In the most capable profile, the node controller is entrusted with a multitude of responsibilities designed to ensure the smooth and secure operation of the node. Although not every node controller undertakes all of these tasks, the most advanced

implementations are expected to manage web API connections, authenticate and authorize clients, and spawn actors upon client request. Additionally, they impose various resource restrictions to maintain optimal performance, oversee registries, monitors, and links, and guarantee that messages sent back to clients adhere to the required format. The node controller also terminates actors that have lost client connections and implements shell utilities such as the \$Var substitution mechanism and the redefinition of I/O predicates.

From an external perspective and when dealing with actors, the intricate workings of the node controller are intended to remain largely concealed, functioning much like an invisible “ether” through which messages seamlessly flow. In this manner, the node controller abstracts away the underlying technical details, allowing users to benefit from its comprehensive management capabilities without the need to engage directly with its multifaceted processes.

3.4.2 The shared Prolog database

A node may host a shared Prolog database accessible over HTTP. It can be:

- *self-contained* – every predicate the exported relations need is available on any node: defined locally, a Web Prolog built-in, or a standard library predicate. Such a dataset is directly consumable by other nodes or clients.
- *dependent* – it relies on a predicate that is not guaranteed to be present wherever the database is consumed: one imported from a custom (non-standard) module, a remote predicate, or one with no provider at all. Portability is then conditional on that dependency being resolvable at the destination.

The predicates – which will be referred to as the *node-resident* predicates – are typically maintained by the owner of the node. External users are not able to make any changes to them, only the owner of the node is allowed to do so. Any client connected with the node has access to these predicates in addition to the Web Prolog built-in predicates, and can pose queries over the web APIs formulated in terms of them.

The contents of the file `mortal.pl` must be valid Web Prolog source code. It may be something like this:

```
mortal(Who) :- human(Who).

human(socrates).
human(plato).
human(aristotle).
```

This is also what a user is presented with when steering an ordinary browser to the URI `http://n7.org`. This is an example of a shared Prolog database that is *self-contained*. In Chapter 4 we will demonstrate how this property makes it possible to achieve what we think of as *instant portability*.

A node also reports this property: its `/node_info` advertises whether its database is self-contained, and the Discovery Hub surfaces the reported value for each registered node, as illustrated in Figure 3.9.

However, a file might also look like this:

```
:- use_module(philosophers, [philosopher/1]).

mortal(Who) :- human(Who).

human(socrates).
human(Who) :- philosopher(Who).
```

This database is *dependent*: it draws `philosopher/1` from a separate `philosophers` module rather than defining it here. It functions at this node only if that module is present, and it is portable to another node only if `philosophers` is deployed there as well. (Had the file simply called `philosopher/1` without providing it anywhere, it would be dependent in a stronger sense – on a predicate with no provider at all.)

3.4.3 Deploying a node

To make this as concrete as possible, let us assume that the owner of a node wants to make the data in the file `mortal.pl` available to clients. The owner uses the following command to set up a Prolog node:

```
$ prolog-node --port=80 --src=mortal.pl --settings=settings.pl
```

As a result, a new node is created and the contents of the file is loaded into its shared Prolog database. The node is configured according to the settings given in the file `settings.pl`. After having performed all the usual moves necessary for the deployment of a web server, access to the node is offered through the set of API endpoints listed in Table 3.1:

API endpoint	Resource or functionality
<code>http://n7.org</code>	The shared Prolog database
<code>http://n7.org/shell</code>	A simple shell environment
<code>http://n7.org/call</code>	The stateless HTTP API
<code>http://n7.org/toplevel.spawn</code>	The semi-stateful HTTP API
<code>ws://n7.org/actor</code>	The stateful WebSocket API

Table 3.1 Proposal for how to link URIs with targets. Note that the stateful and semi-stateful APIs have additional endpoints.

By steering an ordinary browser to the URI `http://n7.org` an authorized user may want to just browse the program. Or perhaps query it by making a visit to

<http://n7.org/shell>. Or perhaps the plan is to build an end-user application talking to the node over any of its web APIs.

Note also that because a Prolog node is also a web server, it may offer other HTTP endpoints as well, some of which may not be relevant to its core functionality.

Other ecosystems show that this strategy works. In the Prolog world, Jan Wielemaker's *Cliopatria* demonstrated how a substantial Web application could be delivered as a ready-to-run system: users provided their data, made modest extensions, and obtained a full Semantic Web portal. In the Elixir world, Phoenix goes even further by generating complete, fault-tolerant applications with supervision, telemetry, routing, and deployment already in place. Developers simply add the behaviour that makes the system theirs. These examples illustrate how a language ecosystem can supply standardised, reproducible scaffolds that capture the best practices of a community.

3.4.4 Settings

As suggested by Figure 3.5, a Web Prolog node is governed by a set of *owner-controlled settings*. Only the node owner may change these settings; some of them may be *readable* by clients over HTTP, so that clients can discover the node's advertised capabilities and operational limits. Collectively, the settings determine which capabilities the node offers, who may access them, and the resource envelope within which actors and sessions are allowed to run.

It is useful to group settings into five categories:

1. **Identity and advertised capability.** The node advertises a *profile* to clients. This is primarily a discovery and compatibility mechanism: it tells clients what kind of service they can expect.
2. **Resource governance.** The node may enforce limits on injected code size, actor-local database size, cache bounds, and the maximum number of concurrent actors that client code is allowed to spawn. At the transport layer it may also enforce connection-related limits (for example, the maximum number of connections, and the maximum number of agents sharing a connection). These limits are part of the node's resource policy and are set by the node owner.
3. **Admission control.** The node may restrict which clients may connect and which goals may be executed, for instance via allow/deny lists and related authorization rules.
4. **Session lifecycle and PTCP timeouts.** Two timeouts are particularly important for PTCP sessions (cf. the PTCP state diagram in Figure 3.4). A *running timeout* bounds how long a session may remain in the working state (i.e. **s2**). If this timeout is exceeded, the node aborts the current goal and returns an error (e.g. `time_limit_exceeded`) to the caller, while keeping the session actor alive. An *idle timeout* bounds how long a session may remain inactive in an idle state (i.e. **s1** or **s3**). If this timeout is exceeded, the node terminates the session actor. If

the session is monitored, the client will subsequently observe a **down** message consistent with normal actor termination.

5. **Bundle loading policy.** The node may require, or merely prefer, that loaded bundles are self-contained, i.e. do not rely on unresolved external dependencies.

Note that readable over HTTP does not mean modifiable: the intention is that the node may expose a read-only subset of the settings that are relevant to interoperability and capacity planning, while keeping sensitive policy (notably admission rules) private.

Several of these settings are best understood as *capability gates* rather than mere tuning parameters. In particular, a node distinguishes between what the owner may do and what external clients may do. Clients are intended to program *session-local* concurrent computations, while the owner configures the long-lived surface of the node and installs durable services. This distinction will later be reflected in which concurrency-oriented predicates and options are admitted for client code, especially for lifetime control (for example whether a spawned actor may outlive its parent) and, in the distributed case, which combinations of remote placement and lifetime options are permitted.

3.4.5 Two transports – three web APIs

The primary responsibility of the node controller is to facilitate web API interactions. The most capable nodes are designed to support two different transports – HTTP and WebSocket – and three distinct API types: a stateless HTTP API, a semi-stateful HTTP API, and a stateful WebSocket API. This architecture enhances flexibility by increasing the likelihood that at least one API will effectively serve a given end-user application.⁶

The HTTP transport

The *Hypertext Transfer Protocol* (HTTP) is a core communication protocol of the World Wide Web, enabling the exchange of resources like web pages and data between clients and servers. Operating as a stateless request–response protocol, it supports methods such as GET, used to retrieve resources, and POST, used to submit data to a server.

A client interacting with a node by making requests over a HTTP connection is illustrated in Figure 3.6.

HTTP typically uses TCP for reliable transmission and supports secure communication through HTTPS with TLS encryption. Evolving since the 1990s, HTTP has advanced through versions like HTTP/2 and HTTP/3, enhancing performance

⁶ Strictly speaking, HTTP and WebSockets both live in the application layer, not the transport layer. But in practice, even if TCP is the real transport protocol under the hood, it makes sense to think of them as alternative ways of transporting data between agents.

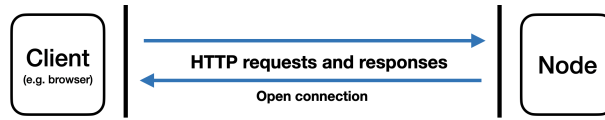


Fig. 3.6 Interaction between a client and a node over an HTTP connection.

and features like caching, authentication, and content negotiation. There is a vast infrastructure for HTTP and HTTPS that already exists (proxies, firewalls, caches, and other intermediaries), ensuring that security requirements of the modern web are fulfilled and the browser and the node can validate each other.

The WebSocket transport

In order to enable a client to control all aspects of a set of toplevels or other kinds of actors, the most capable type of node offers a WebSocket sub-protocol. WebSocket is a real-time, low latency, bidirectional protocol for asynchronous communication between a client and a node. A WebSocket sub-protocol is an application-layer protocol, which operates on top of the WebSocket protocol, leveraging the transport services provided by TCP. The three phases in the life-cycle of a WebSocket connection is illustrated in Figure 3.7.

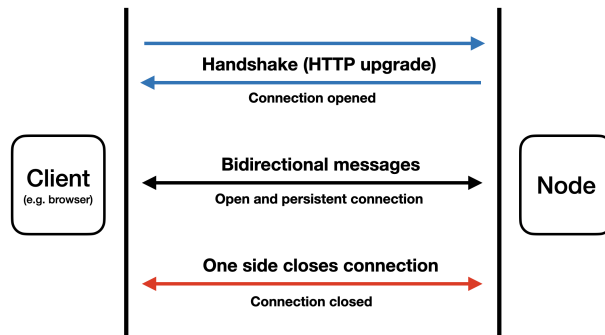


Fig. 3.7 The life-cycle of an interaction between a client and a node over a WebSocket connection.

In the first phase (in blue in the diagram) a WebSocket connection is initiated by the client via an HTTP connection which is then switched over to the WebSocket protocol. In the second phase (in black), the client and the node starts talking to each other. This is where the sub-protocol comes in.

Subject to the value of a HTTP header or parameter, answers and other kinds of messages arriving from the node are returned in the form of either JSON or Prolog

text. Client code written in languages other than Prolog would normally request that they be encoded in JSON.

WebSocket client libraries are available for the most common general programming languages. However, the most common use of WebSockets is in communication between a web browser and a server. In all the major web browsers, the WebSocket object provides a JavaScript API for creating and managing a WebSocket connection to a server, as well as for sending and receiving data on the connection.⁷ The role of JavaScript here is to create a new websocket, to define the necessary handlers for `onopen`, `onmessage`, `onerror` and `onclose` messages, and to call the methods `send` or `close` for sending messages or closing the connection.⁸

In the next section, and with the help of a number of scenarios, the actual APIs will be demonstrated. For details, see Appendix A.

3.5 Node profiles

Not all Prolog nodes are equal. They vary in the capabilities they offer clients, and we use the term *profile* – borrowed in the standards sense – for the declared set of capabilities a node commits to providing. A profile is a tailored subset of the full design: it constrains what a node must offer in order to make that offer verifiable, portable, and predictable. Less capable profiles are easier to specify, implement, and standardise, while still participating in the same ecosystem.⁹

We distinguish two kinds of capability: *language capabilities* (which Prolog fragments and predicates are available) and *interactional capabilities* (which transports and web APIs the node supports). Four profiles are defined: RELATION, ISOBASE, ISOTOPE, and ACTOR. Their nesting is shown in Figure 3.8: features in the inner rings are required by every profile, while features in the outer rings are restricted to more capable ones. For example, owner-defined shared predicates and the stateless HTTP API appear inside the RELATION ring, signifying that these features are supported by all profiles. The WebSocket API appears in the outermost ring, indicating that it is guaranteed to be available only in the ACTOR profile.

The RELATION profile is the base. It requires only the stateless HTTP API and a set of owner-defined predicates; it does not require any Web Prolog built-in predicates. ISOBASE adds the standard built-in predicate set, `rpc/2-3`, and asynchronous RPC. ISOTOPE further adds a session-oriented toplevel actor accessible over the semi-stateful HTTP API, enabling persistent state and interactive I/O within

⁷ See <https://www.w3.org/TR/websockets/>

⁸ For a good general introduction to WebSocket we recommend (Lombardi, 2015).

⁹ Throughout this section *capability* means a client-visible feature or power provided by a node – support for a given web API or built-in predicate. The narrower capability-security sense of an unforgeable authority token – a value whose possession confers the ability to act – is developed in Chapter 4. The two uses are related: node-level capabilities determine what actions are available in principle, while capability values (such as pids) determine which of those actions a client can actually perform.

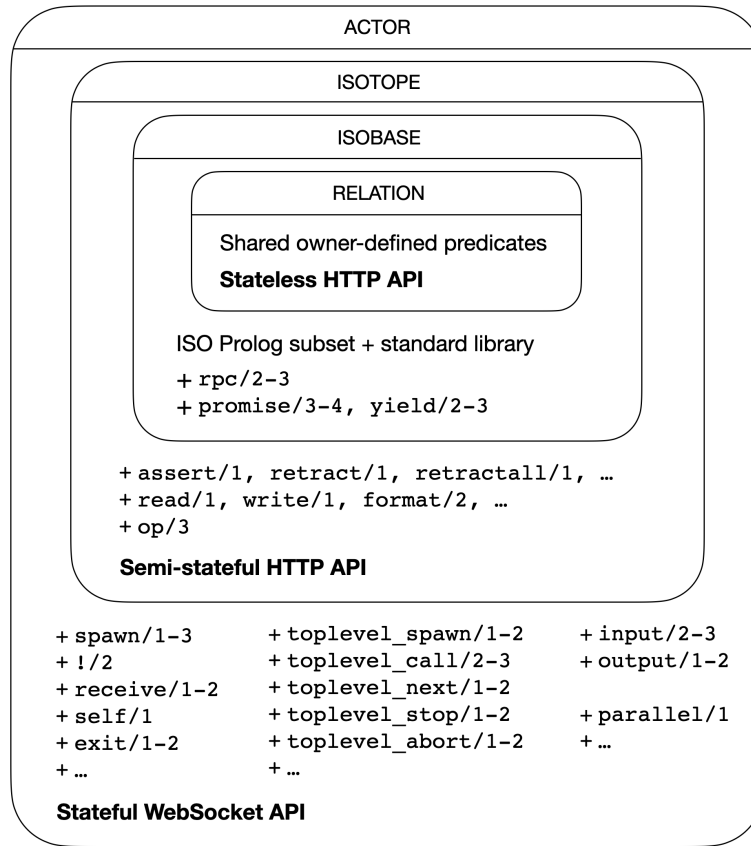


Fig. 3.8 The profile hierarchy. Each ring adds capabilities to those of the profiles inside it.

a session. **ACTOR** adds the full suite of Erlang-style concurrency predicates – `spawn/2-3`, `!/2`, `receive/1-2`, and the derived predicates built on them – and exposes them over a bidirectional WebSocket connection. On the owner side, an **ACTOR** node also provides `register_service/2` and `unregister_service/1` for publishing and withdrawing stable names in the node's public service namespace. These are administrative operations required of an **ACTOR** implementation, not capabilities granted to ordinary clients. A node offering the **ACTOR** or **ISOTOPE** profile can be both queried and programmed; a node offering only **ISOBASE** or **RELATION** can only be queried.

The **RELATION** profile differs from the others in kind, not merely in degree. An **ISOBASE**, **ISOTOPE**, or **ACTOR** node accepts arbitrary Prolog goals – within the predicate subset defined by the profile – and evaluates them. A **RELATION** node does not. It exposes specific named relations through a fixed query interface: a client can ask `wife(H, W)` because the node advertises that relation, not because

it evaluates arbitrary Prolog code. The goal in a request to a `RELATION` node is better understood as a pattern matched against a known schema than as a program to execute. A conformant `RELATION` node need not contain a Prolog engine at all.

Conformance

To be considered *conformant*, a node *must* provide *all* capabilities specified by its profile, to the role for which each capability is defined. For instance, an `ACTOR` node *must* offer clients access to the three web APIs and the complete client-facing `ACTOR` predicate set indicated in Figure 3.8; it must also provide the owner-only service-publication operations `register_service/2` and `unregister_service/1`. Similarly, an `ISOTOPE` node *must* provide clients with the two HTTP APIs and all predicates defined in the `ISOTOPE` subset of Web Prolog. Other profiles follow analogous requirements.

A node *may* provide *more* than what is mandated by its profile, including additional predicates or web APIs. For example, a `RELATION` node may support the `rpc/2-3` predicate as well as both the stateless and semi-stateful HTTP APIs, but it is not obligated to do so. Only the stateless API is required. Clients that depend on such extras sacrifice portability across nodes at that profile level.

It is perfectly reasonable for a node to not be conformant to any profile whatsoever. For example, a node whose only purpose is to serve a fast-paced, web-based game may make good use of the `WebSocket` API, but its owner may choose to block the use of everything that is not required for the support of the game. Such a node makes no portability claim at the level of interaction; it is the act of conforming to a profile that brings a node into the scope of the portability guarantees defined in Section 9.2.

What are profiles good for?

Profiles do two things simultaneously: they give clients a stable contract and give node implementers a scalable commitment. A client talking to an `ACTOR` node knows it may spawn actors, use persistent sessions, and receive push events over `WebSocket`; the same client talking to an `ISOBASE` node knows it may not. A node implementer choosing to deploy at the `RELATION` level knows that a Python microservice wrapping a database is sufficient; one targeting `ACTOR` knows what the full engineering cost entails. The profile a node advertises is thus a public, verifiable statement of what both parties can assume.

Crucially, this contract operates at the level of *observable interaction*, not implementation. Across all four profiles, the same shell-style pattern is preserved: a client submits a goal and receives solutions one at a time, lazily, on demand. What differs is only the infrastructure behind that pattern – stateless HTTP, a semi-stateful session actor, or a live `WebSocket` stream. In the extreme case a `RELATION` node may produce correct answers without any Prolog execution at all; from the client’s perspective the interaction is indistinguishable. This separation of observable contract

from implementation commitment is what allows the profile hierarchy to be both strict (conformance is testable) and open (any conformant implementation qualifies, regardless of how it works internally).

The shell-style interaction pattern deserves emphasis here because it is central to logic programming itself. Presenting one solution at a time, pausing for user input, and resuming search on demand exposes the mechanics of backtracking and makes nondeterministic computation tractable and inspectable. Profiles ensure that this familiar interaction can be offered even by nodes with limited capabilities, while allowing more capable nodes to support richer forms of control, state, and concurrency.

Profiles also carry a security dimension, developed fully in Section 4.5. A node's profile determines not only what a client can *do* but what it can *express*: excluding I/O and dynamic database modification from ISOBASE is not merely a protocol convenience but removes those operations from the client-accessible language fragment entirely. Greater expressiveness at higher profiles therefore comes paired with greater exposure, a tradeoff the security architecture addresses through inexpressibility, name discipline, and lifetime containment. The profile a node advertises is, in this sense, also a declaration of its security posture.

3.5.1 A profile in practice: the Discovery Hub.

The proof-of-concept deployment includes a Discovery Hub at <https://n0.elfenbenstornet.se/discovery-hub>, itself implemented as the Prolog node `n0`. The hub maintains a directory of the registered demonstrator nodes and periodically probes each one. It publishes the resulting information – including current reachability, advertised profile, authorisation mode, database portability, and provided predicates and services – as owner-defined relations over the stateless `/call` API. The filters in the browser interface are translated into ordinary Prolog goals and evaluated against these relations.

Node `n0` advertises the `RELATION` profile. Internally, however, it uses an actor to perform the periodic probes. That actor and the corresponding actor operations are not exposed to clients. The hub therefore provides a particularly compact illustration of the distinction made above: a profile describes a node's observable contract, not everything used to implement it. It also demonstrates that useful ecosystem infrastructure, including node discovery, can be constructed from ordinary relations and the stateless Web Prolog API.

The following subsections work through five scenarios – `RELATION`, `ISOBASE`, `ISOTOPE`, `ACTOR` via browser, and `ACTOR` via social robot – each illustrating the interaction model that its profile enables.

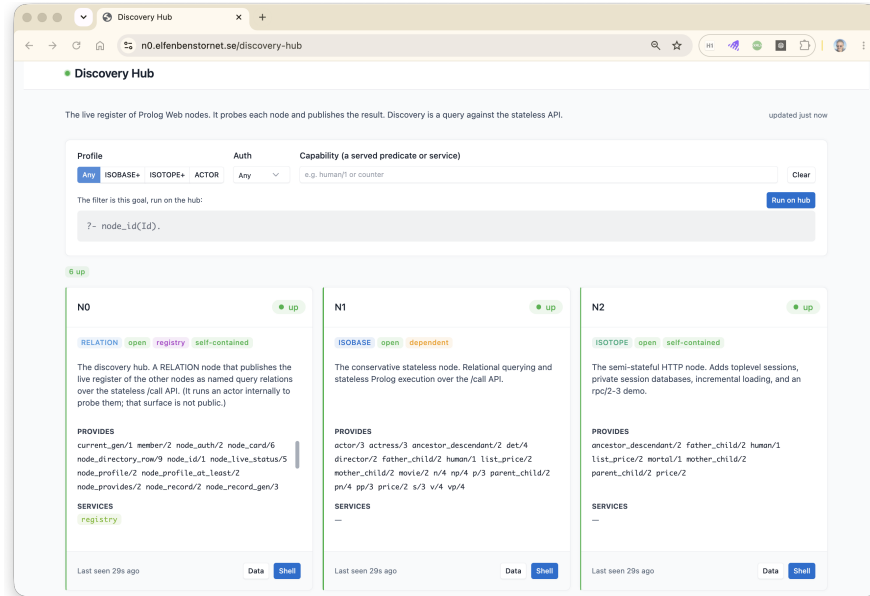


Fig. 3.9 The Discovery Hub at node n0. The browser interface queries a live directory of demonstrator nodes by profile, authorisation mode, and advertised predicate or service. Although n0 uses an actor internally to probe the registered nodes, it exposes the directory through a stateless RELATION interface.

3.5.2 Browser talking to a RELATION node

As indicated by the profile hierarchy in Figure 3.8, the stateless HTTP API is the only web API that *must* be supported by a RELATION node. This does not distinguish it from nodes having a more capable profile, as this is required by them too. What makes it stand out is that it is not required to offer access to any Web Prolog built-in predicates or control constructs *at all*. Even control constructs such as `,/2` and `;/2`, may not be available.

In the scenario depicted in Figure 3.10, a user equipped with an ordinary web browser is talking to the RELATION node at `http://n1.org`. The web-based terminal displays the remnants of a session during which the user executed two queries. The two question marks signify that neither the owner-defined resources nor the way answers are produced is known by the user. The interaction started with the loading of the Prolog shell at `http://n1.org/shell`.

The interaction opened with the welcome banner and a `?~` prompt; a JavaScript loop then began reading queries, translating them into HTTP requests, and writing answers back to the terminal.

Let us look at how the first solution to `?~wife(H,W)` was retrieved by using the stateless HTTP API. The URI pointing to this solution is:

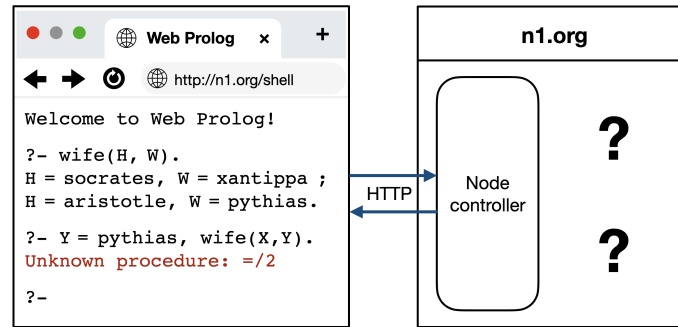


Fig. 3.10 A scenario involving a web browser in conversation over HTTP with a remote RELATION node. The two question marks signify that neither the owner-defined resources nor the way answers are produced is known by the user.

`http://n1.org/call?goal=wife(H,W)&limit=1&offset=0`

Note the use of `limit` and `offset` parameters. Although here they are parameters rather than options, their defaults and the effect that they have on answer terms is the same as for the options accepted by `toplevel_call/3`.

URIs such as these are simple, they are meaningful, they are declarative, they can be bookmarked, and responses are cacheable by intermediates. Most of these desirable properties are there because the client-node interaction involved during the process of resolving the URIs is stateless.

When the user entered the query `?-wife(H,W)`, (and assuming that a relative path was used instead of an absolute one) the following GET request was made to the node:

```
GET /call?goal=wife(H,W)&limit=1&offset=0 HTTP/1.1
Accept: application/json
```

The Accept header requested responses in JSON. (In this case the header could have been left out since JSON is the default.)¹⁰

The node responded with the following JSON structure, indicating that the query succeeded:

```
{ "type": "success",
  "data": [{"H":"socrates","W":"xantippa"}],
  "more": true }
```

The data field contains the variable bindings and the value `true` of the `more` field signals that further solutions may be available. This triggered the printing of the variable bindings and served as a prompt for how to continue the exploration.

¹⁰ A `format` parameter is also supported as this allows us to easily make a request from the address field of a web browser.

The user wanted to know if there were more solutions and typed a semicolon which led to the following request being made, looking exactly like the previous request, except that the `offset` parameter was incremented from 0 to 1:

```
GET /call?goal=wife(H,W)&limit=1&offset=1 HTTP/1.1
```

The node responded like so:

```
{ "type": "success",
  "data": [{"H":"aristotle","W":"pythias"}],
  "more": false }
```

The `more` field was now `false`, indicating that no further solutions existed, and after having printed the next set of bindings and a full stop, the shell again produced the `?-` prompt, inviting the user to enter another query.

At this stage of the interaction, it is reasonable for the user to expect that `wife/2` has the mode `wife(?, ?)` and that queries such as `?-wife(socrates, W)` or `?-wife(H, pythias)` should also succeed. A query such as `?-wife(plato, W)` should fail, however, and the response would be the following:

```
{ "type": "failure" }
```

Finally, an erroneous query, such the attempt in the scenario to query a predicate which is not defined, should raise an error, and show up as a response like so:

```
{ "type": "error",
  "data": "Unknown procedure: =/2" }
```

We can summarize this by saying that a **RELATION** node must offer clients a truly relational query interface over stateless HTTP. Any node failing to meet this expectation is not a conformant **RELATION** node. A **RELATION** node is required to respond to a client as if it were Prolog, thereby allowing the user to lazily back-track over possible solutions. Although a **RELATION** node may not truly perform backtracking, it must behave externally as though it does.

The lack of built-in predicates naturally makes a **RELATION** node very easy to implement. There are two quite different ways a **RELATION** node can come into existence. First, a node that already implements a richer profile can be turned into a **RELATION** node by installing a filter that inspects the `goal` parameter in each `/call` request and only allows it through if it matches a declared relation. Such a filter can be as simple as matching the submitted goal against a whitelist of terms – for instance, accepting only `employee(, , )` and `name(, )`, and rejecting everything else with an error. A node that is supposed to serve a particular application rather than act as a general compute server will often operate this way: it implements ISOBASE or more internally, but presents a **RELATION** interface to its clients.

Second, a **RELATION** node can be implemented from scratch in any programming language. (Recall that we do not require that a Prolog node is written in Prolog, only that it can talk Prolog.) At a minimum, an implementation must parse Prolog terms and produce variable bindings for queries against its data. A Python web

service backed by a relational database, or a microservice wrapping a key-value store, can present itself as a conformant **RELATION** node as long as it implements the stateless HTTP API and responds with the expected JSON structure. This is the profile that makes the Prolog Web interoperable with the rest of the Web: any structured data endpoint can, in principle, participate in the ecosystem.

3.5.3 Browser talking to an ISOBASE node

For a node to adhere to the requirements of the ISOBASE profile, it *must* support the same stateless HTTP API as required by the **RELATION** profile. In our next scenario, illustrated in Figure 3.11, the user is running the same query as in the previous scenario. It means that we do not need to describe the details of requests and responses again. Instead, we focus on the language capabilities of the node and the contents of the shared Prolog database offered by it, and visible in the figure.

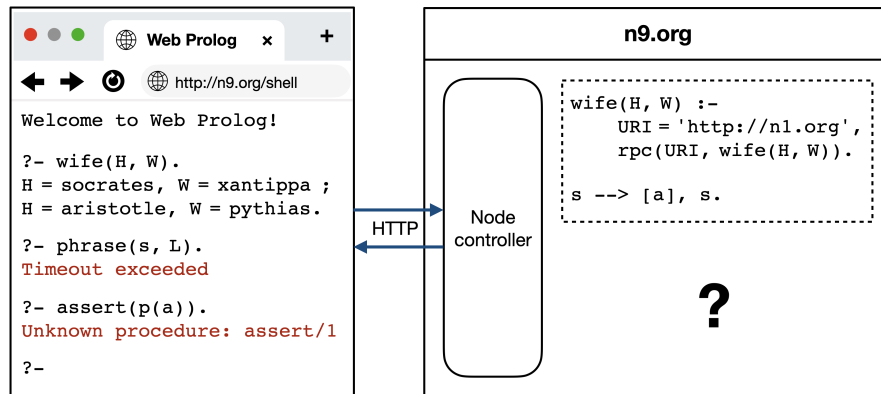


Fig. 3.11 Stateless interaction in a scenario involving a web browser and a remote node.

The main difference compared to the previous scenario is that an ISOBASE node *must* make a fairly large number of Web Prolog built-in predicates available, including a large subset of ISO Prolog predicates (recall the cloud of ISO Prolog predicates in Chapter 2), standard library predicates, definite clause grammars, as well as predicates for working with the Web. (The scenario differs from the previous one in that it allows the user to browse the source code defining the predicates offered by the owner of the node.)

An ISOBASE node must also support `rpc/2-3`, an easy-to-use predicate for making *nondeterministic remote procedure calls* from one node to another over stateless HTTP, communicating in a way that is essentially *synchronous*. In the scenario in Figure 3.11, the URI `http://n1.org` in the first argument of the call

to `rpc/2` shows that it accesses the relation `wife/2` made available by the node `n1.org` in the previous scenario.

In addition to the synchronous `rpc/2-3`, an ISOBASE node must also support *asynchronous* RPC using two built-in predicates; `promise/3-4` and `yield/2-3`. It turns out that these three predicates can all be implemented on top of the stateless HTTP API. They will be dealt with in more detail in Chapter 4.

Stateless interaction has one significant constraint: once a query is submitted, neither the client nor any other party can abort it. Because there is no control channel back to the running computation, ISOBASE nodes must enforce a timeout. This is what happened with the user's second query in the Figure 3.11 scenario: it was met with a `Timeout exceeded` message. Browsing the contents of the node's shared database, the problem seems to be that the node owner forgot to add a DCG rule `s --> []` that would have allowed the call to terminate.

Another limitation is that a truly stateless request-response protocol simply cannot support updates to the dynamic private database of an actor, nor I/O. It means that ISO predicates for modifying the dynamic database (e.g. `assert/1` and `retract/1`), or I/O (e.g. `read/1` and `write/1`), are *not* available. This is what happened with the user's third query, which was met with an `Unknown procedure` error message when trying to assert the clause `p(a)`. The predicate `assert/1` is simply not available to a client talking to an ISOBASE node.

3.5.4 Browser talking to an ISOTOPE node

An ISOTOPE node offers clients a shell backed by a toplevel actor. (Importantly, this does not mean that the internal `toplevel_*` predicates are directly exposed to programmers.) By using the process identifier of this actor as a session key in otherwise stateless HTTP requests, the node provides a semi-stateful API: each request is stateless in itself, but thanks to the pid they address the same long-lived toplevel actor.

The client's initial request can populate the actor's private database, which can then be updated dynamically throughout the session. Furthermore, although the client must initiate all interactions, the toplevel actor can not only respond to queries but can also prompt the client for further input or produce output at arbitrary moments. As long as the client keeps supplying the session identifier, this will work just fine.

In the scenario in Figure 3.12, the client established communication with a toplevel actor on an ISOTOPE node, which is a node that must provide all the capabilities that an ISOBASE node has, and then some.

The request below instructed the node controller to spawn a toplevel actor running as a session, and to populate its private database with the value of the `load_text` option. (Let us assume that the clause `q(X) :- p(X)` was picked up from an editor window that is not shown in the diagram.)

```
POST /toplevel_spawn HTTP/1.1
Content-Type: application/json
```

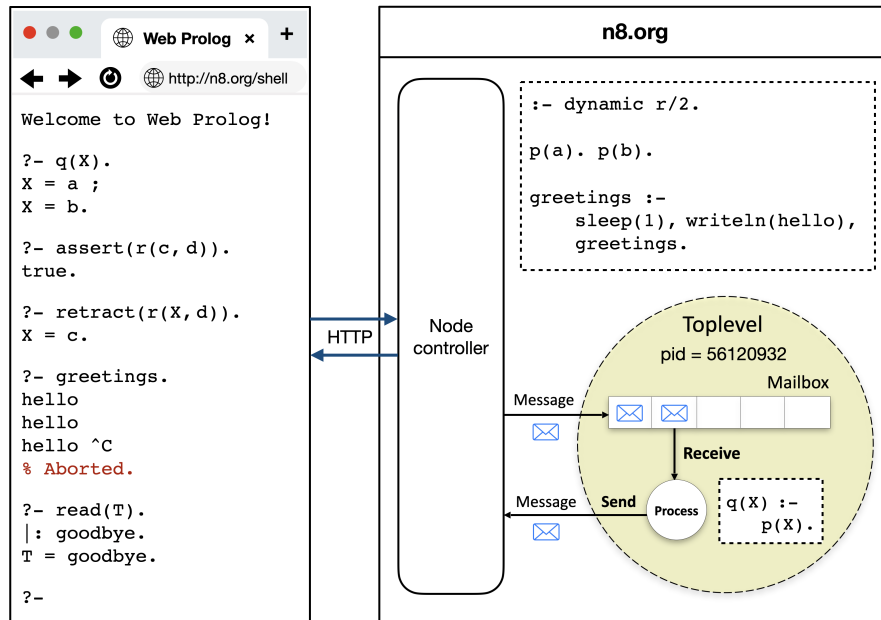


Fig. 3.12 A scenario involving a web browser in conversation over semi-stateful HTTP with a remote ISOTOPE node running a toplevel actor.

```
{ "options": "[session(true), load_text('q(X):-p(X).')]" }
```

The node responded with a JSON structure containing the pid for the spawned toplevel actor:

```
{ "type": "spawned",
  "pid" : 56120932 }
```

This produced the prompt `?-` in the terminal, inviting the user to enter a query. The query `?-q(X)` caused the JavaScript shell to make the following request to the node:

```
GET /toplevel_call?pid=56120932&goal=q(X)&limit=1 HTTP/1.1
```

(Note that the pid must be sent along when querying the toplevel.) The request told the node controller to make a call to `toplevel_call/3` and to convert the resulting answer term into a JSON structure that encodes the first solution:

```
{ "type": "success",
  "pid" : 56120932,
  "data": [{"X":"a"}],
  "more": true }
```


Except for the `pid`, the JSON structure representing this solution is identical to the response when making a stateless request.

The value `true` of the `more` property signifies that more solutions may be available. Asking for the next solution results in a similar message, except that the value of the `data` property becomes `[{"X": "b"}]` and the value of `more` becomes `false`.

When the user asked for another solution, the following request was made:

```
GET /toplevel_next?pid=56120932 HTTP/1.1
```

Note that it took the *combination* of the clause that was loaded into the `toplevel`'s private Prolog database, and data in the shared database to answer the query `?-q(X)`.

Because the `toplevel` persists across requests, it makes sense to allow the client to update its private database using `assert/1` and `retract/1` during the session.

```
GET /toplevel_call?pid=56120932&goal=assert(r(c,d)) HTTP/1.1
```

With this, the clause `r(c,d)` ended up in the private database of 56120932, but was removed again when the user decided to call `retract(r(X,d))`.

The user then called `greetings/0`, which calls `sleep(1)` and `writeln(hello)` in a loop. The `writeln/1` predicate has been redefined by the node controller as `writeln(Term) :- output(Term)` so this resulted in a series of messages of the following form to be sent back to the client, one per second:

```
{ "type": "output",
  "pid" : 56120932,
  "data": "hello" }
```

For this to work over HTTP, a trick somewhat similar to so-called *long-polling* is employed: every time a JSON structure of type `output` reaches the client, it must issue a new request for a response, which can come as more output, or some other response. See Section 3.6.2 for more details.

The call to `greetings/0` will run forever unless the user does something about it. When the user hits Control-C, the nonterminating query is aborted, and the node responds with the following message:

```
{ "type": "abort",
  "pid" : 56120932 }
```

If the client does not exit the remote process, the node may terminate it after a set time with no activity. The `pid` will no longer be valid and any changes made to the workspace of the process will be lost.

3.5.5 Browser talking to an ACTOR node

Unlike an ISOTOPE node, an ACTOR node gives clients access to the full set of Erlang-style concurrency predicates and exposes them over a bidirectional

WebSocket connection. This combination enables push-driven interaction, mixed-initiative communication, and concurrent actor programming – capabilities that the HTTP-based APIs cannot support. Figure 3.13 illustrates a scenario in which a user engages with an ACTOR node over such a connection.

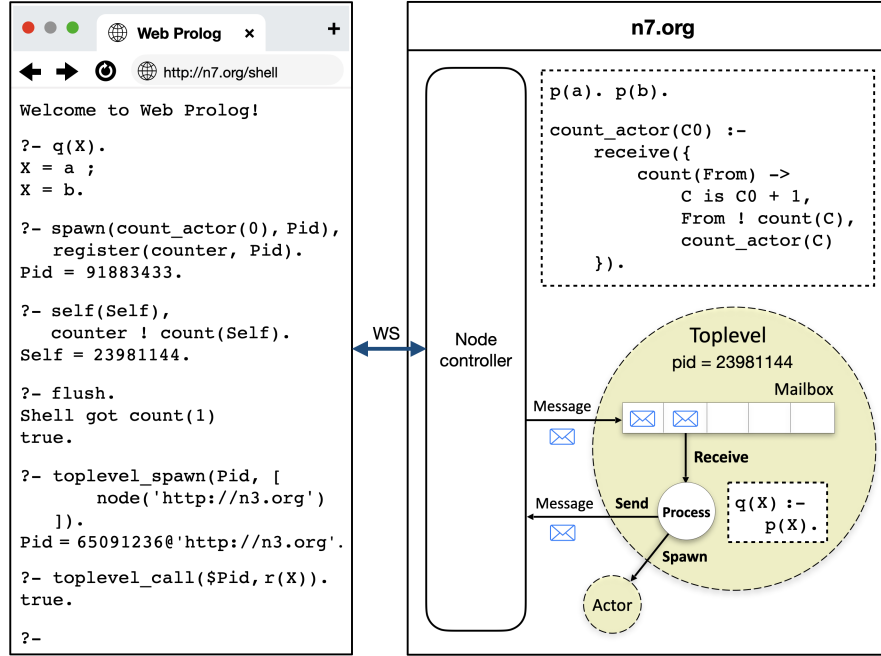


Fig. 3.13 A scenario involving a web browser in conversation with a remote node running a toplevel actor over a stateful WebSocket connection.

After having loaded the shell and established a WebSocket connection with the node, the JavaScript process sent the following command to the node's controller, instructing it to spawn a toplevel actor running a session with the clause `q(X) :- p(X)` loaded into the actor's private database:

```
{ "command": "toplevel_spawn",
  "options": "[session(true), load_text('q(X):-p(X).')]" }
```

As requested by the client, the pid of the toplevel actor was returned to the client, not in the form of a HTTP response, but embedded in a JSON formatted message sent over the WebSocket connection:

```
{ "type": "spawned",
  "pid" : 23981144 }
```

From here on, the solving of the query `?-q(X)` proceeds in a way very similar to how it worked in the ISOTOPE scenario, except that instead of the JavaScript shell

making an HTTP GET request, it sent the following command message to the node over the WebSocket connection:

```
{ "command": "toplevel_call",
  "pid": 23981144,
  "goal": "q(X)",
  "options": "[limit(1)]" }
```

Knowing that the node's shared database offers a definition of `count_actor/1`, the user then decided to spawn a count server of the kind we saw in Chapter 2. Here is the message sent over the WebSocket connection:

```
{ "command": "toplevel_call",
  "pid": 23981144,
  "goal": "spawn(count_actor(0),Pid),register(counter,Pid)",
  "options": "[limit(1)]" }
```

This resulted in:

```
{ "type": "success",
  "pid" : 23981144,
  "data": [{"Pid":91883433}],
  "more": false }
```

While the `toplevel` actor continued to serve shell queries, the spawned count actor ran concurrently and received `count` messages independently of the shell interaction, illustrating the concurrent, actor based nature of the ACTOR profile. With access to the pid of the spawned count server, the user could send it a `count` message in order to try it out, and call `flush/0` to inspect the response.

Finally, the user called `toplevel_spawn/2` with the `node` option. This is yet another way to do network-transparent concurrency that will be dealt with in more detail in Chapter 4.

In this implementation, if the user leaves the terminal at `http://n7.org/shell`, closing the WebSocket causes the controller to terminate the session `toplevel` (via `exit/2`); before exiting, the `toplevel` terminates the spawned count actor. This yields a supervision-like hierarchy, discussed further in Chapter 4.

3.5.6 A terminal talking to a browser node

The scenario in the previous subsection placed the ACTOR node on a *remote* machine, reached across the network over a bidirectional WebSocket. The very same interaction can take place against a node that lives entirely within the user's own browser tab. From the terminal nothing visible changes: the same Erlang-style concurrency predicates are typed, the same answers come back, and the same command and response messages pass to and from the `toplevel` actor. The session is, at the

level of what the user types and reads, indistinguishable from the remote one of Figure 3.13; we therefore do not repeat the traffic here, but draw out what that indistinguishability is telling us.

A browser tab, in other words, can host not a cut-down client but a *full* Web Prolog node – one that spawns actors, runs toplevels and serves queries exactly as a node on a remote machine does. That a tab can play this role at all is the first thing worth noticing; that the terminal cannot tell the difference is the second.

The reason it cannot is location transparency. A query is solved, an actor is spawned, a message is sent and flushed by issuing the same predicates against the same protocol; whether the node answering them sits on another machine or in the same tab changes only the *transport* those messages travel over, never their form. The protocol is the invariant; its carrier is an implementation detail.

The one visible trace the transparency leaves is the shape of a pid. A purely local actor's pid is a bare integer, such as the count server's 11719087; an actor on another node is node-qualified, 20071236@'http://n3.org'. Yet once an actor holds a pid, sending to it is the same operation either way: `toplevel_spawn/2` addresses a local actor and a remote one identically, and only the pid it returns betrays where the actor came to rest.

This is the point the scenario is really drawn to make. The final entry in the terminal spawns a toplevel on a remote node with the `node` option, and the browser node honours it by opening a genuine network WebSocket – now with the browser node itself in the role of *client* to `n3.org`. A tab is therefore not a leaf that can only be talked *to*; it is a first-class peer on the Prolog Web, reaching its fellow nodes over the same WebSocket transport any node uses, even as it talks to its own terminal and its own local actors without a network at all. This is the location transparency the ACTOR profile was designed to provide, now seen to span the boundary between a tab and the wider network.

The one difference transparency cannot paper over is lifetime. A remote node outlives any single session – closing a connection ends a session while the node goes on serving others – which is precisely why a remote session must arrange the disciplined containment of Chapter 4, a toplevel reaping the actors it spawned before it exits. A browser node shares the fate of its page: its actors live and die with the tab, and there is no way to end the local node short of ending the page, which ends everything together. The containment a remote session must arrange has, here, nothing left to arrange. The only fate not sealed by closing the tab is that of any *remote* actor the browser node spawned, whose survival depends on its own node's lifecycle policy. How a browser node actually realises all of this – concurrency, isolation, transport and teardown – is the subject of Section C.3.2 in Appendix C.

3.5.7 Social robot talking to an ACTOR node

In the previous scenarios, the shell and its toplevel actor served as the root of control: it was the first long-lived process from which other actors were spawned, monitored,

and addressed. In general, however, the root of an application need not be a toplevel at all. Any actor can play that role, and a client may interact with such an actor directly, without routing interaction through a shell-style PTCP conversation.

Figure 3.14 sketches a deliberately minimal toy scenario in which a social robot interacts with a remote ACTOR node over the stateful WebSocket API.

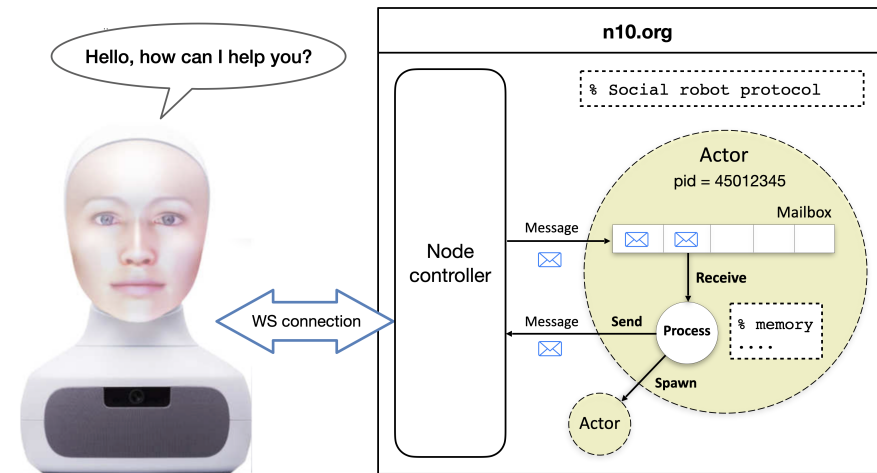


Fig. 3.14 Stateful interaction in a scenario involving a social robot and a remote ACTOR node.

The robot itself provides speech recognition and speech synthesis: it turns the user's spoken input into text, sends that text to the node, receives a reply as text, and speaks it back to the user. In this toy setup the remote side does nothing clever – it simply echoes the input – so the interaction is a “parrot” loop. The purpose is not to propose a useful robot skill, but to isolate the transport-level point: unlike the shell scenarios, the client is not a terminal driving a toplevel, and yet it can still engage in persistent, bidirectional interaction with a remote node.

At the protocol level, the scenario begins with the robot requesting that the node spawn an echo actor. The request is sent as a WebSocket message to the node controller (encoded here in JSON for the benefit of a non-Prolog client), and the node responds by returning the pid of the spawned actor. This pid becomes the capability by which the robot can address the actor in subsequent messages.

```
{ "command": "spawn",
  "goal": "echo_actor",
  "options": "[monitor(true)]" }

{ "type": "spawned",
  "pid": 47110023 }
```

The WebSocket transport matters here because the interaction is push-driven and continuous. The robot establishes one connection and then exchanges messages in both directions as events occur: it sends new utterances as soon as they are recognised, and the node can return responses as soon as they are produced, without requiring a new HTTP request per response. This is the sense in which the interaction is stateful at the connection level: the channel remains open, latency is low, and messages can be delivered whenever either party chooses to speak.

In realistic conversational systems, of course, echoing recognised text is not enough. One typically needs additional components such as natural language understanding, a dialogue manager, grounding in a knowledge base, and response generation, as well as timing-sensitive coordination of speech, gaze, and facial expressions. We return to such agent-level concerns in Chapter 8; the present scenario only serves to show that, with the ACTOR profile, the transport and interaction model already supports the kind of persistent, low-latency, bidirectional exchange that embodied clients require.

3.5.8 Could there be other profiles?

The hierarchy $\text{RELATION} \subset \text{ISOBASE} \subset \text{ISOTOPE} \subset \text{ACTOR}$ is useful because many applications need less than a full ACTOR node. Simpler profiles are easier to implement, govern, and deploy, while still supporting portable client behaviour.

At the same time, the hierarchy is not meant to be a closed taxonomy. The existing profiles mostly specify *what* capabilities are exposed at the node interface. One can also imagine profiles that express *how* a node chooses to do it: restrictions, guarantees, or disciplines that cut across the capability levels.

A PURE profile is one example: clients would be restricted to a declarative subset (excluding cut, non-logical arithmetic, and related extra-logical constructs), while still potentially allowing features such as source injection. Another example is a statechart discipline exposed as an optional ACTOR sub-profile for reactive conversational control. The same profile idea recurs one level down, within the statechart language itself: Chapter 6 defines a *flat* profile of SXML (Section 6.8) that restricts the language to non-hierarchical machines, exactly as a node profile restricts node capabilities.

3.6 Notes on node implementation

This chapter specifies node behaviour primarily in terms of externally observable contracts: profiles describe what a client may rely upon, and the web APIs describe the shapes of requests and responses. Implementation techniques are not part of the contract, and different Prolog systems – and different host platforms – will naturally realise the contracts in different ways. Nevertheless, a few recurring engineering

issues are worth highlighting. They explain several design choices, clarify why multiple APIs are worth supporting, and help readers anticipate the practical costs associated with each profile.

3.6.1 Statelessness as a protocol property

The stateless HTTP API is *stateless for the client*: each request contains all information needed to interpret it, and the node is not required to remember a conversational session in order to respond. This does not prohibit the node from keeping internal caches. It merely means that caches are best-effort optimisations whose presence must not be observable as a behavioural dependency. A node may evict cached state at any time and still remain correct and conformant.

As noted in Section 3.5.2, statelessness brings familiar web virtues: meaningful URIs, replayable requests, and cacheable responses. A corresponding limitation is that a stateless request cannot be *externally* interrupted by the client once evaluation has begun. There is no control channel associated with the computation, and no session actor to signal. In practice, therefore, the stateless API must rely on node-enforced resource policy, most importantly running timeouts, to avoid nonterminating or overly expensive computations.

Two strategies for implementing paged solution retrieval

The `offset` and `limit` parameters make it possible to retrieve solutions in slices. This subsection is concerned only with how a node might implement that contract efficiently; the architectural interpretation of paged retrieval as an instance of Prolog-style backtracking is discussed separately in Chapter 4.

A naïve implementation uses *restart-and-skip*: to compute the slice starting at offset N , the node restarts evaluation from the beginning, discards the first N solutions, and then collects up to `limit` further solutions. This strategy is simple and portable, but it may waste work when later slices are requested, because earlier solutions are recomputed only to be thrown away.

A more efficient strategy uses *save-and-resume*. The node evaluates the goal in an execution context that can be suspended after producing a slice and resumed later to continue search. The node may maintain a cache keyed by a fingerprint of the query (goal, template, and any relevant execution parameters) together with the next slice boundary. When a request arrives for offset N , the node first attempts to resume a cached computation that has already produced N solutions; if none is available, it falls back to restart-and-skip. This improves performance in the common case without changing the protocol contract.

Importantly, the choice between these strategies is not visible at the API level. Clients only observe that slices are returned in Prolog order and that the `more` flag correctly reports whether further solutions may exist. All optimisation is therefore

constrained by the requirement that cached state is disposable. For a stably enumerable goal – a goal over data the client cannot modify – eviction changes only the cost of subsequent responses, never their meaning: restart-and-skip and save-and-resume return the same slices in the same order. For a goal that reads mutable state, neither strategy is sound across requests, and a fallback from save-and-resume to restart-and-skip may change which solutions a slice contains; such goals fall outside the paging contract, as noted in Section 3.3.5.

3.6.2 Semi-stateful HTTP sessions and response retrieval

The semi-stateful HTTP API differs from the stateless API in one crucial respect: each request targets a long-lived toplevel actor, identified by a pid, and the observable behaviour of the conversation depends on the evolving state of that actor. Nevertheless, each individual HTTP exchange is still an ordinary request–response transaction. In this sense, the protocol is *semi-stateful*: the transport remains stateless, but the addressed computation is not.

From an implementation perspective, this style aligns naturally with the architecture of SWI-Prolog’s `library(pengines)` and systems such as SWISH: the server maintains per-session engines, while the client repeatedly issues short requests that either advance the computation, or retrieve the next available response produced by that computation.

Output and prompts fit this model once they are treated as ordinary protocol events emitted by the session actor, rather than as exceptional side effects. Concretely, the toplevel actor can be understood as producing a sequence of *responses* (answers, failure, errors, output fragments, prompts, and acknowledgements) that the client retrieves over a series of short HTTP exchanges.

The statechart in Figure 3.15 extends and specializes the core PTCP in Figure 3.4 with the states and transitions needed for HTTP delivery.

The additional states **s4** and **s5** shown in the figure are implementation states used to emulate push-style output through repeated polling requests. They are part of the HTTP transport mapping rather than part of PTCP itself. When the toplevel emits an `/output` event during evaluation, the client enters a local pull cycle: it retrieves the pending event and immediately issues a follow-up **poll** request for the next pending response. This cycle repeats until the toplevel produces a terminal response (`/success`, `/failure`, `/error`), at which point control returns to the main protocol states. The mechanism resembles long-polling only locally – within the scope of a single goal evaluation that happens to produce intermediate events – rather than as a general-purpose polling loop.

Prompts require one additional step: when the next pending response is a prompt, the client retrieves it, presents it to the user, and sends a follow-up request that delivers the user’s reply back to the session actor. From the protocol’s perspective, this is still a request–response rhythm and no polling is needed here; the client is simply advancing the conversation by alternately retrieving pending events and, when

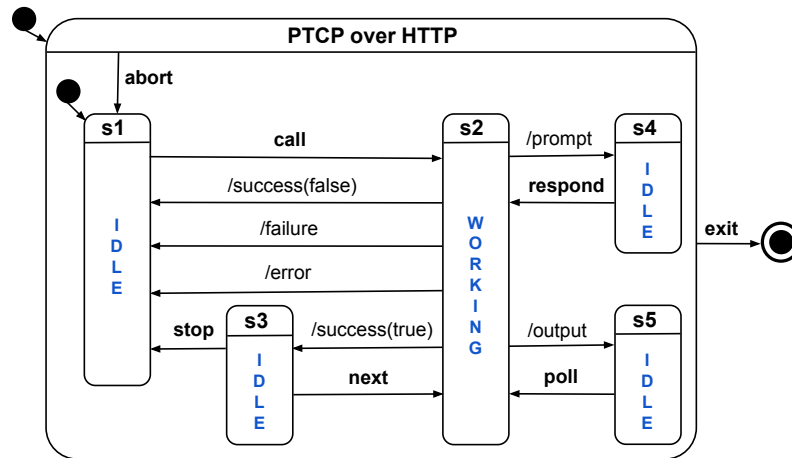


Fig. 3.15 Statechart specifying the PTCP for a successful conversation with a toplevel over HTTP.

prompted, supplying input. Ordering is preserved because responses are dequeued in the order they were produced. Transient transport failures may delay retrieval but do not change the logical meaning of the session.

3.6.3 Stateful WebSockets and push-driven interaction

The stateful WebSocket API exists for interactions that are naturally push-driven. Once a connection is established, the node can deliver answers, output fragments, prompts, and monitor events as soon as they are produced, without requiring a new client request for each response. This improves latency and avoids the request choreography of our long-polling trick. It also supports interleaved traffic from multiple actors over one connection, which is central to ACTOR-profile use cases.

Implementation complexity, however, shifts from request handling to connection lifecycle management. A robust node must maintain per-connection routing state (which pids belong to which client), preserve ordering within each actor stream, and enforce outbound back-pressure so that a slow client cannot cause unbounded buffering on the node. It must also define clear disconnect semantics: when a client vanishes, session actors and child actors are either terminated or detached according to policy, and monitored parties observe the corresponding `down`/`exit` events. In this sense, the WebSocket API is conceptually simple but operationally demanding: the protocol surface is small, while lifecycle and resource handling must be precise.

3.7 Summary

Chapter 2 equipped Web Prolog with actors and concurrency, and this chapter lifted those ideas to the level of Prolog agents and nodes. The central move was to treat toplevels as stateful actors governed by an explicit Prolog Toplevel Communication Protocol (PTCP). By making the conversational contract first-class – including how queries are started, how solutions are streamed, how I/O and aborts are handled, and how sessions are terminated – we turned an informal REPL interaction pattern into a well-defined behavior that can be implemented inside a node or exposed over the network.

On top of this, we introduced Prolog nodes as agents in their own right. A node is more than a mere container for code and data: it offers one or more toplevels, may host arbitrary actors, and is addressable on the Prolog Web through URIs and web APIs. The notion of node profiles (RELATION, ISOBASE, ISOTOPE, ACTOR) makes explicit which capabilities a node must offer to its clients. In particular, the ACTOR profile marks nodes that support the full actor model and PTCP-based toplevels, whereas the more restricted profiles support progressively smaller subsets of those capabilities. This lets implementors scale their commitment, and lets clients reason about what they can rely upon in a heterogeneous network.

A recurring theme in the chapter was the distinction between private state and shared data. Actors and toplevels maintain their own conversational and internal state – their “private beliefs” – while nodes may expose shared knowledge bases that serve as common background for all actors running on that node. The dynamic database, though available within an actor, remains process-local and cannot be used for inter-process communication, preserving the principle that processes should communicate to share memory rather than share memory to communicate.

The three communication paradigms – stateless HTTP, semi-stateful HTTP, and stateful WebSocket – serve complementary purposes. Stateless interactions offer simplicity, cacheability, and horizontal scalability; semi-stateful sessions enable dynamic database updates without persistent connections; stateful WebSockets unlock the full power of bidirectional, real-time interaction and concurrent actor programming. Supporting all three within a single node architecture allows applications to choose the appropriate trade-off between simplicity and capability.

Finally, the chapter touched on security considerations. Like JavaScript, Web Prolog is designed as a sandboxed language: it omits file I/O, socket programming, and direct OS access, relying instead on the host environment for such capabilities. Security in the actor model rests primarily on name distribution – if a process identifier is never revealed, the process cannot be accessed. This principle, combined with the constrained language subset and profile-based capability advertisement, forms the foundation of a security model appropriate for running untrusted code on shared infrastructure.

The overall message of the chapter is that Web Prolog is not just a language but also a way of packaging processes, protocols, and capabilities into networked agents that can be composed and reasoned about at a higher level of abstraction. Chapter 4 extends these local concepts across the network, showing how nodes discover each

other, how actors migrate and communicate across node boundaries, and how the Prolog Web emerges as a federated infrastructure for distributed logic programming.

The motivation for profiles is not only security, but also coordination. FYPB emphasises that portability is not just about language constructs, but also about portable libraries and developer tools – for unit testing, documentation, linting, and package distribution. A profile is a way to make that expectation concrete: it specifies which predicates, libraries, and APIs are guaranteed to exist on a node of a given kind. In other words, profiles separate *observable capability* from *implementation commitment*, enabling tools to target stable capability sets even when different implementations continue to innovate internally.

Chapter 4

The Prolog Web

Imagine the Web wrapped in Prolog, running on top of a distributed architecture – a network of nodes exposing HTTP and WebSocket APIs, speaking web formats such as JSON. Think of it as a high-level Web, with Prolog agents capable of serving answers to queries – answers that follow from what the Web knows. Moreover, imagine it being programmable, allowing Web Prolog source code to flow in either direction: from client to node, or from node to client. This is what **the Prolog Web** is all about.

The Prolog Web – the elevator pitch

The previous chapters introduced Web Prolog as a language and Prolog agents as programmable entities. This chapter turns to the third element of the Prolog Trinity: the *Prolog Web* itself – a network of nodes hosting actors that communicate across node boundaries. In short, we extend Web Prolog into a language for distributed programming.

The model draws on Erlang’s *network-transparent concurrency*: if we know a node’s identity and we are authorised, we can spawn an actor there; if we know an actor’s pid or its registered name, we can send it a message, even across node boundaries. Unlike an Erlang cluster, however, the Prolog Web is as open as the Web itself. Here, *openness* means more than mere reachability. It means heterogeneous ownership, weak trust assumptions, communication over ordinary Web infrastructure, and an environment in which latency, disconnects, and partial failure are part of normal operation. Openness therefore brings both opportunity and risk, so the chapter pays close attention to safety: how lifetime discipline and capability separation prevent orphan processes and resource leaks.

Openness also forces a basic separation of authority. In the Prolog Web there are two roles: *clients*, who temporarily borrow execution resources in order to run queries and coordinate short-lived concurrent work, and *node owners*, who are responsible for the node as an ongoing public resource. For clients, the default discipline is *containment-by-default*: spawned processes remain tied to the client session and are torn down when the session ends, preventing orphans and resource leaks. For owners, the corresponding power is *provisioning*: the ability to install node-resident predicates and start long-lived actor services meant to be shared and kept available.

In practice such services are often managed under supervision, but the generic supervision patterns that support this belong to Chapter 5. This role split will recur in concrete form when we discuss remote placement and the various code-shipping mechanisms via the `load_*` predicates.

The chapter is organised in five sections, moving from the concurrent foundation through progressively higher layers of abstraction. The concurrent Prolog Web (Section 4.1) extends the actor model of Chapters 2 and 3 across node boundaries: remote spawning, cross-node messaging, orphan prevention, and the unified semantics that treats local and remote communication as instances of the same model. With the concurrent substrate in place, we can already speak concretely about node-resident services: long-lived actors started by the node owner, published under stable names, and invoked by clients over the Prolog Web. The reusable engineering patterns that structure such services – servers, supervisors, and other generic behaviours – are introduced in the next chapter and implemented in Chapter 5.

The sequential Prolog Web (Section 4.2) then introduces a synchronous, non-deterministic abstraction built on top of the concurrent layer. The central construct, `rpc/2-3`, can be understood as a disciplined use of remote toplevel actors, though in practice it is implemented over stateless HTTP backed by cached toplevels operating in save-and-resume mode. The dependency is genuine: the sequential interface hides the actor machinery, but it is always, at bottom, built from it.

The programmable Prolog Web (Section 4.3) shows how code and data flow between client and node: clients may inject task-specific predicates into their own execution contexts, while node owners install predicates and service actors that define what the node offers.

A discussion of timing and failure boundaries (Section 4.4) makes explicit where waiting becomes observable and how timeouts attach to different interaction models. Finally, a section on security (Section 4.5) asks what makes it safe to let strangers run code on a node, and answers in terms of three structural mechanisms – inexpressibility, invisibility, and containment – together with the deployment concerns that complete the picture. Chapter 9 later returns to the architectural and paradigmatic questions that these technical sections raise.

4.1 The concurrent Prolog Web

As introduced above, all the mechanisms described in the preceding chapters – spawning, sending, linking, monitoring, registering – work transparently across node boundaries. This section makes that claim concrete by introducing remote placement via the `node` option and showing how actor-style interaction spans nodes. The focus is on *where* an actor runs and how it exchanges messages across the network, but also on the safety and authority questions that network-transparent concurrency raises on an open Web. Programmability mechanisms such as `load_*` options are deliberately postponed to Section 4.3.

4.1.1 Actor talking to remote actor

For specifying the node on which we wish to create an actor process, `spawn/3` can be passed the `node` option:

```
?- spawn(echo_actor, Pid, [
      node('http://n8.org'),
      monitor(true)
    ]),
   register(echo_actor, Pid).
Pid = 99231321@'http://n8.org'.
?-
```

Note that the `pid` is a compound term: an integer paired with a URI using the infix operator `@`. If `n8.org` is a node conforming to the ACTOR profile, and if the node's shared database defines the predicate `echo_actor/0`, then we can have a conversation with our remote echo server like so:

```
?- self(Self),
   echo_actor ! echo(Self, hello).
Self = 71773120.
?- flush.
Shell got echo(hello)
true.
?- whereis(echo_actor, Pid),
   exit(Pid, die).
Pid = 99231321@'http://n8.org'.
?- receive({A -> true}).
A = down(99231321@'http://n8.org',99231321@'http://n8.org',die).
?-
```

4.1.2 Actors playing ping-pong while on different nodes

The actors playing ping-pong back in Chapter 2 kept their game inside one node. Passing the option `node('http://n2.org')` to one of the calls to `spawn/3` allows the game to be played between two nodes:

```
ping_pong :-
  spawn(pong, Pong_Pid, [
    node('http://n2.org')
  ]),
  spawn(ping(3, Pong_Pid)).
```

Figure 4.1 illustrates this scenario.

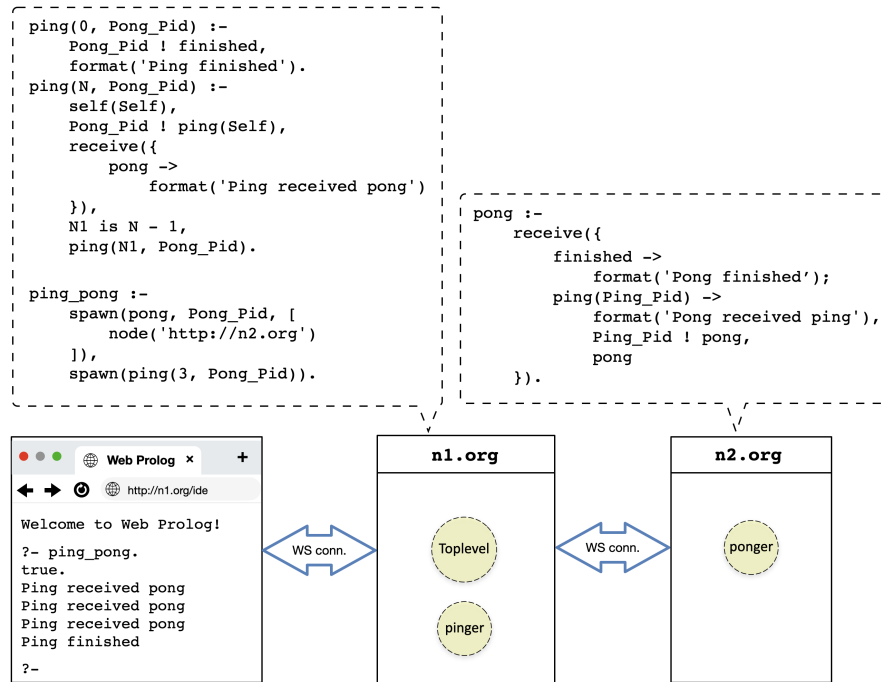


Fig. 4.1 Actors playing ping-pong while on different nodes.

But why do we get the output written by the “pinger” only, and not the output from the “ponger”? The asymmetry in the output is not an accident of the example but a consequence of a deliberate capability rule. A terminal is attached to one particular toplevel actor, and rendering output in that terminal is treated as an *I/O capability* that is scoped to the toplevel’s local process lineage. Concretely, only actors that (i) run on the same node as the toplevel to which the terminal is attached, and (ii) are local descendants of that toplevel, may write output that is forwarded to the client for display. In any case, readers may rest assured that the actors really *do* play ping-pong.

4.1.3 The node option works for toplevels too!

Toplevels are actors, so it should come as no surprise that the `node` option can also be used to create a toplevel process on a remote node:

```
?- toplevel_spawn(Pid, [
    node('http://n1.org')
]).
Pid = 83110912@'http://n1.org'.
```


?-

The query `?-human(Who)` executes on the remote node `n1.org`, with answers returned to the local shell:

```
?- toplevel_call($Pid, human(Who), [
    template(Who),
    limit(1)
]).
true.
?- receive({Msg -> true}).
Msg = success(83110912@'http://n1.org',[plato],true).
?- toplevel_next($Pid).
true.
?- receive({Msg -> true}).
Msg = success(83110912@'http://n1.org',[aristotle],false).
?-
```

Network transparency makes distributed concurrency easy to *use*. The harder question is how to make it *safe* on an open, failure-prone Web: how do we prevent orphan processes, constrain resource usage, and ensure that failure leads to reclamation rather than silent accumulation? The answer is largely about lifetime discipline, supervision, and the separation of authority between clients and node owners.

4.1.4 Node-resident actor processes

In addition to predicates in the shared database, the owner of a node may install *node-resident actor processes* – long-lived actors that are intended to be durable, shared, and continuously available. They are usually of a kind that we do not want unauthorised external clients to be able to create or destroy; doing this is the privilege of the node owner. This is also where supervision trees become most useful: crash recovery for node-resident services is part of the node owner's responsibility, and Chapter 5 introduces the supervisor behaviour that makes this responsibility tractable.

A node-resident count actor

We begin with a deliberately small example. Assume that the owner of the node `n8.org` starts a count actor, detaches it from the owner's own session, and publishes it under a stable name:

```
?- spawn(count_actor(0), Pid, [
    link(false)
]),
```

```

    register_service(counter, Pid).
Pid = 69220912.
?-

```

Two details distinguish this service from an ordinary client-created helper. The first is `link(false)`. Under the default lifetime discipline, the new actor would be tied to the life of the owner's shell and torn down when that session ends – exactly the right behaviour for client-created helpers, and exactly the wrong one for a service meant to outlive any particular session. Detaching an actor from the hierarchy that created it is precisely the privilege that Section 4.5.3 reserves for node owners; here we see what the privilege is *for*.

The second detail is `register_service/2`, a publication operation distinct from ordinary `register/2`; the distinction is developed in Section 4.1.4 below. Publication does not itself alter an actor's lifetime: `register_service/2` neither detaches nor supervises the actor. Lifetime management and service publication are separate owner responsibilities, and the example performs both explicitly.

A published name can be addressed from other nodes by qualifying it with the node's URI, just as pids are: where `69220912@'http://n8.org'` is the globally qualified pid of a process at `n8.org`, the term `counter@'http://n8.org'` denotes whatever process is currently published under the name `counter` there. A client on some other node can therefore interact with the service without ever learning its pid:

```

?- self(Self).
Self = 51230945.
?- counter@'http://n8.org' ! count($Self),
   receive({count(Count) -> true}).
Count = 1.
?- counter@'http://n8.org' ! count($Self),
   receive({count(Count) -> true}).
Count = 3.
?-

```

The example is intentionally simple, but it already shows an important shift in perspective. This actor is not a client-local helper process created for one temporary task; it is a node-resident process with a stable public name, and it therefore behaves like a shared stateful resource. The fact that the second reply is 3 rather than 2 is not an accident of presentation, but evidence that the actor's state is shared across interactions: another client has incremented the counter in the meantime.

A node-resident publish-subscribe service

The same service perspective becomes even clearer in a publish-subscribe example. Consider the following actor:

```

pubsub_actor(Subscribers0) :-
    receive({

```

```

publish(Msg) ->
    forall(member(Pid, Subscribers0), Pid ! msg(Msg)),
    pubsub_actor(Subscribers0);
subscribe(Pid) ->
    pubsub_actor([Pid|Subscribers0]);
unsubscribe(Pid) ->
    (
        select(Pid, Subscribers0, Subscribers)
    -> pubsub_actor(Subscribers)
    ;   pubsub_actor(Subscribers0)
    )
}).

```

This actor maintains a list of subscribers. A `subscribe(Pid)` message adds a subscriber, an `unsubscribe(Pid)` message removes one, and a `publish(Msg)` message broadcasts `Msg` to all currently subscribed processes. On its own, this is just another actor definition. It becomes a service when the owner starts it as a durable, detached, node-resident process and publishes it:

```

?- spawn(pubsub_actor([]), Pid, [
    link(false)
]),
    register_service(pubsub_service, Pid).

```

A client can then subscribe to the service and wait for messages from it:

```

?- self(Self),
    pubsub_service@'http://n8.org' ! subscribe(Self),
    repeat,
    writeln('Waiting for a message ...'),
    receive({
        msg(Message) ->
            format('Received: ~p~n', [Message]),
            fail
    }).

```

Waiting for a message ...

Received: hello

Waiting for a message ...

Note the quiet work that network transparency is doing here: the subscriber list held by the actor at `n8.org` contains *remote* pids, and `Pid ! msg(Msg)` delivers across node boundaries with the same syntax as local delivery. The message `hello` was received because some client published it to the same named actor:

```

?- pubsub_service@'http://n8.org' ! publish(hello).
true.
?-

```

This example illustrates a useful contrast with the counter. The counter service exposes shared *state*: what one client observes depends on what other clients have already done. The publish-subscribe service instead exposes shared *coordination*: clients interact indirectly through a common mediating process. In both cases the essential pattern is the same – a node-resident actor with a stable public name behaves as a service.

The implementation shown here is intentionally minimal. In practice, one would monitor subscribers so that dead ones are pruned from the list, place the service itself under supervision, possibly persist subscriptions in a more durable store, and perhaps attach access control or topic structure.

Naming, discovery, and service publication

The preceding examples published their services with `register_service/2` rather than the ordinary `register/2`, and the distinction deserves comment. Ordinary registration is a convenience for private or client-local actors: a client may use `register/2`, `whereis/2`, and `unregister/1` for actors that it has itself created and named. Service publication is a different act, exercising different authority: the node owner places a name into the node’s public service namespace using the dedicated operations `register_service/2` and `unregister_service/1`, which clients do not hold.

Clients may, subject to the node’s authorisation policy, send to a published service name, but they do not thereby gain the ability to discover the service’s pid, enumerate what else is running on the node, or remove the published name. This raises the practical question: how do clients discover which services a node makes available?

A qualified address `Name@Node` is resolved against the public service registry of `Node`. Ordinary names installed with `register/2` are not considered during this lookup, even if the same atom occurs in both namespaces; conversely, `whereis/2` does not reveal the pid behind a published service name.

The standard design principle is to separate a stable service identifier from the process that currently implements it (van Steen and Tanenbaum, 2023). The Prolog Web inherits this distinction within a node: the published address `Name@Node` does not bind clients to a particular pid. The implementing actor may be restarted or replaced while the published name remains stable.

The node URI remains part of the service’s address. Moving a service to another node therefore requires an additional mechanism – for example, an owner-maintained directory, forwarding entry, or stable front-end URI. The publication mechanism described here provides process-identity independence within the publishing node; it does not by itself provide a global location-independent naming system.

Node discovery operates at a different scope from service publication within a node. A node directory may advertise which public nodes are known, whether they have recently been reachable, which profiles they claim, and which predicates or services they provide. The proof-of-concept Discovery Hub shown in Figure 3.9 implements this pattern as a queryable relation hosted by node `n0`.

The hub should not be understood as a mandatory central authority for the Prolog Web. It is one possible directory implementation: other hubs may coexist, participation may be voluntary, and applications may use owner-maintained peer information or other discovery mechanisms instead. Moreover, a reported liveness status records a recent observation; it does not guarantee that a node will remain reachable.

Service discovery is an owner-curated form of advertisement, distinct from publication itself. A call to `register_service/2` makes a name addressable in the public service namespace; it does not expose a client-queryable enumeration of that namespace. A node may instead advertise selected services through owner-maintained metadata, while leaving other published services unadvertised.

This separation is a design invariant. If clients could enumerate the service registry or all running processes, discovery would collapse into introspection and name discipline would be lost. Ordinary client naming operations must likewise neither expose nor remove published services. The security implications are developed in Section 4.5.2.

In the Trinity, such an advertisement can be expressed as an ordinary relation in the node's shared database. For example:

```
service(counter, meta(actor, protocol(count_v1))).
service(pubsub_service, meta(actor, protocol(pubsub_v1))).
```

The relation is maintained by the owner and may advertise all published services or only a selected subset. The owner is responsible for keeping its entries consistent with the services actually published. Richer metadata – versioning, authorisation requirements, or documentation URIs – can be represented by extending the second argument.

A client discovers advertised services with an ordinary goal-oriented query:

```
?- rpc('http://n8.org', service(Name, Meta)).
Name = counter,
Meta = meta(actor, protocol(count_v1)) ;
Name = pubsub_service,
Meta = meta(actor, protocol(pubsub_v1)).
?-
```

Discovery therefore remains declarative and compositional: clients can enumerate advertised services, filter by protocol, or combine discovery with other constraints. Because `service/2` is owner-controlled metadata rather than an enumeration of the service registry, the node reveals only what it has deliberately chosen to advertise and may apply ordinary authorisation policy to the query.

4.1.5 Uniform messaging across node boundaries

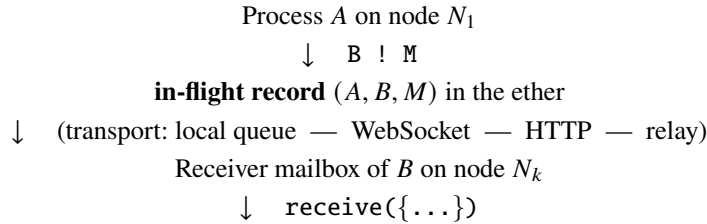
The preceding examples show that the same primitives – spawning, sending, linking, monitoring, registering – work whether the target process is local or remote. This

uniformity is not accidental; it rests on a deliberate design choice inherited from the unified semantics proposed by Svensson et al. (2010) for Erlang-like systems. The key idea is to eliminate the semantic distinction between local and remote communication: all side-effecting operations are mediated by a node controller abstraction, so that placement becomes a configuration detail rather than a semantic one. Chapter 9 returns to the deeper implications of this distribution-first stance; here we spell out the messaging model that makes it concrete.

Two-step messaging model

Following Svensson et al. (Svensson et al., 2010)’s *ether* abstraction, we model messaging as a two-step mechanism: *sending* places an in-flight record into a virtual system queue (the *ether*), and a later *delivery* step appends the message to the receiver’s mailbox. This separation allows us to define program meaning independently of transport.

The in-flight layer is a semantic device rather than an implementation requirement. Locally, a runtime may realize sending by inserting the message directly into the receiver’s mailbox, which can be seen as choosing a schedule in which each send is immediately followed by its delivery. In Web Prolog, the in-flight phase may be implemented using multiple transports, as illustrated below.



The transport realizations in Web Prolog are as follows.

- **Local delivery:** the runtime enqueues (A, B, M) in an internal scheduler structure and later appends M to B ’s mailbox.
- **Direct remote delivery:** (A, B, M) is serialized and shipped from N_1 to N_k as a WebSocket frame and then enqueued for mailbox delivery.
- **Request-based delivery:** (A, B, M) is carried in an HTTP request to N_k (or to a node gateway), which authenticates, deserializes, and enqueues the message.
- **Relay-mediated delivery:** (A, B, M) is sent to an intermediary that forwards it to N_k , after which normal mailbox delivery occurs.

This separation lets us specify one uniform concurrency model while allowing multiple deployment choices. Ordering guarantees are stated over *delivery* into mailboxes: per sender-receiver FIFO when sender and receiver are addressed without mixing addressing modes, with no global order across different senders. Liveness properties depend on the fairness of the delivery schedule rather than on the specifics of the network path.

4.2 The sequential Prolog Web

The *sequential Prolog Web* is the fragment of the Prolog Web in which every client interaction can be explained by ordinary Prolog-style backtracking. More precisely, a node belongs to the sequential Prolog Web if, for every client session, the sequence of answer substitutions returned for each goal is exactly the sequence that a single-threaded SLD-style execution would produce. No reasoning about interleavings, mailboxes, or concurrent actors is needed to understand what happens – the entire observable behaviour admits explanation by a single resumable logical history.

The *concurrent* partition is fundamentally different: it admits multiple simultaneous causal histories, requires reasoning about interleavings, and cannot in general be reduced to a single sequential trace. Yet the two partitions are not independent. The sequential layer of the Prolog Web is built directly on top of the concurrent one, and understanding that dependency is the key to understanding why the sequential interface looks and behaves as it does.

The full layering, from bottom to top, is as follows:

1. Actor processes and asynchronous messaging.
2. Toplevel actors, which add goal evaluation and answer-stream management to the basic actor model.
3. `rpc/2-3` over a remote toplevel – the conceptual implementation that makes the synchronous, nondeterministic interface explicit.
4. `rpc/2-3` over stateless HTTP – the practical implementation, backed by cached toplevels operating in save-and-resume mode, which realizes the same interface more efficiently.

Items 3 and 4 expose the same interface; they differ in realization, not in semantics. Each step in the stack thins the interface and hides more of the underlying machinery.

Because the concurrent layer has already been established, we can now present `rpc/2-3` and its companions `promise/3-4` and `yield/2-3` with the reader already understanding what lies beneath. Not every distributed program needs explicit concurrency – often a caller simply wants to ask another node to solve a goal and wait for answers – but the sequential interface is always, at bottom, a disciplined use of the concurrent one.

4.2.1 Nondeterministic remote procedure calls

In Web Prolog, `rpc/2-3` is a high-level meta-predicate for making *nondeterministic remote procedure calls*. It allows a process running on a node N_1 to call and attempt to solve a goal in the Prolog context of another node N_2 , taking advantage of the programs and data offered by N_2 as if they were local to N_1 . Such calls are synchronous: the caller blocks while waiting for answers. In return, `rpc/2-3` is remarkably easy to use for distributing programs across two or more nodes. It can

therefore be seen as a construct for distributed programming that trades concurrency for ease of programming.

A Web Prolog client performs a remote call by providing a URI identifying the node and a goal to be solved in that node's context. The optional third argument is a list of options. Here is a simple example:

```
?- rpc('http://n1.org', human(Who)).
Who = plato ;
Who = aristotle.
?-
```

Note that solutions are returned on demand and in the familiar one-solution-at-a-time fashion: variable bindings appear as instantiations of variables occurring in the goal.

Using `rpc/3` with the `limit` option

The `rpc/3` predicate supports a `limit` option, inherited from the underlying `toplevel_call/3` predicate. Passing `limit(K)` instructs the remote node to compute at most *K* solutions per network roundtrip. The default is `infinite`, meaning all solutions are fetched in a single exchange. Here is what happens if we override that default with `limit(1)`:

```
?- rpc('http://n1.org', human(Who), [
    limit(1)
]).
Who = plato ;
Who = aristotle.
?-
```

From the caller's perspective, the result is identical. What changed is invisible: instead of retrieving both solutions in a single roundtrip, the call now required *two* – one per solution. In this case, the option adds cost without benefit. Later, we shall see contexts in which limiting the number of solutions per roundtrip matters in a genuinely positive way.

In addition to the `limit` option, `rpc/3` inherits the `load_*` options from `spawn/3` and the `timeout` and `once` options from the `toplevel` mechanism. Examples using `load_*` options will be given in Section 4.3.1. See Appendix A for details about the other options.

The semantics of `rpc/2-3`

For intuition it is useful to note that a call to `rpc(URI, Goal)` is semantically equivalent to the following pattern, which makes explicit that solutions arrive as instantiations of a copy that is then unified back into the original goal:


```
copy_term(Goal, Copy),
call(Copy),           % executed on node at URI
Goal = Copy.
```

The key implication is that no variables are shared across nodes: the remote execution operates on a renamed-apart copy, and only the induced substitution is returned to the caller via unification with the original goal.

This three-line characterisation is intended as a *logical* specification of `rpc/2-3`: it defines what answers the caller may observe, not how those answers are obtained. As a specification it is precise and complete for the intended domain. Web Prolog restricts remote goals to pure Herbrand terms – attributed variables are not supported, and the serialisation protocol ensures that bindings returned to the caller are always meaningful in the caller’s context, ruling out node-local references such as stream handles or database pointers. What the characterisation deliberately abstracts over is the operational machinery required to deliver those answers across a network: the creation of a remote process, the protocol that sustains nondeterminism across multiple solutions, and the handling of timeouts, exceptions, and partial failure. These concerns are addressed by actor infrastructure described in the sections that follow.

A notable property of `rpc/2-3` is that it preserves the logical purity of the goal it calls: if the goal is pure, then the answers returned by `rpc/2-3` are exactly the logical consequences of the remote database – the network introduces no additional bindings, side effects, or observable state changes.¹ This holds at the level of answer substitutions, assuming successful communication; in practice, timeouts, connection failures, and resource limits may prevent all answers from being delivered, but they do not corrupt the answers that are delivered. Later, we return to this observation when discussing the idea of a *pure* Prolog Web.

4.2.2 Implementing `rpc/2-3` on top of a toplevel actor

The PTCP makes it possible to reconstruct the central construct of the sequential layer, `rpc/2-3`, on top of a toplevel actor: spawn a toplevel on the target node, submit a goal, and collect answer messages in a loop. This is a pedagogical reconstruction rather than the implementation provided by the system, which implements `rpc/2-3` over the stateless HTTP API described below. The reconstruction is nevertheless useful because it makes explicit how a sequence of deterministic protocol exchanges can be presented to the caller as ordinary Prolog nondeterminism:

```
rpc(URI, Goal) :-
    rpc(URI, Goal, []).
```

```
rpc(URI, Goal, Options) :-
```

¹ This is a property it shares with the `call/N` family of Prolog predicates.

```

    toplevel_spawn(Pid, [
        node(URI),
        session(false),
        monitor(false)
    | Options
    ]),
    toplevel_call(Pid, Goal, Options),
    wait_answer(Pid, Goal).

```

From the caller's perspective, the interaction is synchronous and nondeterministic: variables are bound on success, and backtracking yields the next solution. The remote toplevel is spawned with `session(false)`, so it terminates when the query is exhausted or explicitly stopped.

On the ordinary protocol outcomes, the answer loop terminates on failure or error, or after the final slice has been consumed:

```

wait_answer(Pid, Goal) :-
    receive({
        success(Pid, Slice, true) ->
            ( member(Goal, Slice)
              ; toplevel_next(Pid),
                wait_answer(Pid, Goal)
            ) ;
        success(Pid, Slice, false) ->
            member(Goal, Slice) ;
        failure(Pid) ->
            fail ;
        error(Pid, Error) ->
            throw(Error)
    }).

```

The nondeterministic behaviour arises from two sources. A `success/3` answer contains a slice of solutions in its second argument. The call `member(Goal, Slice)` enumerates this slice without further network communication. When a non-final slice has been exhausted, execution falls through to the second disjunct, which requests another slice using `toplevel_next/1`. In this way, deterministic message exchanges with the remote toplevel are converted into a nondeterministic, on-demand interface at the caller.

The sketch deliberately omits failure detection and lifecycle management. It does not monitor the toplevel or bound the receive operation, so a node failure or network partition may leave the caller blocked indefinitely. There is also an abandonment case. If an answer reports that further solutions are available and the caller then prunes the `rpc/2-3` choice point using a cut or `once/1`, the discarded local continuation never sends `toplevel_stop/1`. The remote actor may consequently remain suspended in state `s3`, retaining the search state from which another slice could have been produced. The option `session(false)` does not detect such abandonment;

it causes termination only after exhaustion or an explicit stop. A production actor-based implementation would monitor the toplevel, bound each blocking operation, and associate the actor's lifetime with that of the nondeterministic call.

Although actors and messages provide the underlying mechanism, the caller and callee proceed in request–response lockstep: the caller waits while the remote goal is evaluated and resumes only when an answer arrives. Operationally, therefore, this is a synchronous and sequential pattern. The same structure also makes it possible to implement `rpc/2–3` over stateless HTTP.

4.2.3 Implementing `rpc/2–3` on top of the stateless HTTP API

The actor reconstruction reveals a strict request–response alternation: the caller submits a query, receives a slice of answers, and requests another slice only when local backtracking demands it. There are no unsolicited replies, server-initiated notifications, or concurrent answer streams. This traffic pattern maps naturally onto the sequential layer's stateless HTTP interface, allowing `rpc/2–3` to be implemented without requiring the remote node to expose its programmable actor layer.

Each exchange is a self-contained HTTP request–response transaction. This avoids a client-addressed remote session and reduces the risk of leaving an orphan query behind. It also works between any two ISOBASE-conforming nodes, passes through ordinary HTTP infrastructure, and can be inspected using standard HTTP tooling. Blocking can be bounded by a timeout on each request, although a timeout does not by itself establish whether the remote evaluation completed. For the synchronous, slice-at-a-time interaction required by `rpc/2–3`, stateless HTTP is therefore a natural fit.

Statelessness at the protocol boundary does not imply that the server retains no execution state. When save-and-resume caching is enabled, the node may keep a suspended toplevel so that a later request for another offset can resume the search rather than repeat it. If the caller abandons enumeration, no such request arrives, and the cached computation may remain unused. This state is not a client-addressed session but a disposable optimisation: it may be reused by another matching request or evicted without changing the protocol semantics. A production node must therefore bound the cache and terminate cached toplevels on eviction, expiry, or resource pressure. Without such reclamation, caching can itself become a source of resource leaks.

Compared with the actor-based reconstruction, the HTTP version is more involved. The additional complexity lies not in the control structure, which is shared by both versions, but in transport concerns: serialising goals and templates, encoding terms correctly, and handling authentication, redirects, TLS settings, and timeouts:

```
rpc(URI, Goal) :-
    rpc(URI, Goal, []).
```

```
rpc(URI, Goal, Options) :-
```

```

    parse_url(URI, Parts),
    term_variables(Goal, Vars),
    Template =.. [v|Vars],
    format(atom(GoalAtom), "~p", [Goal]),
    format(atom(TemplateAtom), "~p", [Template]),
    option(limit(Limit), Options, 100000000000),
    rpc(Template, 0, Limit, GoalAtom, TemplateAtom, Parts,
        Options).

rpc(Template, Offset, Limit, GoalAtom, TemplateAtom, Parts,
    Options) :-
    parse_url(ExpandedURI, [
        path('/call'),
        search([goal=GoalAtom, template=TemplateAtom,
            offset=Offset, limit=Limit, format=prolog])
    | Parts
    ]),
    setup_call_cleanup(
        http_open(ExpandedURI, Stream, Options),
        read(Stream, Answer),
        close(Stream)
    ),
    rpc(Answer, Template, Offset, Limit, GoalAtom,
        TemplateAtom, Parts, Options).

rpc(success(Slice, true), Template, Offset, Limit, GoalAtom,
    TemplateAtom, Parts, Options) :- !,
    ( member(Template, Slice)
    ; NewOffset is Offset + Limit,
      rpc(Template, NewOffset, Limit, GoalAtom, TemplateAtom,
          Parts, Options)
    ).

rpc(success(Slice, false), Template, _, _, _, _, _) :-
    member(Template, Slice).
rpc(failure, _, _, _, _, _, _) :-
    fail.
rpc(error(Error), _, _, _, _, _, _) :-
    throw(Error).

```

This implementation is provided in the transparent proof-of-concept implementation.

4.2.4 Browser talking to a node talking to a node

In the scenario depicted in Figure 4.2, a user of a Prolog terminal running in a browser interacts with a remote node on the Prolog Web, which in turn interacts with another node.

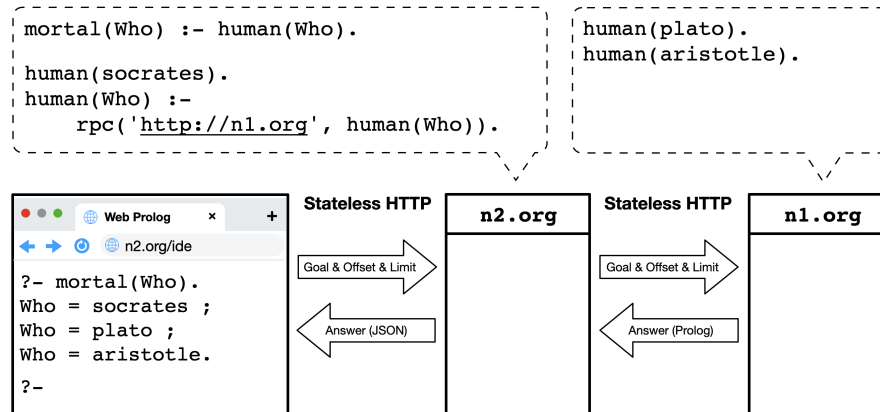


Fig. 4.2 Stateless interaction in a scenario involving a web browser and two remote nodes.

The solutions shown in the terminal follow from the *combination* of two programs: the code on the upper left in Figure 4.2 belongs to n2.org, and it contains a URI pointing to the code offered by n1.org. The browser-to-node interaction as well as the node-to-node interaction take place over HTTP and are stateless in the sense that each request can be understood and processed without relying on server-side session state.

To retrieve the first solution to `?-mortal(Who)`, the browser issues a request to n2.org asking for exactly one solution:

```
GET http://n2.org/call?goal=mortal(Who)&limit=1
```

Since the user requests solutions one-by-one, the response contains a single binding together with a flag indicating that more solutions exist:

```
{ "type": "success",
  "data": [{"Who":"socrates"}],
  "more": true }
```

When the user types a semicolon, the browser requests the next solution. In this example, producing that next solution requires n2.org to consult n1.org. Concretely, n2.org evaluates `mortal/1` and, as part of doing so, triggers an `rpc/2-3` call for `human(Who)` against n1.org. Operationally, this involves an HTTP request to n1.org such as:

```
GET http://n1.org/call?goal=human(Who)&format=prolog
```

Because of the default values of the corresponding slicing parameters, the response in this case contains a complete slice with all solutions to `human(Who)` currently returned in one roundtrip. Since the URI specified `format=prolog`, this was the result:

```
success([human(plato),human(aristotle)], false)
```

At this point the implementation of `rpc/2-3` on `n2.org` must convert the returned slice into the intended nondeterministic behaviour. The usual mechanism is to keep the slice locally and enumerate it using `member/2`. This yields solutions to `human(Who)` on backtracking without further network communication, until the slice is exhausted.

Since the browser client still wants only one solution at a time, `n2.org` returns only the first derived solution, `Who=plato`, to the browser:

```
{ "type": "success",
  "data": [{"Who":"plato"}],
  "more": true }
```

When the user again types a semicolon, the browser requests another solution to `mortal(Who)`. In this particular run, no additional request to `n1.org` is needed: `n2.org` can simply backtrack within the local `member/2` enumeration of the slice already obtained from `n1.org`. The browser request therefore looks like:

```
GET http://n2.org/call?goal=mortal(Who)&offset=2&limit=1
```

and the response contains the next binding, `Who=aristotle`, now with `more:false`:

```
{ "type": "success",
  "data": [{"Who":"aristotle"}],
  "more": false }
```

This completes the description of what happens “under the hood” in the scenario of Figure 4.2: the browser drives one-solution-at-a-time enumeration through stateless HTTP requests to `n2.org`, while `n2.org` uses `rpc/2-3` to consult `n1.org` and reconstructs nondeterminism locally from slices of answers.

Large result sets and the `limit` option

If `n1.org` had millions of clauses of `human/1`, returning a huge slice in one response would be undesirable. In that case `n2.org` can pass a smaller `limit` to `rpc/2-3` to retrieve answers in manageable batches:

```
human(Who) :-
    rpc('http://n1.org', human(Who), [
        limit(1000)
    ]).
```

This avoids swamping the caller with data. A goal with very many solutions will be delivered incrementally over multiple roundtrips, but since `rpc/2-3` is non-deterministic, the call remains logically complete provided the caller keeps asking for more.

As this scenario involves only two nodes, the Prolog Web here is tiny. Later we argue that the same principles can, in principle, scale to arbitrarily many nodes. Key is the combination of statelessness (for robustness and simplicity) with an execution model that avoids shipping excessive data and avoids recomputing answers unnecessarily.

4.2.5 Backtracking as pagination

We now step back from implementation to draw out a correspondence that the rest of the book will build on.

The `offset` and `limit` parameters in the stateless HTTP API may look like a polite concession to web conventions – a way to make Prolog’s multiple solutions fit a request–response protocol. But the connection is deeper than that. Prolog backtracking and HTTP pagination instantiate the same abstract idea: incremental enumeration under external control.

A paginated endpoint does not “return the result set”; it exposes a procedure for generating results and a protocol for consuming them in slices. The server produces a page, then stops. A later request asks for the next page, which amounts to resuming enumeration at a recorded position. Prolog behaves in the same general way. A goal may have many solutions; the toplevel presents the next one (or the next batch) and then waits. Internally, evaluation proceeds by creating choice points, and enumeration can be resumed from a well-defined position. In an interactive toplevel, that position is represented by retained search state, and the user’s semicolon is simply the “resume” signal.

Offset surface, cursor semantics

Web API pagination comes in two principal forms. In *offset pagination*, the client says “skip N , return the next k ”; the server must, at least logically, regenerate the first N results and discard them. In *cursor pagination*, the server hands the client an opaque token that encodes where to resume; the client returns the token to request the next page, and the server picks up where it left off. The token is meaningless to the client – it is a handle to internal state, not a numeric position in a pre-existing result set.

Prolog backtracking is structurally cursor-based. A choice point is opaque retained state that captures exactly where to resume the search: variable bindings, unexplored branches, the position in the clause index. The semicolon hands it back and says “continue.” No re-enumeration from the top occurs; no solutions are skipped and discarded.

On the wire, however, the stateless HTTP API speaks `offset` and `limit` because that is idiomatic REST:

```
GET /call?goal=hotel(Name,paris,Rating)&offset=20&limit=10
```

This returns solutions 21–30. But the implementation need not treat the offset as an instruction to re-enumerate from the beginning and discard. A node can cache a suspended toplevel process keyed by goal and offset, so that a request for page $k+1$ resumes the process from its suspended choice point rather than replaying the search. The offset then functions not as a distance from the start, but as a lookup key into retained state – in effect, a cursor in offset clothing.

Consistency and the update view

The distinction between true offset re-execution and cached-process resumption is not merely one of cost; it has semantic consequences. True offset pagination – re-executing the goal fresh for each page – gives the client a sliding window over whatever the database contains at the moment of each request. If facts are asserted or retracted between pages, solutions may be duplicated, skipped, or reordered. Cursor-based resumption, by contrast, continues a single in-flight enumeration whose view of the database is fixed once, at the point the enumeration began.

In the Prolog literature, this is the distinction between the *logical update view* and the *immediate update view*. Under the logical update view – which the ISO Prolog standard mandates – a goal sees the set of clauses that existed when it was called, regardless of later modifications. A cached toplevel process therefore provides snapshot-consistent pagination *for the lifetime of that process*: so long as the same suspended computation is resumed for each successive page, every page continues a coherent enumeration even as the shared database evolves underneath it. The owner may assert and retract freely; the in-flight scan does not see those changes, and the client receives a consistent sequence.

This guarantee is real but conditional, and it is important to be precise about the condition. It holds exactly as long as the cached process survives and is resumed. A node is free to evict cached computations – under memory pressure, after a timeout, or for any other reason (Section 3.6.1). If a page request arrives after the relevant process has been evicted, the node has no suspended enumeration to resume and must fall back to restart-and-skip against the *current* database. At that point the snapshot is lost: the remainder of the scan is computed against a possibly-changed database, and the cross-page guarantees degrade to those of the naïve case described above – duplicates, gaps, or a stale completion flag become possible. Eviction therefore affects not only the *cost* of subsequent pages but, for a scan that races a concurrent mutation, their *meaning*.

Two consequences follow. First, the portable, unconditional guarantee a client may rely upon across *any* conformant node is the weaker one stated in Chapter 3 (Section 3.3.5): paging by position is well defined for *stably enumerable* goals – goals over data the client cannot modify – and drift caused by owner-side mutation

mid-scan is possible and is not, in the base design, reported to the client. Snapshot consistency via the update view is a stronger property that a node *may* provide for the lifetime of a cached process, not one the protocol contract obliges it to sustain across evictions. Second, a node that wishes to offer snapshot consistency as a dependable guarantee – rather than a best-effort consequence of caching – must either decline to evict scans that are still in progress, or detect the eviction and signal it to the client rather than silently re-deriving. The mechanisms for such detection are discussed as future work in Chapter 11.

A Prolog system that provides the logical update view thus delivers, essentially for free and for as long as it retains the enumerating process, the kind of stable pagination that web backends often must engineer with care. The qualification “for as long as it retains the process” is precisely what separates a convenience from a contract.

Three points on a spectrum

The relationship between Prolog’s backtracking mechanism and HTTP pagination can be understood as a spectrum of three API styles, each exposing more of the underlying process model.

1. **Stateless offset, naïve implementation.** Each request re-executes the goal against the current database, skips N solutions, and returns the next batch. This is true offset pagination: cheap on server memory, idempotent by construction, but vulnerable to inconsistency if the database changes between pages.
2. **Stateless offset, cached processes.** The wire protocol still says `offset/limit`, but the server caches a suspended toplevel process keyed by goal and offset. Offset on the surface, cursor under the hood. Under the logical update view, this gives snapshot-consistent pagination *for as long as the cached process is retained*; if the process is evicted mid-scan, a subsequent page falls back to case (1), and to its weaker consistency, against the current database.
3. **Semi-stateful API with process identifiers.** The server hands back an explicit process identifier, and the client uses it to request continuation. There is no offset at all. This is cursor pagination without disguise, and the closest structural match to interactive backtracking: the pid *is* the choice-point handle, and “next” *is* the semicolon.

The three styles differ in what the client’s handle refers to. In (1), there is no handle; each request reconstructs everything from scratch. In (2), the offset is an implicit handle to a suspended search tree – choice points and bindings against the shared database. In (3), the pid is an explicit handle to a full process: search tree, variable bindings, *and the process’s private database*, including any facts it has asserted locally or code loaded via `load_*` options. Successive “next” requests do not just advance enumeration; they resume a process that may have asserted local facts, updated counters, or built up intermediate structures as part of its computation. This is closer to a database session with a temporary table than to a mere cursor.

When the stateless API grows stateful

The clean separation between stateless offset pagination and stateful pid-based resumption blurs once the stateless API accepts `load_*` options. A request such as

```
GET /call?goal=solve(X)&load_uri=http://n5.org/planner&limit=5
```

causes the node to spin up a process that loads the specified source into its private database before executing the goal. That process now carries local state – the loaded clauses – not just a position in a search tree over the shared database.

If the node caches this process for subsequent page requests, the cached handle points to a process with a populated private database: essentially the same kind of entity that a pid refers to in the semi-stateful API. The only difference is how the client addresses it – an offset key derived from the goal and its options, versus an explicit pid.

If the node does *not* cache – naïve re-execution – then each page request reloads the code, re-executes from scratch, and skips. This is not merely inefficient for enumeration; it re-runs the entire setup for every page. And if the loaded source has changed between requests, different pages may be enumerating over different programs entirely.

The conclusion is that even the API that looks most RESTful is, in the presence of `load_*` options, implicitly creating processes with private state. The question is only whether that state is retained across pages (cached) or reconstructed (naïve), and whether the client gets an opaque handle to it (pid) or addresses it indirectly (goal + options + offset as a composite key). The semi-stateful API simply makes this honest.

Caching and the Web

The web-caching story is cleanest at one specific point on the spectrum, and it is worth being precise about which. Style (1) – stateless offset with naïve re-execution and no `load_*` options – inherits the Web’s caching infrastructure directly. A request for `?goal=mortal(X)&limit=10` is a plain URI with a response determined entirely by the request and the current database. CDNs, reverse proxies, and browser caches can store and serve it without modification. Load balancers can route requests to any backend, since there is no server-side session to preserve. Retry is safe, because the request is idempotent. None of this requires any special support from Web Prolog; it falls out of the decision to expose backtracking as stateless pagination over HTTP.

These properties are exactly what styles (2) and (3) trade away. A cached process lives on one particular backend and embodies a snapshot taken at a particular moment, so the same offset URI is no longer backend-independent or idempotent: a retry routed to a different backend finds no suspended process and restarts; a repeated request after eviction may return a different page. The progression along the spectrum is therefore also a progression in caching character – from a fully cacheable, backend-independent, idempotent request, to a request whose correct interpretation

is pinned to a specific process on a specific node. This is not a defect but a trade: style (1) offers cacheability and gives up cross-page snapshot consistency; styles (2) and (3) offer consistency and richer state, and give up transparent web caching. The surface-level HTTP compatibility is preserved throughout – every style still speaks ordinary requests and responses – but only style (1) is cacheable in the strong sense the Web’s intermediaries assume.

4.2.6 Programming with `rpc/2-3`

Because `rpc/2-3` is a simple, synchronous, nondeterministic interface, it is easy to build small programming schemes on top of it. We sketch a few patterns here.

The `parallel/1` predicate

The first and simplest pattern is a meta-predicate `parallel/1` that takes a list of goals and attempts to solve them concurrently rather than one after another. It is most useful when the goals are independent and each spends significant time waiting – as remote calls typically do – so that overlapping them shortens the total elapsed time. We state its intended behaviour as a small set of requirements. A call to `parallel(Goals)` should

1. block until the outcome is determined, and no longer than necessary,
2. succeed, with variable bindings, if all goals succeed,
3. fail as quickly as possible if any goal fails, and
4. rethrow any errors thrown by a goal, also as quickly as possible.

In other words, running a list of goals in parallel should behave in the same way as when running them sequentially, but potentially complete sooner. For this to work properly, we must require something about the input too, namely that goals in the list are independent, that is, they must not communicate using shared variables or by any other means. From the caller’s perspective, `parallel/1` is a Prolog-style behavior: a pure-looking meta-predicate whose implementation happens to orchestrate actors.

Crucially, `parallel/1` does not introduce a new observable interaction pattern. Provided the listed goals are independent in the sense just described, the call `parallel([G1, . . . , Gn])` is extensionally equivalent to sequential conjunction for deterministic goals.

That is, it succeeds with exactly the same variable bindings, fails under the same conditions, and propagates the same errors. The only intended difference is cost. The qualification *for deterministic goals* is essential: `parallel/1` is semi-deterministic, committing to the first solution of each goal rather than enumerating combinations on backtracking, so for nondeterministic goals it is a genuine restriction of sequential conjunction rather than merely a faster version of it. In this sense, `parallel/1` is an optimisation of sequential conjunction rather than a semantic extension. Its internal

use of actors and asynchronous messaging remains observationally invisible, and it therefore remains within the sequential partition of the Prolog Web.

Making RPC calls in parallel

Two RPC calls in sequence take the running time of the first call plus the running time of the second:

```
?- rpc('http://n1.org', foo(X)),
   rpc('http://n2.org', bar(Y)).
```

If the caller node has ACTOR capabilities, we may combine RPC calls with `parallel/1`:

```
?- parallel([
    rpc('http://n1.org', foo(X)),
    rpc('http://n2.org', bar(Y))
]).
```

The time spent waiting is then approximately the maximum of the two response times. Note, however, that this is still a synchronous pattern: `parallel/1` blocks until both calls have returned. It therefore differs from actor-style interaction, where sends are asynchronous by default.

In its basic form, `parallel/1` commits to the first solution of each goal; it does not explore combinations by backtracking, so if `foo/1` or `bar/1` are nondeterministic, the above returns only one solution from each call. To aggregate lists of solutions, use `findall/3`:

```
?- parallel([
    rpc('http://n1.org', findall(X, foo(X), Xs)),
    rpc('http://n2.org', findall(Y, bar(Y), Ys))
]),
append(Xs, Ys, Solutions).
```

Remote modules

Below we sketch an implementation of *remote modules*:²

```
use_remote_module(URI, ImportList, Options) :-
    maplist(import(URI, Options), ImportList).

import(URI, Options, Functor1/Arity as Functor2) :- !,
    functor(Head1, Functor1, Arity),
    Head1 =.. [Functor1|Args],
```

² What we here call *remote modules* is similar in spirit to Ciao Prolog's *active modules*.

```

    Head2 =.. [Functor2|Args],
    assertz((Head2 :- rpc(URI, Head1, Options))).
import(URI, Options, Functor/Arity) :-
    functor(Head, Functor, Arity),
    assertz((Head :- rpc(URI, Head, Options))).

```

Thanks to `load_*` predicates, an import directive can define local wrappers even for predicates not directly exported by the remote node, provided they can be expressed in terms of predicates that are exported.

Global goals

Our second example takes a step towards even more network transparency and is inspired by (Loke, 2006). The idea is to support *global goals* of the form `?- *Goal`, where a local peer database maps goal patterns to peers and options:

```

peer(foo(_), 'http://n1.org', [limit(10)]).
peer(bar(_), 'http://n2.org', [timeout(1),limit(100)]).
peer(bar(_), 'http://n3.org', [timeout(3),limit(100)]).
peer(baz(_), 'http://n3.org', [timeout(1),limit(50)]).

```

Given such a database,³ `*/1` can be implemented like so:

```

:- op(100, fx, *).

* Goal :-
    peer(Goal, Peer, Options),
    catch(rpc(Peer, Goal, Options), time_limit_exceeded, false).

```

A registry-based variation can be obtained by retrieving `peer/3` information remotely:

```

* Goal :-
    rpc('http://registry.pl', peer(Goal, Peer, Options)),
    catch(rpc(Peer, Goal, Options), time_limit_exceeded, false).

```

4.2.7 Promise and yield over HTTP

The `rpc/2-3` predicate is synchronous: the caller waits until the remote node returns an answer. For many applications this is fine, but sometimes a client wants to initiate a remote computation, continue with other work, and collect the result later. The `promise/3-4` and `yield/2-3` predicates support this split-phase pattern.

A call to `promise/3-4` sends a query to a remote node but returns immediately, handing back a reference that identifies the pending computation:

³ We make no assumptions on how the peer database is updated.

```
?- promise('http://n1.org', human(Who), Ref).
Ref = 19303011.
?-
```

The remote node begins evaluating `human(Who)`. Meanwhile, the caller is free to do other work. When it's ready for the answer, it calls `yield/2-3` with the reference:

```
?- yield($Ref, Answer).
Answer = success([human(plato), human(aristotle)], false).
?-
```

If the answer has already arrived, `yield/2-3` returns immediately. If not, it waits. The answer is delivered to the caller's mailbox, so `yield/2-3` must be called by the same process that issued `promise/3-4`.

The familiar options carry over: `template/1` controls which bindings are returned, while `offset/1` and `limit/1` control slicing. By default, `promise/3-4` requests all solutions; with `limit(K)`, the answer contains at most K solutions, allowing large result sets to be fetched in batches. It is often useful to think of these batches as *pages* of an answer stream: `limit(K)` sets a page size, and `offset` selects which page to retrieve next. In Section 4.2.5 we will return to this idea and argue that pagination is best understood as a web-facing form of Prolog-style backtracking.

Overlapping remote waits

The split-phase nature of `promise/3-4` and `yield/2-3` also makes it possible to overlap the waiting time for several remote computations. The important point is to issue the promises before yielding on any of them:

```
?- promise('http://n1.org', foo(X), R1),
   promise('http://n2.org', bar(Y), R2),
   yield(R1, A1),
   yield(R2, A2).
```

Since both remote computations are initiated before the caller waits for either answer, the elapsed time is approximately the maximum of the two response times, rather than their sum, assuming that the two remote nodes proceed independently. In this respect, the pattern resembles the use of `parallel/1` with synchronous RPC calls:

```
?- parallel([
    rpc('http://n1.org', foo(X)),
    rpc('http://n2.org', bar(Y))
]).
```

The two patterns are nevertheless not equivalent. A call to `parallel/1` behaves like a conjunction executed in parallel: the whole call succeeds only if all component goals succeed. It is therefore a mechanism for speeding up a conjunctive computation whose parts are expected to fit together as a single logical whole.

By contrast, `promise/3-4` and `yield/2-3` do not impose any collective success condition. Each promise denotes one pending remote computation, and each yield retrieves the answer associated with one such computation. The caller may choose to require all answers to be successful, but it may just as well accept partial success, ignore a failed computation, retry a timed-out one, or proceed as soon as one sufficiently useful answer has arrived. The pattern is therefore not a parallel conjunction, but a way of initiating several remote computations and later deciding how their outcomes should be interpreted.

This distinction also clarifies the issue of variable sharing. A parallel conjunction normally requires its component goals to be independent, in particular not to share variables whose bindings make the goals mutually dependent. The `promise/yield` pattern does not impose that restriction in the same way, because the promised computations are not parts of one live conjunction. Variables occurring in several promised goals should not be understood as shared variables spanning the remote computations. Each promise sends its own query, and each answer is later returned as data. Any consistency condition between the answers must therefore be imposed explicitly by the caller after collection.

For example, a caller that wants to combine two independently computed pages can do so locally:

```
?- promise('http://n1.org', foo(X), R1),
   promise('http://n2.org', bar(Y), R2),
   yield(R1, success(Page1, false)),
   yield(R2, success(Page2, false)),
   append(Page1, Page2, Solutions).
```

This is often the right design for web-facing computation: remote nodes compute independently, answers are returned as ordinary terms, and the caller decides how to compare, combine, or discard them.

Thus, `parallel/1` and `promise/yield` both allow waiting time to be reduced, but they express different ideas. The former is a parallelized conjunction with all-or-nothing success semantics. The latter is an asynchronous, split-phase interaction pattern with per-call answer handling.

Bounded waiting and timeouts

A client that cannot afford to block indefinitely can bound the wait performed by `yield/3`:

```
?- promise('http://n1.org', expensive_query(X), Ref),
   yield(Ref, Answer, [
       timeout(2.0),
       on_timeout(Answer = timeout)
   ]).
Answer = timeout,
```

```
Ref = 48468904.
```

```
?-
```

If no answer arrives within two seconds, the `on_timeout/1` goal is called and `Answer` is bound to `timeout`. Timing out does not cancel or consume the promise: the caller may invoke `yield/3` again with the same reference and collect the answer if it has since arrived.

A polling pattern therefore emerges naturally. The following code initiates an asynchronous call and periodically checks for its answer while doing other work:

```
?- promise('http://n1.org', (sleep(1), X=a), Ref),
   repeat,
       yield(Ref, Answer, [
           timeout(0.3),
           on_timeout(Answer = timeout)
       ]),
       ( Answer == timeout
       -> writeln('Still waiting...'),
          writeln('working...'),
          sleep(0.1),
          fail
       ;   !
       ).
Still waiting...
working...
Still waiting...
working...
Answer = success([(sleep(1),a=a)], false),
Ref = 13936890.
?-
```

Each iteration waits briefly for the answer. On timeout, the caller does some work and backtracks to try again. The binding `Answer = timeout` is undone by that backtracking, while the pending promise remains available. Once the answer arrives, the cut terminates the polling loop. The precise number of polling iterations depends on scheduling and network delay.

This is an explicitly operational pattern, but it remains sequential from the caller's perspective: at each step the caller either receives the answer or decides what to do after a bounded wait.

This illustrates a more general rule: on the Prolog Web, timing belongs in the protocol, at the points where a process would otherwise block.

4.3 The programmable Prolog Web

The Prolog Web is programmable in a stronger sense than the familiar ‘Web of APIs’. In its original form as a hypertext Web of documents, the Web was not programmable at all. This changed early with server-side scripts that generated documents dynamically, and with client-side JavaScript. Later, remote procedure calling and open web APIs made it natural to treat the Web as a platform for composition, where independently developed services could be combined into “mashups”, often using a client language as glue.

Web Prolog supports this conventional style of integration, but also generalises it. A node may offer clients not merely a fixed set of endpoints, but a Web Prolog runtime environment that can be programmed directly by queries of varying complexity and, when permitted, by the injection of source code into long-lived processes on the node. In addition, a node owner may install node-resident predicates that become part of what clients can call. In this sense, the Prolog Web is programmable both from the outside, by clients issuing queries and spawning processes, and from the inside, by node owners shaping the execution environment that clients interact with.

This dual perspective aligns naturally with the actor model. A node may restrict which capabilities are present in an actor at birth, for example via `load_*` options that provide an initial private program, and via placement choices (`node`) that determine in which environment the actor should live. Some behaviour is therefore “innate” to the actor, while other behaviour is acquired through interaction. Both are subject to the policy, resource limits, and trust assumptions of the hosting node.

Once code and data can cross node boundaries in either direction, a central systems question arises. Should a computation be moved to where the data resides, or should data (or code-as-data) be imported to where the computation runs? This question has no single correct answer. The optimal direction depends on data volume, update frequency, reuse patterns, latency requirements, and trust. The Prolog Web therefore does not enforce a fixed architecture, but instead exposes a small set of orthogonal mechanisms that let programmers choose, per interaction, how code and data should flow.

The owner’s side of this exchange was illustrated in Section 4.1.4: installing predicates in the shared database and running node-resident actors that define what clients can call. The remainder of this section focuses on the client’s side: how the `load_*` options let a client determine not only *where* to run, but *what* to run.

4.3.1 Injecting code with the `load_*` options

Looking back at the ping-pong scenario in Section 4.1.2, what if the definition of `pong/0` is available at `n1.org`, but not at `n2.org`? The `load_predicates` option ensures that the appropriate source code is shipped from the caller’s database and injected into the private database of the actor that will run at `n2.org`:

```
ping_pong :-
    spawn(pong, Pong_Pid, [
        node('http://n2.org'),
        load_predicates([pong/0])
    ]),
    spawn(ping(3, Pong_Pid)).
```

The `load_predicates` option names predicates that already exist in the caller's context. When the clauses themselves should be supplied literally, the `load_list` option serves that purpose. The following example uses it to populate a remote toplevel actor's private database with a clause that participates in the query:

```
?- toplevel_spawn(Pid, [
    node('http://n1.org'),
    load_list([(mortal(Who) :- human(Who))])
]).
Pid = 83110912@'http://n1.org'.
?-
```

The query `?- mortal(Who)` now executes on `n1.org`, combining the injected rule with the node's own `human/1` facts:

```
?- toplevel_call($Pid, mortal(Who), [
    template(Who)
]).
?- receive({A -> true}).
A = success(83110912@'http://n1.org', [plato, aristotle], false).
?-
```

4.3.2 Code shipping and data locality

On the Internet, the cost of moving code and data across node boundaries is significant, yet unavoidable. The practical question is therefore not whether movement happens, but in which direction it should happen in order to reduce cost and improve responsiveness. In some cases it is more efficient to bring a small computation to a large or frequently updated dataset; in others, it is better to import data once and reuse it locally.

The most direct way to bring code to the data in Web Prolog is to inject a small, task-specific predicate into a process placed near the data, execute it there, and return only the derived result. This corresponds closely to the classical systems idea of *function shipping*: rather than transferring a large dataset across the network, one ships a small computation to where the data resides and retrieves only a compact summary.

Conversely, bringing data to the code can be expressed by running a goal locally while importing a remote program into the local process. This is where a unifying

feature of the design becomes visible: since the default value of the `node` option for `spawn/1-3` and `toplevel.spawn/1-2` is the special-purpose URI `localhost`, the same `load_*` mechanisms used for remote execution also work for local code composition. The `load_*` options are not a “remote” feature; they are a general code-composition mechanism that happens to work transparently across the network. In this way, Web Prolog treats code and data symmetrically: facts and rules can be moved, cached, and reused much like datasets, reflecting Prolog’s natural blurring of the distinction between code and data.

These examples suggest a useful symmetry. Web Prolog code can flow from client to node or from node to client, and the direction can be chosen declaratively by configuring the actor or `toplevel` used for the interaction. In principle, this choice can even be made programmatically at runtime, allowing applications to adapt their strategy based on observed latency, data size, or load.

4.3.3 A small design space

Taken together, the mechanisms described in this chapter form a small but expressive design space structured around a handful of recurring choices:

- **Interface:** `spawn`, `toplevel.spawn`, `rpc`, or `promise/yield`?
- **Placement:** locally or remotely (node)?
- **Code origin:** from the caller’s database, shipped literally, or already resident on the target node (`load_*`)?
- **Interaction style:** pure query, interactive enumeration, or actor-style orchestration?
- **Synchrony:** should the caller block, or initiate work and collect the result later?
- **State:** stateless request–response, or session-oriented exchange?

The point is not that one combination is always best, but that the Prolog Web becomes programmable in a richer sense by letting developers decide where computation happens and how programs and data flow, rather than hard-coding those decisions into the architecture.

Code shipping is also the point where portability stops being an academic concern and becomes operational. If nodes can execute code originating elsewhere, then even small semantic differences or library mismatches become visible immediately. For this reason, the Prolog Web treats mobile code as profile-scoped: a piece of code should state, implicitly or explicitly, which profile it assumes, so that execution is either well-defined or rejected early.

In the limiting case, when a program is self-contained – all predicates it depends on are either included or guaranteed by the target profile – relocation becomes immediate. A client can download a program from one node and begin executing it locally, or ship it to another node, without any adaptation step. We refer to this property as *instant portability*. It is not a special mechanism but a direct consequence of the design: profile-scoped code, symmetric code flow, and the uniform treatment

of local and remote execution combine to make relocation a deployment decision rather than a porting effort.

4.4 Timing and failure boundaries on the Prolog Web

Distributed execution differs from local execution in one decisive respect: waiting becomes observable. A goal that would either succeed or fail immediately in a local Prolog system may, when executed across a network, block for an indeterminate period. Timeouts therefore cannot be treated as mere transport parameters; they define the points at which partial failure becomes visible to the programmer.

In the Prolog Web architecture, two transports are used – HTTP and WebSocket – but three Web APIs are exposed: a stateless HTTP API, a semi-stateful HTTP API, and a stateful WebSocket API. Each API introduces a different blocking surface. Timeouts attach to those surfaces. Their meaning, and therefore their appropriate values, depend on the interaction model.

4.4.1 Three blocking surfaces

Stateless HTTP API – request–response blocking. Blocking occurs at the boundary of a single HTTP request. A client submits a goal and waits for a response. The timeout bounds that roundtrip. Because each call is isolated and resources are allocated per request, timeouts are typically short and strictly bounded. Pagination via `limit` and `offset` exists precisely to control cost and latency.

Semi-stateful HTTP API – split-phase waiting. Work submission and result retrieval are separated. A client initiates a task and later invokes `yield/3` with a `timeout/1` option. The timeout now bounds waiting for a previously initiated computation. Short polling intervals, retries, and fallback actions (`on_timeout/1`) become explicit application logic. Session-level idle or running limits govern resource reclamation independently of per-call waiting.

Stateful WebSocket API – mailbox waiting. Blocking occurs inside `receive/2`. The timeout bounds participation in an interaction protocol rather than a single computation. A mailbox wait may represent a reply, a coordination step, or a conversational turn. Its value is therefore determined by protocol design, not merely transport latency. Longer waits are natural in sustained dialogues, provided recovery behaviour is defined.

4.4.2 A comparative example

Consider the goal `route(göteborg, paris, Route, Cost)`, which computes a travel route together with its cost.

Stateless HTTP. The client poses the goal and waits for answers. The timeout bounds the entire roundtrip. If exceeded, the call fails from the client’s perspective. The interaction remains goal-centric: one request, one response. Expensive computations must be constrained (e.g. by limiting search) or retrieved in slices.

Semi-stateful HTTP. The client submits the goal but separates submission from waiting. The node may continue computing after the initial response. The client later calls `yield/3` with a short timeout, explicitly managing when and how long to wait. Waiting becomes programmable rather than implicit.

WebSocket. The client sends a message and waits in `receive/2`. The timeout now bounds participation in a dialogue rather than completion of a single call. On timeout, the client may retry, redirect, cancel, or renegotiate. The logical goal is unchanged, but the interaction context has shifted from evaluation to conversation.

The difference is therefore not in what is computed, but in where waiting is exposed and how failure is interpreted.

4.4.3 Late replies

Timeouts do not cancel the world. A computation may complete after a client has stopped waiting. The question of what happens to the result – whether it is discarded, logged, or delivered if the session remains alive – is a question about the *semantics of timing*: it determines what the client can observe and what guarantees the API provides. The Prolog Web makes these semantics explicit at the language level through options such as `timeout` and `on_timeout`, rather than hiding them inside transport configuration.

The complementary question – what happens to the *resources* that produced the late reply, and who is responsible for reclaiming them – belongs to lifetime discipline and is addressed in Section 4.5.3.

4.4.4 Design principle

A simple rule emerges:

Place timeouts at the point where a process would otherwise block, and choose their values according to the interaction model – not merely according to transport defaults.

Under this discipline, timing becomes part of the architectural contract. The two transports – HTTP and WebSocket – remain implementation details, but the three Web APIs expose distinct failure boundaries. Making those boundaries explicit is essential for predictable behaviour on an open network.

4.5 Security on the Prolog Web

Don't let your portal to the Prolog Web become a portal to your entire computer!

— *apocryphal advice from a T-800 Model 101*

The Prolog Web is designed to be open: any client that knows a node's URI can submit goals for evaluation. Openness of this kind raises an obvious question: what makes it safe to let strangers run code on your node?

The answer has two components. Three mechanisms – *inexpressibility*, *invisibility*, and *containment* – define the *structural* security posture: the shape of what client code can express, what it can reach, and how long its effects persist. These mechanisms arise directly from the language design, and are developed in detail below.

The relevance of these mechanisms varies with the node profile. For profiles such as RELATION and ISOBASE, where interaction is stateless or request–response over HTTP, the dominant concerns are those of conventional web services: input validation, resource control, authentication, and transport protection. The full capability discipline underlying invisibility and containment becomes essential only for profiles that support long-lived sessions and bidirectional communication, such as ISOTOPE and ACTOR, where process identity and message passing cross node boundaries. In the remainder of this section, we focus on this richer setting, where the full interplay of the three mechanisms becomes visible.

A complete security story also requires *authentication* (to ground the distinction between client and owner), *resource governance* (to prevent abuse within the sandbox), and *transport security* (to protect communication on the wire). These are deployment concerns that Web Prolog inherits from standard web infrastructure; they are discussed briefly in Section 4.5.4.

The layered structure of these mechanisms is illustrated in Figure 4.3. The three lower boxes represent the structural security mechanisms provided by Web Prolog itself; the upper layers correspond to operational and deployment concerns.

4.5.1 Inexpressibility: the sandbox

Web Prolog adopts the same basic strategy as browser-based JavaScript: client-injected code runs in a constrained language subset. The language deliberately excludes file I/O, direct OS access, raw socket programming, and persistent storage.

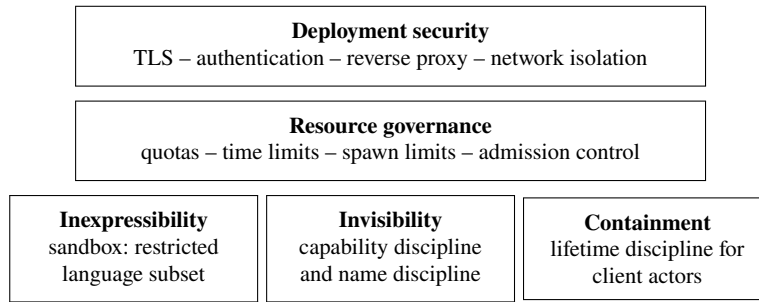


Fig. 4.3 Layered security model of a Web Prolog node. The language sandbox restricts what client programs can express; capability discipline controls which actors and services they may reach; and lifetime containment ensures that client-created computation cannot persist beyond the session. Resource governance limits the amount of computation a client may perform, while deployment mechanisms secure communication and identity at the network level.

These capabilities, if needed, are provided by the host platform and mediated by the node owner through predicates and services that the owner chooses to install – they are not expressible by client-injected code.

This is the price of allowing strangers to run code on your machine: the code must be incapable of causing certain classes of trouble. The restriction is enforced at the language level, not by runtime checks on individual calls, which means that a conforming implementation can reject dangerous code statically rather than catching violations at execution time.

Even when an operation exists in the underlying system, it is exposed to clients only through owner-controlled predicates and services, guarded by authentication and authorisation. The capability policy can therefore be read as part of the node's security boundary: it defines which actions are available to which role, and thereby what the node counts as its public surface.

4.5.2 Invisibility: name discipline

Joe Armstrong articulated the principle clearly: if we do not know the name of a process we cannot interact with it in any way, and the system is secure; once names become widely known, the system becomes less secure. Web Prolog inherits this principle directly.

A pid in Web Prolog is an opaque capability handle. The system does not provide any client-facing process-enumeration interface, and pids are chosen from a sufficiently large, unpredictable space that guessing a live pid is computationally infeasible. A client can therefore interact only with actors whose pids have been explicitly revealed to it, subject to the node's authorisation policy. A pid is a necessary capability – we cannot address what we cannot name – but not always sufficient; nodes may still require authorisation for operations performed via a known pid.

Discoverability creates a natural tension with name discipline. If processes are secure because you cannot talk to what you cannot name, then public services must still provide a way for clients to learn which names are meant to be used. The safe resolution is to treat discovery as an owner-curated *publication mechanism*: discovery should resolve *published* names, not reveal the existence of everything that happens to be running. One concrete realisation of this idea – a declarative service directory queryable via `rpc/2-3` – was given in Section 4.1.4.

Reply channels as capabilities

A subtle but important consequence of treating pids as capabilities concerns the common request–reply idiom. In many actor systems, a client includes its own pid in a request so that the provider can send the reply. In a closed system this is usually harmless, but on the open Prolog Web it grants more authority than intended: a pid is not merely a return address, but a general communication capability.

Revealing a client pid therefore allows the provider not only to reply, but to send arbitrary messages to that actor. This creates a form of unintended capability delegation that may be benign in cooperative settings, but becomes problematic across trust boundaries.

A simple discipline avoids this form of capability leakage. Instead of exposing its own pid, a client should generate a fresh, request-local reply endpoint and include that in the request. Such an endpoint may be realised as a temporary proxy actor or equivalent restricted mechanism, and may be discarded after the reply has been received. In this way, the provider receives authority only to deliver the designated reply, not to invoke the client generally. In capability terms, the reply channel should carry only reply authority, not general send authority.

This pattern refines the capability discipline: communication authority should be as narrow and short-lived as the protocol step that requires it. This refinement is particularly important in distributed interactions, where reply channels cross node boundaries and therefore cross trust domains.

4.5.3 Containment: lifetime discipline

On an open network, a client may disconnect mid-session or vanish without warning. If server-side actors created on that client’s behalf continue to run, they retain memory and send messages into mailboxes that may never be read. Over time, such orphans accumulate into unbounded state and potential security exposure.

The preventive measure is to make process lifetimes *containment-shaped*: every actor spawned by a client is lifetime-dependent on its parent, so that parent termination deterministically tears down the entire subtree. In Web Prolog, `link(true)` is the default; creating an independent actor requires an explicit `link(false)`, and that option is reserved for node owners. A client cannot detach work from its session.

Remote placement sharpens the point. A client may place a child on a remote node via `node(URI)`, but only under lifetime dependence. Allowing a remote detached child would turn `spawn/3` into a deployment primitive and make orphan prevention intractable under disconnects and partitions. For that reason, client-side remote `spawn` rejects `link(false)`.

Containment addresses the *resource* side of the late-reply problem identified in Section 4.4: when a client stops waiting, the computation it initiated may still be running. Lifetime discipline ensures that such processes are reclaimed when the session ends, regardless of whether their results were ever consumed. The *observable semantics* of what the client sees at the timeout boundary – whether a late reply is discarded, delivered, or never generated – is a timing concern and is treated in that section.

Should client-originated detached jobs be needed in the future, they should be introduced as an explicit abstraction – with quotas, lifetime parameters, and auditable handles – rather than as a general relaxation of the containment default.

4.5.4 Deployment concerns

The three structural mechanisms above define the security *architecture* of the Prolog Web, but they do not by themselves constitute a complete deployment story.

First, the node must establish *who* the caller is. Without authentication, the distinction between client and owner has no firm operational basis. Second, the node must enforce *resource governance*. A client may remain entirely within the sandbox and yet still exhaust CPU time, memory, process slots, or network bandwidth. Third, communication between client and node must be protected in transit. Without TLS or equivalent transport protection, a third party may observe or tamper with messages, undermining confidentiality and integrity.

Fourth, the runtime process itself must be constrained by the host environment. Even a correctly specified language sandbox offers limited assurance if the underlying process retains unrestricted filesystem or network access; host isolation – through operating-system-level confinement, WebAssembly runtimes, or WASI-based embeddings – removes entire classes of risk that the language boundary alone cannot prevent.

These concerns are not part of the Web Prolog language design as such. They belong to deployment, and are naturally addressed using standard web infrastructure and operational controls. A production node therefore requires both structural security and secure deployment.

What remains after these mechanisms have done their work is the node's public surface: the predicates it exports, the services it publishes, and the policies under which they may be accessed. A well-configured node should therefore present a small, explicit, and auditable interface: everything else should be either inexpressible, invisible, or contained.

4.6 Summary

This chapter developed the Prolog Web as the third component of the Prolog Trinity, extending Web Prolog from a language for single-node programming into a language for distributed systems.

The *concurrent Prolog Web* introduced network-transparent actor programming: spawning processes on remote nodes, sending messages across node boundaries, and managing lifetimes through links and monitors. Local and remote message passing differ only in latency, not in kind. Node-resident actors demonstrated how an owner can install durable, shared services, and a naming and discovery mechanism showed how such services can be advertised without compromising name discipline.

The *sequential Prolog Web* provided a disciplined request–response abstraction. The key construct, `rpc/2–3`, realises nondeterministic remote procedure calls with familiar Prolog semantics, implemented first over a remote toplevel actor and then, more practically, over stateless HTTP. The split-phase `promise/3–4` and `yield/2–3` predicates extend the pattern to deferred interaction. Although the sequential layer is built on top of the concurrent substrate, it presents a model in which mailboxes and scheduling remain invisible.

The *programmable Prolog Web* showed how code and data flow between client and node. The `load_*` options let clients inject task-specific predicates into remote execution contexts, while the same mechanisms work locally via the `localhost` URI, revealing code composition as a general facility rather than a remote-only feature.

A section on *timing and failure boundaries* made explicit where waiting becomes observable across the three Web APIs – stateless HTTP, semi-stateful HTTP, and WebSocket – and established that timeouts belong at blocking surfaces, chosen according to the interaction model rather than transport defaults.

Finally, a section on *security* asked what makes it safe to let strangers run code on a node, and answered in terms of three structural mechanisms – inexpressibility, invisibility, and containment – together with the deployment concerns (authentication, resource governance, and transport security) that complete the picture. What these mechanisms leave exposed is the node’s *public surface*: the predicates it exports, the services it publishes, and the policies under which they may be accessed.

The chapters that follow build on this foundation. Part II develops generic behaviours – servers, supervisors, toplevels, and statecharts – that structure actor implementations, and later parts return to the architectural and paradigmatic questions that these technical layers raise.

Part II
Behaviors and other generics

Chapter 5

Implementing Web Prolog generics

Generic constructs provide reusable actor patterns that can be specialised, configured, or extended after creation, and that encapsulate common concerns such as message protocols, supervision, and state management.

It is useful to distinguish two broad categories of generics considered in Web Prolog. The first category comprises *Erlang-style behaviours*, which are message-driven components exposing a stable protocol, typically realised as a fixed vocabulary of messages and replies. They do not expose the callee’s proof search directly through caller backtracking. A client wrapper may bind an argument to an explicit reply term, but the interaction itself remains protocol-oriented. The `server_*` predicates provide a request–response service analogous to Erlang’s `gen_server`; the `supervisor_*` predicates provide a behaviour similar to an Erlang supervisor; and statechart actors implement a state-machine model in the spirit of `gen_statem`. The `toplevel_*` predicates also belong to this category, although a Prolog toplevel actor has no ordinary counterpart in Erlang.

The second category comprises what may be termed *Prolog-style generics*. Some are meta-predicates that orchestrate actors internally while presenting a conventional predicate-level interface. Examples include `parallel/1` and `first_solution/2-3`, which spawn worker actors, coordinate them with selective receive, and expose success, failure, exceptions, and variable bindings directly to the caller. They are semideterministic and do not enumerate further solutions on backtracking. By contrast, `rpc/2-3`, introduced in Chapter 4, is nondeterministic: as a distributed analogue of a predicate call, it participates directly in proof search and can return multiple solutions.

This separation reflects a dual identity of the language. Web Prolog inherits explicit message passing and supervision from the actor model while retaining Prolog’s predicate interfaces and variable binding. Generics reconcile the two by factoring communication, lifecycle management, and concurrency out of application logic. Programmers can therefore work either with explicit actor protocols or with ordinary-looking predicates, according to the needs of the application.

The implementations in this chapter are implementation-oriented design sketches, not source listings of the demonstrator. Some fragments are executable in isolation;

others omit helper predicates in order to expose the protocol or programming pattern more clearly. A production implementation must also deal with such matters as module qualification, source transfer, validation, race-free cleanup, resource limits, and transport failure. Where those details do not change the point being explained, we hide them behind small helpers or omit them explicitly. Appendix A defines the normative interfaces adopted by the demonstrator. An interface introduced here but not yet present in the manual should be read as a design proposal.

5.1 The server behaviour

The *server behaviour* implements a classic client-server pattern in Web Prolog: a central server process maintains private state and serves an arbitrary number of clients through message passing. The behaviour is specialised by supplying an application-specific callback predicate, while the generic part provides request-response communication, state threading, monitoring, graceful shutdown, and hot code replacement.

To keep the example focused on the behaviour itself, we assume that callback predicates named in `server_spawn/3-4` or `server_upgrade/2` are already available to the server process. The demonstrator also supports transferring and installing callback source as part of an upgrade; that fuller interface is documented in Appendix A, but its implementation is not needed to explain the server protocol.

A server is started with a callback predicate `Pred/4` and an initial state:

```
server_spawn(Pred, State, Pid) :-
    server_spawn(Pred, State, Pid, []).

server_spawn(Pred, State, Pid, Options) :-
    spawn(server_loop(Pred, State), Pid, Options).
```

The callback is called as

```
call(Pred, Request, OldState, Response, NewState)
```

and therefore describes one state transition of the application. The generic server owns the state and passes it from one callback invocation to the next:

```
server_loop(Pred, State0) :-
    receive({
        '$call'(From, Ref, Request) ->
            (
                call(Pred, Request, State0, Response, State)
            -> From ! Ref-Response,
                server_loop(Pred, State)
            ; exit(false)
        ) ;
        '$upgrade'(From, Ref, Pred1) ->
```

```

        From ! Ref-ok,
        server_loop(Pred1, State0) ;
    '$stop'(From) ->
        From ! reply(true)
    }).

```

A call message contains the client pid, a fresh correlation reference, and the application request. If the callback succeeds, the server sends the response tagged with the same reference and recurs with the new state. If no callback clause handles the request, the server exits with reason `false`; an exception raised by the callback likewise terminates the server with an exception reason.

The upgrade message is likewise correlated by a reference. The server acknowledges it and recurs with the new callback while preserving `State0`. The acknowledgement tells the client that the server has accepted the upgrade, rather than merely that the upgrade message has been placed in transit.

5.1.1 The client interface

A synchronous request is built from two operations: sending a promise and yielding its result.

```

server_request(To, Request, Response) :-
    server_request(To, Request, Response, []).

```

```

server_request(To, Request, Response, Options) :-
    server_promise(To, Request, Ref, MonRef),
    server_yield(Ref, MonRef, Response, Options).

```

The basic three-argument promise operation sends the request and returns its correlation reference:

```

server_promise(To, Request, Ref) :-
    self(Self),
    make_ref(Ref),
    To ! '$call'(Self, Ref, Request).

```

Its reply can later be collected independently:

```

server_yield(Ref, Response) :-
    server_yield(Ref, Response, []).

```

```

server_yield(Ref, Response, Options) :-
    receive({
        Ref-Response -> true
    }, Options).

```

Since each request carries a fresh reference, one client can have several requests in flight simultaneously. Replies may arrive in any order; selective receive associates each reply with the appropriate call.

For example, the following callback implements a small refrigerator:

```
fridge(store(Food), FoodList, ok, [Food|FoodList]).
fridge(take(Food), FoodList, ok(Food), Rest) :-
    select(Food, FoodList, Rest), !.
fridge(take(_Food), FoodList, not_found, FoodList).
```

We can start a server and interact with it synchronously:

```
?- server_spawn(fridge, [], Pid).
Pid = 99491660.
?- server_request($Pid, store(milk), Response).
Response = ok.
?- server_request($Pid, take(milk), Response).
Response = ok(milk).
?-
```

Or we can separate initiation from collection:

```
?- server_promise($Pid, store(meat), Ref).
Ref = 11025019.
?- server_yield($Ref, Response).
Response = ok.
?-
```

5.1.2 Hot code replacement

The upgrade operation uses the same reference-based acknowledgement pattern:

```
server_upgrade(To, Pred) :-
    self(Self),
    make_ref(Ref),
    To ! '$upgrade'(Self, Ref, Pred),
    receive({
        Ref-ok -> true
    }).
```

Assuming that `fridge2/4` is already available to the server, the callback can now be replaced as follows:

```
?- server_upgrade($Pid, fridge2).
true.
?-
```


The operation returns only after the server has accepted the new callback. Its current state is retained. Consequently, `fridge2/4` must accept the existing state representation. If an upgrade also changes the representation of the state, a separate state migration protocol is required; simple callback replacement is not enough.

5.1.3 Monitoring and fail-fast calls

The unmonitored `server_promise/3` interface is useful when several requests are deliberately kept in flight, but a synchronous request should not wait forever after its server has terminated. The four-argument promise therefore installs a monitor before sending:

```
server_promise(To, Request, Ref, MonRef) :-
    self(Self),
    monitor(To, MonRef),
    make_ref(Ref),
    To ! '$call'(Self, Ref, Request).
```

Monitoring an actor that has already terminated does not lose the failure notification: the caller receives a `down` message reporting that the process no longer exists.

The corresponding yield waits for either the reply or the monitor notification:

```
server_yield(Ref, MonRef, Response, Options) :-
    setup_call_cleanup(
        true,
        receive({
            Ref-Response ->
                true ;
            down(MonRef, _Pid, Reason) ->
                throw(server_down(Reason))
        }, Options),
        demonitor(MonRef, [flush])
    ).
```

The cleanup removes the monitor whether the receive succeeds, fails, times out, or raises an exception. The `flush` option also removes a `down` notification that may already have been queued if the server replied and then terminated before the client cancelled its monitor.

A request that the callback cannot handle therefore fails fast:

```
?- server_spawn(fridge, [], Pid).
Pid = 41168156.
?- server_request($Pid, sore(milk), Response).
ERROR: Unhandled exception: server_down(false)
?-
```

The misspelling `sore(milk)` matches no clause of `fridge/4`. The callback fails, the server exits with reason `false`, and the waiting client learns of that exit through its monitor instead of remaining blocked indefinitely.

5.1.4 Graceful shutdown

A server is stopped through an acknowledged message:

```
server_halt(To, Reply) :-
    self(Self),
    To ! '$stop'(Self),
    receive({
        reply(Reply) -> true
    }).
```

5.1.5 From relations to concurrent services

The server behaviour illustrates a principle that runs through the rest of this chapter and, more broadly, through the design of the Prolog Trinity as a whole. It separates the fixed scaffolding of a stateful concurrent service – its message protocol, state threading, monitoring, acknowledged code replacement, and graceful shutdown – from the application-specific logic captured by the callback predicate. The developer writes the part that expresses the domain semantics, while the generic server provides the recurring operational structure.

It is worth comparing this pattern with the simple refrigerator simulation in Chapter 2. There, the behaviour of a refrigerator was modelled directly as a Prolog relation over states. Here, essentially the same relational specification is plugged into a reusable server framework that gives it a process identity, encapsulated state, synchronous and asynchronous interaction, monitoring, and hot code replacement. Developers can thus write ordinary Prolog predicates describing how state evolves and turn them into concurrent services with relatively little additional code. This is one of the distinctive promises of the Prolog Trinity: familiar Prolog programming can be carried into a concurrent and distributed setting without abandoning its relational style.

The demonstrator adds module management, source installation and validation, authorization, and other defensive machinery. Those facilities are important in a production implementation, but they do not alter the separation between the generic server behaviour and its application-specific callback illustrated here.

5.2 The supervisor behaviour

Fault tolerance is essential in software applications because failures are inevitable in real systems, and a fault-tolerant design ensures that individual errors do not cascade into system-wide outages but are contained, recovered from, and rendered harmless to users and neighbouring components.

In Erlang, fault tolerance is achieved through supervision trees – hierarchies of actors that contain and isolate crashes. At the core of this model are *supervisors*, actors whose sole purpose is to start child actors, monitor them, and decide how to recover from crashes. Supervisors themselves can be supervised, yielding supervision trees of, in principle, arbitrary depth.

Underpinning this approach is the so-called *let it crash* principle: worker actors are not burdened with complex error handling but are allowed to crash when something goes wrong. The supervisor detects the crash through a `down` message containing a pid, identifies the crashed actor by this pid, and applies a predefined restart strategy. Erlang's restart policies are uniform and predictable. Depending on the severity and scope of the crash, a supervisor may do nothing, restart the crashed actor process (and perhaps its siblings), or terminate itself so that its own supervisor can take corrective action.

The implementation that follows is deliberately minimal: a one-for-one restart policy with a single global restart budget, enough to show how the actor primitives compose into a supervision structure but far short of a production-quality supervisor. The richer mechanisms that a full implementation would require – alternative restart strategies, a supervisor-wide restart intensity and time window, and dynamic child management – are discussed at the end of this section. Our purpose here is not completeness but demonstration: to confirm that the same `spawn`, `monitor`, `exit`, and selective `receive` primitives that built the server behaviour in the previous section are sufficient to support fault-tolerant process management as well.

The demonstrator implements the fuller supervisor behaviour documented in Section A.5 of Appendix A. The code below is intended to reveal its basic construction, not to reproduce that implementation or define its public interface.

5.2.1 A minimal supervisor loop

A supervisor consists of a *specific part*, which defines which child processes to manage and under what policies they are started and restarted, and a *generic part* – responsible for starting, linking, monitoring, and restarting children. In our implementation, the specific part is given by a list of child specifications and the generic part is captured by the supervisor loop. Each child specification carries an identifier, a start goal, and a restart policy: *permanent* children are always restarted, *transient* children are restarted only if they terminate abnormally, and *temporary* children are never restarted.

The architectural heart of the supervisor is its main loop:

```

supervisor_loop(ChildList, Count0) :-
    receive({
        down(_, Pid, Why) if child_pid(Pid, ChildList) ->
            (
                should_restart(Pid, Why, ChildList)
            -> (
                    Count0 > 0
                -> Count is Count0 - 1,
                    restart_child(Pid, Why, ChildList, ChildList1),
                    supervisor_loop(ChildList1, Count)
                ; exit(gave_up)
            )
        ; remove_child(Pid, ChildList, ChildList1),
          supervisor_loop(ChildList1, Count0)
    }) ;
    '$stop'(From) ->
        terminate(ChildList),
        From ! reply(true)
    }).

```

The guard accepts only notifications from pids represented in the current child list, leaving unrelated `down/3` messages untouched. `should_restart/3` then consults the policy stored in the matching child specification: a permanent child is always restarted, a transient child only after an abnormal exit, and a temporary child never. If the policy does not call for a restart, the child is removed. If it does and the budget has not been exhausted, the child is restarted and its entry is updated with the new pid. If a restart is required after the budget has been exhausted, the supervisor terminates itself, allowing a higher-level supervisor to take corrective action. A stop message instead shuts down all remaining children in an orderly fashion.

5.2.2 An example

Here is a brief example. Given three workers – one that blocks forever, one that terminates normally after three seconds, and one that crashes after five – we start a supervisor with a budget of three restarts:¹

```

?- supervisor_spawn([
    child(steady, (receive({})), permanent),
    child(runner, sleep(3), temporary),
    child(crasher, (sleep(5), exit(an_error)), permanent)
], Pid, [budget(3), name(my_supervisor), monitor(true)]).
Pid = 10039133.
?-

```

¹ The option and child-spec forms used in this sketch (notably `budget/1` and `child/3`) are provisional. For the normative, implemented supervisor API, see Appendix A.

The crasher terminates abnormally and is restarted. After the third restart, the budget is exhausted. When the crasher terminates once more, the supervisor gives up and terminates itself. The remaining child (the steady worker) is killed by the link cascade when the supervisor dies – no orphaned processes are left behind.

5.2.3 From sketch to implemented behaviour

This sketch omits several mechanisms found in a production-quality supervisor. Erlang’s OTP, for example, supports configurable restart strategies – `one_for_all` and `rest_for_one`, which dictate whether sibling processes must also terminate when a crash occurs – as well as a supervisor-wide restart intensity and time window (e.g. three restarts in ten seconds), and an API for dynamically adding or removing child specifications at runtime. These extensions require no new actor primitives. They can be implemented using the same `spawn`, `monitor`, `exit`, and selective `receive` that the sketch already relies on, and in each case the same division of responsibilities remains: the supervisor’s generic loop handles lifecycle management while the child specifications capture the application-specific topology.

The demonstrator implements all three restart strategies, dynamic child management, and supervisor-wide restart intensity tracking. Their public predicates, option forms, and child-specification format are documented in Section A.5 of the manual. The point of the smaller sketch is that the actor substrate introduced in Chapter 2 is sufficient to support these richer supervision patterns. For a detailed account of OTP’s full supervision capabilities, see the Erlang documentation.²

5.3 The toplevel behaviour

The toplevel behaviour turns a Prolog query into a message-driven actor protocol. A client submits a goal, receives a slice of solutions, and may then request further slices, stop the search, abort a running goal, or halt an idle toplevel session.

The implementation developed here follows the core of the demonstrator’s local toplevel implementation closely. To keep the protocol visible, we omit source installation, module isolation, goal validation, inference limits, transport hooks, and other defensive machinery. These do not change the three-state PTCP described here. The normative public interface is documented in Section A.2 of Appendix A.

We begin by spawning an actor running the protocol:

```
toplevel_spawn(Pid) :-
    toplevel_spawn(Pid, []).

toplevel_spawn(Pid, Options) :-
```

² https://www.erlang.org/doc/system/sup_princ.html

```

self(Self),
option(target(Target), Options, Self),
option(session(Session), Options, false),
spawn(ptcp(Pid, Target, Session), Pid, Options).

```

The `target` option determines where answer messages are sent. With `session(false)`, the actor terminates after one query has been exhausted or stopped; with `session(true)`, it returns to its idle state and can accept another query.

5.3.1 Computing slices and answer terms

SWI-Prolog's built-in predicate `findnsols/4` provides the foundation for dividing a query's solutions into slices. It resembles `findall/3`, but returns at most the number of solutions given in its first argument and is itself nondeterministic:³

```

?- findnsols(5, I, between(1, 12, I), L).
L = [1, 2, 3, 4, 5] ;
L = [6, 7, 8, 9, 10] ;
L = [11, 12].
?-

```

The library predicate `offset/2` skips a specified number of solutions before enumeration begins.⁴ Combining the two predicates gives us `slice/5`:

```

slice(Goal, Template, Offset, Limit, Slice) :-
    findnsols(Limit, Template,
              offset(Offset, Goal), Slice).

```

A slice must then be classified as a success, failure, or error. The second argument of `call_cleanup/2` is executed only when the call becomes deterministic, allowing us to distinguish a final slice from one for which further solutions may remain:

```

answer(Goal, Template, Offset, Limit, Answer) :-
    catch(
        call_cleanup(
            slice(Goal, Template, Offset, Limit, Slice),
            Det = true
        ),
        Error,
        true
    ),
    ( nonvar(Error),
      Error == '$abort_goal'
    ).

```

³ https://www.swi-prolog.org/pldoc/doc_for?object=findnsols/4

⁴ https://www.swi-prolog.org/pldoc/doc_for?object=offset/2

```

-> throw('$abort_goal')
; Slice == []
-> Answer = failure
; nonvar(Error)
-> Answer = error(Error)
; var(Det)
-> Answer = success(Slice, true)
; Answer = success(Slice, false)
).

```

The '\$abort_goal' exception is deliberately rethrown. It is a control signal for the surrounding protocol rather than an error answer produced by the submitted goal.

5.3.2 The three inner states

Figure 5.1 shows the Prolog Toplevel Communication Protocol. Its three inner states are **s1**, where the actor waits for a command; **s2**, where a slice is computed; and **s3**, where the actor waits for either a request for another slice or an instruction to stop.

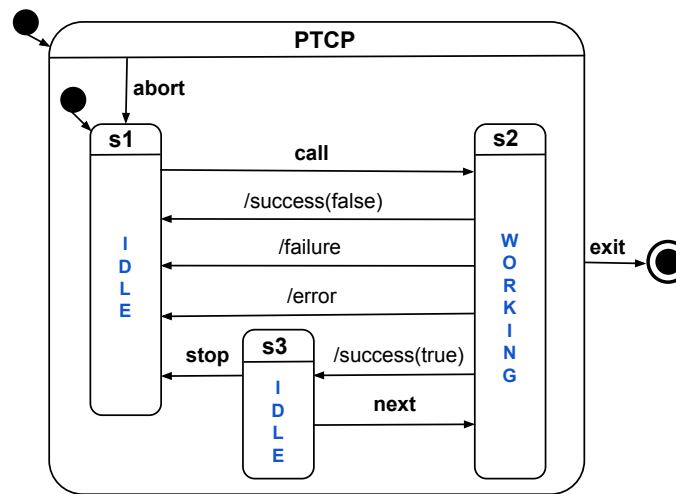


Fig. 5.1 The complete Prolog Toplevel Communication Protocol.

The idle state is implemented by `state.1/3`:

```

state_1(Pid, Target0, Session) :-
    Control = control(continue),
    receive({

```

```

'$call'(Goal, Options) ->
    option(template(Template), Options, Goal),
    option(offset(Offset), Options, 0),
    option(limit(Limit0), Options, 10 000 000 000),
    option(target(Target1), Options, Target0),
    Limit = count(Limit0),
    Target = target(Target1),
    state_2(Goal, Template, Offset, Limit, Pid, Answer),
    arg(1, Target, Out),
    Out ! Answer,
    (   arg(3, Answer, true)
    -> state_3(Limit, Target)
    ;   true
    ) ;
'$halt'(From) ->
    From ! reply(true),
    nb_setarg(1, Control, halt)
}),
(   arg(1, Control, halt)
-> true
;   Session == false
-> true
;   state_1(Pid, Target0, Session)
).

```

The '\$call' message supplies the goal and its options. The limit and target are stored in mutable one-argument compounds so that a later '\$next' message can change either value. The '\$halt' message is accepted while the actor is idle; it is acknowledged before the actor terminates.

The working state calls `answer/5` and adds the toplevel actor's pid to the outward-facing answer:

```

state_2(Goal, Template, Offset, Limit, Pid, Answer) :-
    answer(Goal, Template, Offset, Limit, Answer0),
    add_pid(Answer0, Pid, Answer).

```

```

add_pid(success(Slice, More), Pid,
        success(Pid, Slice, More)).
add_pid(failure, Pid, failure(Pid)).
add_pid(error(Term), Pid, error(Pid, Term)).

```

The `count/1` form accepted by `findnsols/4` acts as a mutable local cell. Its value can be changed between slices:

```

?- Limit = count(2),
   answer(between(1,12,I), I, 0, Limit, Answer),
   nb_setarg(1, Limit, 5).

```



```

Limit = count(5),
Answer = success([1, 2], true) ;
Limit = count(5),
Answer = success([3, 4, 5, 6, 7], true) ;
Limit = count(5),
Answer = success([8, 9, 10, 11, 12], false).
?-

```

When an answer reports that more solutions may remain, the actor enters `state_3/2`:

```

state_3(Limit, Target) :-
    receive({
        '$next'(Options) ->
            (
                option(limit(NewLimit), Options)
                -> nb_setarg(1, Limit, NewLimit)
                ; true
            ),
            (
                option(target(NewTarget), Options)
                -> nb_setarg(1, Target, NewTarget)
                ; true
            ),
            fail ;
        '$stop' -> true
    }).

```

After '`$next`', the call to `fail/0` backtracks into `state_2/6` and resumes the nondeterministic `findnsols/4` computation. The non-backtrackable updates made with `nb_setarg/3` survive that redo, so the next slice uses the new limit or target.

Both mutable cells must therefore be constructed *before* `state_2/6`, where the choicepoint is created. If the target cell were constructed after that call, backtracking would reconstruct it from the original target and silently discard redirection requested through `toplevel_next/2`. The placement of the two assignments in `state_1/3` is thus essential to the protocol.

If '`$stop`' is received instead, the current search is abandoned. A persistent session returns to `s1`; a non-session toplevel then terminates.

Lifecycle management in a production implementation.

The implementation above is deliberately pedagogical and leaves resource governance to the surrounding runtime. A toplevel may wait indefinitely in state `s1` for its first call or in state `s3` for `toplevel_next/1` or `toplevel_stop/1`; a goal may likewise remain indefinitely in state `s2`. In particular, `session(false)` causes termination only after the query has been exhausted or explicitly stopped. It cannot detect that a client has discarded its local choice point using a cut or `once/1`.

A production implementation should therefore combine parent–child lifetime containment with owner-controlled running and idle timeouts. A client that abandons enumeration should explicitly stop the remote search, while cleanup during an executing goal may require aborting or terminating the actor. For calls that require only the first slice, the documented `once(true)` option should commit the search on the remote side and report that no further solutions are available. These mechanisms prevent an abandoned client from leaving a toplevel actor and its saved search state resident indefinitely.

5.3.3 Aborting a running goal

The three predicates above implement the inner states. The surrounding PTCP state catches the private abort signal and re-enters the idle state without terminating the actor:

```
ptcp(Pid, Target, Session) :-
    catch(
        state_1(Pid, Target, Session),
        '$abort_goal',
        ptcp(Pid, Target, Session)
    ).
```

This is why `answer/5` must rethrow `'$abort_goal'` instead of packaging it as an error answer.

In the local SWI-Prolog implementation, interrupting a running computation ultimately requires `thread_signal/2`. A Web Prolog actor pid is not, however, a SWI-Prolog thread identifier. The runtime must first resolve the actor pid to its private thread identifier:

```
toplevel_abort(Pid) :-
    ( actors:resolve_thread(Pid, ThreadId)
    -> catch(
        thread_signal(ThreadId, throw('$abort_goal')),
        error(existence_error(_, _), _),
        true
    )
    ; true
    ).
```

The mapping is an implementation detail: application programs use actor pids and must not assume that they are interchangeable with thread identifiers. The code shown here is the local backend; node-facing requests arrange for the corresponding operation to run on the node that owns the toplevel actor.

5.3.4 The client interface

The remaining public operations are small wrappers around the protocol messages:

```
toplevel_call(Pid, Goal) :-
    toplevel_call(Pid, Goal, []).

toplevel_call(Pid, Goal, Options) :-
    Pid ! '$call'(Goal, Options).

toplevel_next(Pid) :-
    toplevel_next(Pid, []).

toplevel_next(Pid, Options) :-
    Pid ! '$next'(Options).

toplevel_stop(Pid) :-
    Pid ! '$stop'.

toplevel_halt(Pid, Reply) :-
    self(Self),
    Pid ! '$halt'(Self),
    receive({
        reply(Reply) -> true
    }).
```

These operations have distinct purposes. `toplevel_stop/1` abandons a solution stream while the actor is waiting in `s3`. `toplevel_abort/1` interrupts a goal that is currently executing in `s2` and returns the actor to the entry point of the PTCP. `toplevel_halt/2` terminates an idle toplevel actor and waits for its acknowledgment. The general `exit/1-2` predicates may also terminate an actor, but they are not part of this orderly toplevel protocol.

5.4 The parallel/1 meta-predicate

In this section we implement `parallel/1`, the meta-predicate introduced in Chapter 4 (Section 4.2.6). A call to `parallel(Goals)` should wait until its outcome is known and its workers have been cleaned up, succeed with the returned variable bindings if all goals succeed, fail promptly if any goal fails, and promptly rethrow an exception raised by any goal. These guarantees require the goals to be independent: they must not communicate through shared variables, global state, or other side channels.

The predicate is semideterministic. Each goal contributes its first success, if it has one, and `parallel([])` succeeds immediately. From the caller's perspective this is

a conventional predicate-level interface; underneath, one monitored actor is used for each goal.

5.4.1 From two goals to a list

We begin with a deliberately incomplete two-goal version:

```
parallel(Goal1, Goal2) :-
    self(Self),
    spawn((call(Goal1), Self ! Pid1-Goal1), Pid1),
    spawn((call(Goal2), Self ! Pid2-Goal2), Pid2),
    receive({Pid1-Goal1 -> true}),
    receive({Pid2-Goal2 -> true}).
```

Each worker calls its copied goal and, on success, sends its pid paired with the instantiated copy back to the parent. Selecting that message unifies the copy with the corresponding goal in the caller and thereby returns its bindings. The two replies may arrive in either order: selective receive defers the second worker's reply if it arrives before the first one being awaited.

The same pattern generalises naturally from two goals to a list:

```
parallel(Goals) :-
    maplist(par_solve, Goals, Pids),
    maplist(par_yield, Pids, Goals).

par_solve(Goal, Pid) :-
    self(Self),
    spawn((call(Goal), Self ! Pid-Goal), Pid).

par_yield(Pid, Goal) :-
    receive({Pid-Goal -> true}).
```

This version works when every goal succeeds. If a goal fails or throws an exception, however, its worker sends no result and the corresponding `receive/1` waits indefinitely. Monitors supply the missing outcome.

5.4.2 Detecting failure and errors

Each worker is now monitored, and `par_yield/3` accepts either the expected result or an abnormal termination notification from any worker belonging to this invocation:

```
parallel(Goals) :-
    maplist(par_solve, Goals, Pids),
```

```

        maplist(par_yield(Pids), Pids, Goals).

par_solve(Goal, Pid) :-
    self(Self),
    spawn((call(Goal), Self ! Pid-Goal), Pid, [
        monitor(true)
    ]).

par_yield(Pids, Pid, Goal) :-
    receive({
        Pid-Goal ->
        receive({
            down(_, Pid, true) ->
            true
        }) ;
        down(_, FailedPid, false)
        if memberchk(FailedPid, Pids) ->
        tidy_up_all(Pids),
        !,
        fail ;
        down(_, FailedPid, exception(E))
        if memberchk(FailedPid, Pids) ->
        tidy_up_all(Pids),
        throw(E)
    }).

```

On success, the result message is followed by a normal `down/3` notification, which consumes the worker's monitor. A failure or exception from any worker determines the outcome of the whole call, so the remaining workers are cleaned up immediately. Matching only the current `Pid` in the abnormal clauses would unnecessarily delay detection until that particular worker was awaited.

The guards instead admit notifications from every `pid` in `Pids` while leaving unrelated monitor messages in the caller's mailbox. This matters because `parallel/1` may be called by an actor that is monitoring other processes at the same time. The cut in the failure branch prevents backtracking into the receive machinery after cleanup.

5.4.3 Race-free cleanup

Cleanup must do more than kill the remaining workers. It must also ensure that neither result messages nor monitor notifications from this invocation are left behind in the caller's mailbox:

```

tidy_up_all(Pids) :-
    maplist(tidy_up, Pids).

```

```

tidy_up(Pid) :-
    demonitor(Pid, [flush]),
    monitor(Pid, CleanupRef),
    exit(Pid, kill),
    receive({
        down(CleanupRef, Pid, _) -> true
    }),
    drain_mailbox(Pid).

drain_mailbox(Pid) :-
    receive({
        Pid_ ->
            drain_mailbox(Pid) ;
        down(_, Pid, _) ->
            drain_mailbox(Pid)
    }, [timeout(0)]).

```

The creation-time monitor is first removed and any notification it has already produced is flushed. A private cleanup monitor is then installed before the worker is killed. Monitoring an actor that has already terminated still produces a `down/3` notification, so the code also works when the worker has won the race and exited before cleanup reaches it.

Waiting for `down(CleanupRef,Pid,...)` establishes an ordering barrier: the worker has terminated, and any result it sent before terminating has already been enqueued. Only then is a zero-time receive used to drain result or monitor messages associated with that pid. A zero-time drain performed without the barrier could finish just before an in-flight message arrived.

5.4.4 Behaviour and performance

We can now check success, failure, and exception propagation. The following three sleeps overlap, so the call takes approximately as long as the slowest goal rather than the sum of all three delays:

```

?- _Goals = [(X=a,sleep(1)),(Y=b,sleep(3)),(Z=c,sleep(2))],
   time(parallel(_Goals)).
% 189 inferences, 0.000 CPU in 3.006 seconds
X = a,
Y = b,
Z = c.
?-

```

On this example the elapsed time is about three seconds, compared with about six seconds for sequential execution.

If one goal fails, the entire call fails without waiting for slower siblings:

```
?- _Goals = [(X=a,sleep(1)),(Y=b,fail),(Z=c,sleep(2))],
    time(parallel(_Goals)).
% 105 inferences, 0.000 CPU in 0.001 seconds
false.
?-
```

An exception is likewise propagated after the other workers have been cleaned up:

```
?- _Goals = [(X=a,sleep(1)),(Y=b,sleep(a)),(Z=c,sleep(2))],
    time(catch(parallel(_Goals),E,true)).
% 126 inferences, 0.000 CPU in 0.001 seconds
E = error(type_error(float, a), _).
?-
```

Actor creation, monitoring, message copying, and cleanup all have a cost. Goals must therefore perform enough work for their overlap to repay that cost. This implementation also creates one actor per goal; very large goal lists consequently have a correspondingly large resource footprint.

5.4.5 Relation to `concurrent/3`

SWI-Prolog's `library(thread)` provides the related predicate `concurrent/3`.⁵ Its observable success, failure, exception, and variable-binding behaviour is close to that of `parallel/1`, but the interfaces serve somewhat different purposes. The first argument of `concurrent/3` bounds the number of worker threads, and its third argument supplies thread-creation options. By contrast, `parallel/1` deliberately presents a narrower interface and creates one actor for each goal.

The compact implementation developed here should therefore not be read as a replacement for, or a simplification of, `concurrent/3`. Rather, it demonstrates how the actor primitives of Web Prolog can be composed to provide a familiar parallel-conjunction abstraction with predicate-level variable bindings. The two predicates have closely related semantics but make different choices about resource control and the abstractions exposed to the programmer.

5.5 The `first_solution/2-3` meta-predicate

SWI-Prolog's `library(thread)` also provides `first_solution/3`: run several alternative solvers concurrently and return the result produced by the first one to

⁵ <https://www.swi-prolog.org/pldoc/man?section=thread>

succeed.⁶ The Web Prolog version adopts this predicate-level idea but implements the race with monitored actors. Once a winner has been selected, all remaining workers are terminated and their messages are removed from the caller's mailbox.

5.5.1 Racing alternative solvers

Borrowing an example from Wielemaker, suppose that it is unclear whether a graph should be searched breadth-first or depth-first. We can start both strategies and accept whichever finds a path first:

```
search_graph(Graph, Path) :-
    first_solution(Path, [ breadth_first(Graph, Path),
                          depth_first(Graph, Path)
                        ],
                  []).
```

Unlike `parallel/1`, which requires every goal to succeed, `first_solution/2-3` treats its goals as alternatives. Their copies share the same solution template, and the first result message selected by the caller binds that template.

The two-argument form supplies portable default policies: a failed solver is ignored while alternatives remain, whereas an exception terminates the race and is rethrown:

```
first_solution(Solution, Goals) :-
    first_solution(Solution, Goals, [
        on_fail(continue),
        on_error(stop)
    ]).

first_solution(Solution, Goals, Options) :-
    option(on_fail(OnFail), Options, continue),
    option(on_error(OnError), Options, stop),
    maplist(first_solve(Solution), Goals, Pids),
    wait_first(Pids, Solution, OnFail, OnError).

first_solve(Solution, Goal, Pid) :-
    self(Self),
    spawn((call(Goal), Self ! Pid-Solution), Pid, [
        monitor(true)
    ]).

wait_first([], _, _, _) :-
    !,
```

⁶ <https://www.swi-prolog.org/pldoc/man?section=thread>


```

fail.
wait_first(Pids, Solution, OnFail, OnError) :-
    receive({
        Winner-Solution
            if memberchk(Winner, Pids) ->
                tidy_up_all(Pids) ;
        down(_, FailedPid, false)
            if memberchk(FailedPid, Pids) ->
                ( OnFail == continue
                -> select(FailedPid, Pids, Rest),
                    wait_first(Rest, Solution, OnFail, OnError)
                ; tidy_up_all(Pids),
                    !,
                    fail
                ) ;
        down(_, FailedPid, exception(Error))
            if memberchk(FailedPid, Pids) ->
                ( OnError == continue
                -> select(FailedPid, Pids, Rest),
                    wait_first(Rest, Solution, OnFail, OnError)
                ; tidy_up_all(Pids),
                    throw(Error)
                )
    }).

```

Every receive head is guarded by membership in Pids. An unrelated pair in the caller's mailbox must not be mistaken for a winning result, and a `down/3` notification from an unrelated monitor must not affect the race. On success, `tidy_up_all/1` from the implementation of `parallel/1` supplies the same ordering barrier and race-free mailbox cleanup described in the previous section.

If no worker remains, the call fails. The predicate is therefore semideterministic: it returns the first solution selected by the mailbox but does not enumerate later solutions on backtracking.

5.5.2 Failure and error policies

The option `on_fail(continue)` removes a failed solver and keeps waiting; `on_fail(stop)` instead cleans up all workers and fails the entire call immediately. Similarly, `on_error(continue)` ignores an exception from one solver while alternatives remain, whereas `on_error(stop)` cleans up and rethrows it. The accepted values are thus `continue` and `stop`; validation and the associated domain errors are omitted from this design sketch.

The defaults make failure local to an alternative but treat an exception as significant. This is often useful when the solvers are different search strategies: one strategy

may simply find no answer, while an exception may indicate a defect or resource problem that should be reported. Applications can select different policies explicitly through `first_solution/3`.

If all alternatives fail under `on_fail(continue)`, or if all remaining alternatives are discarded under `on_error(continue)`, the worker list eventually becomes empty and the whole call fails.

5.5.3 An example

Using a simple simulation of two long-running goals, here is a trial run that shows that our predicate works as intended:

```
?- time(first_solution(X, [(sleep(2),X=a),(sleep(1),X=b)])).
% 164 inferences, 0.000 CPU in 1.006 seconds
X = b.
?-
```

The second solver wins after approximately one second. Cleanup then terminates the slower worker before returning `X = b` to the caller.

5.6 Summary

The examples in this chapter illustrate two complementary ways of packaging concurrent computation in Web Prolog. Protocol-oriented generics such as the server, supervisor, and toplevel actor expose stable message protocols and hide recurring operational machinery for state management, lifecycle control, monitoring, and query execution. They do not expose their internal proof search through caller backtracking. Predicate-oriented generics such as `parallel/1` and `first_solution/2-3`, by contrast, use actors internally while presenting ordinary predicate-level success, failure, exceptions, and variable bindings.

A recurring design principle is the separation of application logic from coordination. The server callback describes how one request transforms application state; child specifications describe the topology managed by a supervisor; a toplevel actor executes submitted Prolog goals; and the parallel generics receive goals or alternative solvers directly as terms. The surrounding generic supplies the message protocol, process lifecycle, correlation, monitoring, and cleanup appropriate to each case.

That separation does not mean that arbitrary Prolog code can be moved into a concurrent or distributed setting unchanged. Server upgrades must preserve or migrate state representations; parallel goals must satisfy their independence requirements; remotely executed code must be available and authorized on the target node; and implementations must account for resource limits, transport failure, and messages

that race with process termination. The design sketches make the reusable structure visible, while the manual specifies the public contracts adopted by the demonstrator.

Several techniques recur across otherwise different abstractions. Fresh references correlate requests and replies. Selective receive lets a process react to whichever relevant outcome arrives first while deferring unrelated messages. Monitors turn silent process termination into explicit data. Acknowledgements distinguish an accepted operation from a message merely placed in transit. Cleanup monitors provide ordering barriers before mailbox draining. In the toplevel actor, controlled backtracking and mutable cells support demand-driven slices of solutions. These examples show ordinary Prolog mechanisms and actor mechanisms reinforcing one another.

The result is not a claim that Web Prolog replaces OTP or SWI-Prolog's thread library. The server and supervisor show that established actor patterns can be expressed naturally in Web Prolog, while the toplevel and parallel generics show how actor orchestration can be hidden behind interfaces shaped by Prolog's execution model. Together they demonstrate the central point of the chapter: combining goals, unification, and backtracking with process identity, message passing, and monitoring makes it possible to construct abstractions that remain recognisably Prolog while operating concurrently and, where required, across the Web.

Chapter 6

The statechart behaviour

The earlier chapters introduced several generic behaviours for actors in Web Prolog: the toplevel behaviour, the server behaviour, and the supervisor behaviour. Each of these patterns offered a reusable programming model for structuring how actors communicate, manage state, or enforce reliability. In this chapter we introduce a more expressive and visually oriented behaviour: the *statechart behaviour*, which provides a powerful mechanism for defining how a single actor responds to events and evolves behaviour over time.

A statechart behaviour encapsulates the dynamics of an actor in terms of a state machine with hierarchy, parallel regions, and event-triggered transitions. This model is inspired by David Harel’s statecharts, and the result is an abstraction capable of implementing interactive interfaces, conversational flows, control systems, and long-running reactive processes in a structured and compositional manner.

The representation we use in this chapter is based on a compact XML notation. Although XML is not always a popular choice as a general-purpose representation format, it fits statecharts unusually well because its tree structure mirrors hierarchical nesting of states and regions directly. The notation is directly executable, allowing statechart documents to be loaded, spawned, and controlled as Web Prolog actors. For example, a statechart can be launched with `statechart_spawn/2`, participate in supervision trees, and send or receive messages like any other actor. These predicates form the `statechart` family of the ACTOR profile; a node owner may enable or disable that family as part of the node’s advertised offer.

6.1 STATECHARTS

Invented by David Harel, STATECHARTS is a graphical formalism for specifying reactive systems (Harel, 1987). Building on the concept of a state machine, it adds hierarchy, parallel regions (orthogonality), and event visibility across the active states and regions. Like logic, it is a “tool for thinking” with a formal semantics (Harel and Naamad, 1996), but it is also explicitly visual – a way of “tapping the potential

of high bandwidth spatial intelligence, as opposed to lexical intelligence used with textual information” (Samek, 2002).

Statecharts have been used in a wide range of areas, from embedded control and user interfaces to conversational systems and protocols (Samek, 2002; Horrocks, 1999; Skantze and Moubayed, 2012; Cicirelli et al., 2009). In the Web context, the W3C standardised SCXML as a way to represent statecharts in an XML form (Barnett et al., 2015). SCXML has seen industrial use (for example by Genesys¹). Ideas from statecharts are also visible in front-end libraries such as XSTATE, which claims SCXML compatibility while expressing machines in JavaScript.² For the back-end, *Statifier* is a recent Elixir implementation of SCXML with a focus on W3C compliance.³

In this chapter we present a lightweight, XML-based textual profile of the statechart formalism that is directly executable as a Web Prolog actor behaviour, with Prolog as both data model and executable content. The surface syntax resembles SCXML, but it is intentionally not a full SCXML language binding; our goal is to keep the standard execution model while harmonising it with Web Prolog and the Prolog Web. We refer to our dialect as Statechart XML, or SXML for short.⁴ Exactly which SCXML constructs SXML retains, adapts, or omits is set out in Section 6.9.4.

Importantly, the exposition in this chapter is self-contained: readers do not need any familiarity with SCXML. Although SXML’s syntax resembles SCXML superficially, the mechanisms are presented here as a standalone behaviour for Web Prolog actors. A comparative discussion of SCXML 1.0, and of Erlang’s `gen_statem` behaviour, is postponed until the end.

6.2 Statechart actors

A *statechart actor* is a Prolog actor whose control logic is given by a statechart document. Conceptually, it behaves like an ordinary actor with a mailbox, except that incoming messages are interpreted as *events* that drive a structured state machine rather than an ad hoc `receive/1-2` loop.

Events are just messages: any incoming actor message (a Prolog term) can serve as an external event. When an event is processed, it is *visible* to all currently active parts of the machine (including parallel regions), and it may trigger one or more transitions whose actions run as Web Prolog code.

Execution follows the standard statechart *run-to-completion* intuition: processing one external event may trigger a finite cascade of internal work (such as taking condition-only transitions and handling internally raised events) before the actor consumes the next external message. This makes each reaction behave like an atomic

¹ See <https://docs.genesys.com/Documentation/OS>

² <https://github.com/statelyai/xstate>

³ <https://riddler.github.io/statifier/>

⁴ The abbreviation SXML is also used, unrelatedly, for an S-expression representation of XML in the Scheme community; no connection is intended here.

control step at the statechart level, even though the actor itself remains an ordinary concurrent process.

In the examples below, a statechart document can therefore be read as a compact description of (i) which states are active, (ii) which incoming events matter in those states, and (iii) what effects and state changes occur when such an event is observed. SXML takes its transition-selection and run-to-completion model from SCXML, while adapting event matching, executable content, and process invocation to Web Prolog. Section 6.7 states the resulting execution rules precisely.

6.2.1 A playful intro to statecharts

In its simplest form, a statechart is just a deterministic state machine, in which state transitions are triggered by events. The simple statechart in Figure 6.1 serves as our first example. The labelling of one of the transitions with the symbol `play` suggests that the example is drawn from the field of game development.⁵

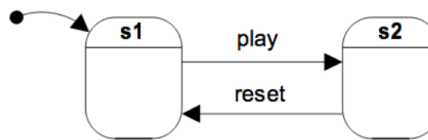


Fig. 6.1 A minimal statechart example: event-triggered transitions in a flat machine.

Here, `s1` is the initial state. If the next selected event is `play`, a transition to `s2` is taken. For any other event, the machine will remain in `s1` and the event will be lost (i.e. it will not be deferred).

Statecharts within the SXML profile can be translated into documents written in an XML-based syntax. Here, a document that captures the statechart in Figure 6.1 is shown:

```

<statechart datamodel="web-prolog" initial="s1">
  <state id="s1">
    <onentry>
      output('IDLE')
    </onentry>
    <go to="s2" on="play"/>
  </state>
  <state id="s2">
    <onentry>
      output('PLAYING')
    </onentry>

```

⁵ We are borrowing some of our examples from (Brusk and Lager, 2008).

```

        <go to="s1" on="reset"/>
    </state>
</statechart>

```

Note that two `<onentry>` elements, the significance of which is not shown in Figure 6.1, have been added to the document. They contain *executable content* – instructions which allow the state machine to *do* things such as modifying its data model or interacting with external entities. Executable content must be specified using Web Prolog source code. In the case of the above example, the Web Prolog built-in predicate `output/1` sends `output(Pid, 'IDLE')` or `output(Pid, 'PLAYING')` to the spawning process each time `s1` or `s2` is entered, exactly as `output/1` does in other actors. Executable content may also appear in `<onexit>` and `<go>` elements; its treatment of solutions, failure, and exceptions is specified in Section 6.7.

6.2.2 A statechart process is an actor in disguise

A major feature of our proposal is that we encourage people to think of a *statechart process as an actor running on the Prolog Web*, either in a browser, or as a process running on a node and accessed from a browser. Indeed, to create a statechart session, we can simply use `statechart_spawn/2`:

```

?- statechart_spawn(Pid, [
    load_uri('http://n6.org/game.xml'),
    monitor(true)
]).
Pid = 78230912.
?- flush.
Shell got output(78230912,'IDLE')
true.
?-

```

When `statechart_spawn/2` is called, a new actor is spawned that interprets the statechart. As specified by the `<onentry>` element in the initial state, the statechart actor immediately sends `output(78230912, 'IDLE')` to its parent. Equipped with the pid, we can use `!/2` to send an event to the actor, causing a transition from `s1` to `s2`. In general, an event may be any Web Prolog term except a bare variable; here, the relevant `on` attribute tells us that the atom `play` will do:

```

?- $Pid ! play.
true.
?- receive({output(_, What) -> true}).
What = 'PLAYING'.
?-

```

We may now send the event `reset` to return to `s1`, and so on. At any point we may also terminate the actor by sending it a signal with `exit/2`. Since the `monitor`

option was set to `true`, the runtime notifies the parent with a `down` message when the actor terminates, exactly as it does for an ordinary actor.

Other options to `statechart_spawn/2` work as expected. For example, we can run the statechart on a remote node by passing a `node(URI)` option, and we can provide the source directly using `load_text` rather than fetching it from a URI.

The examples in the rest of this chapter are organised around a small set of extensions that make statecharts practically useful as an actor behaviour on the Prolog Web. Each extension adds a distinct capability – some at the state-machine level, others at the level of interaction with the surrounding actor system. Concretely, the extensions we rely on are:

- Hierarchy and history states
- Parallelism (aka orthogonality)
- Broadcast communication
- Process invocation
- Inter-process communication
- Data model and scripting language

In the sections that follow, the left column highlights statechart-level structuring mechanisms, while the right column highlights how those mechanisms connect to actors and Prolog in the Web Prolog setting.

6.3 Hierarchy and history

Statecharts may be *hierarchical*, i.e. a state may contain another statechart down to an arbitrary depth. A complex state may contain a *history state*, serving as a memory of which substate *S* the complex state was in, the last time it was left for another state. Transition to the history state implies a transition to *S*.

The statechart in Figure 6.2 exemplifies both hierarchy and the history state by enhancing our simple game with a pause-and-resume functionality.

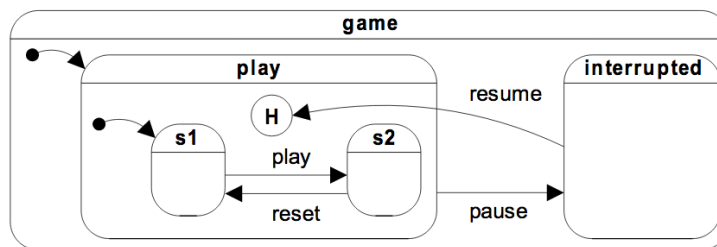


Fig. 6.2 Pause-and-resume via hierarchy and history state in the game controller.

Suppose that the current state is **s2**, and that the next selected event is **pause**. The transitions available in **s2** are considered from the inside out. The **reset** transition in **s2** does not match, but the **pause** transition inherited from the enclosing **play** state

does, so a transition to `interrupted` takes place. If a `resume` event then appears, the system transitions to the history state `H`, which returns it to `s2`.

Documents are structured so that they mirror the topology of the statecharts they represent. A document that captures the statechart in Figure 6.2 is shown below:

```
<statechart datamodel="web-prolog" initial="game">
  <state id="game" initial="play">
    <state id="play" initial="s1">
      <history id="H">
        <go to="s1" />
      </history>
      <state id="s1">
        <go to="s2" on="play" />
      </state>
      <state id="s2">
        <go to="s1" on="reset" />
      </state>
      <go to="interrupted" on="pause" />
    </state>
    <state id="interrupted">
      <go to="H" on="resume" />
    </state>
  </state>
</statechart>
```

A history state is shallow by default: it remembers the immediate child that was active. The optional attribute `type="deep"` instead remembers the complete active descendant configuration. The pause-and-resume example cannot distinguish the two because `s1` and `s2` are already atomic states.

Hierarchy supports two complementary design moves. In *refinement* (top-down), an abstract control state is elaborated into substates whose internal transitions make the behaviour explicit. In *clustering* (bottom-up), a family of closely related states is grouped under a superstate, so that shared transitions and common entry/exit behaviour can be expressed once. Both moves reduce state explosion and make large machines readable: the top level can remain a small “map” of the application, while local detail is pushed down into the regions where it belongs. Together, hierarchy and history give *interruption* and *resumption* a useful first-class representation.

6.4 Parallelism and broadcast communication

A hallmark of statecharts is *orthogonality*: a state may contain two or more regions that execute in parallel. Operationally, the machine is simultaneously in one active state in each region. The point is not CPU-level parallelism, but a way to factor one reactive controller into several facets that evolve together.

6.4.1 A slightly richer orthogonality example: an NPC controller

To see why orthogonality matters, consider a toy controller for a non-player character with two independent emotion dimensions and one behaviour mode. Figure 6.3 sketches the idea: the character is simultaneously in one state in each emotion dimension and in one behaviour state. When an event is processed, it may trigger coordinated changes across several regions.

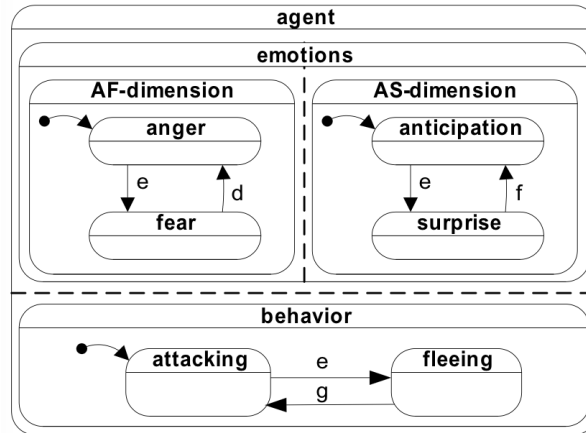


Fig. 6.3 Orthogonal regions: two emotion dimensions plus one behaviour mode in an NPC controller.

In the figure, an *e* event (think: an explosion) moves the character into *fear*, *surprise*, and *fleeing* “at once”, because all three regions observe the same event while they are active. Without parallel regions, we would be forced to encode combinations such as *anger_anticipation_attacking*, *fear_surprise_fleeing*, and so on, and the number of states grows quickly as more dimensions are added.

Here is how to render this machine in XML:

```
<statechart datamodel="web-prolog" initial="agent">
  <parallel id="agent">
    <parallel id="emotions">
      <state id="AF-dimension" initial="anger">
        <state id="anger">
          <go to="fear" on="e"/>
        </state>
        <state id="fear">
          <go to="anger" on="d"/>
        </state>
      </state>
    </parallel>
    <state id="behavior">
      <state id="attacking">
        <go to="fleeing" on="e"/>
      </state>
      <state id="fleeing">
        <go to="attacking" on="g"/>
      </state>
    </state>
  </parallel>
</statechart>
```

```

    <state id="AS-dimension" initial="anticipation">
      <state id="anticipation">
        <go to="surprise" on="e"/>
      </state>
      <state id="surprise">
        <go to="anticipation" on="f"/>
      </state>
    </state>
  </parallel>
  <state id="behavior" initial="attacking">
    <state id="attacking">
      <go to="fleeing" on="e"/>
    </state>
    <state id="fleeing">
      <go to="attacking" on="g"/>
    </state>
  </state>
</parallel>
</statechart>

```

6.4.2 A cautionary broadcast example: an unbounded internal cascade

To make the broadcast mechanism completely explicit – and to expose a hazard that the mechanism makes easy to stumble into – we now switch to a deliberately minimal example: two regions that exchange events. The point is not to model two independent actors, but to show how a single statechart step can coordinate activity across regions. As we shall see, this particular machine also illustrates what goes wrong when that coordination feeds back on itself.

```

<statechart datamodel="web-prolog" initial="Start">
  <parallel id="Start">
    <state id="Pinger">
      <onentry>raise(pong)</onentry>
      <go on="ping">
        writeln('Ping'), raise(pong)
      </go>
    </state>
    <state id="Ponger">
      <go on="pong">
        writeln('Pong'), raise(ping)
      </go>
    </state>
  </parallel>
</statechart>

```

Note the use of targetless `<go>` elements (i.e. elements without a `to` attribute). These transitions consume an event (and may run executable content, such as `raise/1`) without changing the active configuration, so no `<onentry>` or `<onexit>` code is triggered.

This machine does exactly what its two transitions say, and that is the problem. On entry, `Pinger` raises `pong`; the `Ponger` region consumes it and raises `ping`; the `Pinger` region consumes that and raises `pong` again; and so on without end. Because run-to-completion drains the *internal* event queue before the next *external* message is consumed (as described in the next subsection), and because each reaction replenishes that queue, the queue never empties. The actor never completes its first macrostep and never returns to consume an external event: it is livelocked. The lesson is not that broadcast or `raise/1` are unsafe, but that run-to-completion faithfully executes whatever internal cascade the document describes – including a non-terminating one. A cascade is only well-behaved if it is guaranteed to terminate. A guard that counts rounds and stops raising after a given number would fix the problem.

6.4.3 What “broadcast” means here

The key semantic idea is the *configuration*: the set of active states (and regions) that the machine is currently in. Whenever the interpreter selects an event for processing, that event is matched against transitions in *all* active states in the configuration. In this sense the event is “broadcast”: it is visible everywhere in the current machine, even though only some region(s) react to it. Any internal events raised while handling the current input are processed before the next external message is consumed, so the effects within one reaction are deterministic given the current configuration, the data model, and the selected event.

6.4.4 Why not model each region as a separate actor?

A natural objection is that the Prolog Web already provides actor-level concurrency: why not represent each region as its own actor supervised by a parent, and coordinate by ordinary messaging? This is certainly possible, and it is often the right choice when true independence, failure isolation, distribution, or distinct lifetimes are required.

Orthogonality, however, is not primarily a claim about *parallel execution*; it is a way to specify a *single* reactive controller with an atomic, run-to-completion reaction. In a statechart, one selected event is visible to all active regions, transitions are chosen under a single set of priority and conflict-resolution rules, actions run, and the machine commits one coherent update of configuration and data model. An actor-only encoding can approximate the same effect, but typically only by introducing an explicit coordination protocol (disseminate, barrier, commit) and a coordinator that

reimplements the statechart scheduler in ad hoc code, while also making cross-region invariants harder to maintain under message interleavings.

The broader design trade-off between orthogonal regions and separate actors is taken up again in Section 6.9.2.

6.5 The Web Prolog data model

For many applications, a statechart by itself does not suffice. For example, while statecharts may be able to handle reactive aspects of AI-related applications, proactive aspects such as planning, decision making, and many other forms of problem solving and “intelligent” behaviour must also be catered for. Therefore, sophisticated conversational systems require knowledge representation and reasoning capabilities as well as interactional capabilities. In its current form, statecharts do not provide developers with sufficient means for knowledge representation and reasoning. To enable reactive control to be combined with logical inference, rule-based decision-making, and symbolic computation, Web Prolog can be used for data modelling and scripting.

So far, every transition has reacted to an external event. Statecharts also support *eventless* transitions: transitions without an *on* attribute that may fire whenever their guard condition holds in the current configuration. Combined with a private and dynamic Prolog data model, this gives statecharts a form of forward-chaining inference: the machine repeatedly applies condition-action rules until no more apply, then waits for the next event. Here is a simple example, implementing Euclid’s algorithm for calculating the greatest common divisor of a set of numbers:

```
<statechart datamodel="web-prolog" initial="Run">
  <datamodel>
    :- dynamic int/1.
    int(25). int(10). int(15). int(30).
  </datamodel>
  <state id="Run">
    <go if="int(X), int(Y), X > Y">
      Z is X-Y,
      retract(int(X)),
      assertz(int(Z))
    </go>
    <go to="Stop" if="int(X)">
      writeln(X)
    </go>
  </state>
  <final id="Stop"/>
</statechart>
```

The example illustrates another important piece of statechart semantics: if the children of an active state contain more than one `<go>` element, they are tried in document order. In this case, if the order between the `<go>` elements was switched, the statechart would just write 25 (instead of the correct 5) and then transition to the final state `Stop`, and terminate.

The entire calculation takes place within one macrostep as a cascade of eventless transitions – exactly the kind of cascade that the preceding section warned must terminate. Here termination is guaranteed because each subtraction strictly decreases the sum of the positive integers in the data model. Viewed another way, the example is a small forward-chaining production system: guarded rules repeatedly transform a Prolog knowledge base until no higher-priority rule applies.

6.6 Process invocation and communication

Statecharts become much more useful once they can spawn other processes and *communicate* with them. In the Web Prolog setting, the invoked processes are simply actors: toplevels, other statechart actors, or services running on remote nodes. For example, the statechart below spawns a toplevel, submits a query, prints each answer as it arrives, and terminates when the last answer has been received.

```
<statechart datamodel="web-prolog" initial="spawn-ask-collect">
  <state id="spawn-ask-collect" initial="ask">
    <spawn type="toplevel"
      node="https://n9.org"
      session="false">
      p(a). p(b). p(c).
    </spawn>
    <state id="ask">
      <go to="collect" on="spawned(Pid)">
        toplevel_call(Pid, p(X), [
          limit(1),
          template(X)
        ])
      </go>
    </state>
    <state id="collect">
      <go on="success(Pid,Data,true)">
        writeln(Data),
        toplevel_next(Pid)
      </go>
      <go to="f" on="success(_,Data,false)">
        writeln(Data)
      </go>
    </state>
  </state>
```

```

    </state>
    <final id="f"/>
</statechart>

```

How to read the example.

- The `<spawn>` element requests that a child actor be created after the macrostep in which the surrounding state was entered has stabilised. Here the child is a `toplevel` actor started on the node given by `node="https://n9.org"`. SXML currently supports the types `toplevel` and `actor`.
- The text content of `<spawn>` is Prolog source transferred to the child in the same way as a `load_list/1` option. In this example it defines `p/1` privately in the spawned `toplevel`.
- The internal event `spawned(Pid)` reports the `pid` of the spawned actor. This is an SXML invocation convention, not an answer term from the general actor protocol. The transition in state `ask` reacts by calling `toplevel_call/3`, which initiates query evaluation.
- Each answer arrives as an event of the form `success(Pid, Data, More)`. The option `template(X)` makes the successive slices `[a]`, `[b]`, and `[c]`. While `More` is `true`, the targetless transition requests the next slice with `toplevel_next/1` without exiting and re-entering `collect`. When `More` is `false`, the last answer has been delivered and the machine transitions to the final state.

Note that variables bound by matching the `on` pattern and by evaluating the `if` guard are in scope in the executable content of the same `<go>` element.

Also note that the intent of `<spawn>` is not merely to create a child process, but to tie its lifetime to control state: when the state that contains the invocation is exited, the invoked actor is terminated as part of leaving that state. This is the lifetime-containment discipline of Chapter 4 (Section 4.5.3) surfaced as a control construct: the invoked actor is lifetime-dependent on the invoking state in the same way that a client's spawned actors are lifetime-dependent on their session. This makes it easy to express resource ownership ("this control state owns this helper process") without relying on ad hoc cleanup code.

The example also shows how the protocols of earlier chapters compose with the statechart behaviour. Query evaluation is initiated asynchronously, and the ordinary PTCP answer terms become statechart events. No special statechart-specific remote-call protocol is required.

6.7 The execution semantics of SXML, precisely

The examples above convey how the statechart behaviour works in practice. As with `receive/1-2` in Chapter 2, a few finer points deserve explicit statement.

1. **The execution model is based on SCXML's.** SXML follows the transition-selection and run-to-completion model stated in Section 3.13 and formalised in Appendix D of the SCXML 1.0 Recommendation (Barnett et al., 2015). One selected external event may initiate a *macrostep* consisting of one or more *microsteps*. States are exited from the inside out, transition content is executed in document order, and states are entered from the outside in. Eventless transitions and internally raised events are processed before the actor consumes another external message. SXML instantiates this model with Prolog terms and goals and makes the deviations described in Section 6.9.4.
2. **Events are terms and matching is unification.** At the actor level, the behaviour consumes messages in arrival order with an unconditional receive; selectivity lives in the transition rules rather than in the mailbox. A transition's *on* pattern matches an event by Prolog unification, and bindings produced by the match are in scope in the transition's *if* guard and executable content. An external event that enables no transition in the current configuration is discarded rather than deferred.
3. **Executable content is committed.** The Prolog content of an `<onentry>`, `<onexit>`, or `<go>` element is executed once; alternatives are not exposed through backtracking. If the goal fails, the interpreter places an internal `error/1` event on the queue whose argument is `failure(Goal)`. If it throws an exception `E`, the internal event is `error(E)`. These events may themselves enable transitions and are discarded if no transition matches them.
4. **Guards are once-only tests.** An *if* guard is evaluated once against the private Prolog data model. Ordinary failure means that the transition is not enabled. If evaluating the guard throws an exception `E`, the transition is treated as disabled and `error(E)` is placed on the internal event queue. Guards should be free of side effects. The built-in predicate `in(Id)` succeeds exactly when state `Id` belongs to the active configuration and is useful for coordination across parallel regions.
5. **Final states signal completion.** Entering a `<final>` child of a compound state with identifier `P` raises the internal event `done(P)`. When every region of a parallel state `Q` has reached a final state, `done(Q)` is raised. Entering a top-level final state ends the machine normally after orderly exit processing: exit handlers run for the active states and actors still owned by active `<spawn>` elements are terminated. A monitoring process consequently observes `down(Ref,Pid,true)`. Monitoring is what exposes this termination to another actor; SXML does not automatically translate it into a separate `done.invoke` event.
6. **Invocation follows macrostep completion.** A `<spawn>` request is carried out only after the macrostep in which its state was entered has stabilised. A state that is entered and exited within the same macrostep therefore never spawns its child. Leaving an invoking state terminates the actor it owns. Executable content should not block: a call to `receive/1-2`, a synchronous remote call, or a long `sleep/1` prevents the current macrostep from completing and stops the machine from processing further events. Interaction with other actors should instead be

initiated asynchronously in executable content and completed by handling the resulting messages as events, as in Section 6.6.

6.8 A flat profile of SXML

Not every reactive controller needs Harel’s structuring constructs. A protocol handler, a connection state machine, or a simple turn-taking loop is often a handful of states with no nesting and no concurrency, and for such machines the full statechart vocabulary is overhead rather than help. Rather than offer a second, separate behaviour for this case, we define a *profile* of SXML: a conformance-testable subset of the language that omits the constructs a flat machine does not use.

This is the same move, applied to the statechart language, that Chapter 3 applies to nodes. There, a node profile is a declared subset of node capabilities that makes an offer verifiable and portable. Here, *Flat SXML* is a declared subset of the SXML language: it forbids `<parallel>` (orthogonal regions), `<history>` pseudostates, and compound (nested) states, leaving flat states, the full range of `<go>` transition forms (Table 6.1), executable content, and the data model. A Flat SXML document is, by construction, also a valid full-SXML document; it simply uses none of the restricted constructs. The same interpreter therefore runs both, and a profile-aware engine may implement the subset more cheaply, since it need not maintain configuration sets across parallel regions, broadcast events to multiple active states, or track history.

One thing the flat profile retains is the run-to-completion macrostep. RTC is not a structuring construct but part of the execution model: a flat machine still drains its internal event queue – including events raised by `raise/1` – before consuming the next external message. What the restriction removes is the *broadcast* aspect of RTC, since with no parallel regions there are no simultaneously active states for an event to be visible across. Eventless and guarded transitions, condition-action rules over the data model, and internally raised events all remain available; only hierarchy, history, and orthogonality are given up.

The benefit of expressing this as a profile rather than a separate behaviour is economy. There is one language, one interpreter, and one document syntax; the flat profile is reached by writing within a restricted vocabulary, and conformance to it is as testable as conformance to a node profile. A machine that outgrows the flat profile is not rewritten in a different formalism – it simply begins using constructs the full profile already provides.

6.9 Discussion

This chapter has presented statecharts as an executable actor behaviour in Web Prolog, using an SCXML-inspired document syntax and Prolog as both data model and scripting language. The discussion section briefly situates this design relative to

prior Prolog–SCXML work, clarifies what the behaviour buys compared to a hand-written `receive/1-2` loop, and then positions the result relative to Erlang/OTP’s `gen_statem` and the SCXML Recommendation.

6.9.1 Previous work

The idea of using Prolog as a data modelling and scripting language in a statechart formalism is not new. A proof-of-concept implementation of an SCXML system with Prolog as data model and scripting language was developed by a team at TU Darmstadt in Germany (Radomski et al., 2013). Their focus is multimodal conversational systems, and the paper is well worth reading, since it argues – contrary to what one might initially expect – that SCXML and Prolog form a good combination for developers of such systems. However, we take a slightly different, and, we believe, simpler and more lightweight approach to the interface between the two languages. Our aim is a profile that harmonises with Web Prolog and the architecture of the Prolog Web; in other words, although we try to stay as compatible as possible, our goal is to adapt an SCXML-inspired profile to Web Prolog and the Prolog Web, rather than the other way around. For transparency, the present author served as an invited expert and editor in the W3C working group that produced the SCXML 1.0 Recommendation.

6.9.2 Comparison with an ordinary message loop

At this point it is natural to ask why a dedicated statechart behaviour is introduced at all, rather than writing an ordinary message loop in Web Prolog. Small event-driven controllers are easy to code directly, but practical statecharts rely on Harel’s extensions such as hierarchy, history, and orthogonality, which buy *structure* that is difficult to recover from ad hoc `receive/1-2` code once the control problem grows. Hierarchy and history give interruption and resumption a first-class representation instead of burying it in auxiliary flags and continuations. Parallel regions support modular decomposition *within* one reactive process, and run-to-completion execution yields deterministic coordination between regions: given the same event in the same configuration, the same transitions fire in the same order. Explicit invocation ties the lifetime of spawned processes to control states, so that leaving a state can reliably terminate what it invoked. Finally, the data model makes long-lived state a declarative Prolog object rather than an opaque tuple threaded through callback code, and the same structure that guides execution is directly visualisable, making control logic easier to review, communicate, and debug.

Beyond these structural benefits, the statechart behaviour also changes the *event-handling posture*. In a hand-written `receive/1-2` loop, selection is driven by the current mailbox contents and by pattern order, while non-matching messages typi-

cally remain in the mailbox (effectively deferred). The hand-written state machine in Chapter 2 illustrated why this must be a deliberate choice: one state deferred unknown events, while another discarded them. SXML removes that local ambiguity by applying one uniform rule – an unmatched selected event is discarded.

In SXML, event relevance is determined by the current configuration (the active states and regions): one selected event is matched against enabled transitions across the whole configuration, and irrelevant events are discarded unless deferral is modelled explicitly. Conversely, statecharts provide run-to-completion reactions: one external event may trigger a finite cascade of internal work (condition-only transitions, eventless transitions, and internally raised events) before the next external message is consumed. With orthogonal regions, the selected event is visible across the entire active configuration, enabling deterministic broadcast-style coordination within one actor – something that actor-level messaging can emulate only by explicit fan-out and coordination protocols.

Table 6.1 summarises the most common transition forms in the profile. Seen through the lens of `receive/1-2`, the table can be read as an inventory of control moves that are either awkward or easy to lose track of in an unstructured loop: eventless progress, guarded internal reactions, and targetless event handlers that consume messages without changing state.

A simple rule of thumb follows. A statechart is a good fit when the facets of behaviour are tightly coupled parts of one control process and deterministic, inspectable reactions are desired. Separate actors with ordinary message loops remain the right choice when independent scheduling, failure isolation, distribution, or distinct lifetimes are central concerns.

Another operational difference concerns what failure means. `receive/1-2` commits to one selected message and one receive clause, but the fired body may backtrack through its own choice points and, if it ultimately fails, propagate that failure into the surrounding computation. Failure remains an ordinary Prolog control signal. The statechart behaviour draws the line differently: it commits to one external event and executes a deterministic run-to-completion macrostep. Within that macrostep nothing backtracks into the machine; a failing action becomes the internal event `error(failure(Goal))`. Failure-as-control versus failure-as-event is therefore an important operational boundary between the two behaviours.

6.9.3 Comparison with Erlang's `gen_statem` behaviour

Erlang's `gen_statem` behaviour offers a disciplined way to structure long-running reactive processes: the meaning of an incoming event depends on the current state, and a transition may update state data, schedule timeouts, or emit new events. In that sense, `gen_statem` and our statechart behaviour address the same engineering problem: how to express event-driven control in a maintainable way without hand-written receive loops.

Transition form	Typical use / notes
<code><go to="s" on="e">...</go></code>	Triggered state change. The ordinary event-driven case: on event pattern <i>e</i> , transition to <i>s</i> and run actions.
<code><go to="s" on="e" if="c">...</go></code>	Triggered and guarded state change. The standard ECA pattern: a matching event plus a condition enables the transition; convenient when several transitions compete on the same event.
<code><go on="e" if="c">...</go></code>	Guarded, targetless event reaction. Consumes events matching <i>e</i> , checks <i>c</i> , runs actions, and stays in the same configuration. Useful for logging, emitting output, raising internal events, or updating the data model without a state change.
<code><go to="s">...</go></code>	Eventless, unconditional state change. Useful for automatic progress (e.g. completion-style transitions) or compact control-flow steps. Use with care, since chains of eventless transitions may create long run-to-completion cascades.
<code><go to="s" if="c">...</go></code>	Eventless but guarded state change. Useful for decision points driven by the data model, or for encoding automatic progress only when a condition holds.
<code><go if="c">...</go></code>	Guarded, targetless reaction (no control-state change). Useful as a condition-action rule that updates the data model, emits output, raises an internal event, logs, etc., while staying in the same configuration.

Table 6.1 Common `<go>` transition forms and typical uses in the statechart profile.

The statechart behaviour differs in where it places complexity. First, it makes the control structure explicit and inspectable as a hierarchy of states rather than as callback code, which improves readability and supports direct visualisation. Second, it supports hierarchical states, history, and parallel regions as first-class control constructs, whereas `gen_statem` leaves such structure to be encoded by conventions in the callback logic. Third, the statechart run-to-completion macrostep provides deterministic intra-machine coordination (including broadcast effects across parallel regions under fixed priority rules), while Erlang's coordination across processes is intentionally mailbox-driven and concurrent. Finally, statecharts integrate process invocation as a control construct with a precise lifetime rule (invoked processes are tied to the state that invoked them), whereas `gen_statem` relies on external OTP patterns (supervision, linking, monitoring) to manage process lifetimes.

Two `gen_statem` facilities also mark genuine advantages over the present SXML profile. State timeouts provide cancel-on-state-change timing, while the `postpone` action explicitly defers an event for reconsideration after a state change. SXML currently has neither a timed-transition construct nor implicit event deferral: unmatched events are discarded, and applications that need timing or deferral must model it

explicitly. A state-scoped timed transition is a natural future extension, but it is not part of the profile defined here.

This comparison also clarifies what the SXML answer to `gen_statem` actually is. The contrast above is between *full* SXML – with hierarchy, history, and orthogonality – and a flat callback behaviour, and on that comparison the two look like different tools for different jobs. But SXML does not force the full vocabulary on every machine. The flat profile of Section 6.8 is precisely the flat, callback-style state machine that `gen_statem` provides: a small set of states, transitions as ordinary executable content, no Harel constructs. The difference from Erlang is instructive. Erlang offers `gen_statem` for flat machines and leaves hierarchical structure to be encoded by convention or reached for in heavier external libraries; SXML offers a single language in which the flat machine is a *profile* and the hierarchical machine is the *full* language, with a document moving from one to the other simply by using more of the vocabulary. The flat profile is therefore not an additional behaviour competing with the statechart behaviour, but the same language restricted – which is the leaner, lower-overhead choice when the structuring constructs are not needed, and a continuous path to them when they are.

6.9.4 Relationship to SCXML

SXML is intentionally SCXML-inspired, but it is not a full SCXML surface language. Its primary goal is to present statecharts as an executable actor behaviour in Web Prolog, with Prolog as both data model and executable content. For that reason we aim to implement the core of the SCXML 1.0 processing model, while adopting a smaller and more uniform set of constructs that harmonise with actor-style messaging and with Prolog scripting.

Several SCXML constructs are omitted because they are either platform-specific (for example, ECMAScript-oriented scripting facilities), oriented toward browser integration rather than actor systems, or redundant when executable content is restricted to Web Prolog goals. In particular, we do not need a dedicated `<script>` element, since executable content is expressed directly as Prolog code. Likewise, communication with external actors is handled by Web Prolog predicates such as `!/2` (and derived predicates such as `output/1` and `toplevel_call/2-3`), rather than by reproducing the full generality of SCXML's `<send>` element. Moreover, our `<spawn>` element corresponds to SCXML's `<invoke>` but is adapted for actor-style invocation rather than the SCXML service model.

SXML also renames a small number of constructs for readability in Prolog-centric examples: `<transition>` becomes `<go>`, and the attributes `target`, `event`, and `cond` are rendered as `to`, `on`, and `if`. These are surface changes; the intent is to make examples compact and to emphasise that transitions function as rules governing event-driven evolution of the actor.

Three further differences are semantic rather than notational. First, SCXML matches dotted event names by prefix against event descriptors, whereas SXML

matches Prolog terms by unification and carries the resulting bindings into guards and actions. Second, SXML has no SCXML `_event` record: the structure and data of an event are represented directly by the term and the bindings of the `on` pattern. Third, SCXML's `<donedata>` and automatic `done.invoke` protocol are omitted. SXML uses explicit actor messages, internal `done(Id)` events, and ordinary monitoring instead. What remains central is SCXML's transition-selection and run-to-completion model, instantiated by the rules of Section 6.7.

6.9.5 Toward an actor-oriented successor to SCXML

Seen as a language for programming reactive actors, SCXML's core ideas are compelling, but the Web Prolog setting suggests a slightly different "centre of gravity": a declarative data model, a single uniform scripting language and an explicit integration with actor messaging. The statechart behaviour in this chapter can therefore be read as a concrete sketch of what an actor-oriented successor might prioritise: the standard statechart execution model, combined with Prolog-level reasoning and with explicit, programmable interaction with the surrounding actor system. What makes the proposal more than a Prolog convenience is that the same shift applies to other host languages: the state-machine core is genuinely orthogonal to the scripting layer, and trusting that layer to carry more – communication, data modelling, scripting – is a direction a successor could take whether the host language is Prolog or ECMAScript. We develop this argument, and its implications for the SCXML lineage, in Chapter 11.

6.10 Summary

This chapter introduced the statechart behaviour: an actor behaviour in which control is expressed as a hierarchical, event-driven state machine with parallel regions, deterministic transition selection under run-to-completion execution, and executable actions written in Web Prolog. Statechart actors can be spawned and supervised like ordinary actors, can communicate via messages, and can spawn child actors with well-defined lifetimes tied to control states. SXML retains SCXML's central execution model while replacing dotted event names with Prolog terms matched by unification and integrating action failure, final-state completion, and invocation with the actor protocols of earlier chapters. The result is an SCXML-inspired but Prolog-centric profile that supports reactive control, structured coordination within one actor via configuration-wide event visibility, and integration of long-lived knowledge and decision logic through a private declarative Prolog data model. Timed transitions and automatic event postponement remain outside the present profile.

Part III
The broader technological contexts

Chapter 7

The Prolog Web $\approx$ the Semantic Web?

Since the Prolog Web adds logic programming and automated reasoning to the conventional Web, it is natural to ask whether we are approaching something like the *Semantic Web* – the machine-readable Web vision articulated by (Berners-Lee et al., 2001). The gist of their proposal is captured by the following three statements:

The Semantic Web is an extension of the current web in which information is given well-defined meaning, better enabling computers and people to work in cooperation.

For the semantic web to function, computers must have access to structured collections of information and sets of inference rules that they can use to conduct automated reasoning.

Adding logic to the Web – the means to use rules to make inferences, choose courses of action and answer questions – is the task before the Semantic Web community at the moment.

At first glance, words and phrases such as *logic*, *inference rules*, and *answer questions* could be read as an informal description of what Prolog already does. Likewise, the characterisation as an extension of the current Web fits the Prolog Web rather well. This invites a tempting but slightly misleading conclusion: perhaps the Prolog Web is simply a simpler, “poor man’s” Semantic Web?

The resemblance is real, but it is a resemblance of ingredients rather than of engineering goals. The Semantic Web focuses on semantic interoperability by standardising shared meaning – data models, identifiers, vocabularies, and ontology semantics that many independent parties can agree on. The Prolog Web focuses on execution by standardising an executable substrate – a uniform rule language plus a node and process architecture in which predicates can be deployed as running services and agents. Both concern interoperable representations, but the Semantic Web foregrounds representation and shared meaning, whereas the Prolog Web foregrounds turning knowledge into executable behaviour.

The thesis of this chapter is therefore cooperative rather than competitive: the two are best viewed as complementary. In practice, Web Prolog can consume and produce RDF-style data, reuse established vocabularies and identifiers, and apply Prolog’s execution model to “put the data to work” as queries, rules, and long-lived agent behaviour.

The integration patterns described below are architectural patterns, not additional standard Web Prolog APIs. The Web Prolog profiles do not prescribe an RDF store, a SPARQL client, or an OWL reasoner, and the demonstrator does not provide standard

predicates for them. A node owner may supply such capabilities through trusted host libraries and expose them as ordinary predicates or services. Keeping this boundary explicit lets us discuss cooperation without silently enlarging the language defined in the preceding chapters.

Although the Semantic Web serves as our principal reference point, the underlying distinction between interoperable representation and executable computation applies more broadly to knowledge graphs, property graphs, and typed data APIs.

This cooperative framing also speaks to a difficulty that the Semantic Web community has increasingly acknowledged. Verborgh and Vander Sande (2020) observes that as Web development shifted toward front-end engineering, Semantic Web technologies remained largely inaccessible to the developers now building most applications – which helps explain the rise of simpler, more approachable stacks such as GraphQL. His diagnosis is pointed: the GraphQL community, having prided itself on simplicity, is gradually rediscovering the problems that SPARQL federation research addressed years ago, since queries with purely local semantics break down once data is drawn from multiple sources. Rather than reusing those lessons, it is reinventing alternatives to ontologies and federation. Verborgh’s conclusion is one we take seriously here: bringing SPARQL-level expressivity within reach of front-end developers is itself an unsolved research problem, not merely an engineering inconvenience.

We read this as further reason to treat the two stacks as complementary rather than competing. If expressive interoperable querying is hard to make accessible, then a substrate that lets application logic be written as ordinary, composable rules close to the data – and exposed through web-native APIs that front-end code can consume directly – is addressing exactly the gap Verborgh identifies. We return to this developer-facing question in Section 7.8.

7.1 The Semantic Web tower

The Semantic Web is often presented as a layered architecture, where each layer builds on the layers below it. The lower layers are shared with the conventional Web, while the upper layers introduce machine-processable representations of data, query mechanisms, and ontology-level semantics. A commonly used illustration is the so-called *Semantic Web tower*, one variant of which is shown in Figure 7.1.

The diagram is a historical snapshot rather than a current inventory of standards. In particular, SWRL remained a 2004 W3C Member Submission,¹ whereas the RIF family later produced W3C Recommendations, including the Core, Basic Logic, and Production Rules dialects.²

The tower diagram is necessarily impressionistic, but it provides a useful scaffold for our comparison. At the bottom, URIs play the familiar role of locating resources (as URLs), but on the Semantic Web they also serve as global names for entities, relations and concepts (as IRIs). The Prolog Web uses URIs operationally to locate

¹ <https://www.w3.org/Submission/SWRL/>

² <https://www.w3.org/TR/rif-overview/>

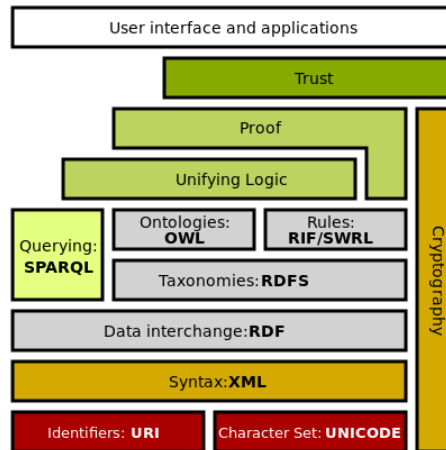


Fig. 7.1 One historical variant of the Semantic Web tower.

nodes and source; globally qualified pids then combine a process identifier with the URI of its node.

Unicode can likewise be viewed as a naming layer, ensuring that text and identifiers can be represented consistently across languages. Not all Prolog systems fully support Unicode, but a web-facing Prolog system effectively has to.

Many variants of the tower place XML at the syntactic level. In this chapter we treat XML mainly as a data interchange format among several. In the Prolog Web setting, Prolog terms provide a natural internal representation, and JSON is often the most convenient interchange format for client-facing APIs, while RDF serializations such as RDF/XML and Turtle remain important for interoperability with existing Semantic Web tooling.

7.2 RDF as `rdf/3`

Right above the XML level of the historical Semantic Web tower we find the levels devoted to semantics proper, where RDF forms the basis. RDF represents information as subject–predicate–object triples. More complex structures are built by connecting resources and blank nodes through multiple triples. Reification has a narrower purpose: it allows an RDF graph to describe a particular triple token, but does not itself assert the triple being described.³

From a Prolog point of view, the triple pattern maps naturally to a three-argument relation. The SWI-Prolog Semantic Web libraries and ClioPatria therefore use `rdf/3` as their elementary query predicate (Wielemaker et al., 2016). Current SWI-Prolog applications normally use `library(semweb/rdf11)`, whose literal notation follows

³ <https://www.w3.org/TR/rdf11-mt/#reification>

RDF 1.1 more closely than the older `literal(type(...))` representation. The following self-contained fragment creates a small example store:⁴

```
:- use_module(library(semweb/rdf11)).

:- rdf_register_prefix(ex,      'http://example.org/').
:- rdf_register_prefix(foaf,    'http://xmlns.com/foaf/0.1/').
:- rdf_register_prefix(schema, 'http://schema.org/').

load_example :-
    rdf_assert(ex:bob, rdf:type, foaf:'Person'),
    rdf_assert(ex:bob, foaf:knows, ex:alice),
    rdf_assert(ex:bob, foaf:knows, ex:john),
    rdf_assert(ex:alice, schema:birthDate,
               date(1990,7,4)^xsd:date),
    rdf_assert(ex:john, schema:birthDate,
               date(1992,9,6)^xsd:date).
```

Once these triples have been loaded, ordinary Prolog clauses can define a task-specific interface to the store. For example, the following relation finds the birth dates of people somebody knows:

```
known_with_birthdate(Me, Person, Date) :-
    rdf(Me, foaf:knows, Person),
    rdf(Person, schema:birthDate, Date).
```

Prefixes registered with `rdf_register_prefix/2` allow IRIs to be written in the compact form `Prefix:LocalName`; the calls to `rdf/3` still operate on the corresponding full IRIs. A similar namespace mechanism would be valuable in Web Prolog, both for readability and for disciplined use of interoperable identifiers. It is important, however, to keep the layers distinct: `rdf/3` and the prefix declarations above are supplied by the SWI-Prolog host library, not by the standard Web Prolog predicate set. On a public node they would normally be installed by the owner and exposed through selected predicates rather than loaded by an untrusted client.

7.3 SPARQL-style querying in Prolog

Instead of using SPARQL, querying Semantic Web data represented as above can be done in Prolog. The interactive session below illustrates the use of `known_with_birthdate/3` to query the triple store:

```
?- known_with_birthdate(Me, He, Date).
Me = 'http://example.org/bob',
```

⁴ The data is adapted from the RDF 1.1 Primer: <https://www.w3.org/TR/rdf11-primer/>.

```

He = 'http://example.org/alice',
Date = date(1990, 7, 4)^^
      'http://www.w3.org/2001/XMLSchema#date' ;
Me = 'http://example.org/bob',
He = 'http://example.org/john',
Date = date(1992, 9, 6)^^
      'http://www.w3.org/2001/XMLSchema#date'.
?-

```

The key point is modularity. Wielemaker et al. (2016) argue that the Prolog version using `known_with_birthdate/3` has several advantages over the SPARQL version. First, because the Prolog version has an explicit name, it is easy to reuse it in other queries. This allows developers to incrementally build more sophisticated functionality on top of existing, proven and tested building blocks. This can greatly simplify maintenance of complex queries.

Secondly, application logic written in Prolog is not confined to the filters and functions standardised by SPARQL. Any available predicate can serve as a guard, transformation, or derived relation at any point in the clause, and the same definition can combine RDF triples with other relations – for example, views backed by an RDBMS or calls into domain-specific code.

The query above is simple, but more complex relational queries are possible too. An ISOBASE node guarantees the usual Prolog control constructs, including conjunction, disjunction, and Negation As Failure, together with the ISO all-solutions predicates `bagof/3`, `setof/3`, and `findall/3`. SWI-based nodes may additionally expose `library(aggregate)`,⁵ which provides operators such as `count`, `sum`, `min`, and `max`. These library predicates are useful implementation facilities, but are not guaranteed by the Web Prolog ISOBASE profile.

The same qualification applies to the SWI-Prolog library `solution_sequences`,⁶ which was developed to support direct formulations of many database-style queries and provides `distinct/1-2`, `limit/2`, `order_by/2`, and related predicates. On a node whose owner has made these two libraries available, consider the following relation `r/2`:

```

:- use_module(library(aggregate)).
:- use_module(library(solution_sequences)).

r(a, foo).  r(a, bar).  r(a, bar).  r(c, bar).  r(c, bar).
r(b, bar).  r(a, foo).  r(d, baz).  r(a, foo).  r(c, baz).

```

Here is an example of how a SPARQL-like query can be formulated using such predicates, and what the solutions would look like:

```

?- limit(10,
      order_by([desc(C)],

```

⁵ <https://www.swi-prolog.org/pldoc/man?section=aggregate>

⁶ <https://www.swi-prolog.org/pldoc/man?section=solutionsequences>

```

                                aggregate(count, X, r(X,Y), C))).
C = 3, Y = bar ;
C = 2, Y = baz ;
C = 1, Y = foo.
?-

```

The call `aggregate(count, X, r(X,Y), C)` groups on the free variable `Y`. Its discriminator `X` removes duplicate `X` values within each group, so the result corresponds to `COUNT(DISTINCT X)` rather than a count of all matching clauses. The call to `order_by/2` sorts the resulting `Y-C` pairs by descending `C`, and `limit/2` truncates the sequence – conceptually similar to a SPARQL query with `GROUP BY`, `ORDER BY`, `COUNT(DISTINCT ...)`, and `LIMIT`.

The resemblance should not be mistaken for semantic identity. SPARQL has defined multiset, unbound-variable, error, and negation semantics that do not coincide in every case with Prolog’s solution sequences and Negation As Failure. The example illustrates how familiar query operations can be composed in Prolog; it is not a general semantics-preserving translation of arbitrary SPARQL.

If the owner has exposed `r/2`, `aggregate/4`, and `order_by/2` on a remote SWI-based node, the same goal can be submitted through `rpc/3`:

```

?- Goal = order_by([desc(C)], aggregate(count,X,r(X,Y),C)),
   rpc('http://n7.org', Goal, [limit(10)]).

```

The `limit(10)` option bounds the solutions returned through the RPC interface; it does not prevent `order_by/2` from collecting and sorting its complete input on the remote node. It is therefore a transport-level bound, not necessarily a computational optimisation.

An ISOBASE node is sufficient for this stateless call. ISOBASE also permits source to accompany a call through options such as `load_text/1` and `load_uri/1`. ISOTOPE becomes necessary when definitions or data must persist across a sequence of requests in a toplevel session. None of these profile guarantees includes the SWI-specific RDF or solution-sequence libraries; those remain owner-provided capabilities.

The bottom line is that few mainstream general-purpose languages offer an interface to RDF that is as direct and idiomatic as Prolog’s: triples map to relations, and rule layers can be expressed in the same language as the queries that consume them.

Wielemaker et al. (2016) summarise the position we adopt:

We do not consider SPARQL adequate for creating rich semantic web applications. SPARQL often needs additional application logic that is located near the data to provide a task-specific API that drives the user interface. Locating this logic near the data is required to avoid protocol and latency overhead. RDF-based application logic is a perfect match for Prolog and the RDF data is much easier queried through the Prolog RDF libraries than through SPARQL.

This is precisely the role the Prolog Web embraces: Prolog predicates become the task-specific API layer that lives near the data, while remaining compositional, testable, and reusable as ordinary code.

7.4 OWL as an external capability

Moving up the Semantic Web tower, we reach OWL, a family of ontology languages based on description logics. OWL's purpose is not merely to provide more rules, but to support a particular kind of interoperability: shared vocabularies with explicit semantic commitments – class hierarchies, property characteristics, domain and range axioms, cardinality restrictions, and so on – that can be processed by standard OWL *reasoners*. This matters whenever independent producers and consumers of data need to agree not only on identifiers, but on the intended interpretation of those identifiers.

The Semantic Web community makes a clear distinction between knowledge representation languages (such as RDF, RDFS) and query languages (such as SPARQL). As we have seen, Prolog (including Web Prolog) can, to some extent, fill both roles – but with different semantic commitments. Web Prolog is most naturally used as an *executable rule layer*: named predicates encode task-specific queries and derived relations, composable and testable as ordinary code. OWL, by contrast, targets standardisable ontology semantics under open-world assumptions and decidable reasoning fragments, enabling interoperability through shared class and property commitments rather than shared procedural conventions. Prolog therefore excels at turning relational data into reusable executable services; OWL excels at making ontology-level meaning explicit in a form that independent tools and parties can agree on. The Prolog Web is not “an OWL replacement,” and should not be presented as one. Web Prolog is grounded in Horn clause logic and, in typical use, relies on closed-world idioms including Negation As Failure. The OWL 2 DL language and OWL profiles provide standard, decidable fragments under open-world assumptions, supporting entailment and consistency reasoning that plain Horn clauses cannot reproduce directly without changing the semantics or moving to richer logic programming extensions.

OWL also makes no unique-name assumption: two different IRIs may denote the same individual unless their distinctness is stated explicitly.⁷ This differs from ordinary Prolog term semantics, in which distinct ground terms do not unify. The contrast therefore concerns both what may be inferred from missing information and how identity is treated.

Ontology reasoning should also be distinguished from data validation. Under open-world semantics, a missing value does not normally constitute a violation. Applications that need to validate an RDF graph against structural conditions can use a shapes language such as SHACL,⁸ again treating the corresponding validator as a specialised external capability.

This does not make the Prolog Web irrelevant to ontology-heavy applications – it merely suggests a division of labour. When OWL is the right tool for defining a shared ontology, Web Prolog can still play at least three valuable roles: *consuming* OWL-derived results (such as classifications from an external reasoner) and combining

⁷ <https://www.w3.org/TR/owl2-primer/>

⁸ <https://www.w3.org/TR/shacl/>

them with application-specific rules; *mediating* between heterogeneous sources by translating Semantic Web representations into executable relations exposed through the Prolog Web APIs; and providing an *agent substrate* in which long-lived actors use ontology-backed knowledge as one input among many while the agent logic itself remains straightforward Prolog.

Ontologies are thus one of the places where the Semantic Web stack contributes something genuinely distinct: not just inference, but shared semantic commitments and a mature ecosystem of reasoners and tooling. The Prolog Web complements this by making it easier to turn ontology-backed data into running services, executable behaviour, and web-native agents.

7.5 Positioning the Prolog Web alongside the Semantic Web

The preceding sections have examined individual layers of the Semantic Web tower and shown how each relates to the Prolog Web. It is now worth stepping back to compare the two ecosystems as wholes. Figure 7.2 places the Prolog Trinity alongside what, by analogy, might be called a “Semantic Trinity”: a comparable ecosystem assembled from Semantic Web technologies – RDF and SPARQL for data and querying, and a general-purpose language such as Java for application logic (and possibly OWL for ontology-level semantics, and an external reasoner for inference).

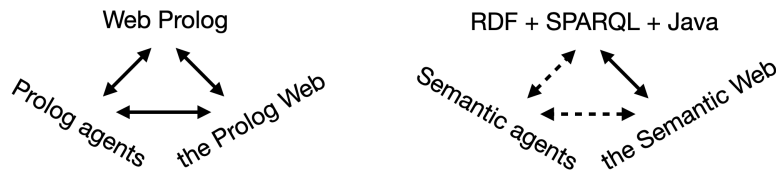


Fig. 7.2 Comparing the Prolog Trinity with a “Semantic Trinity.” Solid arrows indicate tight integration; dashed arrows indicate looser coupling.

The comparison is schematic. Integrated Semantic Web platforms certainly exist; the dashed arrows describe boundaries between independently standardised layers, not the quality of every implementation. Two differences nevertheless stand out. The first is integration. In the Prolog Trinity the arrows are solid, suggesting that the same language is used for knowledge representation, querying, rule-based computation, and agent scripting, and the execution substrate (nodes, actors, APIs) is part of the proposal rather than an afterthought. In the Semantic Trinity, the data layer (RDF), query layer (SPARQL), and application layer (Java or another host language) have distinct interfaces, and an application framework connects them.

The second difference is architectural. The Semantic Web standards specify representations and query protocols, but they do not prescribe a process infrastructure: there is no standard notion of a Semantic Web node, no standard agent lifecycle,

and no standard way to host long-lived services that maintain state and coordinate over time. These concerns are left to the chosen application framework. The Prolog Web, by contrast, includes an architecture as part of the design. Nodes serve answers over standard web protocols, calls compose local and remote resources, source can accompany calls and spawned processes, and actors coordinate over time. This is what makes the Prolog Web a *platform* for executable knowledge rather than merely a representation format.

These observations suggest a pragmatic positioning. Semantic Web standards provide interoperable representations and shared semantic commitments, but rich applications still require a host language to implement task-specific APIs, workflows, and integration with heterogeneous data sources. A common assumption is that any general-purpose language will do. In practice, however, there is often an impedance mismatch between graph-shaped, relational data and imperative application code (Wielemaker et al., 2016). This is where Prolog has an unusually good fit. RDF triples map directly to relations, and rule layers can be expressed in the same language as the queries that consume them. Application logic that would otherwise live as opaque client-side code or as ad hoc service glue can instead be expressed as named, reusable predicates close to the data. This echoes the earlier argument by Wielemaker et al. (2016).

From the Prolog Web perspective, then, Web Prolog can act as a programming and agent layer around Semantic Web assets: consuming RDF and SPARQL endpoints when appropriate, delegating ontology-level reasoning to OWL reasoners when needed, and exposing the resulting functionality as web-native services hosted on Prolog Web nodes. The next section makes this complementarity concrete by outlining three integration patterns that recur in practice.

7.6 Cooperation patterns

The complementary roles identified above lead to three recurring integration patterns, referred to as A, B, and C below.

7.6.1 A: Web Prolog as a SPARQL client and post-processor

In many applications the most pragmatic role for Web Prolog is to act as a consumer of Semantic Web data offered by existing endpoints. A Web Prolog program issues a SPARQL query to a remote endpoint, parses the result, and then applies ordinary Prolog predicates for filtering, aggregation, explanation, or task-specific inference. The important point is that the *logic that makes the results useful* is written as reusable predicates rather than being forced into a single monolithic SPARQL query.

Conceptually, the pattern looks like this:

1. Query a SPARQL endpoint and obtain a sequence or table of bindings.

2. Convert the bindings to a relational representation.
3. Run Prolog rules over that representation and expose the result as a Prolog Web service.

The first step is performed by an owner-installed library predicate or service that implements HTTP and one of the SPARQL result formats. The remaining steps are plain relational programming. If the application is interactive, the derived predicate can be offered through the Prolog Web APIs, allowing clients to consume its Prolog solutions incrementally. This does not imply that the underlying SPARQL endpoint is itself consumed lazily: an adapter may fetch or buffer its complete result before the Prolog relation begins producing answers.

A useful boundary case is to place a RELATION adapter directly in front of a SPARQL endpoint. The adapter advertises a fixed relation such as `rdf/3`, translates calls to that relation into SPARQL triple-pattern queries, and returns the resulting bindings through the stateless HTTP API. A client can then write:

```
?- rpc('http://sw-node.org', rdf(S, P, O)).
```

Here `rpc/2` is a client-side interface to the stateless call; it does not imply that the target accepts arbitrary ISOBASE goals. As stressed in Chapter 3, a RELATION node need not contain a Prolog engine at all. The adapter is conformant because it exposes only its advertised relation and behaves externally as a relational endpoint. In this way an existing SPARQL service can become a Prolog Web citizen without changing its storage or query engine.

7.6.2 B: RDF as `rdf/3`, Prolog rules as the API

A second pattern is to treat RDF as a storage and interchange format, but to treat Prolog as the *programming interface* to that data. In this pattern, RDF triples are ingested into a node-local indexed store queried through `rdf/3`, while the public surface consists of named Prolog predicates that encode the relevant domain.

The advantage over exposing a SPARQL endpoint is not merely syntactic. Named predicates become stable, reusable building blocks, and they can be composed with arbitrary auxiliary relations, foreign data sources, and task-specific computations. The service remains readable, testable, and evolvable: if a predicate is refactored, callers can remain unchanged as long as the predicate interface is preserved.

In a Prolog Web setting, a fixed collection of such predicates can be advertised through a RELATION node. An ISOBASE or stronger node may additionally accept general calls through `rpc/2-3`, while an ISOTOPE or ACTOR node can support longer conversations. The result is a service that is recognisably “Semantic Web” in the sense that it is grounded in RDF identifiers and vocabularies, but “Prolog Web” in the sense that its interface consists of executable relations.

A minimal bridge example

To make the idea more concrete, suppose a node maintains a local RDF store and provides the above derived predicate `known_with_birthdate/3`. A client can then query this service lazily using `rpc/3`, taking advantage of the Prolog Web's support for bounded slices of answers:

```
?- Goal = known_with_birthdate(Me, He, Date),
   rpc('http://n7.org', Goal, [limit(10)]).
```

The point of the example is not the particular relation, but the division of labour. The use of established RDF identifiers and vocabularies makes the underlying data interoperable with Semantic Web tooling, while the exposed predicate provides an application-friendly interface that can be maintained and composed like ordinary Prolog code. If more logic is needed, it can be layered on top of `known_with_birthdate/3` rather than being duplicated across multiple SPARQL queries.

7.6.3 C: OWL reasoning as a service, Web Prolog for orchestration

A third pattern is to treat OWL reasoning as an external capability rather than as something to be reimplemented inside Web Prolog. This is often the most realistic approach when an application depends on ontology-level commitments such as class subsumption and property characteristics that are supported by mature OWL reasoners.

In this pattern, OWL plays the role of *semantic backbone*, while Web Prolog plays the role of *executable glue*:

1. An OWL reasoner (local or remote) is used to compute classifications, entailments, or consistency checks.
2. The computed results are imported into Web Prolog as ordinary relations (e.g. `class/2`, `subclass/2`, `entailed/1`, or additional `rdf/3` facts).
3. Prolog rules and, if needed, long-lived actors use these relations as inputs to application-specific decisions, workflows, or interactive services.

This arrangement respects the strengths of both stacks. OWL reasoners provide well-understood semantics and tool support for ontology-level inference, while Web Prolog provides a convenient substrate for integrating that inference with other sources of information, additional rule layers, network calls, and agent-style control logic. Where RDF data must also be validated, a SHACL processor can be integrated in the same way. The resulting system is typically easier to engineer than an attempt to force all application logic into SPARQL and OWL alone.

7.6.4 Summary

Pattern A consumes and post-processes existing endpoints. Pattern B places a named Prolog rule layer over an RDF store. Pattern C delegates specialised ontology reasoning or validation while Web Prolog orchestrates the application. In every case, Semantic Web assets retain their standard meanings and Web Prolog supplies the executable service and process layer.

7.7 Proof and trust

The upper layers of the Semantic Web tower are often labelled *Proof* and *Trust*. The labels point to real concerns, but the standards themselves do not prescribe a single, broadly adopted mechanism for either.

For the Prolog Web it is useful to read “trust” in a pragmatic engineering sense. A simple starting point is transparency: a node can let clients inspect the Web Prolog source code behind a published predicate or service, for example through a browsing facility, and optionally accompany it with version identifiers, digital signatures, or provenance metadata. Source visibility does not establish correctness, and a signature authenticates provenance rather than truth. These mechanisms nevertheless support auditability, reproducibility, and accountability – which is often what developers actually need.

This stance is also operationally natural in a Prolog setting. A node need not publish only answers; it can also publish the predicate definitions and provenance information required to reconstruct how those answers were obtained. Chapter 8 develops the idea further by showing how proof trees can be collected and inspected as a basis for explainable reasoning in Prolog agents.

7.8 User interfaces and applications

The cooperation patterns above are easiest to exploit in the familiar web application architecture: a JavaScript front end plus a back end that exposes task-specific predicates as services.

Semantic Web research has long emphasised the importance of user interfaces (Hachey and Gasevic, 2012). The Prolog Web perspective is pragmatic: conventional Web front ends are typically implemented in HTML, CSS and JavaScript, while Web Prolog services provide a relational and rule-based back end accessed over HTTP or WebSocket. This arrangement is relevant to both approaches discussed in this chapter: a JavaScript UI can talk to Web Prolog services that in turn query RDF stores, SPARQL endpoints, or OWL reasoners, effectively using Web Prolog as an application-logic layer around Semantic Web assets.

The demonstrator already takes the further step of running Web Prolog in the browser through SWI-WASM. Its JavaScript bridge connects the browser-resident runtime to remote calls and actor messaging; a stable, application-facing JavaScript API over the same primitives would be a natural extension. This is a partial answer to the accessibility gap noted at the start of the chapter: front-end code can reach expressive, rule-based services through a small call-and-message interface rather than embedding all federation and inference logic in the UI. It does not dissolve Verborgh’s research problem – making interoperable querying genuinely easy remains hard – but it lowers the entry cost at the point where, in practice, most application code is written.

This is also where agents enter the picture in a concrete, engineering sense. Some agents are *UI agents*: long-lived client-side processes that maintain conversational or task state, trigger background queries, coordinate multiple services, and present results incrementally rather than as a single request–response transaction. In a conventional stack, such behaviour is implemented ad hoc in JavaScript and application glue. In the Prolog Web, the same interaction model can be carried across the client–server boundary: UI agents and back-end services can be written against the same set of message-passing and call abstractions, with the agent logic expressed as stateful actors that communicate by messages, consume RDF/OWL-backed knowledge when relevant, and expose their functionality through web-native APIs. We develop this agent perspective and the disciplined control structures needed to keep it predictable in Chapter 8.

7.9 Related work: towers of logic programming

As noted above, many variants of the Semantic Web tower have been proposed, and some align more closely with logic programming than others. The two-tower architecture of Horrocks et al. (2005), reproduced in Figure 7.3, contrasts two plausible instantiations of a layered Web stack.

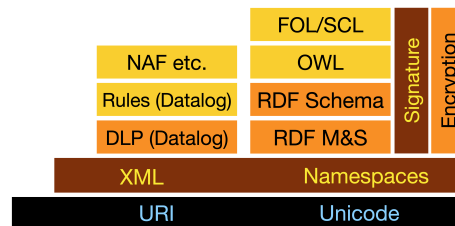


Fig. 7.3 The two-tower architecture, with the tower proposed in (Horrocks et al., 2005) to the left and a simplified version of the “original” tower to the right.

The right-hand tower captures essentially the same idea as Figure 7.1. The left-hand tower, by contrast, is more explicitly grounded in the logic programming paradigm.

Here DLP denotes Description Logic Programs, a fragment designed to connect Datalog-style rule reasoning with part of the ontological expressiveness found in OWL. The later OWL 2 RL profile is especially relevant: it is a standardised, rule-oriented subset of OWL 2 designed for rule-based implementations and explicitly inspired by DLP and pD*.⁹ Above DLP, one can formulate rules and enrich them with features such as Negation As Failure. The “etc” in the diagram is not mere hand-waving: it points to the broad family of logic programming formalisms surveyed in Section 2.1, each with its own semantic foundations and practical strengths.

The key observation is that this left-hand tower is not a single monolithic stack but a family of possible stacks, each grounded in a different fragment or extension of logic programming. The Prolog Web’s profile hierarchy (Chapter 3) provides a common interaction envelope for this diversity. A Datalog engine may expose fixed relations through the RELATION profile or a broader goal language through ISOBASE. A WFS-capable or probabilistic system can likewise remain addressable through standard Web Prolog protocols.

The profile name alone does not advertise the engine’s entailment regime, termination properties, treatment of negation, or probabilistic semantics. Those logical commitments require separate service metadata. The profiles instead say how a client may interact with the node – as a relation endpoint, a stateless Prolog evaluator, a persistent toplevel, or an actor system – while the protocol layer provides the common ground.

In this reading, the left-hand tower of Horrocks et al. (2005) maps directly onto the architecture described in earlier chapters. The broader vision – a Logic Programming Web rather than merely a Prolog Web – is taken up again in Chapter 10, where we consider what this architecture offers to the wider LP community.

7.10 Outlook: Prolog Trinity towers

Where does Web Prolog fit? One view places it as middleware between a conventional HTML, CSS, and JavaScript application layer and the Semantic Web stack beneath it, as suggested in Figure 7.4.

The same kind of arrangement can be built on top of the second tower, as suggested in Figure 7.5.

Or it might be regarded as a *third* tower, independent but interoperable with the other towers. Figure 7.6 suggests this option.

These pictures are complementary views rather than mutually exclusive implementation plans. Pattern B often makes Web Prolog look like middleware over an RDF store, whereas a network of independent Prolog Web nodes is better captured by the third-tower view. The latter is therefore the most faithful architectural picture of the proposal in this book: an independent but interoperable execution stack, able to sit beside Semantic Web stacks and to incorporate their assets when useful.

⁹ <https://www.w3.org/TR/owl2-profiles/>

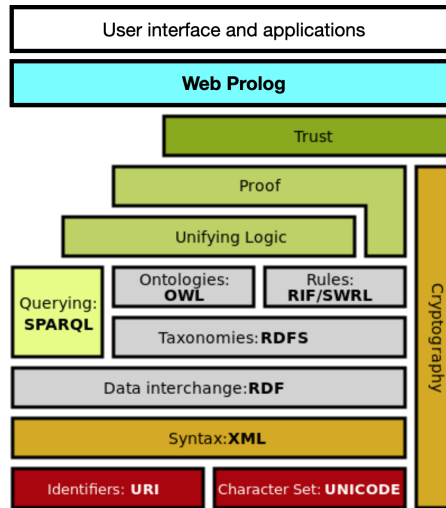


Fig. 7.4 The Semantic Web tower with an explicit Web Prolog middleware layer.

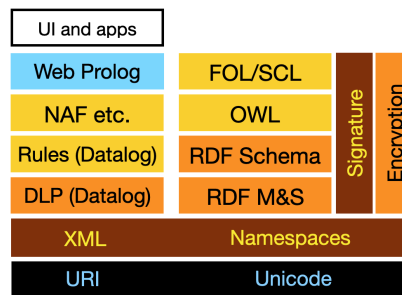


Fig. 7.5 Two-tower architecture (after (Horrocks et al., 2005)): logic-programming-oriented tower alongside the 'original' tower.

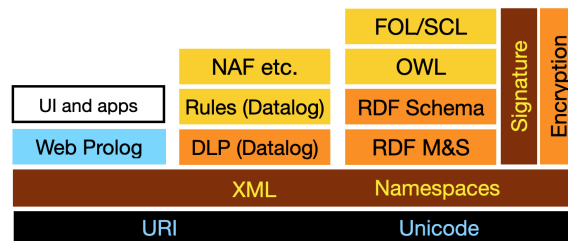


Fig. 7.6 Three-tower view: adding a Prolog Web tower alongside the two Semantic Web towers (after (Horrocks et al., 2005)).

While the Semantic Web might be described as “layered,” the Prolog Web might be described as “integrated” – not only because the same language is used for knowledge representation and querying, but because Web Prolog is also a general-purpose programming language with a process architecture.

7.11 Summary and discussion

The Prolog Web and the Semantic Web are similar in spirit: both treat the Web as a computational environment rather than merely a document store, and both rely on global naming (URIs) and on logic-based machinery to make data more useful than it would be in a purely syntactic web of documents. Both also aim at decentralisation and federation, in the sense that useful functionality should arise from composing resources offered by many independent parties rather than from a single central database.

At the same time, the two stacks emphasise different things. The Semantic Web places its centre of gravity on interoperable data models and shared semantic commitments, while the Prolog Web places its centre of gravity on executable relational services, process-level structure, and web-native agents. Table 7.1 summarises these differences.

One way to state the complementarity is that unification disciplines structure at the point of call, whereas shared vocabularies and ontologies make semantic commitments explicit. Neither substitutes for the other: structurally compatible terms can still be interpreted differently, and shared meaning does not by itself provide an execution model. Chapter 9 returns to the structural side of this distinction when treating unification as a universal data contract.

Although the discussion in this chapter uses the Semantic Web as a concrete point of comparison, most of the underlying issues are not specific to RDF, OWL, or W3C standards. Over the past decade, a range of graph-based technologies – including property-graph systems and large-scale knowledge-graph platforms – have converged on similar concerns: graph-structured data, pattern-based querying, distributed evaluation, and the tension between shared representation and executable computation. The Prolog Web is intended to address this broader design space, offering an executable logic-based substrate that can sit alongside, or on top of, different graph representations rather than competing with any single one.

It should be clear by now that we do not aim for the Prolog Web to *replace* the Semantic Web, or for Web Prolog to replace the current Semantic Web languages and the technologies surrounding them. On the contrary, we hope and believe that we will be able to carve out a place for the Prolog Web alongside the Semantic Web, and that the languages involved can live happily together. This also appears to be consistent with the multi-stack architecture for the Semantic Web proposed in (Kifer et al., 2005; Horrocks et al., 2005).

Interestingly, Kifer et al. (2005) uses the following analogy when arguing against a single-stack architecture for the Semantic Web:

The Prolog Web	The Semantic Web
Uses the same language for querying and rule-based computation; queries can be packaged as reusable predicates	Typically separates data representation (RDF/OWL) from querying (SPARQL) and application logic
Provides a process and service architecture (nodes, APIs, long-lived actors) as part of the proposal	Provides standards for interoperable data and ontology semantics, but does not prescribe a single process architecture
Supports agent-style computation naturally through long-lived actors and message passing	Has strong support for shared ontologies and open-world reasoning through standardised languages and reasoners
Shared meaning is typically implicit in executable predicates and local conventions; shared ontologies are optional	Interoperability is centred on shared identifiers, vocabularies, and ontology axioms
Well suited as a programming and orchestration layer around heterogeneous services (including Semantic Web services)	Well suited as a lingua franca for knowledge interchange across independent parties

Table 7.1 Comparing the Prolog Web and the Semantic Web

While a single-stack architecture would hold aesthetic appeal and simplify interoperability, many workers in the field believe that such architecture is *un-realistic* and *unsustainable*. For one thing, it is presumptuous to assume that any technology will preserve its advantages forever and to require that any new development must be compatible with the old. If this were a law in the music industry (for example) then MP3 players would not be replacing compact disks, compact discs would have never displaced tapes, and we might still be using gramophones.

The authors have a point. We are tempted to think of Web Prolog programs as the vinyl records of the logic programming world, and of actors and nodes as the web-compatible gramophones on which to “play” them. Vinyl supports techniques such as *scratching* that newer media do not reproduce. Likewise, Prolog combines knowledge representation and querying with general-purpose programming, whereas the usual Semantic Web languages rely on a separate application language for that role. This integration has costs, but it is at the heart of Prolog. Without it, this would no longer be a Prolog Web.

Chapter 8

Prolog AI agents on the Agentic Web

Figure 8.1 illustrates a familiar way of viewing the history of artificial intelligence since the late 1950s: as a sequence of booms and busts, or “AI summers” and “AI winters.” Prolog and Lisp played prominent roles during the second boom, when expert systems were widely seen as the flagship technology of applied AI. In the current boom, by contrast, the centre of gravity has shifted toward non-symbolic methods, especially deep learning and large language models (LLMs).

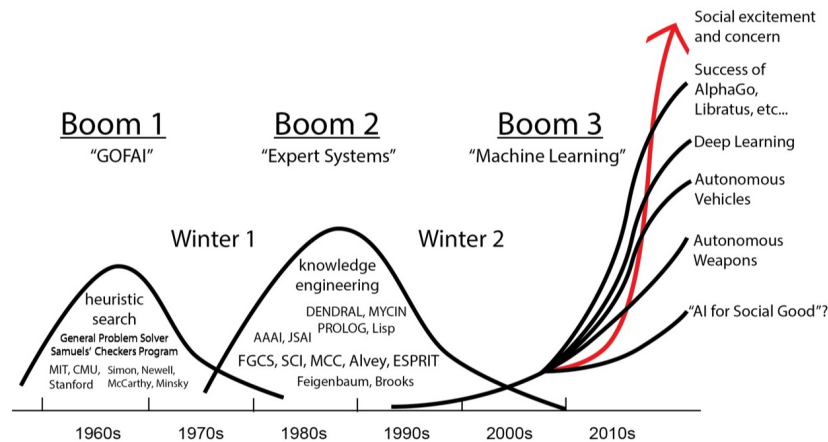


Fig. 8.1 Cycle of AI booms and busts. Chart inspired by a similar chart by Yutaka Matsuo.

This chapter argues, however, that the symbolic tradition remains highly relevant – not as a rival to contemporary AI, but as a complement to it. The reason is architectural. As the Web becomes increasingly populated by long-lived software processes that communicate, maintain state, use tools, and act over time, the need for controllable behaviour, inspectable reasoning, and explicit constraint handling re-emerges as a first-class engineering concern. This broader development is often

described in terms of an *Agentic Web*: a Web in which autonomous software processes operate alongside documents, services, and human users as first-class participants.

This development intersects naturally with logic programming. Prolog’s declarative foundations, together with its strengths in knowledge representation, inference, and structured communication, make it a plausible basis for agents that are not only capable of acting, but also capable of reasoning in explicit and auditable ways. Earlier in the book, and especially in Chapter 4, we argued that the Prolog Web already constitutes a multi-agent system in a minimal but precise sense: a network of autonomous processes interacting through messages, evolving over time, and embedded in a shared computational environment. This chapter builds directly on that observation. Rather than treating agents as an emergent property of the infrastructure, we now design them deliberately. The question is no longer whether agents exist on the Prolog Web, but how Prolog can be used to structure their behaviour, govern their interaction, and integrate reasoning, learning, and tool use in a principled and inspectable way. Because the vocabulary and products surrounding LLM-based agents change unusually quickly, specific descriptions in this chapter should be read as current in July 2026; the architectural claims are intended to outlast them.

8.1 Why Prolog agents now?

The classical Web is primarily a publishing and integration substrate for humans: documents, hyperlinks, and request–response endpoints. A plausible next phase – one that is already taking shape – is a Web populated by long-lived software processes that communicate, maintain state, and take action over time. The technical heart of this shift is a change in the unit of composition: from pages and endpoints to interacting agents.

The term *agentic* is often used loosely for systems that combine conversational interfaces, tool use, and large language models. In this chapter we use it in a stricter architectural sense. An *Agentic Web* is a Web in which autonomous, communicating processes are first-class participants. By the *Prolog Web* in its agentic aspect, we mean the part of that environment in which agents are implemented as Prolog processes with knowledge bases and reasoning procedures, exposed as web-native services and capable of interacting through message passing.

The word *agent* itself carries three relevant senses. This book’s default, established in Chapter 3, is architectural and deliberately modest: an agent is defined by a persistent interface and behaviour, without any claim about intelligence. Classical AI uses a richer vocabulary of beliefs, desires, intentions, goals, and speech acts (Wooldridge and Jennings, 1995). Contemporary LLM engineering often uses *agent* for a model embedded in a tool-using control loop. The latter two are things that may be built on the first: a BDI agent may use an actor’s private database for beliefs and its behaviour for deliberation; an LLM agent may delegate part of that deliberation to a model. Unless qualified as *BDI agent* or *LLM agent*, the word retains the book’s architectural meaning.

Historically, the Web has standardised naming, transport, and representation far more successfully than it has standardised process structure. We have mature machinery for addressing resources and exchanging messages, but much less agreement about the lifecycle of long-lived services, the discipline of stateful interaction, the handling of partial failure, or the way autonomous components should compose. Early discussions of web agents within the Semantic Web community made this gap explicit: representation and inference were advancing, but the process infrastructure in which agents would live and cooperate remained largely an application-level concern (Hendler, 2001).

The Prolog Web treats this gap as a design target. It preserves the Web's strengths – global naming, standard protocols, and interoperability – while adding an explicit execution model. Nodes host long-lived actors with mailboxes and supervised lifecycles, and predicates can be exposed as message-driven services. This is why the chapter can move from “agents as an emergent property” to “agents as a deliberate design”: the substrate already has the right shape for autonomous processes, and the remaining problem becomes behavioural engineering – how to specify, constrain, explain, and govern what agents do.

That an architectural gap exists does not, by itself, explain why Web Prolog should help fill it. Two contemporary pressures make the case. The first is the need for enforceable semantics. Large language models have made fluent natural-language interaction cheap and widely available, and they are rapidly becoming an interface layer for tools and services. But fluency is not the same as behavioural reliability. Whenever an agent is expected to satisfy non-negotiable constraints, follow explicit rules, or produce auditable justifications, we need a component whose behaviour has clear meaning and can be validated. Web Prolog offers precisely this: executable symbolic representations with inspectable derivations, well-defined failure modes, and a clear distinction between what is entailed and what is merely suggested. It is therefore a natural place to locate the parts of an agent that must be reproducible, enforceable, and open to inspection.

The second pressure is that the modern Web removes much of the old deployment bottleneck for symbolic AI. Symbolic systems have often remained laboratory artefacts because deployment required bespoke infrastructure, heavy integration work, and awkward maintenance. Standard protocols, cloud practices, and ubiquitous client platforms change this equation. A Prolog engine can now sit behind stable interfaces as a verifier, planner, policy engine, dialogue manager, or explanation service, and can be composed with other services without losing its semantic character.

The result is not a call for “Prolog everywhere,” but for Prolog where semantic clarity and constraint satisfaction matter most. Web Prolog is best understood as a *behavioural backplane* for agentic systems: the component in which constraints, state, control, and explanations are represented explicitly as programs, and can therefore supervise more fallible components. This positioning requires candour about Prolog's limitations. Its learning curve is steep, its implementation landscape remains fragmented, and its mainstream ecosystem is limited. The claim is only that these costs may be justified when the alternative is to reconstruct constraint handling,

control, and auditability in ad hoc form on top of tools that do not natively support them.

Having framed the architectural motivation, the remainder of the chapter develops the argument in four steps. We begin with Prolog’s symbolic competence on a single machine. We then distribute those idioms across the Prolog Web by treating engines as communicating actors and predicates as message-driven services. Next, we introduce a neuro-symbolic layer in which large language models provide linguistic robustness while Web Prolog supplies the authoritative control envelope. Finally, we turn to user-interface and embodied agents, where dialogue is treated as a problem of explicit interaction control.

8.2 Symbolic competence in one engine

Before we distribute Prolog across the Agentic Web, it is useful to recall what a single Prolog engine already provides as an AI substrate. The point of this section is not to teach Prolog, but to fix a compact capability map: a handful of idioms that repeatedly recur in symbolic AI, and that will later reappear as message-driven services and interacting agents.

8.2.1 Knowledge representation and inference

A Prolog program can function simultaneously as a declarative knowledge base and as an executable inference procedure. A canonical example is a small relational theory of kinship, where recursion expresses transitive closure:

```
ancestor_descendant(X, Y) :- parent_child(X, Y).
ancestor_descendant(X, Z) :-
    parent_child(X, Y), ancestor_descendant(Y, Z).

parent_child(X, Y) :- mother_child(X, Y).
parent_child(X, Y) :- father_child(X, Y).

mother_child(trude, sally).

father_child(tom, sally).
father_child(tom, erica).
father_child(joe, tom).
```

A query such as `?-ancestor_descendant(joe, Who)` asks for logical consequences of the clauses and enumerates answers via backtracking. This combination – explicit relational structure plus a proof procedure – is the basic engine behind many symbolic systems, and in Prolog the proof procedure is built in.

8.2.2 Planning as search in a state-transition model

Many agent tasks are naturally expressed as planning: find a sequence of actions that transforms an initial state into a goal state. A small meta-program suffices when actions are represented as declarative transition rules:

```
plan(P, P, []) .
plan(P0, Pf, [A|As]) :-
    step(P0, P1, A),
    plan(P1, Pf, As).

step(start, position(monkey, bananas, chair), start).
step(position(Y,Y,Y), final, climb).
step(position(X,Y,X), position(Y,Y,Y), carry(X,Y)).
step(position(X,Y,Z), position(Z,Y,Z), walk(X,Z)).
```

Here the entire world state is a first-order term, and each `step/3` clause specifies an action's preconditions and effects. Prolog's search then discovers action sequences by exploring transition alternatives under backtracking.

8.2.3 Meta-interpreters as a control and explanation lever

A distinctive Prolog idiom is to treat programs as data and write interpreters for restricted logics, alternative control regimes, or explanation-producing variants. A minimal meta-interpreter mirrors Prolog's own proof search:

```
prove(true) :- !.
prove((A,B)) :- !, prove(A), prove(B).
prove(A) :- clause(A,B), prove(B).
```

Small changes to such an interpreter can add substantial functionality without changing the object-level knowledge base. For example, adding a second argument allows the interpreter to construct a proof object – a structured explanation that records which rules were used:

```
prove(true, true) :- !.
prove((A,B), (PA,PB)) :- !, prove(A, PA), prove(B, PB).
prove(A, A/Proof) :- clause(A,B), prove(B, Proof).
```

Proof terms of this kind are evidence-carrying data structures that can be inspected, summarised, or shown to a user. In Section 8.3 we will exploit this idiom across nodes, so that a single explanation can span distributed computation.

8.2.4 An expert system with a query-the-user facility

Expert systems were a flagship application of symbolic AI during the 1980s. They did not become a lasting mainstream paradigm, but the core pattern remains instructive, especially in Prolog: it shows how deduction can be interleaved with interaction in a disciplined way.

An expert system typically consists of a knowledge base plus an inference engine. A practical system must also be able to acquire facts that are not already stored. In Prolog, a simple way to realise this is to extend the meta-interpreter from Section 8.2.3 with a single additional clause that queries the user whenever a goal is declared askable:

```
prove(A) :- askable(A, Q), writeln(Q), read(T), T == yes.
```

The accompanying knowledge base supplies both domain rules and a set of askable/2 declarations that map predicates to human-readable prompts:

```
good_pet(X) :- bird(X), small(X).
good_pet(X) :- cuddly(X), yellow(X).

bird(X) :- has_feathers(X), tweets(X).

askable(tweets(_), 'Does it tweet?').
askable(small(_), 'Is it small?').
askable(cuddly(_), 'Is it cuddly?').
askable(has_feathers(_), 'Does it have feathers?').
askable(yellow(_), 'Is it yellow?').
```

When prove/1 encounters an askable subgoal, it suspends deduction, poses the associated question, reads the reply, and then resumes the derivation. For example:

```
?- prove(good_pet(tweety)).
Does it have feathers?
|: yes.
Does it tweet?
|: yes.
Is it small?
|: no.
Is it cuddly?
|: yes.
Is it yellow?
|: yes.
true.
?-
```

The important point is structural: proof search determines which information is needed when. The resulting interaction is mixed-initiative in a precise sense. The

engine advances the derivation whenever its knowledge suffices, and it consults the user exactly at the proof obligations that require additional axioms. The askable declarations therefore act both as an interaction interface and as a lightweight user model: they mark which facts are expected to come from dialogue rather than from stored knowledge.

This is, of course, a toy shell. In more realistic systems one would add typed answers, richer question forms, explanation facilities, and a persistent dialogue state. For a fuller treatment of Prolog-based expert systems, see (Merritt, 1989) and (Sterling and Shapiro, 1994).

8.2.5 Parsing as deduction: DCGs

Definite Clause Grammars (DCGs) turn parsing into a form of proof search: a sentence is grammatical exactly when it can be derived from the grammar rules. DCGs are also reversible, so the same grammar can be used for generation. Operationally, naïve top-down execution makes left recursion problematic, but in tabled Prolog systems this restriction largely disappears: tabling turns parsing into dynamic programming while preserving a declarative rule notation. (Tabling is a system feature rather than part of the ISO core, so it should be read here as an optional acceleration of the same modelling stance.)

8.2.6 Summary

The examples above form a compact capability map. A single Prolog engine already supports (i) explicit relational knowledge representation, (ii) planning as systematic search in state-transition models, (iii) meta-level control and explanation via interpreters that construct proof objects, (iv) interactive acquisition of missing facts, and (v) grammar-based language processing as deduction.

The next section lifts these idioms onto the Agentic Web. Instead of treating predicates as internal procedures, we treat them as services: message-driven actors with interfaces, state, and failure modes. The result is an Agentic Prolog Web in which symbolic reasoning is not only executable, but deployable, composable, and inspectable across the network.

8.3 Prolog AI agents on the Agentic Prolog Web

The central claim of this section is that the idioms (i)–(v) do not need to remain local. When Prolog engines are exposed as web-native actors, the same programs can be deployed as distributed services and composed into interacting agents.

The Agentic Prolog Web is therefore not a new symbolic formalism. It is an execution and deployment stance: treat Prolog engines as networked processes with mailboxes, state, and interfaces, and let predicates become message-driven services. In this setting, what used to be a local `read/1` becomes a `prompt/2` message expecting a response; what used to be a local meta-interpreter becomes a mobile proof-producing component; and what used to be an internal predicate call may become a remote call whose provenance is part of the explanation.

We develop the idea through four ingredients: an expert-system agent that acquires missing facts as a distributed dialogue; a proof-producing meta-interpreter whose explanations span multiple nodes; a positioning of Web Prolog relative to earlier Prolog-based agent frameworks; and a brief discussion of scalability and interoperability on the Web.

8.3.1 An expert-system agent and a simulation

Classic expert systems faced two persistent bottlenecks: the difficulty of extracting knowledge from humans and the lack of a ubiquitous infrastructure for deployment. A web-native actor substrate addresses both. By treating the Web as the infrastructure, symbolic agents can be deployed as services; and by treating interaction as message passing, knowledge acquisition becomes an incremental, mixed-initiative dialogue rather than a local read/write loop.

To adapt the askable meta-interpreter pattern from Section 8.2.4 to an agent setting, we replace terminal I/O with an interaction predicate. Concretely, the clause that previously wrote a question and read an answer becomes:

```
prove(A) :- askable(A, Q), input(Q, T), T == yes.
```

The purpose of `input/2` is to decouple deduction from any particular user interface. In a local setting, `input/2` might still be implemented on top of `read/1`. As we saw in Chapter 4, in the Prolog Web it is implemented by sending a `prompt` to some conversational partner and waiting for a reply via `respond/2`.

We can now package the expert system as an expert-system agent by spawning a remote toplevel actor and preparing a call to `prove/1`:

```
expert_ask(Query, Pid, Options) :-
    toplevel_spawn(Pid, Options),
    toplevel_call(Pid, prove(Query), [
        limit(1)
    | Options
    ]).
```

The `limit(1)` option bounds each returned solution slice to one answer. This keeps terminal answer handling simple, but it does not limit the number of prompts: those arise while the single remote query is running, and each is resumed with `respond/2`.

To demonstrate the pattern, we construct a small simulation in which a second agent – in this case, the shell process – plays the role of the user. The simulation answers questions stochastically, illustrating that the expert system does not control the dialogue by a script; rather, the proof search itself determines which information must be acquired next.

```
simulation(Options) :-
    random_member(Name, [tweety,pingu]),
    format('A1: Is ~w a good pet?~n', [Name]),
    expert_ask(good_pet(Name), Pid, Options),
    interact(Pid, Name).

interact(Pid, Name) :-
    receive({
        failure(Pid) ->
            format('A2: No, ~w is not a good pet.~n', [Name]);
        success(Pid, _, _) ->
            format('A2: Yes, ~w is a good pet.~n', [Name]);
        prompt(Pid, Question) ->
            format('A2: ~w~n', [Question]),
            random_member(Answer, [yes,no]),
            respond(Pid, Answer),
            format('A1: ~w~n', [Answer]),
            interact(Pid, Name)
    }).
```

One may run the simulation, for example, by spawning the expert-system agent on a remote node, loading the relevant predicates, and providing an initial fact base:

```
?- simulation([
    node('http://n9.org'),
    load_predicates([prove/1]),
    load_list([yellow(tweety)])
]).
A1: Is tweety a good pet?
A2: Does it have feathers?
A1: yes
A2: Does it tweet?
A1: yes
A2: Is it small?
A1: no
A2: Is it cuddly?
A1: yes
A2: Yes, tweety is a good pet.
true.
?-
```

The architectural point is that the reasoning process is encapsulated as a mobile, interactive component: an actor whose control flow is governed by logical necessity rather than by a fixed dialogue script. The Web substrate supplies distribution and composability, while Prolog supplies the proof structure that drives which questions must be asked.

8.3.2 Explainable AI across nodes: distributed proof trees

A defining advantage of logic-based, symbolic AI is transparency: every conclusion rests on rules and facts, and an explanation can be constructed as a proof object. On the Agentic Prolog Web, explanations must survive distribution. If a subgoal is proved on another node, its justification should be folded into the same overall proof structure rather than relegated to an opaque remote call.

A compact meta-interpreter can achieve this by treating remote calls as proof-producing sub-derivations. When the interpreter encounters a remote goal, it ships itself to the remote node and requests a proof there, then folds the result back into the local derivation:

```
prove(true, true) :- !.
prove(rpc(URI,A), Proof) :- !,
    prove(rpc(URI,A,[]), Proof).
prove(rpc(URI,A,Options), Query@URI/Proof) :- !,
    rpc(URI, prove(A, Query/Proof), [
        load_predicates([prove/2])
    | Options
    ]).
prove((A, B), (ProofA, ProofB)) :- !,
    prove(A, ProofA),
    prove(B, ProofB).
prove(A, A/Proof) :-
    clause(A, B),
    prove(B, Proof).
```

The `load_predicates([prove/2])` option is what makes the explanation discipline recursive: the meta-interpreter is transmitted to the remote node along with the goal, so the remote node runs `prove(A, Query/Proof)` rather than the bare goal `A`. Proof construction therefore applies at every hop, not just the first.

The key idea is that a cooperating remote computation need not be treated as a black box. When the remote node accepts the transmitted interpreter and exposes the clauses it may inspect, the same explanation discipline is applied at each node, and remote contributions are marked in the resulting proof term by the `@` operator.

Suppose, as in Section 4.2.4, that the local node states that every human is mortal by means of a rule `mortal(X) :- human(X)`, and defines `human(X)` by consulting

a remote node. If the fact `human(plato)` is stored on `n1.org`, we obtain a proof object that explicitly records the distribution boundary:

```
?- prove(mortal(plato), Proof).
Proof = mortal(plato)/
      (human(plato)/
       (human(plato)@'http://n1.org'/
        true)).
?-
```

The annotation `human(plato)@'http://n1.org'` documents not just what was concluded but where. The proof term is ordinary Prolog data: it can be inspected, stored, transmitted, or rendered into a human-readable explanation by a further layer of code.

One practical requirement is that predicates inspected through `clause/2` must actually be visible to the interpreter. Permission to inspect compiled predicates varies between Prolog systems and security profiles. A portable Web Prolog service should therefore expose the relevant clauses deliberately – for example as dynamic clauses in the actor’s private database – rather than assume that arbitrary host code can be reflected upon.

Proof-producing interpreters can of course generate very large derivations. On the Prolog Web this is best treated as a controllable engineering choice rather than an unavoidable pathology: one may bound proof depth, abstract away local details and retain only a node-level skeleton, cache recurring subproofs, or compress proof objects into summaries suitable for user-facing explanations. The benefit is that distribution does not force a loss of transparency. Explanations remain first-class data, even when computation spans multiple machines.

8.3.3 Meta-interpreters as generic behaviours

The two preceding subsections followed the same architectural pattern. In each case, a meta-interpreter defined the reasoning strategy – askable deduction, proof-term construction – while the actor substrate provided the process container: a mailbox, a lifecycle, and a network address. The domain knowledge was supplied as an ordinary Prolog knowledge base, loaded into the actor at spawn time. The result was a reusable, deployable component whose control flow was determined by logical structure rather than by a hand-written message loop.

This is precisely the separation that generic behaviours are designed to capture. In Erlang/OTP, a generic behaviour such as `gen_server` factors a concurrent service into two parts: a reusable process skeleton (message reception, state threading, supervision hooks) and a set of application-specific callbacks. The programmer supplies the callbacks; the behaviour supplies the process discipline.

Meta-interpreters on the Prolog Web admit the same factoring. The generic part is the interpreter itself – its search strategy, its interaction policy, its proof-construction

discipline – together with the actor wrapper that exposes it as a networked service. The application-specific part is the knowledge base: the clauses, the `askable/2` declarations, and any auxiliary predicates loaded at spawn time. Swapping the knowledge base yields a different expert in the same conversational frame; swapping the meta-interpreter yields a different reasoning regime over the same knowledge.

The two examples already presented illustrate the point concretely. The askable interpreter of Section 8.3.1 defines a behaviour in which each proof step may trigger an interactive prompt; the proof-producing interpreter of Section 8.3.2 defines a behaviour in which each proof step extends an explanation term. Both accept the same knowledge-base interface. A third variant could combine the two, producing explanations for askable derivations, by straightforward meta-interpreter composition – a standard technique in the Prolog literature that requires no new infrastructure on the actor side.

What the actor substrate adds is deployment discipline. Each meta-interpreter behaviour, once packaged as an actor, inherits the supervision, addressing, and lifecycle machinery described in the previous chapters. A supervisor can restart a failed proof-producing agent exactly as it would restart any other actor; a client can address an askable agent by URI without knowing which meta-interpreter it runs internally. The generic behaviour boundary thus serves the same purpose on the Prolog Web as it does in OTP: it localises the reasoning policy inside the behaviour, and exposes only a uniform actor interface to the rest of the system.

The claim here is modest. We are not proposing a fixed catalogue of meta-interpreter behaviours analogous to OTP’s behaviours. We are observing that the meta-interpreter tradition already supplies the raw material for such a catalogue, and that the Prolog Web’s actor model provides the deployment frame that makes the analogy precise. Whether a richer library of standard behaviours – bounded-resource interpreters, tabled interpreters, interpreters with confidence or provenance tracking – proves useful in practice is an empirical question that the architecture leaves open rather than forecloses.

8.3.4 Parsing as remote deduction

A client may offload parsing to a node that hosts a grammar, without embedding that grammar locally:

```
?- rpc('http://n6.org', phrase(s(PT), [joe,saw,a,telescope])).
PT = s(np(pn(joe)), vp(v(saw), np(det(a), n(telescope)))).
?-
```

This vignette illustrates the broader pattern: symbolic competencies that are expensive to ship as local libraries or knowledge bases can be deployed as web-native agents and accessed by clients through uniform remote calls.

This pattern is, in fact, especially well motivated for grammars, because the honest case for shipping a grammar as a remote service is partly a case about DCGs’

limitations. As a formalism for writing natural-language grammars, DCG has never been a favourite among computational linguists. A term-based representation of syntactic categories can feel insufficiently flexible, and naïve top-down execution makes left-recursive rules such as `s --> s, conj, s` or `np --> np, pp` nonterminating, so the grammar engineer must contort otherwise natural rules to avoid them.

This is precisely where remote deduction earns its keep. A client need not embed a grammar it did not write, nor match the implementation assumptions the grammar depends on; it issues an `rpc/2-3` call and receives a parse. The grammar's requirements become the *host node's* concern, hidden behind the uniform call interface. The division of labour generalises beyond DCGs: for serious natural-language work one would not write a large grammar in raw DCG at all, but expose an established resource such as Grammatical Framework (Ranta, 2011), with its grammars for more than forty languages, behind the same remote-call contract.¹ Whether the node runs a tabled DCG or fronts an external grammar system, the client sees the same thing: a goal sent, a structured parse returned.

And because a parse tree is itself a proof – a record of which grammar rules were applied, and how they combine – the returned PT is not merely the result of a remote deduction but its trace. Remote parsing is thus already a special case of the proof-carrying remote calls of Section 8.3.2: the proof object simply happens to be the syntactic structure we wanted all along.

8.3.5 Implementing agent frameworks in Web Prolog

Logic-based agent languages and frameworks have been implemented in Prolog for decades. Systems such as DALI (Costantini and Tocchio, 2004) and LPS (Kowalski and Sadri, 2015) show that one can retain a Horn-clause core while adding structures for events, actions, reactive rules, and persistent state. Their appeal lies in combining logic programming virtues – declarative clarity, inspectability, and verifiability – with practical mechanisms for autonomy and interaction.

This tradition is richer than an actor substrate alone. BDI architectures model deliberation in terms of beliefs, desires, and intentions, while agent communication languages attach speech-act semantics to messages. Web Prolog standardises neither mental attitudes nor an agent communication language. Its private databases, behaviours, and term messages provide places in which such notions can be represented, but their meaning belongs to the framework layered above the substrate. This distinction prevents a mailbox-bearing process from being mistaken for a complete cognitive architecture.

Web Prolog does not present itself as yet another high-level cognitive agent language with built-in planners, ontologies, or a fixed BDI-style architecture. Instead, it provides a lower-level actor substrate in which the primary abstraction is a process with a private database and a mailbox, encapsulated behind a process identifier and

¹ <http://www.grammaticalframework.org>

supervised by reliability patterns. In this respect it is closer to an Erlang-inspired concurrent logic substrate than to a monolithic agent language.

Precisely because the substrate is low-level and concrete, many classic agent-language ideas can be reconstructed. A receive loop that pattern-matches on incoming messages, together with an explicit representation of pending goals, already suffices to encode reactive rules of the form “on event then action” or “on event and if condition, then action” as ordinary Prolog clauses over the mailbox and the private database. In statechart actors, this becomes even more structured, allowing rich reactive and mixed-initiative behaviours to be expressed as statechart specifications interpreted by Web Prolog rather than as ad hoc control code.

Seen from this perspective, Web Prolog is again best understood as an enabler rather than a competitor. It supplies the web-native execution fabric on top of which more specialised agent frameworks can be layered. LPS-style engines, DALI-like event-driven agents, and more elaborate frameworks can all be realised as conservative extensions that treat Web Prolog actors as their concrete embodiment on the open Web.

8.3.6 Scalability and interoperability on the Web

Two traits of the Web are decisive for symbolic agents: elastic scalability and protocol-level interoperability.

Scalability

Web Prolog agents are processes that can be spawned, suspended, or terminated on demand. Because interaction can be mediated by standard web protocols, ordinary deployment practices can be used around node-level services: containers can replicate nodes, load balancers can distribute connections, and observability tools can monitor the host processes. Inside a node, actor supervision supplies the finer-grained failure boundary, allowing individual agents to be restarted or isolated. These two scales are complementary and should not be conflated.

Interoperability

The Web rewards components that speak stable protocols. On the Agentic Prolog Web, terms travel as structured data, which makes it natural to expose symbolic services as typed message interfaces and to interoperate with non-Prolog components through defined protocol encodings. Conversely, Prolog agents can call REST services, interact with browser clients, or orchestrate external systems, while keeping their internal reasoning symbolic and inspectable. A single canonical Prolog–JSON

mapping is not assumed; as Chapter 11 discusses, representation choices remain part of the interoperability contract.

Distributed symbolic agents already provide transparency and coordination at Internet scale, but many deployed agents are not purely agent-to-agent components. They are hybrid systems that must incorporate neural components for language, perception, and retrieval, while keeping authority in a checkable substrate. The next section therefore introduces neuro-symbolic agents: architectures in which Web Prolog acts as control and constraint envelope, and LLMs act as pluggable language modules.

8.4 Neuro-symbolic AI agents

To build a robust, knowledge-driven approach to AI we must have the machinery of symbol manipulation in our toolkit. Too much useful knowledge is abstract to proceed without tools that represent and manipulate abstraction, and to date, the only known machinery that can manipulate such abstract knowledge reliably is the apparatus of symbol manipulation.

Marcus and Davis, 2019

Large language models are powerful generators of fluent text, but sampled generation does not by itself provide hard behavioural guarantees. When facts, constraints, obligations, or provenance are missing from the prompt, a model can still produce a plausible continuation, and the result may read as confident even when it is wrong. In low-stakes settings this is often acceptable and sometimes useful, but in settings where actions are irreversible or correctness is externally defined (healthcare, compliance, infrastructure automation), we need architectures that can enforce invariants, expose assumptions, and fail safely. Neuro-symbolic agents address this by combining neural components for perception and language with symbolic components for rules, search, and verification, so that generative flexibility is grounded in checkable structure rather than treated as the final authority.

Sections 8.2 and 8.3 established, respectively, the symbolic competence of Prolog programs and their web-native, actor-oriented execution model. Building on that foundation, this section shows how neural components (LLMs) and symbolic components (Web Prolog) can be composed into coherent agents, and why Web Prolog is a natural host language for such hybrids when the agent is expected to be concurrent, auditable, and distributed. Because the landscape of agentic systems is evolving quickly, the section deliberately rests on a small set of stable architectural ideas – control (statecharts), constraints (symbolic rules), and supervision (actors) – that remain valuable even as specific tools and model capabilities change.

One consequence of that pace is worth stating directly, because it shapes how the rest of this section should be read. The boundary between description and implementation is itself shifting: it is no longer implausible that one could feed the text of this book – specifications, examples, protocol sketches, and semantic constraints – to a capable LLM and obtain a working implementation of a Prolog node in return.

Far from undermining the architectural material that follows, this raises its value. Faster code generation increases the premium on precise behavioural specifications, because it lets us generate implementations quickly while still knowing what, exactly, they are supposed to implement – and that requires stating the intended semantics clearly, separating invariants from conventions, and making the control and safety envelope explicit.

8.4.1 Why hybridize?

It is worth naming, at the outset, a tension that many logic-programming researchers feel acutely: the rise of LLM-centred AI can be read either as a threat to the symbolic tradition or as an opportunity for it. The threat reading is largely sociological. LLMs are an existence proof that a great deal of apparently competent reasoning can be produced without explicit symbolic structure – unreliably, but well enough to capture most of the field’s attention, funding, and incoming students – and a tradition that once justified itself as *the* route to machine reasoning may understandably feel displaced. The opportunity reading is the mirror image. Logic programming was never weak at inference; it was weak at acquisition, that is, at turning messy human input into clean logical form and turning derivations back into readable prose. These are precisely the boundary tasks at which LLMs excel, so the model can be seen less as a rival than as the long-missing front-end that finally lets a symbolic system do what it was always good at. This section takes the view that the threat/opportunity dichotomy is itself the wrong frame, because it presupposes that the two paradigms compete for the same ground. They do not. The more durable position is that they occupy complementary ground, and the productive question is not whether neural methods help or harm logic programming, but *who holds authority over what*. We should be candid that placing decision authority with the symbolic layer is the conclusion a logic programmer would prefer, and that how much of a verifier’s role a sufficiently capable model might eventually absorb remains an open empirical question. The architectural bet of this section is that, for domains where correctness is externally defined, an explicit and checkable substrate will remain the responsible place to put it – and that this is a more honest and more defensible position than the totalising claims that have burned the field before.²

With the question framed as one of authority rather than rivalry, the constructive task is to say how the two layers should be combined. Neural models excel at perception, linguistic robustness, and learning from heterogeneous data, but they do not natively preserve the invariants that a symbolic program can enforce: deductive consistency, traceability, structured explainability, and predictable failure modes.

² The cautionary precedent is the field’s own history: the expert-systems boom and Prolog’s association with the Fifth Generation project left logic programming once positioned as *the* path to AI and then blamed when the wider enterprise stalled. Positioning symbolic methods as the reliability substrate beneath a successful paradigm, rather than as the paradigm itself, is the more modest and more durable claim. See the retrospective discussion in (Körner et al., 2022).

Hybridization therefore combines (i) neural flexibility for perception and language understanding, (ii) symbolic rigour for logic, planning, and verification, and (iii) actor concurrency for scalable orchestration across the Web – assigning each the work it is suited to rather than forcing one to stand in for the other. The aim is not to replace neural methods but to embed them in an architecture where the non-negotiables of a domain can be represented and checked continuously.

Table 8.1 makes this division concrete at the level of engineering posture: what kind of semantics each layer offers, how it fails, and what can or cannot be guaranteed. It is meant to show why hybrid designs place authority with the symbolic layer even when language is handled by an LLM.

Posture	Prolog-based reasoning systems	LLM-centred systems
Semantics and guarantees	Precise operational meaning; behaviour can be constrained by rules, proof procedures, and resource bounds.	Implicit and statistical; hard to guarantee correctness beyond narrow, externally enforced constraints.
Correctness posture	Supports explicit checking: constraints, invariants, and entailment can be validated and reproduced within the formalised fragment.	Optimised for generation: can be accurate, but may also hallucinate or drift unless outputs are checked externally.
Explainability	Can return structured reasons (derivations, rule firings, failed constraints) and support audit trails.	Explanations are generated narratives; may be persuasive but are not inherently evidence-carrying.
Failure modes	Typically explicit: non-termination, incompleteness from control choices, or constraint inconsistency; failures can be instrumented.	Often implicit: silent errors, confident wrong answers, instruction-following failures, and sensitivity to prompt phrasing.
Data and learning	Knowledge is explicit and editable; learning is external (rule authoring, induction, or updates to facts/rules).	Knowledge is latent in parameters; updating behaviour often requires retraining, fine-tuning, or additional scaffolding.
Integration shape	Natural fit as a verifier, planner, policy engine, or reasoning service; can sit behind APIs with stable semantics.	Natural fit as a generator and interface layer; needs surrounding machinery for validation, safety, and state consistency.
Strength under constraints	Strong when requirements must be satisfied (safety, compliance, configuration, scheduling).	Strong when requirements are soft and language-driven (drafting, summarising, proposing options).
Cost model	Inspectable per inference step; performance depends on search space and control but can often be bounded.	Token- and model-size-driven; costs can be high and scale with context length; behaviour may vary across model versions.

Table 8.1 Contrasting Prolog-based reasoning systems with LLM-only systems: engineering posture and guarantees.

The table is best read not as a scorecard but as a guide to where each paradigm should carry engineering weight in a deployed agent. Prolog is attractive wherever

correctness is externally defined and can be formalised: verification and enforcement of constraints, invariants, permissions, and policy; structured search where explicit pruning, tabling, or constraint propagation turns a large discrete space into an executable plan; and settings where auditability is first-class, since Prolog can return evidence-carrying artefacts. LLM-centred systems are attractive where the objective is linguistic or heuristic rather than deductive: fluent generation, heuristic proposal of candidate solutions, and robustness to messy or vaguely specified natural-language input. The design takeaway is a division of authority: *the model proposes; the logic disposes*. Generation supplies candidate interpretations, plans, or utterances, while the symbolic layer polices the commitments actually made – but only for the fragment represented in its rules and constraints.

8.4.2 What is neuro-symbolic AI?

Neuro-symbolic AI is a broad family of approaches that combine learned representations with explicit symbolic structure or inference. In the architectural sense used in this chapter, neural and symbolic modules exchange intermediate representations at run time under the control of an agent framework. The coupling may be loose (black-box tool calls with structured I/O), semi-tight (shared constraints, schemas, or retrieval paired with symbolic checks), or tight (a single differentiable pipeline in which parts of the symbolic procedure are compiled into a learnable computation graph). These three modes correspond to three engineering postures: orchestration, co-representation, and end-to-end training.

8.4.3 Architectural patterns for hybrid agents

Four patterns recur in contemporary neuro-symbolic agent design.

Tool delegation

A model emits a structured request for an external tool to perform an objective sub-task such as retrieval, arithmetic, symbolic query answering, or constraint checking. The surrounding orchestrator validates and executes that request, then returns the result to constrain the next generation step. The distinction matters: the model proposes the call, but a conventional program holds the capability and performs the effect.

Planning with verification

The LLM proposes an explicit action sequence while the symbolic layer checks preconditions, postconditions, and resource constraints before execution. The LLM is treated as a planner and summariser, while Web Prolog plays the role of verifier and dispatcher. This pattern is especially useful when actions are partially ordered, when actions can fail, or when the environment imposes hard constraints that must never be violated.

Multi-agent decomposition

Sub-goals are assigned to cooperating actors so that each specialist works locally while the overall system remains concurrent and robust to partial failure. The neural component may participate as one actor among others, or it may be called by several actors as a shared service. In both cases, actor boundaries provide a natural granularity for supervision, timeouts, and audit trails.

Finite-state or statechart cage

The LLM is embedded inside a deterministic control wrapper that constrains what can happen next. The wrapper can accept, reject, or repair model outputs, and it can route the dialogue through explicit states such as `collect`, `confirm`, and `abort`. In Web Prolog this wrapper can be implemented directly as a statechart actor, giving the system a concrete behavioural skeleton rather than an implicit, prompt-only one.

It is worth connecting this pattern to a term that has become idiomatic in contemporary agent engineering: the *harness*. In current usage, an agent harness is the surrounding layer that decides when to call the model, validates what it returns, enforces limits, and routes control – in other words, the machinery that turns a bare model into a usable agent. The word is well chosen, because a harness is what constrains something powerful so that it can do useful work safely. The difficulty is that, in most systems built today, the harness is itself an informal artefact: a tangle of prompts, conditionals, and retry logic in which the control rules are implicit and the safe envelope is asserted rather than specified. This is precisely the situation the present argument warns against, in which control becomes an emergent property of prompts and glue code rather than something one can point to and inspect.

The Trinity claim is not that harnesses are unnecessary – they plainly are – but that a harness should be a *program* with explicit semantics rather than a prompt with implicit ones. A statechart actor is what the harness becomes when it is made rigorous: its states, transitions, guards, and stopping conditions are written down, executable, and auditable, so that the question “what can happen next?” has an answer in the code rather than in the model’s disposition. A closely related stance is

articulated by Adam Charlson in his 2025 talk³ on building multi-agent systems with finite state machines, where statecharts are presented as a governance layer around LLM-driven systems: the model may propose actions or emit events, but an external statechart remains authoritative, enforcing valid transitions via guards and refusing to act on unmodelled events, which are logged for remediation rather than executed. In this framing, the LLM supplies flexible language and task-specific heuristics *inside* a supervised control envelope, while the harness – now a statechart, not a prompt – supplies the predictability, observability, and control that unconstrained text-generation loops typically lack.

That this case is made from within the JavaScript ecosystem – Charlson’s statecharts are built with XState, not Prolog – is itself telling: the argument for a statechart as the authoritative envelope around an LLM arises independently of the host language, which is exactly what the host-neutrality of the statechart model (Chapter 6, Section 6.9.5) would predict. The present book makes the case in Web Prolog; that a practitioner reaches the same architecture in a different language stack is evidence that the governing idea – explicit, inspectable control around a fallible generator – is not parochial to either.

8.4.4 Web Prolog as symbolic backend and agent layer

Python dominates ML practice for good reasons, and the point of this section is not to argue against Python. The point is that a hybrid agent needs more than tensor libraries: it needs an orchestration substrate with first-class support for search, symbolic constraints, concurrency, supervision, and network-transparent messaging. Web Prolog places these capabilities in the language model itself.

Web Prolog offers built-in backtracking and nondeterminism, which lets an agent represent alternatives and explore them under logical control rather than by ad hoc sampling. It supports guarded message reception and mailbox-driven coordination, which makes it natural to express reactive, concurrent control flows without collapsing everything into a single event loop. Its actor primitives and supervision patterns provide a principled way to handle partial failure, retries, and escalation. Finally, because messaging over the Web is part of the language-level model, the boundary between local and remote computation can be treated uniformly, which matters when a hybrid agent is distributed across nodes and services.

The result is a coherent programming model in which symbolic reasoning, orchestration, supervision, and network-transparent messaging share one substrate. Neural components then become pluggable co-processors rather than the place where control logic silently accumulates.

³ Adam Charlson, “Finite State Machines: A Foundation for AI Governance” (video presentation, 2025), <https://youtu.be/OD13PiXw60o> (accessed 2026-07-14).

Normative reasoning as executable policy

Legal, ethical, and organisational constraints can be modelled as rule sets in Prolog, including defeasible rules, exceptions, and priority schemes. This makes it possible to treat compliance not as post-hoc documentation but as executable policy: before an action is taken, the agent can attempt to prove that it is permitted under the current facts, and if no proof exists it can either refuse, ask for missing information, or escalate to a human.

For computational-law treatments in a Prolog setting, see, for example, the discussion in *Prolog: The Next 50 Years* and related work on executable norms and obligations (Kowalski et al., 2023a; Kowalski and Dato, 2022).

Bridging, not siloing

Hybrid architectures are not an either/or choice between Prolog and Python. Modern Prolog systems increasingly support tight, bidirectional interoperation with Python, and Janus provides a fast two-way interface adopted by more than one Prolog implementation. From an agent-design perspective, a common and often natural direction is for Prolog to call Python: the symbolic layer retains control and delegates vectorised numerics, deep-learning inference, or GPU-backed tensor algebra to Python libraries, while Prolog remains responsible for search, constraint enforcement, supervision, and auditability. This yields a pragmatic division of labour: Python for tensors, Prolog for control (Andersen and Swift, 2023; Wielemaker, 2023).

8.4.5 Interaction patterns between LLMs and Prolog

This subsection describes three progressively stronger integration styles. They are not mutually exclusive, and real systems often combine all three.

LLM → Prolog (tool invocation)

In a tool-augmented architecture, an LLM is embedded in an orchestrator that can route structured requests to external components. If we expose a Prolog engine as a tool, the model can propose that a subproblem be delegated to symbolic evaluation; the orchestrator decides whether to permit the call and performs it. The result then becomes a grounded intermediate value rather than a fabricated continuation. The essential design point is that the Prolog side must be sandboxed: allowed predicates are whitelisted, resource limits are enforced, and the result is returned in a typed, machine-checkable format.

Conceptually, one may imagine a helper predicate such as `prolog_tool/2` that evaluates a restricted query and returns a structured answer term that the LLM can

incorporate into its next step. This is the lightest-weight integration, and it already delivers substantial benefit when the delegated subtask has crisp semantics.

A concrete bridge now exists in SWI-Prolog’s experimental `mcp` pack.⁴ A declaration `mcp_tool/3` exposes a Prolog predicate as a Model Context Protocol tool with a description and JSON Schema. This complements rather than replaces the Prolog Web protocols. MCP gives model-oriented orchestrators a bounded, schema-described tool call; the PTCP preserves a Prolog session with incremental, nondeterministic answers, prompts, and control messages. One knowledge base may usefully offer both interfaces, each carrying the interaction shape it can express.

Prolog → LLM (API delegation)

Conversely, Web Prolog can delegate open-ended language tasks to an LLM and treat the model call as just another effectful predicate, guarded by timeouts, budgets, and policy checks. The exact API surface will evolve, but the architectural point remains stable: the symbolic layer controls when to call the model, how to constrain the prompt, how to validate the reply, and what to do when the call fails or returns something unsafe.

The following sketch shows the shape of an HTTP call to a contemporary text-generation endpoint. It is owner-side SWI-Prolog adapter code, not portable client code in the Web Prolog ACTOR profile; a node owner would normally expose such an adapter behind an authorised predicate or service. Endpoint details should be aligned with the provider’s current documentation.

```
:- use_module(library(http/http_client)).
:- use_module(library(http/http_json)).
:- use_module(library(option)).

call_llm(SystemMsg, UserMsg, Options, Reply) :-
    option(endpoint(Endpoint), Options),
    option(authorization(Auth), Options),
    option(model(Model), Options),
    Payload = json([
        model=Model,
        input=[
            json([role=system, content=SystemMsg]),
            json([role=user, content=UserMsg])
        ]
    ]),
    http_post(Endpoint, json(Payload), Reply,
        [request_header('Authorization'=Auth)]).
```

⁴ Version 0.1.1 in July 2026; <https://github.com/JanWielemaker/mcp>. The repository describes support for MCP tools, resources, and prompts, and explicitly marks the implementation experimental.

In a Web Prolog setting, this call is typically wrapped in a control shell: a timeout guard, a retry policy, a trace hook that records prompts and replies with provenance tags, and a validator that checks the reply against a schema before it re-enters the agent’s control flow. This control shell is the harness of Section 8.4.3 in its smallest form – a single guarded predicate rather than a full statechart, but the same principle of explicit, inspectable control around an opaque model call.

Containing latency and failure

The HTTP predicate above blocks while the provider responds. Blocking is acceptable in a dedicated wrapper actor whose only job is to wait for that service; it is not acceptable in SXML executable content, which must initiate long-running work asynchronously and consume the result as a later event. A statechart should therefore send a correlated request, enter a waiting state, and handle reply, failure, and timeout events on separate transitions. Because the SXML subset of Chapter 6 has no timed-transition construct, a helper actor or wrapper must arrange for an explicit timeout event rather than invent an `after` attribute.

The wrapper is also the correct failure boundary. It may be monitored or run under the supervisor behaviour of Chapter 5; a network exception can then terminate and restart the wrapper without corrupting the dialogue governor. Correlation references prevent a late reply from an earlier request from being accepted in a later waiting phase. This is the practical payoff of combining the preceding chapters: supervision below, asynchronous messages between, and an explicit statechart above.

Tight coupling and differentiable logic

In tight neuro-symbolic systems, parts of the symbolic procedure participate directly in gradient-based training. The core idea is to replace crisp truth values with differentiable scores, and to compile proof search into a computation graph that can be optimised. Projects such as DeepProbLog and NeurASP demonstrate that this can be engineered in practice by compiling logic programs into weighted proof graphs or tensor expressions executed in a deep-learning framework (Manhaeve et al., 2021; Yang et al., 2020).

From the Trinity perspective, the key observation is not that every agent should be differentiable, but that Prolog already provides a disciplined representation of structure, and that structure can sometimes be made learnable where it is genuinely useful – for example when learning soft constraints, priors, or perception modules. Concretely, a Web Prolog node would expose the learnable component as a co-processor behind a predicate boundary: the differentiable scores feed proof search, but the node retains behavioural authority in explicit rules and policies, so the statistical part influences *which* allowed derivation is preferred without widening *what* is allowed. This is the tightest of the three integration styles, and the one to reach for most selectively.

8.4.6 Statechart-wrapped medical triage agent

To make the cage pattern concrete, consider a simplified medical triage agent implemented as a statechart actor supervising an LLM. The example is deliberately schematic and is intended to illustrate the architecture only; it is not a clinical recommendation, and a deployed triage system would require regulatory, clinical, and human-oversight guarantees well beyond what is sketched here.

1. A Web Prolog actor initialises a statechart with states `collect`, `confirm`, `assess`, `route`, and `abort`.
2. In `collect`, the LLM proposes patient-facing questions; each model request is initiated asynchronously, and replies, errors, and explicit timeout signals return as statechart events. Answers are parsed by DCGs and stored as structured facts.
3. The transition `collect` $\rightarrow$ `confirm` fires when required slots such as `age/1` and `symptom/1` are instantiated.
4. In `assess`, reviewed Prolog rules classify routing urgency under explicit assumptions; the example delegates neither diagnosis nor clinical advice to the model.
5. If $\text{risk} \geq \theta$, the statechart transitions to `route(escalate)`; otherwise to `route(non_urgent_review)`.
6. All model outputs pass through a validator before they re-enter the state machine, rejecting disallowed content (for example, prohibited advice classes, missing disclaimers, or policy violations) and forcing the agent into `abort` or `escalate` when appropriate.

The central benefit is architectural: the LLM provides language, but the statechart provides behaviour. The system therefore has states, transitions, stopping conditions, and escalation paths, which are difficult to guarantee in a free-form prompt loop. It is exactly the harness-as-program of Section 8.4.3, instantiated for a domain where the cost of an unmodelled transition is high.

8.4.7 Challenges and research directions

- **Tracing and audit.** Bridging symbolic traces (choice points, proof trees, constraint stores) with neural traces (prompts, tool calls, sampling settings) into a unified audit log remains a practical challenge, but it is also a major opportunity for web-native agent engineering.
- **Proof artefacts as user-facing explanations.** Proof trees and derivation traces are precise but not always pleasant to read. An LLM could be used as a commentary layer that turns such traces into structured explanations (step-by-step narratives, minimal justifications, or counterfactual “what would have to change” analyses), while the Prolog artefact remains the authoritative source of truth.
- **Intermediate representations.** Standardising typed schemas for the boundary between neural and symbolic modules reduces brittle prompt engineering. JSON

with explicit types is often sufficient, but richer formats such as JSON-LD can help when provenance and linking matter.

- **Safety envelopes.** A reliable agent needs a policy layer that is external to the model and enforceable by construction. In Web Prolog this naturally lives in the symbolic backplane and in the supervisory wrapper.
- **Traffic optimisation on the Prolog Web.** Because Web Prolog makes messaging and distribution explicit, it naturally exposes telemetry (latency, queue depths, error rates) that could support learning-based optimisation of higher-level traffic patterns on the Prolog Web, such as pacing, batching, replica selection, or load-aware routing. The architectural twist is that this can be done with hard safety and policy constraints enforced by the symbolic and actor-level substrate: learning chooses among allowed actions, while Prolog defines what is allowed.

8.5 User-interface agents

Content is king. – *Bill Gates*

Conversation is king. Content is just something to talk about. – *Cory Doctorow*

The Agentic Prolog Web is a substrate for multi-agent interaction in general, but many of the most common and economically important agents are not peers in a negotiation protocol. They are user-interface agents: components that mediate between a person and a computational environment. In such settings, the engineering problem is not only what the agent can infer, but how it manages an interaction over time: when to ask, when to confirm, how to repair misunderstandings, how to handle silence and timeouts, and when to refuse or escalate.

The social-robot “parrot” of Section 3.5.7 can now be given a brain without changing its transport architecture. The robot node may continue to own speech recognition, synthesis, gaze, and embodiment, while a remote brain actor owns the dialogue governor, symbolic knowledge, and any model wrapper. Utterances, interpretations, and action requests cross the boundary as messages. This separation keeps device-specific timing and effects near the body while making the conversational policy independently inspectable, replaceable, and supervisable.

This section therefore shifts emphasis from agent-to-agent coordination to dialogue as control. The central claim is that conversational behaviour should not be treated as an emergent property of a prompt. It should be treated as a program with states, transitions, and failure handling. Web Prolog is well suited to this stance because it can combine (i) symbolic representations of dialogue-relevant facts, (ii) executable parsing and interpretation, and (iii) an actor-oriented control shell, most naturally expressed as a statechart.

8.5.1 UI agents versus MAS agents

In classic multi-agent systems, interaction is often modelled as peer-to-peer exchange: negotiation, coordination, and distributed problem solving among relatively symmetric participants. User-interface agents are asymmetric. They exist to serve a person, and they are evaluated by human-facing criteria such as clarity, trust, timing, politeness, and error recovery. The success condition is not only that the agent eventually produces a correct plan, but that it maintains an intelligible and safe interaction while doing so.

On the Web, this role frequently manifests as a conversational web agent: an agent that accepts input in natural language (text or speech) and produces responses that guide the user through a task. In such systems, the boundary between reasoning and interaction is not clean. Questions, confirmations, clarifications, and refusals are themselves action choices that must be governed by policy.

8.5.2 Turn-taking and repair as control structure

Conversation, in the technical sense used in linguistics, is a jointly managed activity. Participants alternate turns under a largely implicit turn-taking system; many contributions occur in adjacency-pair patterns (question–answer, offer–acceptance, request–grant); and when misunderstanding occurs, participants engage in repair sequences to re-establish common ground.

For an engineered UI agent, the practical implication is that interaction cannot be reduced to a single request–response call. The agent needs persistent state and control over what may happen next. It must be able to suspend a task to ask a question, resume after an answer, back up after a misunderstanding, and time out or escalate when interaction fails. These are precisely the kinds of behaviours that state machines and statecharts are designed to express.

A useful mental model is to separate the conversational system into a *flow layer* and a *content layer*. The flow layer governs when the system may advance, when it must confirm, what to do on silence, and how to recover after a misunderstanding. The content layer is where parsing, belief update, and reasoning live. In a Web Prolog setting, both layers can be expressed as programs, but the flow layer benefits from an explicit statechart representation because the control surface is the point where reliability and user trust are won or lost.

8.5.3 Dialogue as proof: the missing-axioms view

A useful way to connect dialogue management to symbolic AI is the Missing Axioms Theory (Smith et al., 1995). In this view, the agent’s task can be phrased as an attempt to prove a goal. When the proof fails, the reason is often not that the rules are wrong,

but that some required facts are missing. Dialogue then becomes a controlled process for acquiring the missing axioms.

This perspective is already latent in the askable expert-system pattern from Section 8.2.4 (and, in web-native form, Section 8.3.1). There, proof search determines which question must be asked next. The limitation of the toy meta-interpreter approach is not the idea, but the control envelope: real dialogue involves interruptions, repairs, topic shifts, and timeouts. To handle those robustly, we need an interaction controller with persistent state and well-defined failure handling.

8.5.4 Statecharts as dialogue managers

A practical architecture is to separate flow from content. The flow concerns turn-taking, timeouts, confirmations, and repair. The content concerns parsing, storing extracted facts, and reasoning about constraints and next steps. Statecharts are a good fit for the flow layer because they support hierarchy and orthogonality, events, guarded transitions, actions, and a history mechanism.

In Web Prolog, a statechart actor can therefore serve as the authoritative dialogue manager, while Prolog code in the statechart's data model provides the symbolic substrate for interpretation and reasoning. A minimal illustration is an agent that collects two attributes (colour and size) and uses a history state to recover after a misunderstanding. Operationally, this means that the dialogue manager needs a representation of where it is in the interaction, what information is still missing, and what repair strategies are permitted.

```
<statechart datamodel="web-prolog" initial="Start">
  <datamodel>
    :- dynamic color/1, size/1.
    np(Attr) --> [a], adj(Attr), [apple].
    adj(color(red)) --> [red].
    adj(size(big)) --> [big].
  </datamodel>

  <state id="Start" initial="Color">
    <history id="H">
      <go to="Color"/>
    </history>
    <state id="Color">
      <onentry>
        writeln('What color do you want?')
      </onentry>
      <go to="Size"
        on="input(Input)"
        if="phrase(np(color(C)), Input)">
        assert(color(C))
```

```

    </go>
  </state>
  <state id="Size">
    <onentry>
      writeln('What size do you want?')
    </onentry>
    <go to="Final"
      on="input(Input)"
      if="phrase(np(size(S)), Input)">
      assert(size(S))
    </go>
  </state>
  <go to="H" on="input(_)">
    writeln('I did not understand.')
  </go>
</state>
<final id="Final">
  <onentry>
    size(S), color(C),
    format('You want a ~p ~p apple!~n', [S,C])
  </onentry>
</final>
</statechart>

```

Three elements matter here.

Repair via history

The history state supports a simple repair policy for unrecognised input. When an `input(_)` event fails to match any more specific transition, the machine emits a complaint and transitions to the history pseudostate. This re-enters the most recently active substate of the dialogue, causing its `<onentry>` prompt to run again and thereby re-asking the current question. In contrast to restarting the whole interaction, the repair stays within the current plan, resembling how human repair sequences typically preserve the larger conversational trajectory.

Parsing as a guard

Transition guards call DCG parsing directly, so the controller advances only when the incoming `input(Input)` can be parsed into the intended structured form. This makes the interface between interpretation and control explicit: the grammar determines what counts as understood, while the statechart determines when understanding is sufficient to commit to a control move.

Belief updates as explicit effects

Executable content makes belief updates explicit and localised. When an input is successfully parsed, the extracted fact is asserted as a named predicate in the statechart's data model. These stored facts can later be queried, audited, and constrained by ordinary Prolog rules, rather than remaining implicit in conversational context.

Together, history-based repair, guard-based interpretation, and explicit data-model updates yield a compact pattern for dialogue control: recognise → commit → store, otherwise complain → repeat.

8.5.5 Dialogue as a governed interface

The dialogue manager of the previous subsection is not a new mechanism. It is the harness of Section 8.4 seen from the user-facing side: the same statechart-as-program that constrains an LLM's actions in a hybrid agent also constrains what that agent may say and do in conversation with a person. What changes is only the audience for the control envelope – peers and services in the agent-to-agent case, a human interlocutor here – and with it the criteria of success: clarity, timing, repair, and the principled placement of refusals and escalation. The model may be invoked for paraphrasing, interpretation, and question generation, but its outputs re-enter the system only through guards, schemas, and policies that are authoritative and auditable. The statechart thereby constrains the observable interaction pattern – turn-taking, repair, refusal, and escalation – even if the model behind it is replaced. Such portability depends on the shared SXML and profile semantics and on stable service contracts; it is not guaranteed by the diagram alone.

8.6 Summary

This chapter framed Prolog's relevance in an AI-driven software landscape as an operational question: can we make symbolic constraints, search, and explanations deployable and composable at Internet scale, so that they can supervise more fallible components in agentic systems? We first recalled Prolog's symbolic competence on a single machine: representing knowledge as executable clauses, planning as search, constructing proof objects for explanation, acquiring missing axioms through interaction, and using DCGs to connect language and structure. We then lifted these idioms onto the Agentic Web, treating Prolog engines as networked actors and letting proofs and explanations span nodes.

On that foundation, we distinguished the book's architectural agents from classical cognitive agents and contemporary LLM agents, and argued for a neuro-symbolic stance: treat large language models as powerful but fallible language modules, embed them in a symbolic control and constraint envelope, and make behavioural

authority explicit. We also made the failure boundary concrete: potentially blocking model calls belong in supervised wrapper actors, while statecharts initiate work asynchronously and react to replies, errors, and explicit timeout events. The ecosystem changes rapidly, but that makes precise semantics and explicit invariants more valuable rather than less.

Finally, we turned to user-interface agents as the most common and user-visible application class of these ideas. Dialogue is where control matters: turn-taking, repair, confirmation, timeouts, refusals, and escalation must be governed by something more concrete than a prompt. Statecharts provide this governance, and in Web Prolog they can be executed as actors whose guards and effects are symbolic programs – the same control envelope that supervises neural components in a hybrid agent, now turned toward a human interlocutor.

The chapter’s central claim is therefore not that Prolog should replace neural AI, but that agentic AI needs an explicit behavioural substrate – and that Web Prolog offers one way of making such a substrate symbolic, distributed, inspectable, and executable.

Part IV
From architecture to paradigm

Chapter 9

The Trinity as a web-native symbolic paradigm

The preceding chapters have introduced the elements of the Prolog Trinity ecosystem. We began with Web Prolog as a carefully delimited profile of Prolog, extended with Erlang-style message passing and concurrency-oriented semantics. We then examined Prolog agents – actors and nodes – as concrete computational entities, each equipped with clearly defined capabilities and lifecycle properties. From there, we explored the Prolog Web itself: the network that arises when such entities are allowed to interact across transport boundaries, using both sequential query-answer patterns and long-lived concurrent conversations. Subsequent chapters introduced reusable behavioural patterns, statechart actors, and the positioning of the Trinity in relation to the Semantic Web and contemporary AI architectures.

Taken individually, these components may appear as incremental extensions of existing technologies. Web Prolog resembles Prolog with additional concurrency primitives. The actor abstraction draws inspiration from Erlang. The sequential layer echoes long-established patterns of remote querying. Statecharts are extensions of ordinary event-driven state machines.

Chapter 1 introduced several of these elements – the two partitions, the structural affinities between Prolog and the Web, the mutual dependence of the Trinity’s components – as motivations for the project. This chapter returns to those observations and asks what paradigmatic pattern they jointly constitute when developed in full.

When considered together, these elements form a coherent architectural stance – one that may reasonably be viewed as a candidate paradigm for executable symbolic systems on the Web. And perhaps not *only* symbolic. Neuro-symbolic integration is a growing area across the AI community. The Web is becoming agentic, and it seems evident that a language that can serve both as a logic programming language and as an agent programming language may have something to contribute.

Here, *paradigm* is used in the ordinary programming-language sense, not to claim a Kuhnian revolution. Chapter 2 used Van Roy’s taxonomy (Van Roy, 2009) to locate Web Prolog among relational and logic programming, imperative search programming, and message-passing concurrency. The claim made here is that adding nodes, node-qualified process identities, profiles, and remote nondeterministic calls produces more than a collection of facilities: web distribution becomes part of the

computation model, and the inherited and added concepts constrain one another strongly enough to deserve consideration as a further point in that design space.

With that framing in place, this chapter asks a disciplined question:

What conceptual pattern emerges when logic programming, actor-based concurrency, Web transport semantics, and structured behavioural abstractions are deliberately integrated into a single design?

The answer, we will argue, is a paradigm characterised by a small number of interlocking commitments: a foundational separation between sequential and concurrent computation; a notion of portability grounded in interaction rather than implementation agreement; the agent as the primary unit of structure; a logical foundation that survives distribution; a distribution-first programming model; and the treatment of interaction governance as a first-class design dimension.

These commitments are mutually constraining. The language profile determines what can be exposed safely and portably; the agent model determines which long-lived behaviours can be made robust; the network substrate determines which forms of composition are realistic at web scale. It is precisely this mutual constraint that prevents the proposal from collapsing into “Prolog over HTTP”, “actors in Prolog”, or “a distributed Prolog database”. No single strand can be redesigned in isolation without consequences for the others.

9.1 The two partitions as a foundational principle

One of the central structural decisions in the Trinity architecture is the separation between two distinct partitions of computation: the sequential and the concurrent.

The *sequential partition* is the world of queries and relations. A client poses a query, a node evaluates it against a clause database, and solutions are produced nondeterministically. Over the Web, this partition manifests as remote querying, paged solution retrieval, and the disciplined mapping of backtracking onto HTTP-level interactions. The programmer’s mental model remains close to that of classical Prolog, even though transport boundaries intervene.

The *concurrent partition* is the world of actors and interactions. Computation is structured around long-lived processes equipped with mailboxes, identities, and lifecycle semantics. Communication is explicit, asynchronous by default, and potentially distributed across nodes. Communication failures and recovery are first-class design concerns.

In the Trinity, the two partitions are deliberately kept orthogonal at the programming model level. A sequential query may be realised as an actor conversation underneath, but the programmer need not know or care; conversely, an actor managing a long-lived protocol is not forced to reason about backtracking and choice points. Each mode has its own transport assumptions, blocking surfaces, and communication-failure semantics.

This separation provides a stable design axis. Systems can be built primarily in one partition while selectively invoking the other. Sequential queries may spawn concurrent actors. Actors may internally perform sequential reasoning. Yet the boundary remains conceptually intact.

Beyond the partition boundary, several orthogonal axes further structure the design space. The most important is the axis of *purity*. If the remote goal in an `rpc/2-3` call is pure, the call itself is pure; answers are logical consequences of distributed knowledge bases. The impure Prolog Web incorporates procedural and extra-logical features – cut, dynamic database updates, I/O – as well as the inherently procedural character of actor-based concurrency. A related axis is *openness versus risk*: the extent to which a node exposes itself as a shared execution environment, with the attendant security and resource-governance pressures. The profile hierarchy and the capability posture developed in earlier chapters are best read as disciplined responses to both axes.

In practice, systems mix pure and impure freely: pure queries for declarative reasoning, actors for coordination and lifecycle management. The guiding principle is *incremental relaxation*: begin with the purest model that fits the problem, relax purity only when necessary, and prefer explicit structure over implicit behaviour. Section 9.4 develops the purity axis in full.

A word is needed on how this orthogonality coexists with the mutual constraint developed in Section 9.3.1, since the two might appear to pull in opposite directions. They operate on different objects and at different levels. Orthogonality is a separability property of the two *partitions*, and it holds at the level of the programming model: a sequential computation and a concurrent one can be reasoned about, and composed, without either intruding on the other. Mutual constraint is an inseparability property of the three *components* – language profile, agent model, network substrate – and it holds at the level of architecture: none can be redesigned without consequences for the others. The two claims are compatible precisely because partitions and components are different cuts through the design space. Indeed, the orthogonality of the partitions is itself one of the things the architecture has to *secure*: Section 9.5 shows that it rests on the separation of Prolog engine state from actor process state, together with a committed interface between them. Orthogonality at the programming-model level is thus a *product* of mutual constraint at the architectural level, not a counterexample to it.

9.1.1 Two levels of abstraction

The sequential partition gives rise to a striking property: the Prolog Web can be read simultaneously at two levels of abstraction. On the upper level lies a *level of logic programs* – interlinked Prolog clauses distributed across nodes, where URIs serve as references between knowledge bases and `rpc/2-3` calls serve as the operational links. On the lower level lies a *level of communicating processes* – often realised as toplevel actors that are created, exchange messages, and eventually terminate.

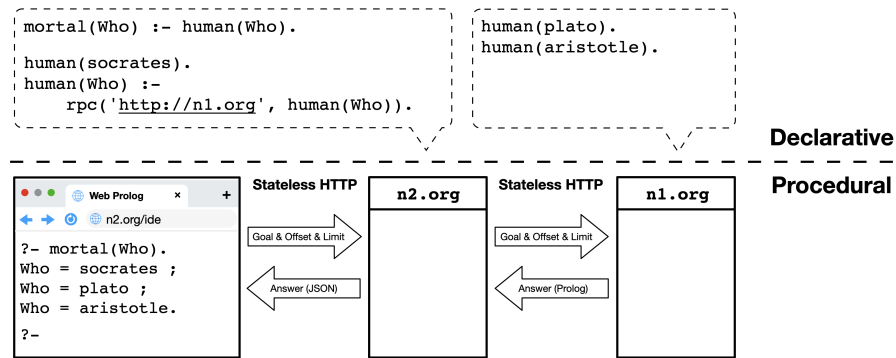


Fig. 9.1 Two levels of abstraction: interlinked logic programs above the line and communicating processes below it.

Figure 9.1 revisits the scenario of Figure 4.2, now drawing the distinction between the two levels explicitly. On the lower level, processes are created and destroyed; while alive, they send messages to each other. Nothing of this kind takes place on the level of logic programs. Entities such as processes, messages, and mailboxes do not exist there – they have been abstracted away. Drawing a line between these two levels is another way to express the distinction between declarative and procedural that so often appears in the story of Prolog. “What?” is specified at the level of logic and the corresponding “How?” is handled on the level of communicating toplevel processes.

The analogy works because pure Prolog explores each branch of a search tree independently of the others. Web Prolog, running pure Prolog on the Prolog Web, explores each *node* independently from other nodes. From the caller’s point of view, a call to `rpc/2-3` in the body of a rule is just another subgoal to process, another search tree branch to traverse, only this time in a Prolog context different from the context of the caller.

This two-level reading is a paradigmatic property. When the sequential partition is pure, the Prolog Web admits both a *logical* reading – in which distributed computation is understood as inference over distributed knowledge bases – and a *procedural* reading – in which the same computation is understood as coordinated message exchange among network processes. Sometimes, when actors and side effects are involved in a fundamental way, only the procedural reading remains. That both readings are available at all in a distributed setting, and that the programmer can move between them, is one of the defining characteristics of the Trinity as a web programming paradigm.

9.1.2 Unification as the universal data contract

The two-level structure would remain an architectural curiosity if the interface between client and node required the kind of “contract work” that characterises conventional web integration. In a typical web API, one reads documentation to learn an endpoint’s URL and parameters, inspects a response schema (often JSON with nested fields), writes deserialisation code that maps responses into application structures, and then maintains that glue as the API evolves. The fragility is structural: three logically coupled things are kept separate – the request shape, the response shape, and the rule that determines whether a response satisfies a request. Conventions such as OpenAPI, JSON Schema, and versioning try to restore this coupling, but only indirectly and after the fact.

In Prolog, unification makes the coupling direct. A goal sent to a node is simultaneously a request for information, a specification of the expected structure of answers, and the compatibility test between the two. The query

```
?- employee(Name, Dept, salary(Min, Max)).
```

both asks for employees and states that answers must have exactly this structure. If the node contains `employee(alice, engineering, salary(80000, 120000))`, the goal unifies and bindings flow back. If the node defines no `employee/3` at all – because the relation has been renamed, removed, or has changed arity – the intended Web Prolog error contract reports an existence error at the point of invocation: the mismatch is not a downstream runtime error buried in deserialisation code, but an immediate signal that names the missing relation. The query, as Chapter 1 put it, *is* the contract.

It is worth being precise about how divergence manifests, since this is where the contract earns its keep. A relation that has changed arity or disappeared should raise an error; a goal that fails to terminate may surface as a timeout; only the genuine absence of matching data should yield failure. This three-way distinction must be part of the node contract rather than left to a host Prolog’s configurable treatment of undefined predicates; Section 10.1.3 returns to this as an example of why profiles must settle implementation latitude. A poorly specified JSON API may collapse all three into a silent null, whereas the Prolog protocol has the vocabulary to keep them distinct. Projection is trivial: a client can ask only for what it needs. Evolution is structural: if a node moves from `employee/2` to `employee/3`, old clients do not silently receive nulls; they receive errors that say why.

Composition is native: constraints combine by conjunction, for example

```
?- employee(Name, engineering, salary(_, Max)), Max > 100000.
```

The combined query can be executed entirely on the node or split between remote and local evaluation. And because a Prolog predicate is inherently relational, a single relation supports many access patterns depending on which arguments are instantiated. The client chooses the mode at query time; the node simply unifies and enumerates. A conventional HTTP API needs separate endpoints for each of these patterns; in Prolog they are all the same contract, expressed once, as a goal.

The contract is structural, not semantic. Unification can ensure that terms have compatible shapes and that shared variables satisfy the same equality constraints, but it cannot determine whether two parties attach the same meaning to a predicate or to its arguments. That agreement must still come from documentation, a shared vocabulary, or a shared ontology. Chapter 7 develops the complementary division of labour: unification disciplines structure at the point of call, while Semantic Web vocabularies and ontologies make intended meaning explicit.

9.1.3 Backtracking as pagination

The structural correspondence between Prolog backtracking and HTTP pagination is not a surface-level adaptation but a genuine affinity. Both instantiate the same abstract idea: incremental enumeration under external control. A paginated endpoint exposes a procedure for generating results and a protocol for consuming them in slices; a Prolog toplevel actor does the same, with choice points serving as the resume mechanism and a “next” message as the page-turn signal.

Chapter 4 developed the operational details: the spectrum from naïve re-execution through cached-process resumption to explicit pid-based continuation, and the role of the logical update view in providing snapshot-consistent enumeration. What matters here is the paradigmatic observation. Unification provides the data contract, backtracking provides the enumeration model, and HTTP or WebSockets provides the transport – with each layer reinforcing the others rather than working against them. This three-way alignment is one of the reasons we regard the Trinity not merely as a practical engineering effort, but as a paradigmatic fit between logic programming and the architecture of the Web.

9.2 From compatibility to interaction portability

The Prolog community has long understood portability as a matter of *compatibility*: the extent to which different systems happen to accept the same programs and produce similar results. Efforts such as the ISO Prolog standard have addressed this at the level of the core language. Yet, as surveyed in *Fifty Years of Prolog and Beyond* (Körner et al., 2022), portability issues remain widespread beyond the core – in libraries, constraint systems, concurrency models, and interfaces to the outside world.

Compatibility, even when it covers both syntax and behaviour, is an *empirical* property: it depends on the contingent alignment of implementations, and it is easily broken by innovation. It asks whether two systems happen to agree, not whether they are obliged to. The Prolog Web sharpens the problem further: in a network of nodes implemented on top of different Prolog systems, the question is no longer just whether two systems accept the same program and produce the same answers,

but whether they preserve the same observable *interactions* – the same patterns of queries, answers, and message exchanges as seen by clients and peer agents.

In this work, we adopt a different perspective. Rather than treating portability as empirical compatibility between systems, we treat it as a designed property of programs, defined over their observable interactions.

Interaction portability. A program is portable across a class of execution environments if its observable interaction behaviour – the patterns of queries, answers, and message exchanges it can produce – is invariant under relocation between those environments.

This shift has several important consequences. First, portability becomes relative to a well-defined class of environments: a design decision, not an accident of history. Second, interaction rather than textual acceptance becomes central: two systems may accept the same program and yet differ in observable interaction behaviour, and under this notion of portability such differences determine whether portability holds. Third, portability must be *designed* rather than *inferred*: it requires explicit control over the aspects of execution that affect observable interactions.¹

The Trinity provides the ingredients from which this stronger notion can be built. Web Prolog restricts the client-supplied fragment so that self-contained programs need not depend on uncontrolled host features; owner-defined predicates and optional libraries remain explicit dependencies. Agents and nodes structure execution around message passing and explicit interaction, making behaviour visible and comparable across environments. Node profiles then provide the first reference points relative to which interaction portability can be assessed.

The same logic reappears, in miniature, at the user-facing edge of the system. The statechart-governed dialogue of Chapter 8 (Section 8.5.4) defines an interaction contract of this kind: the statechart constrains the observable pattern of turns, confirmations, repairs, and refusals, while the language model behind it may be swapped, retrained, or re-prompted. What can be held invariant is the governed structure of the interaction, not the implementation that produces it. Where node profiles support the assessment of interaction portability across *relocation* between execution environments, the dialogue governor supports it across *substitution* of a fallible component – the same principle of designed interactional invariance, applied to a different axis of variation.

¹ The distinction drawn here has a parallel in linguistics, where *semantics* concerns the meaning of expressions in isolation, while *pragmatics* concerns how meaning is realised in interaction – including phenomena such as turn-taking, feedback, and conversational structure. What we call interaction portability is closer to a pragmatic than a semantic criterion: it requires not merely that two environments assign the same meaning to a program, but that they produce the same patterns of exchange when that program is used. Traditional compatibility, by contrast, is already partly semantic; the real gap it leaves is at the interactional level.

9.2.1 Node profiles as interaction contracts

Returning to the primary axis – relocation across execution environments – the notion of interaction portability is meaningful only relative to a well-defined class of such environments, and those classes must therefore be made explicit. Chapter 3 introduced node profiles as capability declarations – specifications of what language features and web APIs a node provides. A profile is not, by itself, a claim that any two nodes bearing its name have identical data, libraries, policies, or owner-defined predicates. It does, however, delimit the observable surface on which a stronger interaction contract can be stated and tested.

What varies across profiles is therefore both what a node can do and which aspects of its behaviour become relevant to portability. At each level the observable surface grows wider:

- **RELATION.** Observable interaction consists of answer substitutions delivered through the paged stateless API. Each node must provide an order stable enough for its own pages to be meaningful, as described in Section 3.3.5, but the profile specifies no canonical order shared by independent backends. Portability between two **RELATION** nodes therefore requires equivalent advertised relations and answer sets; pages from different nodes cannot in general be spliced into a single enumeration.
- **ISOBASE.** Observable interaction extends to the evaluation of goals in the **ISOBASE** fragment, including Prolog answer order, control constructs, and synchronous and asynchronous RPC. Incremental delivery is already present at **RELATION**; what **ISOBASE** adds is a language-level basis for asking whether answer order and continuation behaviour agree. Such agreement is required for a particular program to be interaction-portable, but it does not follow from the profile alone when owner code, libraries, or policies differ.
- **ISOTOPE.** Observable interaction extends to side effects within a session: I/O, dynamic database updates, and the execution of client-supplied code. Portability must therefore cover not merely answers and their order, but the effects produced along the way. This is the most demanding level within the sequential partition.
- **ACTOR.** Observable interaction extends to message exchanges, mailbox semantics, and process lifecycle. Portability here depends on the delivery guarantees, the selection semantics of `receive/1-2` (Section 2.3.4), the failure-notification vocabulary, and the lifetime discipline for spawned processes. These are precisely the parts of the actor contract that must be pinned down independently of any one implementation.

The profiles thus form a hierarchy of increasingly rich observable surfaces, and interaction portability becomes more demanding as one climbs it. What may be sufficient for a **RELATION** program – agreement on answer sets and paging behaviour – is not sufficient for an **ACTOR** program, which also requires agreement on the dynamics of interaction itself. Future conformance specifications can turn selected parts of these requirements into normative profile guarantees; the present profiles should not be read as having completed that work already.

Under this view, portability is no longer a fragile byproduct of partial standardisation. It becomes a first-class design principle: programs are written against a profile rather than a specific implementation, their meaning is assessed in terms of observable interactions, and systems are free to innovate internally as long as they preserve the relevant contract. As shown in Section 4.3.3, when a program is self-contained and the target environment implements the required profile, relocation may require no adaptation. Programs that rely on owner resources, optional libraries, or particular service semantics must carry those dependencies explicitly.

9.3 Agents as the primary abstraction

At the centre of this design space stands the agent. In the Trinity, the agent is not an optional library construct layered atop another abstraction. It is the primary unit of structure.

Actors are agents. Toplevels are agents. Nodes are agents. Statechart processes are agents in structured disguise. Even external systems – whether web browsers, services, or AI components – are treated as agents when interfacing with the Prolog Web. A node is not merely a web server that happens to execute Prolog goals; it is a situated participant that exposes capabilities under a policy, hosts durable knowledge and services, and mediates between clients, resident actors, and the surrounding network.

This agent-centric stance contrasts with other dominant structuring principles. In object-oriented systems, the object is primary. In functional architectures, the function dominates. In microservice ecosystems, deployment units often eclipse semantic structure. The Trinity instead adopts a process-and-message discipline in which identity, mailbox semantics, and lifecycle policies define the core. Logical reasoning occurs *within* agents; distribution occurs *between* agents; behavioural templates constrain *classes* of agents; and statechart hierarchies govern their reactive control.

The choice has far-reaching implications. It aligns naturally with distribution-first semantics: because the agent is already a process with a mailbox, the step from local to distributed execution does not require a change of abstraction. It supports fault containment and supervision: agents fail individually, and their failure is observable through links and monitors. It allows sequential reasoning and reactive control to coexist: the same agent may answer queries in its sequential capacity and participate in actor protocols in its concurrent capacity.

The agent-centric stance also supports a literal multi-agent reading. Once nodes host durable processes that interact over published protocols, the Prolog Web becomes a web-native multi-agent system in the architectural sense: a population of autonomous processes coordinating by message exchange. This is not a claim about rationality or intention in the AI sense. It is a concrete decomposition principle: each agent has a mailbox, a lifecycle, and a locally scoped program and database;

global behaviour emerges from explicit communication under failure and timing constraints.

We saw in Section 2.2.1 that Armstrong characterised the Erlang approach as structuring applications from large numbers of communicating parallel processes – an architectural infrastructure of well-defined, independently implementable sub-components. The same principle applies directly to the Prolog Web. But in the Trinity, the process-and-message discipline is not the whole story – it is held in place by a tighter structural property.

9.3.1 Mutual constraint

The three components of the Trinity are mutually constraining – a property more characteristic of a paradigm than a toolkit. The language profile constrains the agent model: because clients execute a restricted fragment, agents need not defend against arbitrary meta-calls, and the security posture can be structural rather than procedural. The agent model constrains the network substrate: because agents have explicit identities, mailboxes, and lifecycles, the interaction protocol must support naming, session tracking, and failure notification – requirements that are reflected in the design of the HTTP and WebSocket layers. And the network substrate constrains the language profile: because programs must cross transport boundaries, the language is shaped so that goals, answers, and code fragments are serialisable as Prolog terms without loss of meaning.

No single component can be redesigned independently without affecting the others. This tight coupling is not a fragility; it is the source of the design’s coherence. It is also why the Trinity is not well described as “Prolog plus actors plus HTTP”. The components do not merely coexist; they shape each other.

9.3.2 Capability-oriented security

One consequence of this mutual shaping is the security model. Because the language profile, the agent model, and the network substrate constrain each other, part of the security posture can be structural rather than imposed only by external checks.

The structural mechanisms that make the Prolog Web safe to open – inexpressibility, invisibility, and containment – are developed in Section 4.5. What matters paradigmatically is that these mechanisms constitute a *capability-oriented* layer. Opaque pids and narrowly scoped reply channels convey the ability to address actors. Possession of a pid is necessary but not always sufficient: node policy may still require authorisation for its use. Public node URIs and published service names, by contrast, are discoverable addresses governed by ordinary access policy, not secret capabilities. Inexpressibility is enforced at the language boundary: dangerous operations cannot be named in the client fragment. Invisibility is enforced by the address

space: a pid that has never been revealed cannot feasibly be guessed. Containment is enforced by lifetime dependency: when a session ends, its actor subtree disappears. These mechanisms reduce what scattered runtime checks must accomplish, but they complement rather than replace authentication, authorisation, resource governance, auditing, and transport security.

This structural character aligns the Prolog Web with a family of contemporary execution environments – browser-based JavaScript, WebAssembly, WASI – that share the same posture: code executes inside a deliberately bounded environment and acquires authority only through explicitly provided references. Traditional Prolog systems are designed around maximal expressiveness; Web Prolog inverts the default, treating expressive power as owner-controlled capability that can be selectively granted rather than silently assumed.

9.3.3 Five complementary views

The agent-centric design refracts into several complementary perspectives, each emphasising different linked entities and activities:

The Prolog Web of source code. URIs link programs to programs. Nodes may expose source code over HTTP; users and tools can “view source” by following links.

The Prolog Web of query solutions. URIs link callers to answer streams. A client asks a node to solve a goal and incrementally enumerates solutions. Interaction is sequential and pull-based.

The Prolog Web of actors. Pids link live processes with mailboxes. Interaction is asynchronous: `!/2` sends, `receive/1-2` receives. Linking and monitoring make lifetimes explicit.

The Prolog Web of behaviours. Actors implement behaviours – protocols specifying message syntax, semantics, and timing. The behaviour is the contract; the actor is the running entity.

The Prolog Web of nodes. Nodes are the “places” of the system: they store and serve code, host actors, enforce policy, and advertise capabilities. They are points of execution and accountability.

These are not competing architectures but complementary perspectives on the same system. A Prolog Web application may involve several simultaneously: source code that is browsable, goals that are queryable, actors that are addressable, behaviours that are composable, and nodes that are accountable.

9.4 The logical foundation: purity, belief, and scoped inference

The agent-centric view supplies the structural principle; the logical foundation supplies the semantic one. The Trinity is not merely a concurrent system that happens to use Prolog syntax. Logic programming plays a constitutive role, and the key to understanding that role is the notion of a *pure Prolog Web*.

9.4.1 Pure Web Prolog and the pure Prolog Web

As noted in Chapter 4, *rpc/2-3* retains the logical purity of the predicates it calls. This allows us to form the notion of *pure Web Prolog*:

$$\text{Pure Web Prolog} = \text{Pure Prolog} + \text{rpc/2-3}$$

Purity here means purity of answer substitutions as logical consequences; timeouts and connection failures may truncate the answer stream but do not corrupt it. The union of all fragments of the Prolog Web written in pure Web Prolog forms what we think of as the *pure Prolog Web*. With a large number of nodes running pure Web Prolog programs interlinked to other pure Web Prolog programs, we may *in theory* create a Web of Logic on top of the conventional Web – a layer that can *in principle* grow as large as we want it to grow while still adhering to the formal semantics of the pure subset of the Prolog language. In practice, this may never happen, but the *architecture* is there.



Fig. 9.2 A pure Prolog Web spanning the globe remains aspirational, but the architecture does not preclude it.

9.4.2 Epistemic reading and scoped inference

If agents are the primary abstraction and the pure Prolog Web gives them a logical character, then a call such as `rpc(URI, Q)` admits a natural epistemic reading: it asks whether the agent at URI *believes* that Q. A clause such as

```
human(X) :- rpc('http://n1.org', human(X)).
```

can, using a belief operator – equivalently, a predicate over reified formulas – be formulated as

$$\forall x[\text{believes}(n_1, \text{human}(x)) \rightarrow \text{human}(x)]$$

It makes perfect sense to say that one agent believes that `human(socrates)` is true while another does not:

```
?- rpc('http://n2.org', human(socrates)).
true.
?- rpc('http://n1.org', human(socrates)).
false.
?-
```

And the following query asks whether two agents *agree* that `human(X)`:

```
?- rpc('http://n1.org', human(X)),
   rpc('http://n2.org', human(X)).
X = plato ;
X = aristotle.
?-
```

The epistemic reading is not the only one available. More conservatively, the URI can be understood as specifying the *scope* of inference – the knowledge base with respect to which a query is to be evaluated. As argued in (Kifer et al., 2005), scoped inference is mandatory for realising Negation As Failure on the Web, because applying any form of the closed world assumption requires knowing the entire knowledge base first – something that is not possible for the Web since its boundaries cannot be clearly delineated. A third and more practical view is simply that `rpc/2-3` invokes a goal in the context of a remote *module*, with data encapsulation that prevents the mixing of predicates with the same name and arity. For pure relations on suitably aligned nodes, the three views – epistemic operator, scoped inference, remote module – can coincide operationally, although they remain conceptually distinct.

What about `rpc/3`? Code injection via `load.*` options does not jeopardise purity provided the final composition of the program executed is itself pure. The `limit` option is a *pragma*: it specifies the granularity of the conversation between client and node but has no effect on the meaning of the query.

9.4.3 The less pure Prolog Web

Not all of the Prolog Web is pure. ISOBASE nodes may make use of control constructs such as cut, and ISOTOPE nodes go further with side-effecting predicates, I/O, and dynamic database updates. What statelessness motivates is the exclusion of persistent state modification from the client-facing fragment – a protocol-driven restriction enforced, in keeping with inexpressibility, at the language level.

More fundamentally, the concurrent Prolog Web – the world of the ACTOR profile – is inherently procedural. Actor-based programming involves process creation and destruction, message ordering, selective receive, and failure propagation – none of which have declarative readings. A realistic application will typically mix profiles freely, using pure queries for declarative reasoning and actors for coordination and state.

This spectrum from pure to impure is itself a paradigmatic feature. Rather than forcing a choice between declarative and procedural – a tension that has shadowed Prolog since its inception – the Trinity makes the boundary explicit and navigable. The programmer can reason about which parts of a system admit logical readings and which require procedural ones, and can move between the two with architectural awareness rather than accidental drift.

9.5 The distribution-first model

Web Prolog adopts a distribution-first view: concurrency and distribution are not anomalies to be managed but the fundamental reality of computation. The same actor primitives apply locally and remotely, but remote placement additionally introduces serialisation, latency, authentication boundaries, disconnects, and partial failure. The actor model views computation as inherently distributed, with independent actors communicating asynchronously; sequential processing is a special case. For Web Prolog, treating distribution as normal rather than exceptional simplifies the programming model and aligns naturally with web-scale deployment.

The formal foundation for this stance comes from Svensson et al. (2010), who proposed a unified semantics for Erlang-like systems. Their key insight is to eliminate the distinction between local and remote communication: all messages travel through a single conceptual mechanism, and side-effecting operations are handled uniformly by a node controller abstraction. In that abstract account, placement can be treated as a configuration detail rather than as a split between two languages, even though it remains operationally observable. Web Prolog can adopt this cleaner semantic story from the outset, rather than being constrained by decades of backward compatibility as Erlang is. Svensson, Fredlund, and Benac Earle themselves leave open whether their semantics might ultimately fit a language other than Erlang. Web Prolog is a candidate for that role.

A useful way to read the distribution-first stance is through the classic goal of *distribution transparency*: making the distribution of processes and resources

invisible to applications. The Prolog Web does not aim for transparency by hiding the network behind a magic API; rather, it aims for a disciplined form in which the *same* conceptual operations – `spawn`, `send`, `monitoring`, `rpc/2-3` – apply locally and remotely. Latency, serialisation, security boundaries, and failure modes change with distance, but the programming model does not bifurcate into a “local” and a “remote” language. The profile hierarchy identifies the capabilities and protocol surfaces available across the boundary; precise conformance rules must say which observable guarantees travel with them. Distribution-first therefore means uniform primitives with explicit operational differences, not an attempt to make the network disappear.

The combination of Prolog’s execution model with this actor semantics suggests a modular route to a formal account. Conceptually, each process has both an actor component – including its mailbox and lifecycle state – and a *Prolog engine state* consisting of a goal stack and a current substitution. Most rules respect that separation, but the components meet at the actor primitives. A `send` constructs a message from the current substitution; a `receive` removes a selected message and extends the substitution by unification; `spawning`, `monitoring`, and `termination` likewise connect goal execution with process lifecycle. The crucial discipline is that these crossings are committed: backtracking never unsends a message or returns a selected message to the mailbox. Backtracking therefore remains local to an actor’s engine even though the two state components interact. A full development and proof of composability lie beyond this chapter and belong to the research agenda of Chapter 11. The separation and committed interface make orthogonality a plausible design objective; they should not be mistaken for a completed formal result.

9.5.1 The browser as a Web Prolog runtime

The distribution-first model extends naturally to the client side. A browser equipped with a Web Prolog runtime hosts what Chapter 3 called a *browser node*: a node in architectural terms, but not a public endpoint. It accepts no incoming network connections and therefore participates as a client and as a host of local actors. It can nevertheless use the same interaction primitives available to server-side code. The programmer should be able to use synchronous calls when blocking is acceptable, non-blocking request–response calls when responsiveness matters, and actor-style interaction for cases that genuinely require long-lived conversational state – all without being exposed to transport-level details.

The longer-term ambition suggested by this design is Prolog on both sides of the Web. At the same time, this should not be confused with an attempt to hide the Web or to force all clients into a single language. Many developers will continue to prefer writing front-ends in JavaScript and communicating with Prolog back-ends through explicit web APIs. The Prolog Web should make such APIs visible, well-specified, and robust. A Prolog-everywhere experience may make them largely invisible in the common case, but engineering robustness requires that they remain accessible.

9.6 Control as a first-class dimension

Consider the famous equation coined by Robert Kowalski (Kowalski, 1979):

Algorithm = Logic + Control

The kind of control we have in mind here is different from search control. Conversational and reactive systems are hard not primarily because they require sophisticated algorithms, but because they require fine-grained *interaction control* – the kind that governs turn-taking, orchestration, and long-lived dialogue flow. In the Trinity, this motivates the combination of three programming models: logic programming for knowledge and inference; actor-based programming for concurrency, distribution, and interaction; and statecharts for explicit, inspectable control structure. The three are complementary: each addresses a dimension of system design that the others leave open, while – as we show below – remaining tightly coupled to them.

9.6.1 Governance of interaction

If the Trinity were only a story about distributed queries and actor concurrency, it would leave an important practical question unanswered: how does one *govern* complex interaction so that it remains inspectable, auditable, and robust? Protocol sequencing, timeouts, retries, cancellation, interruption, recovery, and the disciplined handling of exceptional cases – these are the concerns that dominate real web applications and agent systems, and none of them are addressed by logic or concurrency alone.

Statechart actors are the Trinity’s response. The idea is not to replace actor programs with statecharts, but to offer a structured substrate for reactive behaviour in which states, transitions, events, timers, guards, and recovery paths are explicit and therefore reviewable. Statechart actors turn the actor model from a concurrency substrate into a *governable interaction platform*.

It is worth being explicit that control enters the paradigm as a genuinely interlocking commitment, not merely as a third orthogonal facility. The control layer is constrained by the other two, and constrains them in turn, in exactly the manner of Section 9.3.1. It is constrained by the language profile: a statechart’s guards and actions are Web Prolog goals, and because executable content is restricted to that fragment, the control notation inherits the same inexpressibility posture as the rest of the system – there is no separate scripting language to secure, which is why the design omits a dedicated `<script>` construct entirely. It is constrained by the agent model: the run-to-completion macrostep that gives a statechart its deterministic intra-machine coordination is built on the actor’s atomic-step semantics, and the lifetime of an invoked process is tied to the control state that invoked it – the very containment discipline of Section 9.3.2, now surfaced as a control construct rather than a session property. And it constrains the agent model reciprocally: once interaction governance is a first-class concern, the actor abstraction must expose the hooks

that statecharts require – monitoring, linking, and lifetime-dependent termination – as part of its public contract rather than as optional extras. The result is that control is woven into the same web of mutual constraint as the two partitions and the network substrate, which is what distinguishes treating it as a paradigmatic dimension from treating it as a convenient add-on library.

Chapter 6 committed to one realisation: SXML, an SCXML-inspired statechart notation with Web Prolog as its guard, action, and data-model language. The wider design space still admits alternatives, including SCXML itself with Web Prolog as a scripting layer. Either way, the relationship follows a familiar web pattern: a declarative control layer whose behaviour is scripted in a general-purpose language.

9.6.2 A platform for the Agentic Web

Chapter 8 placed the Trinity in a broader movement: software is becoming increasingly agentic. The Trinity’s claim is that the Prolog Web is a natural substrate for such systems because it integrates three capabilities that agentic applications repeatedly need: a web-native network of nodes, a concurrency model for durable interactions, and a symbolic language for explicit representation and constraint.

The Agentic Web framing supplies the *application pressure*: conversational and tool-using agents need explicit protocol state, recovery from partial failure, and coordination under bounded waiting. Statecharts supply the *governance mechanism*: inspectable control with timeouts, retries, and fallback paths as first-class structure. Logic programming supplies the *symbolic substrate*: knowledge, constraints, and goal-directed inference remain available inside agents and can be shipped, composed, and queried across node boundaries.

This synthesis motivates a pragmatic neuro-symbolic stance. Statistical language models are useful components in agentic systems, but they are not reliable as autonomous governors of protocol and state. The Trinity treats statistical components, when present, as powerful but fallible modules, and places responsibility for durable behaviour in explicit symbolic structures. “Agentic” does not merely mean “chatty”. It means *durably interactive under explicit rules*, in an open network where failure and delay are normal.

9.7 Executable logic on the Web

A recurring posture in web architecture is to treat logic as primarily *representational*: a way to publish structured meaning, express constraints, and support inference over shared data. The Semantic Web strand exemplifies this orientation. It emphasises common identifiers, interoperable data models, and well-defined entailment regimes. These are substantial achievements, but they leave open a practical question: where

does executable reasoning live, and how is it composed across independently hosted sites?

The Trinity’s distinctive move is to treat logic on the Web as *executable* in a disciplined sense. A node is not only a publisher of facts; it is a host for relational services, rules, and interaction protocols that can be invoked, combined, and enumerated as computations. In the sequential partition, a goal can be shipped to a node and answered nondeterministically, with unification as the data contract and backtracking as the enumeration model. In the concurrent partition, long-lived agents coordinate by message exchange while embedding explicit symbolic structure in the messages they exchange and the constraints they apply.

Once this substrate exists, “logic on the Web” is no longer confined to representation and offline reasoning. It becomes a first-class component of live systems: callable, composable, inspectable, and governable. This executable substrate is what makes it natural to treat the Trinity as a form of symbolic middleware for the Web.

9.7.1 Positioning relative to the Semantic Web

The Prolog Web and the Semantic Web are similar in spirit: both treat the Web as a computational environment, both rely on global naming and logic-based machinery, and both aim at decentralisation. But they emphasise different things. The Semantic Web places its centre of gravity on interoperable data models and shared semantic commitments; the Prolog Web places its centre of gravity on executable relational services, process-level structure, and web-native agents. The detailed comparison belongs to Chapter 7, where Table 7.1 sets the two architectures side by side. Here the paradigmatic distinction can be stated more compactly.

The Semantic Web asks: *how can independently published data be understood and combined?* The Prolog Web asks: *how can independently hosted logic be executed and composed?* The two questions are complementary. RDF, OWL, and SPARQL provide machinery for representing and querying structured knowledge. The Trinity adds an execution substrate: a way to run queries, host rules, ship code, and maintain long-lived interaction under explicit operational rules. The paradigmatic claim is that the Trinity fills a gap the Semantic Web has long acknowledged: the gap between representation and execution, between knowing and doing.

Although this section uses the Semantic Web as a concrete reference point, most of the underlying issues are not specific to RDF, OWL, or W3C standards. Property-graph systems and large-scale knowledge-graph platforms have converged on similar concerns. The Prolog Web addresses this broader design space, offering an executable logic-based substrate that can sit alongside different graph representations rather than competing with any single one. This is consistent with the multi-stack architecture for the Semantic Web proposed in (Kifer et al., 2005; Horrocks et al., 2005).

9.8 Comparison to other approaches

The claim that the Trinity constitutes a candidate paradigm invites a natural sceptical response: has this not been attempted before? Three precedents are close enough to warrant explicit comparison.

9.8.1 Linda and tuple-space coordination

Linda (Gelernter, 1985) introduced coordination through a shared associative memory, with pattern matching over tuples as the retrieval mechanism. The Trinity shares the emphasis on pattern matching as coordination, but the Prolog Web is a network of active agents with explicit identities and lifecycles, not a passive shared store. Coordination is directional: a client submits a goal to a named node and receives answers through a well-defined protocol. This directedness makes reasoning about authority, lifetime, and failure tractable.

9.8.2 Concurrent Prolog and the committed-choice languages

Concurrent Prolog (Shapiro, 1987), Parlog (Clark and Gregory, 1986), and GHC (Ueda, 1986) showed that SLD-resolution and concurrency are not incompatible. Their commitment mechanisms deliberately change the usual Prolog relationship between search and backtracking, and backtracking is absent from the concurrent partition. The Trinity makes a different trade-off. The sequential partition retains ordinary nondeterministic Prolog execution; the concurrent partition adopts Erlang-style message passing, making the non-declarative character of concurrency explicit. The two are deliberately orthogonal: backtracking does not roll back mailboxes, and actor communication does not interfere with proof search.

9.8.3 Oz/Mozart and the multiparadigm kernel language

Oz (Smolka, 1995), implemented in the Mozart system (Roy et al., 2003), is the closest and most instructive precedent. Like the Trinity, it combines logic programming, concurrency, and distribution in a single coherent design. Distributed Oz (Roy et al., 1997) adds network-transparent mobility and fault tolerance, and the textbook (Roy and Haridi, 2004) provides an unusually clean formal account through a hierarchy of kernel languages. The comparison therefore cannot rest on missing features; it must rest on design stance.

The decisive difference is that Oz uses a fully compositional, higher-order kernel language that accommodates its paradigms uniformly, rather than preserving Pro-

log’s characteristic combination of Horn-clause syntax, unification-driven clause selection, backtracking, and the program-as-database identity. Oz therefore belongs to a different language tradition, not to a compatibility profile of Prolog. The Trinity, by contrast, is defined as a *profile* of Prolog. It extends ISO Prolog with concurrency primitives and distribution infrastructure while remaining syntactically and semantically recognisable to anyone who already knows the language. This is a deliberately narrower ambition – but it is the right ambition for a proposal that aims to unify an existing fragmented community rather than replace it with a new one.

A second difference concerns the Web. Oz/Mozart achieves genuine network transparency through a custom protocol over TCP, but nodes are not URIs, processes are not addressable by HTTP clients, and services are not discoverable by web tooling. The Trinity takes the opposite stance: HTTP and WebSocket are the primary transport substrate, and the addressability of nodes by URI is a design commitment. The result is a system less general than Oz in the space of distribution models but more naturally embedded in the Web as it actually exists.

9.9 Why the Trinity fits Prolog

The Trinity architecture is not merely implemented *in* Prolog; it resonates with properties of the language that make the fit unusually tight.

Unification gives the ecosystem a universal data contract: structured terms serve simultaneously as data structures, messages, and query templates, and the same mechanism that matches a goal against a clause head also matches incoming messages against expected patterns. Nondeterministic computation provides a disciplined enumeration model that maps naturally onto Web-style incremental consumption. The proximity between code and data means that predicate shipping and remote goal execution can be expressed using ordinary language constructs rather than elaborate serialisation frameworks. And Prolog’s strengths in symbolic reasoning and explanation-oriented computation complement the broader ambition of supporting agents capable of reasoning about structured knowledge.

What tends to disappear when the pattern is transplanted to other languages is the degree of *semantic unity*. In Prolog, terms, clauses, goals, and substitutions form a single conceptual framework supporting knowledge representation, communication, and computation simultaneously. In most other languages these concerns are handled by separate mechanisms: data structures, serialisation formats, rule engines, and orchestration frameworks. The resulting systems may be powerful, but they lack the conceptual compression that Prolog provides. The Trinity is therefore best viewed as a web-native expression of the logic programming paradigm – the paradigm in which Prolog has historically found its most direct realisation.

9.10 The dual process revisited

Chapter 1 introduced a dual design process – *webizing Prolog* and *prologizing the Web* – and identified a set of concrete requirements in each direction: addressing, infrastructure, security, and culture on the webizing side; logic-driven querying, broader symbolic capabilities, Erlang-style concurrency, and browser execution on the prologizing side. Those requirements were not a wish list; they were a forcing function. The paradigm described in this chapter is, in large part, what emerged when those requirements were pursued simultaneously and allowed to constrain one another.

It is worth tracing the connection explicitly, not as a checklist but as evidence that the interlocking commitments described above – the two partitions, interaction portability, the agent-centric stance, the logical foundation, distribution-first semantics, and control as a first-class dimension – are not architectural accidents. They are responses to the dual process, shaped by the requirement that Prolog become genuinely web-native while the Web becomes a host for executable logic.

The four webizing requirements from Section 1.5.1 map onto the paradigm as follows.

Addressing and identification is not merely a naming convention; it is the mechanism by which scoped inference, the epistemic reading of `rpc/2-3`, and the five complementary views of the Prolog Web become possible at all. Without URIs as first-class identifiers for nodes, actors, and knowledge bases, cross-node interaction would require out-of-band configuration rather than being part of the ordinary programming model.

Web infrastructure and transports – alignment with HTTP, WebSocket, and JSON – is what makes the two-partition architecture viable as a web-scale design rather than a theoretical curiosity. The sequential partition maps onto stateless HTTP with paged retrieval; the concurrent partition maps onto stateful WebSocket connections; and neither requires a bespoke transport layer. The requirement that nodes be reachable by any HTTP client, not only by other Prolog systems, ensures that the Prolog Web remains an open layer accessible to the Web at large.

Security and scalability is answered jointly by the capability-oriented layer of the security model and the distribution-first stance. Nodes are autonomous; composition crosses node boundaries; and pure, appropriately identified stateless requests are amenable to caching and replication by ordinary web infrastructure. The structural security mechanisms – inexpressibility, invisibility, and containment – arise from the mutual constraint between language profile, agent model, and network substrate rather than from bolted-on runtime checks.

Standards and web culture is addressed by treating openness, documentation, and precise interface boundaries as part of the paradigm's commitments rather than as afterthoughts. The deliberate adjacency to W3C and IETF practices – from the transport choices in the Prolog Web to the statechart connection with SCXML and the complementary positioning relative to the Semantic Web – follows naturally from the design's boundary-oriented character.

The four prologizing requirements from Section 1.5.2 map onto the paradigm as follows.

Bringing logic-driven querying to the Web is the sequential partition itself, and the substance of the paradigm's most distinctive claim: that unification functions as a universal data contract, backtracking maps onto paginated retrieval, and the query is the interface description. This is the most immediate and broadly accessible contribution that Prolog makes to the Web, and it requires no concurrent infrastructure at all.

Exploiting Prolog's broader symbolic capabilities – knowledge representation, inference, meta-interpretation, planning, parsing – is the substance of the logical foundation and the reason the Trinity treats executable logic on the Web as its distinctive contribution. Chapter 8 develops this further by positioning Web Prolog as a *behavioural backplane* for agentic systems: the component in which constraints, control, and explanations are represented as programs and can therefore govern more fallible components.

Leveraging Erlang-style concurrency for persistent interaction is the concurrent partition: actors with mailboxes, failure propagation, supervision, and reusable behavioural patterns. The key architectural point, as argued throughout this chapter, is that concurrency *extends* the sequential foundation rather than replacing it; the two partitions are deliberately orthogonal.

Ensuring that Web Prolog can run in web browsers is addressed in principle by the distribution-first model, in which placement is a configuration detail and a browser-hosted runtime participates in the Prolog Web using the same interaction primitives as server-side code.

Two requirements deserve candid acknowledgment as open fronts. Security is the most significant. The paradigm identifies the right structural posture – capability-oriented authority, inexpressibility, invisibility, containment – and Section 9.3.2 identifies the part of the security posture that arises from the architecture. But a robust, standardisable security story demands more engineering detail than has been supplied: authentication, authorisation, auditing, and sandboxing across deployment environments remain areas where the architectural sketch must be filled in with concrete protocol-level work. Browser execution has progressed beyond a conceptual gap: the SWI-WASM demonstrator and the architecture described in Section C.3.2 show a worker-per-actor browser node. What remains open is a portable profile-level account, isolation hardening, resource governance, and systematic performance evaluation. Both gaps are real, and neither can be closed by argument alone; they require implementation and specification work that lies ahead.

What matters for the paradigmatic argument is that these requirements did not produce independent solutions. They produced a single interlocking design – the mutual constraint described in Section 9.3.1 – in which addressing one requirement simultaneously advanced others: the URI requirement enabled scoped inference, the transport requirement established the two-partition architecture, the concurrency requirement created the agent-centric stance. The paradigm is a trace of the design process that produced it.

9.11 Summary

This chapter has stepped back from the technical machinery developed in earlier chapters to ask what conceptual pattern emerges when the components of the Prolog Trinity are considered as a whole.

The answer is a candidate paradigm for web-native symbolic computation, characterised by several interlocking and mutually constraining commitments. Two orthogonal partitions – sequential and concurrent – provide a stable design axis that separates declarative reasoning from actor-based interaction. Interaction portability supplies a designed, contract-based criterion for asking which observable behaviours survive relocation; node profiles delimit the capability classes within which that criterion can be assessed. The agent serves as the primary abstraction, unifying actors, toplevels, nodes, and statechart processes under a single process-and-message discipline – and the five complementary views of the Prolog Web show how this abstraction refracts into a rich but coherent set of perspectives. A logical foundation preserves the possibility of pure, declarative reasoning even in a distributed setting, with the epistemic reading of `rpc/2-3` and scoped inference giving formal substance to what it means to query a web of knowledge bases. A distribution-first model applies common primitives locally and remotely while retaining the operational differences introduced by the network. Control, in the sense of interaction governance, is treated as a first-class design dimension through the integration of statecharts alongside logic and actors. And the mutual constraint among language profile, agent model, and network substrate – each shaping and being shaped by the others – is what gives the design its coherence and prevents it from decomposing into a loose collection of independent ideas.

Whether this constellation of commitments deserves the standing of a paradigm – a named and inhabited point in the space of programming models, taught, implemented more than once, and built upon – is ultimately a judgement that the community, not we, must make. What we can say is that these commitments are not accidental; that they emerged from a deliberate attempt to integrate logic programming, actor-based concurrency, and web architecture into a coherent whole; and that the result, for better or worse, does not reduce to any of its components.

Part V

What's in it for you and your community?

Chapter 10

Communities, interoperability, and adoption

Coming together is a beginning; keeping together is progress; working together is success.

Henry Ford

The earlier chapters developed the Prolog Trinity ecosystem as a technical proposal: Web Prolog as a disciplined profile, Prolog agents as programmable entities, and the Prolog Web as a network of nodes that supports both sequential query interaction and long-lived concurrent conversations. The remaining chapters turn from architecture to value: what, concretely, would this ecosystem make easier or better for the communities that might choose to build with it?

This chapter begins with the Prolog community itself: implementers, tool builders, teachers, and users who already care about Prolog's future. It then widens the lens to neighbouring communities – logic programming, the Semantic Web, Artificial Intelligence, web development, and Erlang – and asks where the Trinity might offer a useful point of contact. These are not two separate questions. A federated technology has value only if it strengthens the community from which it grows while making useful boundaries with communities beyond it. Our aim is therefore to be explicit about trade-offs: which problems the Trinity is meant to address, what it might enable, and which costs it would impose.

For more than five decades, Prolog has demonstrated the power of logic in computation. Yet, as Chapter 1 noted, its community has also identified persistent difficulties concerning portability, systems fragmentation, tooling, and Prolog's place in the post-desktop Web. The Trinity is offered as one possible response to those four concerns.

Before asking what the proposal might contribute, it is important to say what it is not. The Trinity is neither a rescue operation nor a replacement for existing Prolog systems. Web Prolog is a set of profiles and interaction contracts, not a new implementation to which users are expected to migrate. Its subject is the space *between* systems: the externally observable behaviour by which independently developed

Prologs may face the network and cooperate. What each system does behind that surface – its extensions, optimisations, licence, tools, and community – remains its own.

Participation is optional, graded, and additive. A system may implement only the *RELATION* profile and stop there; that is conformance at one rung, not failure to reach another. Higher profiles make larger demands, and *ACTOR* conformance is a substantial undertaking, but no profile asks an implementation to discard its distinctive facilities. The proposal therefore depends on existing systems far more than any of them depends on it. What follows should be read as an invitation to experiment with shared surfaces, not as a verdict on the systems, tools, or communities already in place.

We will again turn to *Fifty Years of Prolog and Beyond* (FYPB), which surveys the state of Prolog around its fiftieth anniversary. Figure 10.1 offers an impressionistic reminder of the plurality of implementations and communities discussed there, together with the more recent Trealla Prolog.



Fig. 10.1 Marks of thirteen Prolog systems represented in this chapter: B-Prolog, Ciao, ECLiPSe, GNU Prolog, JIProlog, Stryer Prolog, SICStus Prolog, SWI-Prolog, Tau Prolog, Trealla Prolog, tuProlog, XSB, and YAP. The collage illustrates plurality; it is not a ranking or a measure of current project activity. The marks remain those of their respective projects.

As a background to our discussion, the detailed SWOT analysis presented by the authors of FYPB proves particularly valuable. Their analysis systematically examines the strengths, weaknesses, opportunities, and threats characterizing the Prolog ecosystem, with the aim of reinforcing its strengths, addressing its weaknesses, seizing available opportunities, and mitigating potential threats.

Among Prolog's strengths, FYPB recalls not only unification and backtracking but also the tabling, constraint, concurrency, indexing, and tooling innovations developed by different systems. These strengths are not distributed uniformly, nor need they be. XSB is closely associated with tabling, ECLiPSe with constraint programming, SWI-Prolog with a broad library and integration environment, while other systems place different emphases on standards, compactness, portability, or execution performance. The diversity is technically valuable. The difficulty is that applications cannot readily combine these strengths across implementation boundaries.

Among the *threats* and *weaknesses*, FYPB gives particular attention to fragmentation and to Prolog's comparatively small user base. The plurality pictured in Figure 10.1 is not itself the problem: multiple implementations are a source of experimentation and resilience. Fragmentation becomes a problem when differences prevent code, tools, knowledge, and running services from crossing the boundaries between them.

10.1 A middle-way approach to systems fragmentation

According to the authors of FYPB, the fragmentation of the Prolog *systems* landscape is indeed a concern since while most of these systems adhere to the ISO Prolog core standard, many differences appear in the added features that are not handled by ISO.

While the diversity is a testament to Prolog's adaptability and the ingenuity within its community, it has historically presented a barrier to collaboration and code reuse. Applications developed in one Prolog system are often difficult to port to another. The traditional pursuit of complete "portability" – the dream of writing Prolog code that runs flawlessly on any system – while laudable, has proven elusive.

Fragmentation also leads to difficulties writing textbooks and tutorials, might be a threat to further language standardization efforts, and hinders the development of a healthy ecosystem.

This section explores three distinct strategies for addressing this long-standing challenge. The first is a bottom-up, pragmatic approach that focuses on incremental improvements and standardization of specific features to gradually enhance portability across systems. In stark contrast, the second approach is a top-down, visionary effort to create a single, unified Prolog that would integrate the best features of all existing systems. Finally, we will examine a third, middle-way approach that champions interoperability over uniformity, proposing a federated model where different Prolog systems can communicate and collaborate in real-time without losing their individual identities. This third approach, which is the focus of this book, is our own proposal.

10.1.1 Reducing systems fragmentation one bite at a time

One response is the *Prolog Implementers Forum*, which provides a place for implementers to discuss divergences and build consensus about shared features. Its *Prolog Improvement Proposals* (PIPs) offer a structured means of documenting, discussing, and evaluating language and library changes. Although PIPs are not themselves part of the ISO Prolog process, they can record implementation experience and prepare proposals that may later inform standardisation.

This is a realistic approach to increasing the portable part of Prolog: bottom-up, incremental, and grounded in implementation experience. Agreement on such

matters as library interfaces can narrow the portability gap without requiring systems to converge wholesale, and proposals tested in several implementations can provide useful evidence for later standardisation.

The limitation is one of scope rather than merit. Incremental convergence can reduce the cost of moving programs between systems, but it does not by itself provide a runtime architecture through which different systems can cooperate. Nor should a proposal mechanism be expected to supply an entire long-term vision. The Trinity therefore treats PIPs and federation as complementary: one can narrow avoidable dialect differences while the other specifies the boundaries across which differences may remain.

10.1.2 One Prolog to rule them all

For an especially visionary perspective, we may turn to one of the contributions in *Prolog: The Next Fifty Years*, where Gupta et al. answer the question – asked in FYPB but never answered – of whether it makes sense to aim for a unified language with a resounding “yes.” They propose an ambitious vision of a system that seamlessly integrates tabling, constraints, concurrency, and parallelism within a single coherent framework.

The idea of a unified system is not new. In a 2016 discussion on Stack Overflow,¹ Jan Wielemaker proposed:

Ideally we should sit together and assemble a new system that combines the speed of YAP with the tabling of XSB, the constraints of ECLiPSe, and the environment and interfaces of SWI-Prolog.

The approach of Gupta et al. goes even further than Wielemaker though, as they also aim to incorporate *Answer Set Programming* (ASP) into Prolog, or more specifically s(CASP), which combines the strengths of ASP and constraint logic programming in a top-down, query-driven model of computation that includes support for explanation generation. The hope is that this leads to a sophisticated evolution that consolidates various extensions and paradigms into a cohesive programming environment even capable of commonsense reasoning. This integration is claimed to be designed not only to expand Prolog’s expressive power but also to align it more closely with the evolving demands of artificial intelligence.

The integrative impulse is already visible in existing systems. SWI-Prolog, for example, combines broad libraries and interfaces with facilities such as constraint programming, tabling, Just-In-Time indexing, and an implementation of s(CASP).² This is substantial integration, but Gupta et al. set the stronger research goal of making advanced features available *efficiently and seamlessly* inside one coherent system.

¹ Jan Wielemaker, Stack Overflow discussion, 2016.

² <https://github.com/SWI-Prolog/sCASP>

The approach of Gupta et al. is ambitious and intellectually attractive, but seamless integration of all these facilities is a long-term research programme. It also answers a different question from the one pursued here. A unified system seeks to assemble capabilities inside one implementation; the Trinity asks how existing and future implementations can cooperate while remaining distinct. These directions need not compete, and progress in either could benefit the other.

10.1.3 The middle way: interoperability as a federating force

The history of programming language standardization is largely a history of unification attempts: efforts to define a single language, a single type system, a single runtime, or a single ontology that all participants must adopt. Such efforts have produced valuable results – ISO Prolog, the W3C Semantic Web stack, and the JVM are notable examples – but they also carry a characteristic risk. Unification demands consensus on internal design decisions; the broader and deeper the consensus required, the harder it is to achieve and maintain. In the Prolog world specifically, the fragmentation of implementations and the difficulty of converging on shared standards have been persistent challenges.

The Trinity takes a different approach. It defines shared *interaction surfaces* – profiles, protocols, and APIs – while leaving internal implementation choices free. A node need not run a particular Prolog system; it need only conform to a profile that specifies which predicates, transports, and behavioural contracts it supports. Different nodes may implement the same profile in different ways, with different optimisations, different libraries, and different internal architectures. What is standardised is the *interface*, not the *engine*.

This is federation in a precise sense: independent parties agree on how to communicate without agreeing on how to think. The agreement is operational (we will exchange messages in this format, over these transports, with these semantics) rather than ontological (we will represent the world in the same way). This is also why the Trinity can position itself as scaffolding around existing Prolog systems rather than as a replacement for any of them.

Figure 10.2 illustrates the concept in concrete terms. Rather than collapsing the distinctive strengths of SWI-Prolog, ECLiPSe, XSB, and YAP into a single uniform system, we envision an ecosystem where each system is wrapped in a node implementation that allows it to converse with others. The diversity of approaches is preserved, yet they can all contribute to a common fabric of cooperation. While participation is voluntary, a node interface of this kind is indispensable for any system aspiring to take part in this shared Prolog Web.

As demonstrated in the proof-of-concept implementations accompanying this book, Web Prolog nodes owned by different entities and running in different locations can offload computations to one another and engage in mutual communication. This demonstrates the interaction model, but not yet cross-implementation interoperability. The discussion so far has been limited to homogeneous cases where nodes

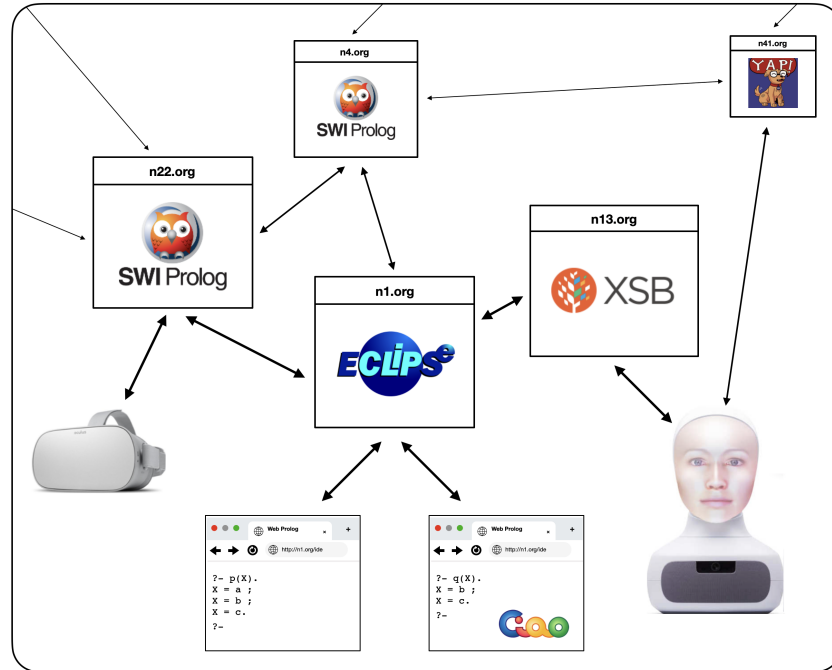


Fig. 10.2 The network introduced in Figure 1.2, now imagined with heterogeneous Prolog implementations behind its nodes. Clients and nodes share profile-defined interaction surfaces while implementations retain their own internal strengths. Later in this chapter the topology remains fixed while the picture opens to neighbouring ecosystems.

run on the same underlying Prolog system. The next challenge is to extend this capability to heterogeneous settings, where nodes may rely on different Prolog systems yet must still achieve predictable, real-time interoperability.

Portability vs. real-time interoperability

Much like the work pursued by the Prolog Implementers Forum (PIF), the advancement of this middle way requires cooperation among system implementers. However, while the PIF emphasises *portability* – the ability to move code between platforms offline – our proposal extends this focus to *real-time interoperability*. Portability enables code to move between systems; interoperability enables the systems themselves to interact during execution.

Because runtime conformance issues cannot be easily fixed on the fly, the demands placed on Web Prolog systems must be strictly defined. The Prolog Trinity ecosystem offers a vision of *cross-system interoperability as a federating force*. Instead of striving to make all Prolog systems behave identically, Web Prolog functions

as a *lingua franca* that enables distinct systems to communicate, collaborate, and interoperate through shared contracts.

Profiles must fix the latitude

Interface agreement must reach further than a common list of predicate names. ISO Prolog deliberately leaves some behaviour implementation-defined, and systems also differ in areas outside the standard. Two conforming systems may therefore disagree about matters that become observable at a network boundary: what happens when an undefined predicate is called, what a double-quoted literal denotes, which errors are raised, or how numeric limits are handled.

A Web Prolog profile must settle every such choice that affects its interaction contract. In this sense, a profile is partly a decision over the latitude left by the host language: it turns implementation-defined behaviour into profile-defined behaviour for the fragment it covers. The choice should follow from the contract, not merely from an implementation's default. For example, the ISO `unknown` flag permits different treatments of an undefined predicate. The Trinity's distinction between an error for a missing relation, failure for the absence of a solution, and timeout for a computation that does not answer requires the profile-equivalent of `unknown = error`, rather than silent failure. Similarly, portable source loading requires the `double_quotes` setting – and therefore the meaning of double-quoted text – to be fixed rather than assumed.

The design requirement is firm even where the final setting still belongs to standardisation. The present demonstrator's profile inventory primarily specifies routes, predicates, source-loading facilities, and actor behaviour; a standards-quality profile must additionally enumerate these language-level choices and test them. Conformance is consequently not a declaration of ancestry or intent but a claim about observable behaviour, checked by a suite shared among implementations. This is how a narrow RELATION surface and a much richer ACTOR surface can both be precise without requiring the engines behind them to be alike.

Reframing fragmentation as diversification

We emphasise that this approach allows Prolog systems to *retain their differences* on many levels. We encourage a “let a thousand flowers bloom” philosophy: implementers are free to add extensions and optimisations, provided they do not interfere with the Web Prolog layer. Through calls to the host language, a Prolog Web node can still offer the functionality of these unique extensions; indeed, their presence might be exactly why a developer chooses that specific node implementation.

Imagine a future where an application orchestrated through Web Prolog can dispatch a deductive query to a remote XSB engine, feed the results into an ECLiPSe solver, and manage the user interface via SWI-Prolog. The names are illustrative rather than commitments about a particular integration. The point is that profile

conformance could allow specialised services to be composed without requiring their implementations to merge.

From this vantage point, the current landscape of the Prolog community changes. What is often labelled as fragmentation can be reinterpreted in a more constructive light: as *diversification*. This shift in framing underscores the presence of a dynamic, albeit loosely coupled, constellation of systems that collectively sustain innovation and resilience.

Engineering a collaborative future

Much like the grand vision for a Unified Prolog, our approach entails substantial work. However, it demands less in terms of foundational research and more in terms of disciplined *engineering*. We already possess mature building blocks that need to be consolidated rather than re-invented. Furthermore, *inclusivity* is a core principle. To ensure that the barrier to entry is not insurmountably high – particularly for systems lacking multi-threading – the profile hierarchy proposed in Chapter 3 allows even the least powerful Prolog systems to integrate by conforming to simpler profiles.

The viability of this middle way would benefit from a constructive relationship with the Prolog Implementers Forum. Work that catalogues divergences and clarifies semantics can inform profile design; executable profiles and conformance tests can, in turn, expose disagreements that are otherwise easy to overlook. This offers a gradual path in which implementation experience precedes or accompanies standardisation, without presuming that the Forum must adopt the Trinity’s particular programme.

If the Prolog Web succeeds, it will not be because every Prolog system has been replaced by a single implementation, but because many implementations have adopted compatible interaction surfaces. Success is measured by interoperation and reuse, not by replacement. The same principle extends to neighbouring communities: the Trinity does not ask the Erlang community to adopt Prolog, or the Semantic Web community to abandon RDF. It asks whether there is value in a shared execution substrate for logic-based agents on the Web – and offers a concrete proposal for what that substrate might look like.

10.2 How to deal with community fragmentation

The Web is more a social creation than a technical one. I designed it for a social effect – to help people work together – and not as a technical toy. The ultimate goal of the Web is to support and improve our weblike existence in the world.

Tim Berners-Lee, 2000

In addition to systems fragmentation, the authors of *Fifty Years of Prolog and Beyond* identify fragmentation of the Prolog *community* as a concern for standardisation and long-term growth. Some grouping around implementations and research traditions is natural and productive, but it can make knowledge and effort harder to share across the community as a whole.

The technical Trinity cannot solve a social problem by itself. It can, however, provide shared objects around which collaboration may form: profiles to debate, conformance tests to improve, nodes and services to reuse, and web-based environments that are not tied to a single installation.

The rest of this section organises the possible contribution under three practical headings: interaction, collaboration and sharing, and learning. A stronger shared identity may follow from those activities, but it cannot be engineered directly.

10.2.1 Coming together through interaction

A programming language ecosystem, defined as a dynamic network of software, developers, and system implementers, relies on continuous interaction for its health and sustainability. The advent of the World Wide Web fundamentally transformed these ecosystems, shifting them from localized groups to interconnected global communities via platforms like GitHub and Stack Overflow. While the Prolog community has leveraged these tools to some extent, the authors of *Fifty Years of Prolog and Beyond* highlight the absence of a strong, unified online presence compared to the vibrant communities surrounding languages like Python or JavaScript.

The “All Things Prolog” website, sponsored by the Association for Logic Programming, is a valuable system-independent homepage.³ A complementary possibility is a dynamic *portal* for the Trinity. We must distinguish between the *technical* Prolog Web – the infrastructure of nodes and agents – and the *social* Prolog Web – the network of users and developers. A portal could connect the two without claiming to replace the sites and forums that existing communities already use.

Such a portal could combine documentation and tutorials with the discovery service already described in this book, making public nodes and services visible and immediately usable. It could also link to community news and discussion, and host the playground proposed in Section 10.2.3. These are possible functions, not requirements for the protocol itself; they would have to earn their place by complementing rather than duplicating existing community infrastructure.

The underlying philosophy is that technical interoperability can be a powerful catalyst for community cohesion. If we can make the many different Prolog systems talk to each other over the Web, there is hope that the Prolog sub-communities will start talking to each other too.

10.2.2 Coming together through collaboration and sharing

Collaboration is only made possible through interaction. To strengthen the community, we propose a culture of “dogfooding” – where the community actively uses the

³ <https://prolog-lang.org/>

Prolog Web to build the Prolog Web. The most immediate early adopters should be Prolog users and implementers themselves.

This extends to the concept of *sharing*. Currently, we share code and ideas. The Prolog Trinity ecosystem enables us to share *computational resources* and *live processes*. Imagine a future where a developer can not only share a library on GitHub but also spawn a live demonstration agent on a public node, allowing others to interact with it immediately. This shifts the paradigm from social coding to *social computing*.

FYPB notes that portability issues extend beyond the language core to libraries and tools such as testing and documentation infrastructure. A profile-governed node would not replace package managers or system-specific tools, but it could provide a common execution target for examples, tests, and teaching material, and a stable surface to which cross-system tools can attach.

10.2.3 Coming together through learning

Ah, Prolog. Sometimes spectacularly smart, other times just as frustrating. You'll get astounding answers only if you know how to ask the question. Think *Rain Man*. I remember watching Raymond, the lead character, rattle off Sally Dibbs' phone number after reading a phone book the night before, without thinking about whether he should. With both Raymond and Prolog, I often find myself asking, in equal parts, "How did he know that?" and "How didn't he know that?" He's a fountain of knowledge, if you can only frame your questions in the right way.

Bruce Tate

Education is central to the future of any language. The interactive and distributed character of Web Prolog suggests a useful role for browser-based "Prolog playgrounds" that complement textbooks and standalone interpreters with immediately runnable, shareable examples.

Prolog playgrounds

Figure 10.3 shows three of the currently existing web-based Prolog playgrounds: SWISH (based on SWI-Prolog), the Ciao Prolog playground, and the Tau Prolog sandbox. These tools allow visitors to program in Prolog directly in the browser without installing software, and they have proven invaluable for education and quick prototyping.

These playgrounds serve different systems and audiences, and their differences are often deliberate. They also reveal a portability boundary: a program written for one environment may depend on its dialect, libraries, or interface conventions and therefore require changes in another. This is not an argument that the existing work should have been centralised. It is an opportunity to add a shared layer. A common Web Prolog profile could make a useful core of exercises and queries portable while allowing each playground to retain the facilities that distinguish it.

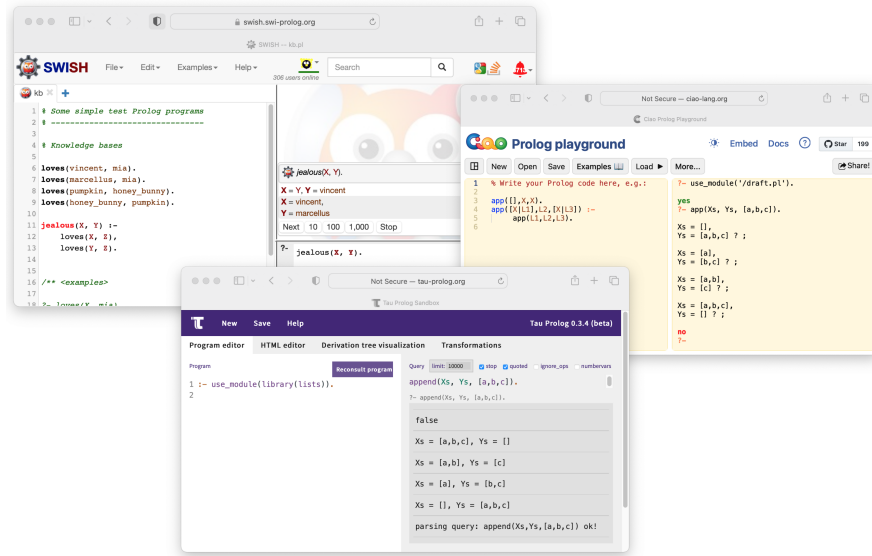


Fig. 10.3 A snapshot of three browser-based environments: SWISH for SWI-Prolog, the Ciao Prolog playground, and the Tau Prolog sandbox. Their different interfaces and dialects reflect their different purposes; a shared Web Prolog profile could add a portable teaching surface between them.

A Web Prolog playground served by an ISOTOPE node could provide one such shared environment for general instruction. When backed by an ACTOR node, the same interface could extend from ordinary queries to actor programming and distributed interaction. It should be viewed as a complement to SWISH, the Ciao playground, and the Tau Prolog sandbox: a place to teach the common profile and its network model, not a universal replacement for system-specific environments.

Web delivery removes installation friction, but it does not remove Prolog’s conceptual learning curve. Existing Prolog programmers can begin with the sequential profiles and encounter actor concepts only when they need them. Readers familiar with Erlang will recognise much of the concurrent vocabulary, while those new to actor programming can draw on the substantial Erlang literature. The profile ladder thus provides a pedagogical sequence as well as an implementation sequence.

Interactive, browser-based examples can present Prolog as a current language for queries, services, and distributed symbolic applications. Whether this attracts new users is an empirical question, but it lowers the cost of trying the language and of sharing a runnable example.

Learning materials

Staying close to traditional Prolog makes it possible to reuse the many good textbooks and tutorials already available. Erlang literature can likewise explain several principles behind Web Prolog's actor layer. Syntax, failure, logical variables, and the toplevel protocol remain specifically Prolog concerns, however, so dedicated Web Prolog tutorials are still needed. The demonstrator manual and browser tutorial are first steps rather than substitutes for such material.

10.3 Extending Prolog's user base

There are people who don't like popularity. It's much better to be exclusive and remote.

Robert Indiana

10.3.1 The imperative for growth

Prolog *used* to be somewhat popular. In the eighties, during the second AI summer, the Japanese Fifth Generation Computer Systems (FGCS) project placed Prolog at the center of the computing world. Communities formed, excellent textbooks were written, and the language became more efficient and expressive.

Today, the landscape is different. FYPB describes Prolog as having a "comparatively small user base." Exact counts from forums, surveys, and user groups are difficult to compare and quickly become obsolete, but the broad asymmetry is not controversial: Prolog occupies far less educational, industrial, and public attention than mainstream languages.

FYPB identifies this comparatively small user base as a threat. The concern is not a lack of technical capability, but limited visibility and the resulting scarcity of contributors, teaching material, libraries, and deployment experience. A web-facing profile cannot reverse that situation by decree. It can make Prolog easier to encounter, run, embed, and demonstrate, which are modest but necessary preconditions for growth.

Why does this matter? Prolog is an excellent language, but being good does not by itself make a language popular. In fact, the causality often works the other way around: a popular language is more likely to improve and become better. Popularity is not merely an intrinsic merit or a vanity metric; it is the engine of ecosystem health. Widespread use brings crucial benefits: more libraries, better tooling, more critical feedback, and greater pressure for innovation. Paul Graham articulated this dynamic in *Hackers & Painters* (2004):

So whether or not a language has to be good to be popular, I think a language has to be popular to be good. And it has to stay popular to stay good. The state of the art in programming

languages doesn't stand still. And yet the Lisps we have today are still pretty much what they had at MIT in the mid-1980s, because that's the last time Lisp had a sufficiently large and demanding user base.

While the Lisp ecosystem has since seen a revival through Clojure, Graham's central point holds true: a large user base fuels the ecosystem that, in turn, increases the utility and appeal of the language itself.

10.3.2 Key factors for popularity

In an essay on what makes a programming language popular, Antonio Cangiano's factors range from circumstantial advantages such as being the default language of an existing ecosystem or enjoying the backing of a tech giant, through inherited virtues such as a familiar syntax and a track record of years of active development, to qualities a language design can deliberately pursue: a killer application in a specific domain, good documentation and onboarding, a welcoming community, and technical innovations that make code more productive and maintainable.⁴

The first two are, as Cangiano himself notes, out of reach for almost any language, and they are out of reach for us: we cannot make Web Prolog the incumbent default of an existing ecosystem, nor can we summon a corporate patron. A handful more we can claim only by inheritance. Web Prolog is young, but its parent languages are not – Prolog dates to the early seventies and Erlang to the mid eighties – so the maturity of the underlying ideas comes with the lineage rather than from anything we have done.

What is worth dwelling on is the smaller set of factors that the Trinity is specifically designed to move. As Graham observes, a language rarely succeeds in isolation: one must either supply more than a language, or design the language to replace the scripting language of some existing system.⁵ The Trinity does the former. Web Prolog is not offered as a language in a vacuum but as the language of the proposed Trinity ecosystem.

The most potent version of domain usefulness, in Cangiano's account, is the *killer application*. His example is Ruby, which was ignored for the better part of a decade until Rails made it compelling for web development; people were willing to learn an obscure language precisely because something built with it was worth having. This is exactly the kind of pull the Trinity hopes to exert – not merely that Prolog is serviceable on the web, but that a sufficiently compelling web application surface could draw developers *into* logic programming who would otherwise never have approached it. To aim for the dominance Rails achieved would be shooting for the stars, but the target is the right one.

Cangiano is candid that this path carries a cost, and we should be equally candid. He observes that Rails was both a blessing and a curse for Ruby: a blessing because

⁴ Antonio Cangiano, "So You Want Your Programming Language to Be Popular".

⁵ <http://www.paulgraham.com/popular.html>

Ruby owes its popularity to it, a curse because the community's resulting fixation on web development narrowed the language's perceived applicability elsewhere – a fate that Python, never tied to a single killer app, escaped. This is precisely the danger that Moura and Wielemaker raise later in this chapter, that Prolog comes to be “downplayed as a Web thing” and that Web Prolog drifts free of the systems serving other applications. The risk is real, and we take it up directly in Section 10.5. Our defence against it is structural rather than rhetorical: Web Prolog is positioned as a *profile* of general-purpose Prolog, not as a replacement for it, so that a web-facing entry point need not become a web-only ceiling.

Documentation and onboarding can begin by aggregation rather than from scratch: existing Prolog documentation and textbooks explain much of the sequential language, Erlang literature supplies useful actor concepts, and browser-based environments remove the installation barrier. The Trinity still needs documentation of its own contracts. Its proposed portal should connect readers to existing forums and project communities rather than attempt to subsume them.

Familiarity is the factor on which we are weakest, and it is worth saying so plainly. Cangiano rates it close to decisive: a language must be innovative enough to be worth the switch yet familiar enough not to provoke a negative reaction on first contact, and he attributes much of Lisp's marginality to the foreignness of its syntax. By that measure Prolog's syntax is a genuine liability for the mainstream developer raised on Python or JavaScript, to whom it will look alien. The honest qualification is narrow: for the Erlang programmer, Web Prolog's syntax will present few obstacles, and that affinity matters for the specific audience the actor-based half of the Trinity addresses. But we should not pretend that familiarity counts in our favour with the larger population; it does not.

Cangiano's survey concerns languages in general. For a web technology such as Web Prolog, where the point is interoperability between independent implementations, we would add one factor of our own: a shared specification. Profiles, protocol documents, and conformance tests allow documentation, clients, libraries, and services to accumulate around more than one implementation. Formal standardisation might eventually strengthen that effect, but no institutional backing should be assumed. Chapter 11 discusses possible processes and venues; the immediate task is to make the technical contracts precise enough to be tested and criticised.

The factor we can claim most directly is technical innovation in service of productivity and maintainability. That, after all, is what logic programming has always been for.

10.4 Opening the network to neighbouring communities

The case for federation does not stop at the boundary between Prolog implementations. Rule engines, triple stores, ASP solvers, Erlang services, model-serving stacks, and web applications need not adopt Prolog or surrender their own semantics in order to participate. They may instead expose selected capabilities through a specified

surface, or remain behind an owner-side adapter. The adapter work and semantic mapping required at such a boundary are real engineering costs. As Section 10.1.3 emphasised, profiles fix observable contracts; they do not prescribe the internals behind them.

Figure 10.4 is the third view of the network first shown in Chapter 1 and revisited earlier in this chapter. The topology and clients have not changed. What has changed is what some nodes expose and what external components may stand behind them. At `n4.org`, the general SWI-Prolog mark has given way to *cplint*, indicating probabilistic logic programming offered through that implementation. Database cylinders suggest RDF stores behind logical interfaces, while the Python and model-provider marks suggest an owner-side adapter to a neural service. These marks are illustrative components, not additional node types, protocol dependencies, or endorsements.

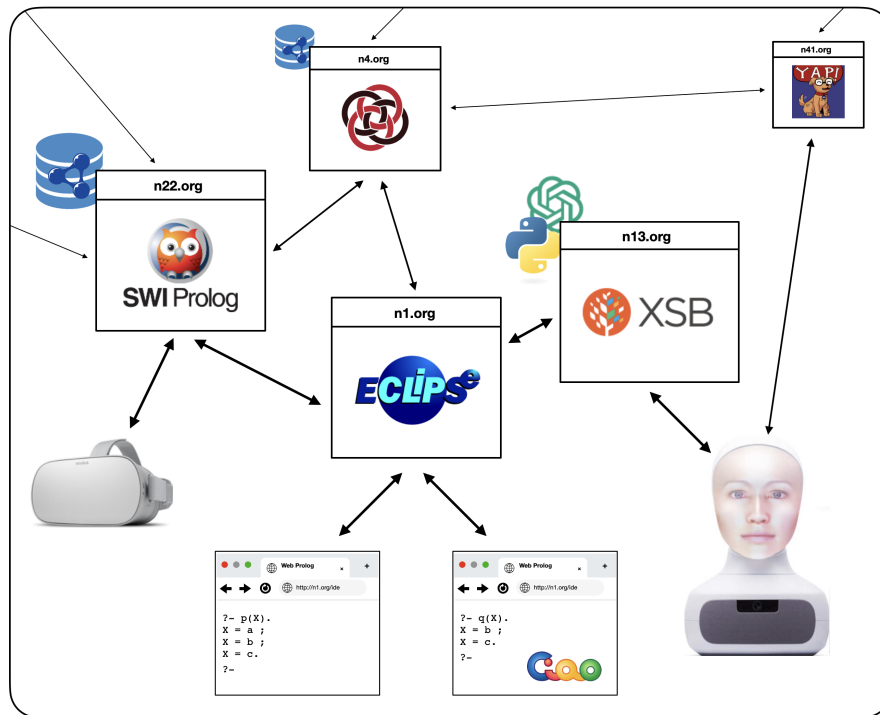


Fig. 10.4 The network of the opening chapter, now opened to neighbouring ecosystems. Node `n4.org` exposes probabilistic reasoning through *cplint*; the database, Python, and model-provider marks indicate possible backing services. The network and its clients remain unchanged.

The following examples ask what such boundaries might offer to logic programming, the Semantic Web, AI, web development, and Erlang. They do not claim that all the illustrated integrations already exist. Where the demonstrator supplies evidence, we say so; elsewhere the text describes an integration path or a research possibility.

10.4.1 For the broader logic programming community?

The logic programming landscape includes ASP, Datalog variants, constraint systems, probabilistic languages, and multi-paradigm systems that are not Prolog implementations in the sense required by ISOBASE. Such a system should not advertise a profile whose language contract it does not implement. It can instead expose a deliberately small logical API through RELATION, or place a profile-conforming controller in front of its native query interface. The wrapper must translate both data and meaning: what constitutes one answer, how alternatives are ordered, how errors are represented, and which calls are authorised.

Probabilistic and specialised services

The cplint system, introduced in Section 2.1, provides a useful intermediate case. It is a SWI-Prolog library and is available through cplint on SWISH. Because SWISH builds on Pengines, the following existing Pengine interaction can call it directly:

```
?- pengine_rpc('http://cplint.ml.unife.it', prob(blue,Prob), [
    src_text("
        :- use_module(library(pita)).
        :- pita.
        :- begin_lpad.
           blue : 2/3; red : 1/3 :- urn1.
           blue : 25/55; red : 30/55 :- urn2.
           urn1 : 0.5 ; urn2 : 0.5.
        :- end_lpad.
    ")
]).
Prob = 0.5606060606060606.
?-
```

This demonstrates remote access through the protocol lineage from which Web Prolog grew; it does not by itself establish conformance to a current Web Prolog profile. A node owner could expose an equivalent, bounded relation such as `urn_probability/2` through RELATION without exposing cplint's entire programming environment. LPS, also available through SWISH, presents a similar opportunity, though its visual results require an explicit agreement about how an execution trace is represented.

ProbLog, implemented primarily as a Python toolbox, is a different case. Its declarative character does not make it an ISOBASE implementation: ISOBASE fixes a Prolog language and toplevel contract, not merely the absence of side effects. A more honest first integration would therefore publish selected ProbLog computations as RELATION predicates. The same pattern applies to Picat's constraint solving, planning, and optimisation facilities. A richer profile could be justified later if multiple implementations converge on a larger common contract.

One stable model as one solution

Answer Set Programming makes the semantic mapping especially visible. For the program

```
cross :- not wait.
wait  :- not cross.
```

a straightforward adapter convention is to expose each stable model (Gelfond and Lifschitz, 1988) as one solution of an owner-defined `RELATION` predicate:

```
?- model(M).
M = [cross] ;
M = [wait].
?-
```

The backend retains ASP semantics; the wrapper defines only how its results cross the boundary. It must choose a canonical representation and an ordering stable enough for the pagination contract. Brave and cautious consequences could be exposed as separate relations. This is an illustrative adapter convention, not a claim that ASP has been absorbed into Prolog or that `model/1` belongs to the `RELATION` profile.

The larger opportunity is federation without forced unification. A probabilistic service, an ASP solver, and a constraint solver could be composed through small declared surfaces while remaining different systems. Achieving that requires more than a thin transport shim: profiles, schemas, error vocabularies, authorisation, resource bounds, and conformance tests all matter. The Trinity offers a place to specify those agreements; adoption must show whether the cost is justified.

10.4.2 For the Semantic Web community?

Chapter 7 compared the Prolog Web with the Semantic Web in detail. The modest claim here is complementarity. RDF, OWL, and SPARQL provide standard ways to identify, describe, and query data. A Web Prolog service can sit above such resources when an application also needs rules, stateful behaviour, or coordination with other services. It may front a SPARQL endpoint, translate selected RDF structures into terms, apply domain rules, and publish the result through a narrow logical API.

That boundary should not be mistaken for a universal representation. RDF graphs and Prolog terms have different semantics; an adapter must state how identifiers, literals, blank nodes, entailment regimes, and failures are mapped. Nor is every triple store thereby a Prolog node. The node is the component that implements the advertised Web Prolog contract; the store may be one of its private backing resources.

The practical offer is thus a programmable agent and orchestration layer that can reuse Semantic Web infrastructure without replacing it. Standardisation is discussed separately in Chapter 11; the present chapter need only establish why the communities have something concrete to discuss.

10.4.3 For the AI community?

Chapter 8 developed the case for symbolic, neural, and hybrid agents, including concrete interaction patterns between Prolog and large language models. It also described an experimental Model Context Protocol bridge. The point need not be repeated here. From the outside, the Trinity offers a control layer in which a model call is one effectful service among others: it can be authorised, timed out, checked against symbolic constraints, combined with stored knowledge, and incorporated into a longer-lived actor.

This does not turn every model endpoint into an actor. A node owner must still write an adapter, choose a stable request and result vocabulary, protect credentials, impose budgets, and decide what model output is permitted to affect. Conversely, a Prolog relation exposed to a model-oriented orchestrator may use MCP when a bounded, schema-described tool call is appropriate, while PTCP remains useful when the interaction requires a Prolog session, nondeterministic answers, or actor messaging. One knowledge base may therefore offer both interfaces without pretending that they express the same protocol.

Agents and the Web

Hendler’s early discussion of agents and the Semantic Web (Hendler, 2001), followed by his later question “Where Are All the Intelligent Agents?” (Hendler, 2007), identified an enduring problem: capable components do not by themselves supply an open architecture for long-lived, accountable interaction. Today’s model-based agents make that problem more visible, not obsolete. Some maintain state and use tools; others are short-lived wrappers around a model call. The useful contrast is therefore not “stateless neural agent” versus “stateful symbolic agent,” but whether state, policy, communication, and failure are explicit parts of the architecture.

Web Prolog actors are stationary rather than mobile. They can nevertheless reach remote knowledge by fetching a permitted representation, querying a resident service, or spawning a child actor on the node that owns the relevant resource. Mobility could be built above that substrate, but is not required merely to delegate work.

An active W3C Community Group on Autonomous Agents on the Web studies Web-aligned, transparent, and accountable multi-agent systems.⁶ Its work is a relevant neighbour, not evidence that the Trinity already implements a Hypermedia MAS architecture. A useful research programme would compare the group’s hypermedia interaction model with ACTOR messaging, identify the missing agent abstractions, and test interoperation rather than assume equivalence.

⁶ <https://www.w3.org/community/webagents/>

10.4.4 For web programmers?

For web programmers, the proposal is not a replacement for JavaScript or a general-purpose web stack. It is an additional tool for applications dominated by rules, constraints, symbolic data integration, or long-lived reactive behaviour. In those domains, relations can make decision logic explicit, while actors can maintain conversations and respond to events without reducing every interaction to an isolated request.

There is a learning cost. Unification, logical variables, and backtracking do not behave like assignment, mutable variables, and loops. The case for Web Prolog must therefore be demonstrated in working tools and examples rather than asserted in general terms. The browser ACTOR node described in Section 3.5.6, with implementation details in Section C.3.2, is one such piece of evidence.

Constructing user interfaces with statecharts

User interfaces must govern asynchronous events, legal operations, and transitions between modes. Statecharts make this control structure explicit. Chapter 6 combined that model with Web Prolog: guards are goals, actions are predicates, and the data model can be queried relationally. This is particularly attractive when an interface must check policies, maintain a structured domain model, or coordinate remote services. It is not a claim that every interface should be a statechart.

The relationship with JavaScript statechart tools is complementary. XState, for example, uses JavaScript functions for guards and actions; a Trinity statechart actor uses Web Prolog for executable content. Both make control state visible. The difference is the scripting and data model available inside the machine, and that difference matters most when logic and constraints are part of the application.

Conversational and embodied agents

The demonstrator's portal provides a concrete starting point for a browser environment in which statechart-driven and receive-loop actors control avatars. A fuller IDE could combine SXML and Web Prolog editors with execution, logging, and state visualisation. This continues earlier work on the *Dialog Web Lab*, a browser environment for authoring and executing SCXML dialogues with a simulated embodied agent (Junger et al., 2012).

Such an IDE would let a developer inspect the state machine while the underlying artefact runs as a genuine actor on an ACTOR node. The same message-level control could also be connected, through suitable adapters, to avatars in an immersive environment or to a social robot.

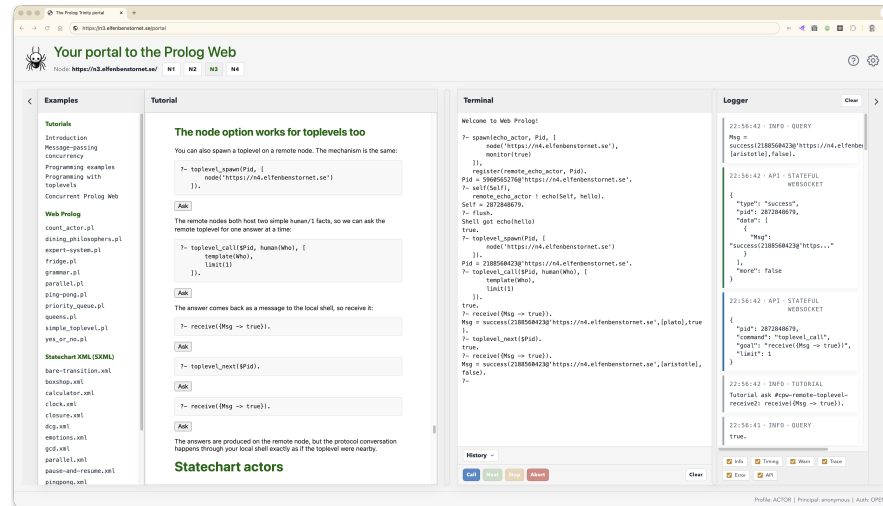


Fig. 10.5 The Trinity demonstrator equipped with two avatars that can be controlled by statechart actors or raw receive loops.

Controlling a social robot.

Furhat, a tabletop social robot head developed by Furhat Robotics, provides a physical example. Its realtime API uses WebSockets and permits an external client to send commands and receive events. An owner-side adapter could translate between that JSON protocol and the terms used by a Web Prolog actor or statechart actor.⁷ The control logic for an on-screen avatar may then be reusable, but the embodiment mapping is not free: gaze, speech, timing, errors, and physical capabilities need an explicit adapter. That qualification identifies the precise boundary at which the reuse claim can be tested.

10.4.5 For the Erlang community?

Erlang's influence is present throughout Web Prolog's actor layer: isolated processes, asynchronous messages, selective receive, monitors, and generic behaviours all acknowledge that lineage (Lager, 2019). Web Prolog is not, however, Erlang with Prolog syntax. It retains logical variables, relational guards, failure, and backtracking, while changing several concurrency choices to suit an open Web in which node owners and remote clients are distinct. Its links express a default one-way parent-child lifetime dependence rather than Erlang's bidirectional relation; a receive commits to a selected message but then executes an ordinary Prolog goal; monitoring distinguishes

⁷ Furhat Realtime API documentation: <https://docs.furhat.io/realtime-api/intro>. The author is not affiliated with Furhat Robotics, but knows members of the team personally.

logical failure from successful completion and exceptions; and profiles, authorisation, sandboxing, and containment mediate an open trust boundary. These semantics were developed where the relevant operations were introduced, rather than repeated here as a comparison table.

The differences still leave room for hybrid systems. An Erlang service can call a bounded Web Prolog relation over HTTP, and a Web Prolog owner can place an adapter in front of an Erlang service. A genuinely profile-conforming ACTOR node on the BEAM would be a larger project: it would have to implement PTCP, the term and error contracts, authorisation, and the specified Web Prolog semantics rather than merely exchange messages with an existing node.

There is also a research question here. A runtime combining BEAM-like process scheduling and fault isolation with unification, backtracking, and meta-level execution would occupy a different point in the language-design space from both Erlang and today's demonstrator.

The outward-looking examples share one principle: the offer is a boundary, not annexation. A system may expose one `RELATION` predicate, provide a richer profile-conforming controller, or remain outside the network and be called through an owner-side adapter. In every case, the important questions are observable: what is named, what one answer means, how alternatives and failures are represented, who may call the service, and which resources bound the call. Whether communities will invest in those boundaries is the adoption question to which we now turn.

10.5 Risk analysis

I've heard that the Web is dead. But all the applications that have killed it are accessing services using HTTP over port 80.

*Nat Torkington*⁸

In a thread on SWI-Prolog's Discourse forum addressing the "marketing" of Prolog, Paulo Moura voiced some scepticism regarding our approach:⁹

As important as the Web is, and therefore the significance of Prolog being a good programming language for Web applications, the rebranding you suggest is overreaching and doesn't make sense to me. Prolog, and logic programming in general, is much more than a specific application field. It's already bad that, in some circles, Prolog is being downplayed as a DSL tool. The last thing we need is for Prolog to be downplayed as a Web thing. Prolog is a general programming language and that's the vision a lot of people in the community is working hard to establish. Trying to sell Prolog was a Web silver bullet would only backfire in the same way Prolog association with the hype surrounding AI backfired in the past. While the Web can be a stage to increase logic programming marketshare and mindshare, is not and should not be, in my view, the end game.

Upon reflection, describing Web Prolog as a "special-purpose language" as we did back then was too narrow. Prolog should be recognised as a general-purpose

⁸ As quoted in (Nash and Loukides, 2015), attributing the remark to Nat Torkington.

⁹ Paulo Moura, SWI-Prolog Discourse, "Encouraging industry about Prolog".

language, while Web Prolog is better characterised as a specialised *profile* – a carefully delimited subset of the larger Prolog language. The right way to state the intent is therefore not that Prolog should be redefined as a web language, but that it should gain a clear web-facing profile and an interoperable surface. Prolog remains a general-purpose language; Web Prolog is a specialised profile that makes certain choices explicit at the boundary, so that nodes and clients can interoperate across the open network. The Web, on this view, is not an end game or a silver bullet, but a particularly effective stage on which Prolog systems can be made discoverable, composable, and runnable by others.

The deeper point is that “web-facing” need not mean “web-only.” A language may gain visibility through a particular application surface without surrendering its uses elsewhere. A credible entry point is not a proof of generality, but neither is it a loss of generality.

The Web is a particularly plausible entry point for Prolog because many web problems have an implicit logical shape: query and transformation over structured data, policy and access control, constraint-based selection, and reasoning over graphs. Presenting Prolog as a rules-and-constraints component inside a polyglot web architecture is therefore not a marketing trick; it is an alignment between strengths and an enormous application surface.

The main failure mode to avoid is not “being on the Web,” but making vague claims. The Trinity’s stance is deliberately modest: specify clear interfaces, provide runnable nodes, and let adoption be driven by concrete utility rather than by rhetoric.

Wielemaker agreed with Moura, and followed up with the following statement:

Prolog is quite suitable for the web, but in my experience Prolog is used for a very diverse set of tasks, many of which need Prolog more than the web. I’d love to see Web Prolog fly, in which case it is not unlikely to become more or less detached from the Prolog systems we see targeting other applications.

While we do agree that Prolog is used for many different tasks not related to the Web, we believe that there are ways to ensure that Web Prolog does not become detached from systems targeting other applications. The key here is to regard Web Prolog as a *profile* of the general-purpose Prolog language, and as a profile of a *future* Prolog. We refer to this future language as ISO Prolog 2.0.

Figure 10.6 presents one possible long-term relationship between ISO Prolog, Web Prolog, and a future general-purpose language. It extends the profile diagram introduced in Chapter 3 with an outer ring provisionally called *ISO Prolog 2.0*. The name denotes a design hypothesis in this book, not an existing standard or a claim over what a future ISO project must contain.

The outer ring restores precisely the kind of capability that Chapter 2 excludes from an open, untrusted Web Prolog execution environment: ambient file I/O and unrestricted access to sockets, streams, threads, and the host system. Those exclusions are therefore not presented as amputations from Prolog, but as stratification by trust and deployment context. A future general-purpose standard might also consider tabling, constraint logic programming, and other mature developments, but this book does not attempt to settle that inventory.

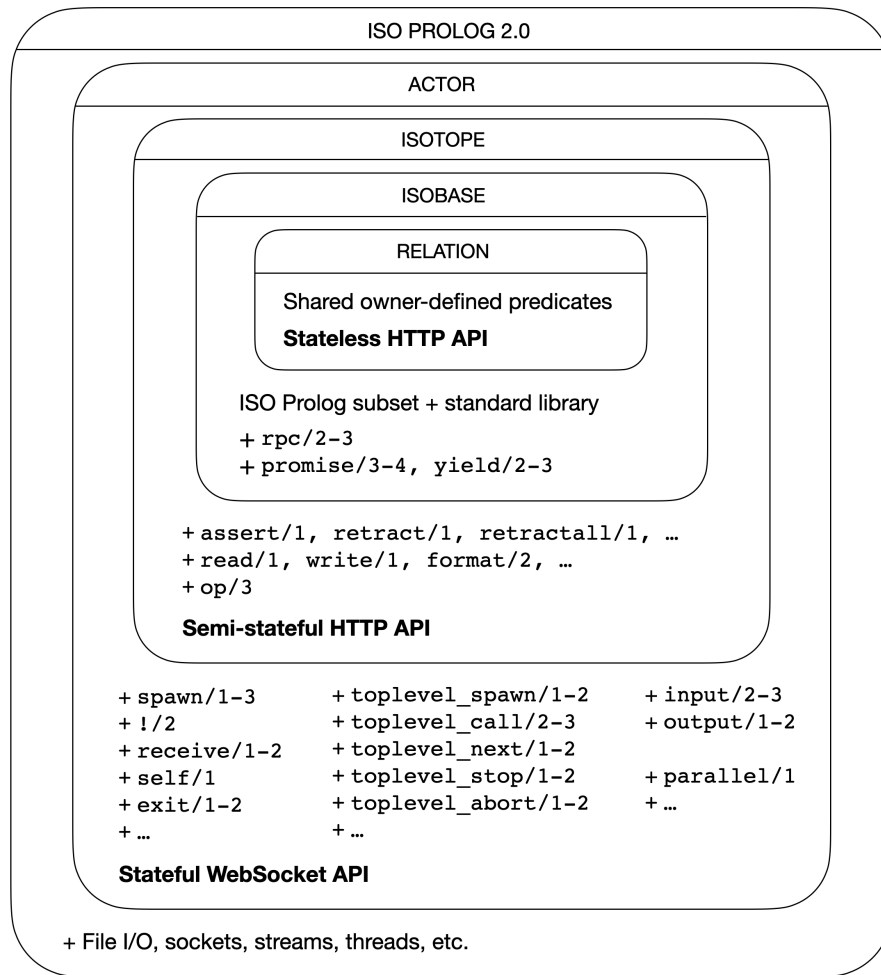


Fig. 10.6 The profile hierarchy completed by a hypothetical outer ring for ISO Prolog 2.0. Read from the inside out, the figure moves from open-web interaction surfaces towards a trusted, general-purpose environment that restores facilities such as file I/O, sockets, streams, and threads. The predicate lists are representative rather than exhaustive; the manuals define the current profile inventories.

The nesting in the diagram expresses the strongest version of the proposal: a future system conforming to the outer profile would also expose the Web Prolog profiles, including their actor and distribution contracts. That is a possible standardisation direction, not a present conformance rule. Other arrangements are possible – for example, a separately standardised Web Prolog layer implemented by adapters around otherwise conforming Prolog systems. The important claim is narrower: web-facing

profiles and a future general-purpose Prolog should be designed so that they compose rather than drift apart.

This suggests that work on Web Prolog could fit inside, or run alongside, work on a future Prolog standard. Chapter 11 considers that institutional and technical path in more detail.

10.6 Summary

The case made in this chapter is deliberately conditional. For portability, the Trinity proposes a profile-defined fragment and conformance tests rather than the portability of arbitrary host-system programs. For systems fragmentation, it proposes runtime interoperability without asking implementations to converge internally. For tooling, it offers common targets for clients, documentation, discovery, tutorials, and playgrounds. For the post-desktop Web, it supplies browser-accessible nodes, services, and actors rather than treating the browser merely as a remote terminal for a desktop system.

Beyond Prolog, the proposal is equally limited and concrete. A probabilistic program, an ASP solver, an RDF store, a model service, an Erlang process, or a robot does not become Web Prolog merely because a node can reach it. What can be shared is a carefully described boundary: the meaning of a request and an answer, the representation of alternatives and failures, the authority to call, and the resources consumed. Some such boundaries have precursors or demonstrations; others remain research proposals.

Protocols do not manufacture communities. Multiple independent implementations, careful specifications, conformance suites, useful applications, and sustained maintenance would all be required before the benefits described here became real. The demonstrator establishes that a substantial part of the design can be built; it does not establish that other Prolog systems or neighbouring communities will adopt it, or that new users will follow.

The proposal is therefore neither that every Prolog become Web Prolog nor that the Web become Prolog's only field of application. It is that Prolog should have a well-defined way to participate in the Web while remaining recognisably Prolog, and that independently developed systems should have a practical surface on which their diversity becomes composable rather than isolating. The same surface can create points of contact with neighbouring ecosystems without absorbing them. Turning that invitation into shared infrastructure is the subject of the final chapter.

Part VI
Final words

Chapter 11

From vision to execution

Vision without execution is just hallucination.

Thomas Alva Edison

At the beginning of this book, the Prolog Trinity ecosystem was introduced by means of a mind map. Its purpose was primarily orienting: to sketch the conceptual territory, name its principal components, and hint at the relationships between them. That diagram accompanied the reader through the early stages of the argument, serving as a visual shorthand for a still-emerging design.

In Figure 11.1 we return to the same mind map – structurally unchanged, but now adorned with three banners of the kind commonly used by the W3C.

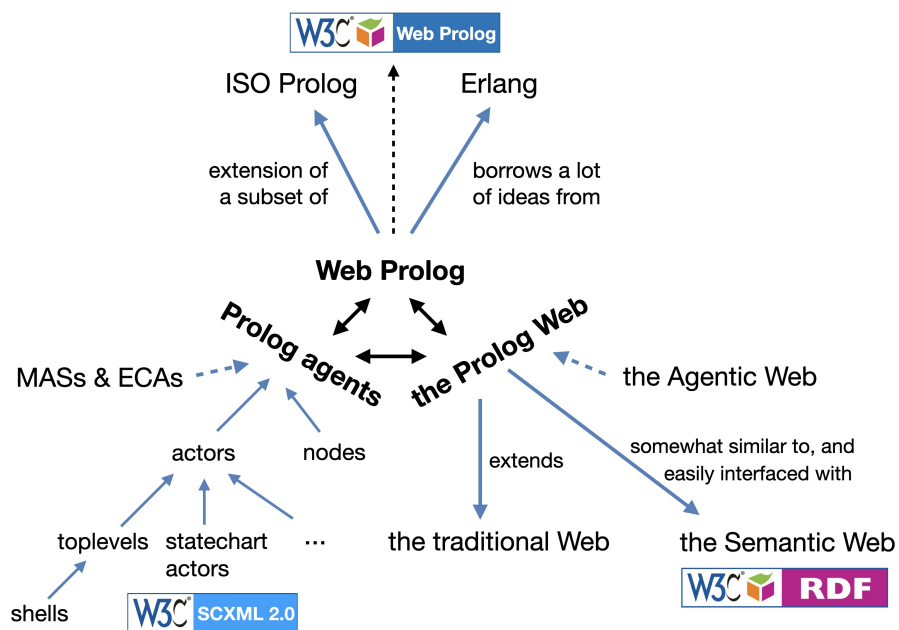


Fig. 11.1 The Prolog Trinity mind map revisited, now framed as shared infrastructure and adorned with three W3C-style technology banners. Of the three, only RDF is an actual, existing W3C standard; the other two are provisional marks of intent.

Such banners are not mere decoration. They are compact visual signs of collective agreement: indications that a technology has passed from an individual proposal to shared infrastructure, and that its definition, maintenance, and evolution are now matters of public process rather than private intent. A banner can compress years of discussion, specification work, experimentation, revision, and implementation experience into a single mark.

Of the three standards implicitly invoked by the Trinity vision, only RDF has completed that journey. Web Prolog and the statechart language have not. This asymmetry is neither accidental nor embarrassing. It clarifies the status of the proposal. RDF has been specified, standardised, implemented, deployed, criticised, and revised. Web Prolog and SXML remain forward-looking designs – argued for and partly implemented, but not socially ratified.

Nor is there only one possible destination. The demonstrator may develop into a serious platform centred on SWI-Prolog, with production quality valued above cross-system portability. The profile and protocol work may instead, or in parallel, become the basis of a federated programme involving several Prolog systems and perhaps standards bodies. The statechart work has yet another lineage through SCXML. These paths share much engineering, but they are not the same project and need not advance at the same pace.

This chapter therefore treats execution as a set of choices rather than a predetermined roadmap. It begins with the work still required by the design, then separates two plausible development routes. It considers what a standardisation programme would require if federation is pursued, what the first specification targets might be, how implementation and AI assistance can provide evidence, and how shared ownership could eventually arise. The two provisional banners in Figure 11.1 should be read in that conditional spirit: not as claims of authority, but as reminders of the work that would have to occur outside this book.

11.1 A research and engineering agenda

The demonstrator has converted several architectural claims into running experiments, including public services, browser-hosted actors, statechart actors, and cross-node PTCP interactions. It has also made the remaining gaps more precise. Some require ordinary product engineering, some require a normative specification, and some remain research questions.

Production hardening and operations.

The demonstrator is not yet a production platform. A serious deployment needs repeatable installation and upgrades, configuration management, persistent state and recovery policies, observability, resource accounting, overload behaviour, performance measurement, compatibility guarantees, and operational documentation. These requirements matter even if no second Prolog implementation ever joins the network.

Layered operational semantics.

Chapter 9 separated a Prolog engine's goal-and-substitution state from an actor's mailbox-and-lifecycle state and identified the crossings between them as committed effects: backtracking never unsends a message or returns a selected message to a mailbox. Extending the unified actor semantics of Svensson et al. (2010) with Prolog engine states, followed by a proof of suitable composability properties, is a research problem rather than a completed result.

Profiles and language latitude.

The profile inventories must be completed by the language-level decisions identified in Section 10.1.3: undefined-predicate behaviour, double-quoted terms, numeric limits, error terms, and every other host choice visible at the network boundary. Each decision needs a normative statement and, if interoperability is claimed, a conformance test.

Security and resource governance.

Chapter 4 established the structural principles of inexpressibility, invisibility, authorisation, and lifetime containment. A deployable public-node profile still needs baseline requirements for identity, transport security, quotas, logging, audit, credential handling, denial-of-service resistance, and adversarial testing.

Browser-hosted ACTOR behaviour.

SWI-WASM demonstrates that browser actors are possible, and Section C.3.2 documents the worker-per-actor design. A portable browser account must say what isolation and resource guarantees a client may rely on, which operations are unavailable, and how behaviour is tested across browser engines.

A Prolog-to-JSON mapping.

The architecture repeatedly uses JSON at web-facing boundaries, but the transformation from Prolog terms to JSON still needs a stable contract. It must cover atoms and strings, variables, compounds, dictionaries, character and numeric rules, cyclic or otherwise unrepresentable terms, and error representations. A canonical mapping is not the only possible answer; negotiated media types may be preferable where fidelity matters.

Extensible answer metadata.

Answer terms and their JSON representations may need a structured extension field for implementation-specific or diagnostic information. Generation identifiers, execution statistics, warnings, provenance, and resume tokens should be addable without changing the meaning of the core success, failure, and error protocol. The design must also say which metadata is stable enough for applications to depend on and which remains advisory.

Paged retrieval and consistency.

Offset/limit paging assumes a stably enumerable solution sequence. The design must decide whether mutations merely invalidate a continuation, whether point-in-time snapshots are supported, and how a client can distinguish these guarantees. Generation identifiers carried as answer metadata, or opaque resume tokens, are plausible mechanisms rather than settled API.

Standard datatypes and web resources.

Richer interaction with JSON would benefit from a portable dictionary-like

datatype. A web-facing library also needs URI parsing, resolution, composition, and prefix registration, together with decisions about XML support and HTML generation. These are valuable for an SWI-Prolog platform and become essential language work if several systems are expected to share source and protocols.

Further work on behaviours.

The server, supervisor, toplevel, and statechart examples demonstrate the value of reusable interaction abstractions, but the status of a behaviour remains underdeveloped. More work is needed on callback contracts, syntax, semantics, versioning, introspection, failure behaviour, and the relationship between a behaviour and the protocol it implements. For SXML specifically, Sections 6.7 and 6.9.4 provide a worked starting point rather than a finished standard.

Independent implementations and conformance evidence.

The SWI-Prolog demonstrator is one implementation lineage. If the goal is a federated Prolog Web, at least one independently engineered implementation of each proposed interoperability layer is needed to reveal hidden assumptions. A shared test suite and implementation report should record feature-by-feature agreement rather than reduce conformance to a single badge. This item is a condition of a federation claim, not a prerequisite for improving the SWI-Prolog platform itself.

This is an agenda, not a backlog for one implementation. Its items have different owners, evidential standards, and time horizons. Treating them as a single undifferentiated project would make the whole programme harder to judge and easier to overstate.

11.2 Two routes from prototype to platform

The distinction between a production platform and a federation is important enough to make explicit. Both routes begin with the same demonstrator and can share protocols, tests, documentation, and much implementation work. They diverge in what counts as success.

11.2.1 A production-oriented SWI-Prolog platform

On the first route, the immediate goal is a dependable SWI-Prolog-specific platform. SWI-Prolog's threads, engines, module system, sandbox, HTTP stack, WebSocket support, browser port, libraries, and development tools are assets to be used fully rather than hidden merely because another Prolog system may not offer the same facilities. Success is measured by security, predictable semantics, performance, operability, useful services, and the experience of people building real applications.

This route does not make profiles or protocols irrelevant. A stable boundary between clients and nodes remains valuable even when every node uses the same

engine. Profiles separate trusted from untrusted execution, make browser and server behaviour intelligible, support versioning, and give tests a public contract. What changes is the freedom to state explicitly that a facility is SWI-Prolog-specific and to optimise the platform around that fact.

A successful platform could later supply the implementation experience from which a portable core is extracted. It need not postpone useful engineering until consensus exists elsewhere.

11.2.2 A federated and standardised Prolog Web

On the second route, the defining goal is cross-system interoperability. Implementation latitude must then be constrained wherever it becomes visible at a node boundary. Profiles must be independent of SWI-Prolog defaults, term and error encodings must be portable, protocol state must be specified, and a second implementation becomes indispensable evidence.

Federation also changes the social problem. The work can no longer be owned by one implementation community alone. Other Prolog implementers must be able to alter the contract, reject assumptions, and demonstrate different internal strategies. If formal standardisation is eventually pursued, public drafts, issue tracking, conformance tests, and implementation reports become part of the engineering rather than administrative decoration.

This route is more ambitious institutionally and may be better developed in a separate specification project. It nevertheless remains technically connected to the production route: a well-run SWI-Prolog platform can be its first source of experience, while a carefully designed portable boundary can prevent that platform from becoming needlessly isolated.

11.2.3 A decision that need not be made all at once

The routes are neither mutually exclusive nor automatically compatible. A feature may begin as an SWI-Prolog extension, acquire a documented remote contract, and only later prove suitable for a shared profile. Another may rely so deeply on one runtime that standardising it would destroy its advantage. The discipline required now is therefore classification: mark what is platform-specific, what is experimentally portable, and what is claimed as a stable interoperability contract. That classification is more honest, and more useful, than pretending that the eventual institutional form has already been decided.

11.3 Standardisation and shared ownership

The remainder of this section is conditional. It describes a plausible path *if* Web Prolog is developed as shared, cross-system infrastructure. A production SWI-Prolog platform can and should proceed without waiting for that path to be chosen.

11.3.1 What should be standardised, and when?

It is too early to standardise the whole Trinity. It is not too early to publish small observable contracts against which independent implementations can experiment: profile boundaries, term and error encodings, node discovery, request and answer protocols, actor lifecycle rules, and the tests that reveal disagreement. If these surfaces remain implicit, they may still become de facto standards, but through accidental dependence on the first implementation.

The caution is close to home. ISO/IEC 13211–2:2000 standardised Prolog modules, yet widely used systems continued to expose materially different module facilities.¹ The lesson is not that module standardisation was pointless, but that a paper contract alone does not create convergence. Proposed surfaces should be incubated beside implementations, tests, and migration experience; speculative features should not be frozen in the hope that implementations will follow.

11.3.2 Institutional coordinates

The existing Prolog standards belong to ISO/IEC JTC 1/SC 22, whose Prolog working group is WG 17. W3C, by contrast, supplies the institutional machinery for web architecture, protocols, formats, and browser-facing standards. A W3C Community Group could provide an open place to incubate specifications, tests, and use cases, but its reports are not W3C Recommendations. A later standards track would require broader support, a chartered group, implementation evidence, and public review.

The name *ISO Prolog 2.0* used in this book remains a design hypothesis, not the title of an approved ISO project. Likewise, there is no W3C Web Prolog project. The institutional names identify possible conversations, not work already accepted by those organisations.

¹ ISO/IEC 13211–2:2000, <https://www.iso.org/standard/20775.html>.

11.3.3 Who should do it?

Neither ISO nor W3C naturally owns the whole problem. ISO has the established language remit and Prolog expertise; W3C has the web-architecture, protocol, format, and Recommendation-track machinery. If formal standardisation becomes appropriate, a division of labour is more credible than assigning the Trinity wholesale to either body.



Fig. 11.2 Turning the Trinity design into shared infrastructure may require a liaison. The image is prospective, not an indication of institutional endorsement.

The division is conceptual before it is bureaucratic. ISO is the natural forum for questions such as a future common Prolog base, solution cursors, dictionary terms, and profile-defined language choices. W3C is the natural forum for HTTP and WebSocket APIs, media types, term encodings, discovery, and the statechart lineage. The actor and resumable-toplevel layers cross that boundary and would require joint review.

In terms of banners, the possible division can be pictured as two related projects:



Fig. 11.3 Provisional banners for a future general-purpose Prolog project and a web-facing Web Prolog project. Neither denotes an existing standard or an approved standards activity.

The image deliberately makes the aspiration visible. Its presence is not a prediction that either project will take this form.

11.3.4 The statechart language and SCXML

The statechart strand has a clearer ancestor than Web Prolog itself. SCXML 1.0 became a W3C Recommendation in 2015. Chapter 6 implemented its transition-selection and run-to-completion model while making the scripting language, event matching, failure semantics, and actor invocation explicit. SXML is not a conforming SCXML binding, but neither is it merely a name for an aspiration: Section 6.7 gives execution rules and Section 6.9.4 enumerates the deviations.

The deeper proposal is that a successor should retain a declarative, inspectable state-machine core while allowing a capable host language to carry more of the executable content and communication. The current author served as an invited expert and editor in the W3C Working Group that produced SCXML 1.0, as noted in Section 6.9.1. That experience motivates the criticism, but confers no present W3C status on SXML.

This material should not become an extended re-litigation of individual SCXML elements in the final chapter. The important research question is whether the smaller division of responsibility demonstrated by SXML remains intelligible, inspectable, and portable with more than one host language. A detailed proposal for an actor-oriented SCXML successor would be substantial enough to stand as work in its own right.

11.4 The first specification targets

If the federation route is pursued, the targets should be layered so that each stage yields an independently testable boundary:

1. Publish the profile inventories, language-choice ledger, common term and error vocabulary, extension rules for answer metadata, and conformance-test format.
2. Specify RELATION over HTTP as the smallest cross-system query surface, including paging, consistency claims, and JSON representation rules.
3. Specify the resumable toplevel and PTCP lifecycle: start, call, continue, stop, abort, success, failure, error, output, and prompt.
4. Specify the ACTOR substrate and its WebSocket binding: identity, send and selective receive, monitors, lifetime dependence, termination, local names, public service names, authorisation, and containment.
5. Incubate SXML separately against its stated SCXML relationship and a dedicated conformance suite.

This order does not imply that implementation must occur sequentially. It says only that later specifications should reuse earlier contracts rather than define competing vocabularies.

11.4.1 The actor substrate as the distinctive target

`spawn/1--3` is the point at which a Prolog goal becomes an addressable, communicating entity. The ACTOR specification must define the observable contracts of `self/1`, `!/2` and `send/2--3`, `receive/1--2`, monitoring and demonitoring, parent–child lifetime dependence, termination, local registration, and public service publication. It must preserve the deliberate differences from Erlang established in the actor chapters rather than silently inherit the semantics of one host implementation.

11.4.2 Actor identity, messaging, and lifecycle

Identity must remain globally meaningful when a pid crosses a node boundary; message delivery must define per-sender ordering and no-process behaviour; receive must define matching, guards, deferral, timeout, and committed mailbox effects. Monitoring must standardise the `down/3` vocabulary. Local registration and the owner-controlled `register_service/2` and `unregister_service/1` namespace must remain distinct. Authorisation and lifetime containment are part of the observable Web contract, not merely deployment advice.

11.4.3 The semantics of `spawn/3`

The `spawn/3` options expose several of the design’s trust decisions. Monitoring is atomic when requested at creation. Parent–child dependence is the default, while an unlinked remote actor requires owner permission. Source loading, module or knowledge-base selection, time limits, and sandbox policy determine the execution context. A specification should classify each option as portable, profile-specific, owner-only, or implementation-specific and say which failures occur before a pid is returned.

11.4.4 The resumable toplevel actor

The toplevel protocol is the bridge between nondeterministic Prolog execution and deterministic network messages. A call establishes one computation; success messages carry slices; `toplevel_next/1--2` resumes that same computation; stop discards the continuation; abort interrupts a running goal. Recomputation with a larger offset is an implementation strategy only when its stability and side-effect restrictions satisfy the contract. It is not the semantic definition of continuation.

This modest protocol is rich enough to support remote shells, notebooks, browser clients, RPC, teaching environments, and distributed services. It is also valuable in a

single-implementation platform: resumability is an interaction property, not merely an interoperability concession.

11.4.5 Solution cursors and the ISO liaison

The protocol points toward a possible language-level abstraction: a solution cursor that creates a resumable all-solutions computation, returns the next slice, and can be closed. ISO WG 17 is a plausible forum for deciding whether such an abstraction belongs in a future Prolog language standard. Web-facing work would specify how a cursor-backed toplevel is represented, controlled, secured, and tested. The protocol creates a use case for the language abstraction; it does not predetermine predicate names or implementation.

11.4.6 Layering the first standards

The central rule is that shared contracts fix observable behaviour while leaving implementations free behind the boundary. Actors may be implemented using threads, lightweight processes, event loops, or browser workers. Resumable queries may use engines, continuations, virtual-machine support, or a carefully bounded recomputation strategy. Those choices become standards matters only when they change what another participant can observe.

11.5 Implementation and interoperability

We are not analyzing a world, we are building it. We are not experimental philosophers, we are philosophical engineers.

Tim Berners-Lee

11.5.1 Why implementation comes early

Implementation is a design instrument, not a ceremony performed after the specification. Running systems expose ambiguous errors, lifecycle races, unstated ordering assumptions, performance cliffs, and security boundaries that prose can hide. The demonstrator has served that role throughout this book. It should not be frozen as the standard merely because it arrived first, nor dismissed as “only a prototype” when its experiments reveal real design constraints.

On the production route, the next evidence comes from sustained use: workloads, failure injection, upgrades, monitoring, and applications maintained over time. On

the federation route, the decisive evidence comes from another implementation meeting the same boundary. Both kinds of evidence can overturn the design, but they answer different questions.

11.5.2 Independent implementations and test suites

For federation, the most useful next artefact is a second independent implementation of a narrow layer, preferably *RELATION* and its term/error encoding before the full *ACTOR* profile. The conformance suite should link every normative requirement to tests and publish an implementation matrix. Disagreement then becomes evidence: the prose is ambiguous, a requirement is overconstrained, or an implementation is wrong.

For the SWI-Prolog platform, redundancy means something different: separate clients, deployments, browser engines, security reviews, and applications exercise the same implementation under varied conditions. A platform can become robust without becoming portable. It should simply avoid presenting that robustness as proof of cross-system interoperability.

11.6 AI-assisted implementation and standardisation

The demonstrator and this manuscript were developed with extensive assistance from language models, including ChatGPT, Claude, and Gemini. That experience supports a bounded claim: AI assistance can reduce the cost of producing and reviewing code, tests, documentation, and alternative designs. It does not transfer responsibility for architecture, protocol semantics, security, or standards decisions. Several model-generated implementations also do not count as independent evidence when they share the same specification assumptions and human direction.

11.6.1 Implementation support

AI assistance was most effective once the intended boundary was explicit: constructing interfaces, generating routine code, proposing variants, debugging transport interactions, and keeping the demonstrator aligned with revised contracts. It was less reliable when asked to infer the contract from an incomplete implementation. This reinforces the need for small stated surfaces and executable tests.

11.6.2 Test generation and adversarial experimentation

Language models can generate protocol variations, boundary cases, and hostile inputs cheaply, but generated tests require an independently stated oracle. For sandboxing in particular, test generation supplements rather than replaces threat modelling, code review, and deployment hardening.

11.6.3 Specification drafting and editorial assistance

An AI editorial agent could check terminology, cross-references, normative keywords, requirement identifiers, test coverage, and implementation reports. It could also help keep code, manuals, and manuscript examples aligned while the design changes. Human participants must still resolve issues and own every normative decision. The attractive reflexivity of using Trinity agents to help maintain Trinity specifications should not obscure that division of responsibility.

11.7 Community and adoption

A technically sound platform or specification can remain inert. Adoption begins with people who have a reason to use it: Prolog programmers building applications, implementers testing boundaries, educators publishing runnable examples, and users willing to host services. The portal and Discovery Hub proposed in Chapter 10 can make the technical layer visible: nodes, profiles, services, documentation, tests, and projects in one place.

Dogfooding remains the most credible first strategy. Sharing should mean not only publishing source code but also hosting queryable relations and runnable actors that others can test, stress, and compose. Learning material can then be organised around visible web interactions rather than asking newcomers to absorb decades of Prolog context before doing anything tangible.

The route chosen affects the first community being invited. A production SWI-Prolog platform can begin with SWI-Prolog users, web programmers, and application builders. A federation requires the active participation of other Prolog implementers from the beginning. The neighbouring communities considered in Chapter 10 are not merely audiences: formal-semantics researchers, Semantic Web practitioners, agent researchers, Erlang implementers, browser engineers, security reviewers, and web developers possess expertise that can change the proposal.

Protocols do not manufacture a community, but shared objects can give one something to gather around: a useful platform, a public service, a failing conformance test, a disputed requirement, or an application that could not easily have been built before.

11.8 Closure

This chapter began by returning to a familiar diagram, not to introduce new ideas, but to reconsider the existing ones from a different vantage point. Viewed in this way, the Prolog Trinity is no longer only a conceptual synthesis. It is a set of commitments: to certain abstractions, to certain interfaces, and to a particular way of embedding logic and agency into the Web.

Execution, as the term has been used here, is not a final step that follows design. It is the continuing process through which designs are tested, revised, constrained, and sometimes abandoned as they encounter real users, real implementations, and real operational limits. Some parts of the Trinity may mature quickly; others may remain speculative for a long time; still others may prove mistaken. That is not a defect of the enterprise, but a normal feature of any attempt to build shared infrastructure.

The chapter has also resisted collapsing every ambition into one programme. A production platform can be worthwhile even if federation never follows. A portable profile can be valuable without determining the internals of every Prolog system. A statechart language may find its most natural future in work separate from both. Keeping these paths distinct makes cooperation possible without making the largest vision a precondition for every smaller success.

An ecosystem, in the end, is not something one simply invents. It is something one manages, if one is fortunate, to invite into existence.

The banners with which this chapter opened were therefore never claims of authority. They were reminders of the work that must occur outside the pages of this book if any part of the Trinity is to become shared infrastructure: implementation, specification, conformance testing, public criticism, and the slow accumulation of trust.

This returns us, quietly but insistently, to the warning implicit in the epigraph. Vision without execution is not merely incomplete. It is a condition in which ideas cannot be stress-tested, useful boundaries cannot be stabilised, and responsibility cannot be shared. Execution is the process through which design becomes answerable: to users, to implementers, to hostile environments, and to the practical constraints of the Web.

Part VII

Appendices

Appendix A

Manual

A.1 Predicates for programming with core actors

Predicate: `self/1`

ACTOR

`self(-Pid) is det.`

Binds `Pid` to the process identifier of the calling process.

Note: The runtime uses globally qualified pids of the form `Id@Node`. A shell presentation setting may omit `@Node` when displaying a pid whose node is the shell's own node. This abbreviation affects only printing; the pid remains globally qualified in messages and protocol representations.

Predicate: `spawn/1-3`

ACTOR

`spawn(+Goal) is det.`

`spawn(+Goal, -Pid) is det.`

`spawn(+Goal, -Pid, +Options) is det.`

Creates a new Web Prolog actor process running `Goal`. Valid options are:

- `node(+URI)`
Creates the process locally or on a remote compatible node. Default is `localhost`.
- `monitor(+Boolean)`
If `true`, monitoring is installed as part of process creation. Default is `false`.
- `link(+Boolean)`
If `true`, installs directional parent-to-child link cleanup. Default is `true`.
- `load_text(+AtomOrString)`
Loads the clauses specified by a Web Prolog source text into the actor's private Prolog database before calling `Goal`.

- `load_list(+ListOfClauses)`
Converts the clauses to source text and loads them into the actor's private Prolog database before calling `Goal`.
- `load_uri(+URI)`
Loads the clauses specified by a file path, file URI, or HTTP(S) URI into the actor's private Prolog database before calling `Goal`.
- `load_predicates(+ListOfPredicateIndicators)`
Loads the local predicates denoted by `ListOfPredicateIndicators` into the actor's private Prolog database before calling `Goal`.

Note: In `spawn(+Goal, -Pid, +Options)`, the option `monitor(true)` installs monitoring as part of process creation. When the child terminates, the parent receives a message `down(Ref, Pid, Reason)` where `Pid` is the terminated actor, `Reason` is its exit reason, and `Ref` identifies the monitor instance. For monitoring created via `monitor(true)`, the current implementation uses `Ref = Pid`.

Note: The semantics of `link(true)` are directional: if a parent spawns a child with linking enabled, termination of the parent causes linked children to be terminated. The link does not imply symmetric bidirectional exit propagation.

Note: It is possible to pass an arbitrary number of the `load_*` options to `spawn/3`, and possibly more than one instance of each variant. To ensure that clauses end up in a well-defined order, they are converted into Prolog source text before being loaded into the database. The order of clauses and directives in the resulting source text is determined by the order of the `load_*` options in the option list.

Predicate: `monitor/2`

ACTOR

`monitor(+PidOrName, -Ref)` is det.

Installs a monitor and returns a fresh reference in `Ref`. The first argument may be a pid or a registered local name. When the monitored process terminates, the monitoring process receives a message of the form `down(Ref, Pid, Reason)`.

Predicate: `demonitor/1-2`

ACTOR

`demonitor(+Ref)` is det.

`demonitor(+Ref, +Options)` is det.

Stops monitoring identified by `Ref`. This is idempotent. There is only one valid option:

- `flush`
Non-blocking removal of one pending `down(Ref, _, _)` message from the mailbox.

Using `monitor(true)` in `spawn/3` is the safest variant for short-lived children, since monitoring is established during `spawn`. Installing monitoring later with `monitor/2` is a separate step and may miss a child that exits immediately.

Predicate: `register/2`

ACTOR

```
register(+Name, +Pid) is det.
```

Register a local process under a name, where `Name` is an atom and `Pid` identifies the actor process. The association between the name and the pid is removed when the process terminates.

Predicate: `whereis/2`

ACTOR

```
whereis(+Name, ?Pid) is det.
```

Determine the identity of the actor process associated with the name. Binds `Pid` to undefined if the process does not exist.

Predicate: `unregister/1`

ACTOR

```
unregister(+Name) is det.
```

Remove the association between the name and the process.

Predicate: `register_service/2`

ACTOR, OWNER ONLY

```
register_service(+Name, +Pid) is det.
```

Publishes the live local actor identified by `Pid` under `Name` in the node's public service namespace. `Name` must be an atom, and `Pid` must identify an actor on the publishing node. There may be at most one published service with a given name on a node. If `Name` is already present in the public service namespace, `register_service/2` raises `permission_error(register, service, Name)`, even when the existing entry denotes the same actor. This makes replacement explicit: the owner must first withdraw the old mapping, unless it has already been removed because the actor terminated.

The public service namespace is distinct from the ordinary registration namespace used by `register/2`. The same atom may therefore occur in both namespaces without a collision. A qualified address `Name@Node` is resolved against the public service namespace of `Node`; `whereis/2` consults only the ordinary namespace and never reveals the pid behind a published service name.

Publication neither changes the actor's links nor places it under supervision. In particular, `register_service/2` does not detach the actor from its parent. When the actor terminates, the node controller removes the service mapping atomically, so a published name never remains bound to a dead pid. A supervisor that starts a replacement actor must publish that actor again under the stable name.

Only code executing with node-owner authority may call this predicate. A call by any other principal raises `permission_error(register, service, Name)`. A non-atom `Name` raises `type_error(atom, Name)`; a `Pid` that does not identify a live actor raises `existence_error(process, Pid)`; and an attempt to publish an actor on another node raises `permission_error(register, service, Pid)`.

Predicate: `unregister_service/1` ACTOR, OWNER ONLY

`unregister_service(+Name)` is det.

Withdraws `Name` from the node's public service namespace. This operation removes only the name; it neither terminates nor otherwise changes the actor that the name denoted. The operation is idempotent: if no published service called `Name` exists, it succeeds without effect.

Only code executing with node-owner authority may call this predicate. A call by any other principal raises `permission_error(unregister, service, Name)`, and a non-atom `Name` raises `type_error(atom, Name)`.

Predicate: `actors/1` ACTOR

`actors(-Pids)` is det.

Binds `Pids` to the list of actor pids visible from the current execution context. In ordinary public client execution, this means the current actor or toplevel together with any actors that have been spawned on behalf of the same client interaction. Actors belonging to other clients are not included.

Note: Outside public client execution, for example in node-owned runtime code or administrative code, `actors/1` returns the pids of all active local actors on the node.

Predicate: `exit/1` ACTOR

`exit(+Reason)` is det.

Exit the calling process with `Reason`.

Predicate: `exit/2` ACTOR

`exit(+Pid, +Reason)` is det.

Exit the process identified by `Pid` with `Reason`. For remote actors, the runtime routes this request through the remote node.

Predicate: `!/2, send/2-3` ACTOR

`+PidOrName ! +Message` is det.

`send(+PidOrName, +Message)` is det.

`send(+PidOrName, +Message, +Options)` is det.

Sends `Message` to the mailbox of the process identified as `PidOrName`. `PidOrName` may be a local pid, a local registered name, a global pid of the form `Id@Node`, or a published service address of the form `Name@Node`. A send to a published service is subject to the destination node's authorisation policy. Sending remains non-revealing: if the service is absent or the sender is not authorised to use it, the send succeeds without delivery, just as it does for an unknown pid. Valid options for `send/3` are:

- `delay(+Number)`
Delays the sending by a specified number of seconds. Default is `0`.
- `id(+Id)`
`Id` is a user-supplied identifier that can be used by `cancel/1` to stop the sending from taking place.

Predicate: `cancel/1`

ACTOR

`cancel(+ID) is det.`

Tries to cancel the sending of *all* delayed messages with the specified `Id`. This is best-effort only, since a message may already have been sent by the time the call is made.

Predicate: `output/1-2`

ACTOR

`output(+Data) is det.`
`output(+Data, +Options) is det.`

Sends a message `output(Pid, Data)` to the target process. `Pid` is the pid of the current process. Valid option:

- `target(+Pid)`
Send the message to `Pid`. Default is the parent process.

Note that this is just a convenience predicate. A `oplevel`, like any other actor, may use `!/2` to send any term to any process to which it has a pid.

Predicate: `input/2-3`

ACTOR

`input(+Prompt, -Data) is det.`
`input(+Prompt, -Data, +Options) is det.`

Sends a message `prompt(Pid, Prompt)` to the target process and waits for its input. `Prompt` may be any non-variable term. `Pid` is the pid of the current process. `Data` will be bound to the term that the target process sends using `respond/2`. Valid option:

- `target(+Pid)`
Send the `prompt` message to `Pid`. Default is the parent process.

Predicate: `respond/2`

ACTOR

`respond(+Pid, +Input)` is `det`.

Sends a response in the form of the term `Input` to a process that has prompted its parent process for input.

Predicate: `receive/1-2`

ACTOR

`receive(+Clauses)` is `nondet`.

`receive(+Clauses, +Options)` is `nondet`.

Waits for, selects, and consumes one message from the calling process's mailbox. `receive/1-2` is semi-deterministic with respect to message selection: it commits to the first accepted message – the oldest one whose pattern matches and whose guard, if any, succeeds – and to the first receive clause accepting it, and backtracking never selects another message or another clause. The selected message is consumed even if the clause body later fails. The body of the selected clause, however, is ordinary Prolog code, and the call as a whole has the determinism of the fired body: choice points the body leaves remain available on backtracking, and if the body ultimately fails, so does the call – with the message already gone.

`Clauses` is a term `{C1 ; ... ; Cn}` in which each `Ci` has the form `Pattern -> Body` or `Pattern if Guard -> Body` – ordinary term syntax under the operator declarations of Section [operators]. A clause accepts a message if the message unifies with `Pattern` and `Guard`, evaluated as if by `once/1` and expected to be free of side effects, succeeds; bindings from pattern and guard are in scope in `Body`. A guard that raises an exception propagates it as the outcome of the call, the message having been consumed. Messages accepted by no clause are *deferred*: they remain in the mailbox, in arrival order, available to later calls. `receive({})` accepts nothing, and waits (subject to `Options`). Valid options for `receive/2` are:

- `timeout(+Number)`
Bounds the wait, in seconds; `0` inspects the mailbox without waiting. Default is no bound.
- `on_timeout(+Goal)`
Called if the bound expires with no message accepted; its success, failure, or exception becomes the outcome of the call. Default is `fail`.

Select the best formulation!

Predicate: `receive/1-2`

ACTOR

`receive(+Clauses)` is `semidet`.

`receive(+Clauses, +Options)` is `semidet`.

`Clauses` is a sequence of receive clauses delimited by a semicolon:

```
{ Pattern1 [if Guard1] ->
    Body1 ;
    ...
    PatternN [if GuardN] ->
    BodyN
}
```

Each pattern in turn is matched against the first message in the mailbox. If a pattern matches and the corresponding guard succeeds, the matching message is removed from the mailbox and the body of the receive clause is called. If no message is accepted, the process waits for new messages, checking them one at a time in arrival order. Messages that are not accepted are *deferred*, i.e. left in the mailbox without any change in their contents or order. Valid options:

- `timeout(+Number)`
If nothing appears in the current mailbox within `Number` seconds, the predicate succeeds anyway. Default is no timeout.
- `on_timeout(+Goal)`
If the timeout occurs, `Goal` is called.

Predicate: `flush/0`

ACTOR

`flush` is det.

Removes all messages currently pending in the mailbox of the calling process. Each removed message is rendered through the actor terminal-output path as a line of the form `Shell got Message`, where `Message` is the full Prolog term.

Note: `flush/0` is primarily a shell-level inspection utility. It does not wait for future messages: once the mailbox is empty at the time of the call, it succeeds immediately.

A.2 Predicates for programming with toplevel actors

Predicate: `toplevel_spawn/1-2`

ACTOR

`toplevel_spawn(-Pid)` is det.
`toplevel_spawn(-Pid, +Options)` is det.

Spawns a toplevel actor and binds `Pid` to its pid. All options accepted by `spawn/3` are also accepted by `toplevel_spawn/2`. In addition, `toplevel_spawn/2` accepts the following options:

- `session(+Boolean)`
If set to `false`, the toplevel actor terminates after having run a goal to completion. If `true`, further interaction is expected. Default is `false` in the actor API.
- `target(+Pid)`
Send all answer terms to `Pid`. Default is the pid of the parent.
- `name(+Name)`
Register the toplevel actor under `Name`.

Predicate: `toplevel_call/2-3`

ACTOR

`toplevel_call(+Pid, +Goal) is det.`
`toplevel_call(+Pid, +Goal, +Options) is det.`

Asks the `toplevel` `Pid` for solutions to `Goal`. Valid options are:

- `template(+Template)`
`Template` is a term sharing variables with `Goal`. By default, the template is identical to the goal.
- `offset(+Integer)`
Collect only the slice of solutions starting from `Integer`. Default is 0.
- `limit(+Integer)`
Restrict the length of the returned list of solutions to `Integer`.
- `once(+Boolean)`
When true, the list of solutions in a `success` answer term may not be the only solutions that `Goal` has, but the answer term may still indicate that it is. Default is false. Makes sense only in combination with the `limit` option.
- `target(+Pid)`
Send the answer term to `Pid`. Default is the value of `target` when passed as an option to `toplevel_spawn/2`.

Variables in `Goal` are not bound directly in the caller. Instead, solutions and other kinds of output are returned in the form of answer messages delivered to the mailbox of the target process.

- `success(Pid, Data, More)`
`Pid` is the pid of the `toplevel` process that succeeded in finding solutions to `Goal`. `Data` is a list holding instantiations of `Template`. `More` is either true or false, indicating whether the `toplevel` can return more solutions if `toplevel_next/1-2` is called.
- `failure(Pid)`
`Pid` is the pid of the `toplevel` process that failed for lack of solutions.
- `error(Pid, Data)`
`Pid` is the pid of the `toplevel` throwing the error. `Data` is the error term.

Predicate: `toplevel_next/1-2`

ACTOR

`toplevel_next(+Pid) is det.`
`toplevel_next(+Pid, +Options) is det.`

Asks `toplevel` `Pid` for more solutions. Valid options:

- `limit(+Integer)`
If omitted, the actor API keeps using the most recent limit value from the previous `toplevel` call state.
- `target(+Pid)`
Send the answer term to `Pid`. Default is the value of `target` from the previous call state.

The messages delivered to the target mailbox are the same as for `toplevel_call/2-3`.

Predicate: `toplevel_stop/1` ACTOR

`toplevel_stop(+Pid) is det.`

Asks `toplevel Pid` to stop searching for more solutions.

Predicate: `toplevel_abort/1` ACTOR

`toplevel_abort(+Pid) is det.`

Tells `toplevel Pid` to abort the execution of the currently running goal.

Predicate: `toplevel_halt/2` ACTOR

`toplevel_halt(+Pid, -Reply) is det.`

Asks the `toplevel actor Pid` to halt gracefully and binds `Reply` to its acknowledgement.

A.3 Predicates for programming with server actors

Predicate: `server_spawn/3-4` ACTOR

`server_spawn(+Pred, +State, -Pid) is det.`

`server_spawn(+Pred, +State, -Pid, +Options) is det.`

Spawns a generic server actor using callback predicate `Pred/4` and initial state `State`. The callback receives `(Request, OldState, Response, NewState)`. All options are forwarded to `spawn/3` except:

- `name(+Name)`
Register the server under `Name`.

Predicate: `server_request/3-4` ACTOR

`server_request(+Pid, +Request, -Response) is det.`

`server_request(+Pid, +Request, -Response, +Options) is det.`

Makes a synchronous request-response call to the server `Pid`. `server_request/4` accepts the options of `receive/2`, for example `timeout(+Seconds)`. Monitoring is installed automatically so the call fails fast if the server terminates before replying.

Predicate: `server_promise/3-4` ACTOR

`server_promise(+Pid, +Request, -Ref)` is det.
`server_promise(+Pid, +Request, -Ref, -MonRef)` is det.

Sends `Request` to the server `Pid` and returns a correlation reference `Ref`. The reply must later be collected with `server_yield/2-4`. The four-argument variant additionally installs a monitor and returns its reference in `MonRef`.

Predicate: `server_upgrade/2-3`

ACTOR

`server_upgrade(+Pid, +Pred)` is det.
`server_upgrade(+Pid, +Pred, +Options)` is det.

Replaces the callback predicate of the running server `Pid` without disturbing its current state. `Pred` must denote a callback predicate of arity four.

The two-argument form transfers no code and therefore requires the callback to be present already in the server's private database. The three-argument form accepts the following source-loading options:

- `load_text(+Text)`
Load callback source from `Text`.
- `load_list(+ListOfClauses)`
Convert the clauses to source text and load them into the server.
- `load_predicates(+ListOfPredicateIndicators)`
Copy the caller's indicated predicates into the server's private database.
- `load_uri(+URI)`
Fetch callback source from `URI` and load it into the server.

The operation succeeds only after the new callback has been loaded and validated. If installation fails, the server retains its old callback and state and `server_upgrade/2-3` rethrows the reported error. Callback replacement preserves the existing state value; it does not perform state migration.

Predicate: `server_upgrade/2`

ACTOR

`server_upgrade(+Pid, +Pred)` is det.

Replaces the callback predicate of the running server `Pid` without disturbing its current state. `Pred` must be arity 4.

Predicate: `server_halt/2`

ACTOR

`server_halt(+Pid, -Reply)` is det.

Asks the server `Pid` to stop gracefully and binds `Reply` to its acknowledgement.

A.4 Predicates for programming with statechart actors

Predicate: `statechart_spawn/2`

ACTOR

`statechart_spawn(-Pid, +Options)` is det.

Spawns a statechart actor and binds `Pid` to its pid.

`statechart_spawn/2` requires exactly one source option:

- `load.uri(+URI)`
Load and run a statechart from a URI or file path.
- `load.text(+Text)`
Load and run a statechart from an XML text string.
- `name(+Name)`
Register the statechart actor under `Name`.

All remaining options are passed through to `spawn/3`.

`load_list/1` and `load_predicates/1` are rejected for `statechart_spawn/2`.

Predicate: `statechart_halt/2`

ACTOR

`statechart_halt(+Pid, -Reply)` is det.

Asks the statechart actor `Pid` to halt gracefully and binds `Reply` to its acknowledgement.

Predicate: `raise/1`

STATECHART BUILTIN

`raise(+Event)` is det.

Enqueues `Event` on the *internal* event queue of the current statechart interpreter.

Important: `raise/1` is only meaningful inside executable statechart content such as `<datamodel>`, `<onentry>`, `<onexit>`, and `<go>`.

Predicate: `in/1`

STATECHART BUILTIN

`in(+Id)` is semidet.

Succeeds if the state identified by `Id` belongs to the current active configuration of the statechart interpreter.

Important: `in/1` is only meaningful inside statechart guards or executable statechart content such as `<onentry>`, `<onexit>`, and `<go>`.

A.5 Predicates for programming with supervisor actors

Predicate: `supervisor_spawn/2-3`

ACTOR

`supervisor_spawn(+ChildSpecs, -Pid)` is det.
`supervisor_spawn(+ChildSpecs, -Pid, +Options)` is det.

Spawns a supervisor actor and starts its children. Supported options are:

- `strategy(+Strategy)`
One of `one_for_one` (default), `one_for_all`, or `rest_for_one`.
- `intensity(+Integer)`
Max restarts in period. Default 1.
- `period(+Integer)`
Window size for restart intensity. Default 5.
- `name(+Name)`
Register the supervisor under Name.

Other options are passed through to `spawn/3`.

Child specifications take the form `child(Id, ChildOptions)`. Child options are:

- `start(+Goal)`
Required. Start goal for the child.
- `restart(+Policy)`
One of `permanent` (default), `transient`, or `temporary`.
- `shutdown(+Shutdown)`
One of `brutal_kill`, `infinity`, or a timeout number.
- `type(+Type)`
`worker` (default) or `supervisor`.

Predicate: `supervisor_spawn_child/3`

ACTOR

`supervisor_spawn_child(+Pid, +ChildSpec, -Reply)` is det.

Dynamically adds and starts one child. Reply is one of `ok`, `error(already_present)`, or `error(start_failed)`.

Predicate: `supervisor_terminate_child/3`

ACTOR

`supervisor_terminate_child(+Pid, +Id, -Reply)` is det.

Stops child Id but keeps its specification. Reply is `ok` or `error(not_found)`.

Predicate: `supervisor_delete_child/3`

ACTOR

`supervisor_delete_child(+Pid, +Id, -Reply)` is det.

Removes a non-running child specification. Reply is `ok`, `error(running)`, or `error(not_found)`.

Predicate: `supervisor_respawn_child/3`

ACTOR

`supervisor_respawn_child(+Pid, +Id, -Reply)` is det.

Restarts a previously terminated child from its stored specification. Reply is either `ok(NewPid)`, `error(running)`, `error(not_found)`, or `error(start_failed)`.

Predicate: `supervisor_which_children/2`

ACTOR

`supervisor_which_children(+Pid, -Children)` is det.

Returns a list of:

`info(Id, Pid, Type, Restart)`

where `Pid` may be undefined for stopped children.

Predicate: `supervisor_count_children/2`

ACTOR

`supervisor_count_children(+Pid, -Counts)` is det.

Returns:

`[specs-N, active-N, supervisors-N, workers-N]`

Predicate: `supervisor_halt/1`

ACTOR

`supervisor_halt(+Pid)` is det.

Stops the supervisor and shuts down children in reverse start order.

Note on synchronous supervisor calls

The dynamic/query API above uses monitored synchronous calls and may throw:

- `supervisor_down(Reason)`
- `supervisor_call_timeout(Sup, Request)`

A.6 Actor-based predicate generics

Predicate: `parallel/1`

ACTOR

`parallel(+Goals) is semidet.`

Runs the proper list of independent goals `Goals` concurrently, using one monitored worker actor per goal. The call succeeds when every goal has produced its first solution and returns the resulting variable bindings. `parallel([])` succeeds immediately. Further solutions are not enumerated on backtracking.

If any goal fails, the remaining workers are terminated and the call fails. If any goal throws an exception, the remaining workers are terminated and the exception is rethrown. Result and monitor messages belonging to the workers are removed before the predicate returns, fails, or throws.

The goals must not depend on bindings or side effects produced by sibling goals. Actor creation and message copying add overhead, and the one-worker-per-goal model gives large goal lists a correspondingly large resource footprint.

A non-list `Goals` raises `type_error(list, Goals)`. An element that is not callable raises the corresponding callable type error from its worker, which is propagated to the caller.

Predicate: `first_solution/2-3`

ACTOR

`first_solution(?Solution, +Goals) is semidet.`

`first_solution(?Solution, +Goals, +Options) is semidet.`

Runs the proper list of alternative goals `Goals` concurrently in monitored worker actors and unifies `Solution` with the result produced by the first solver to succeed. Once a winner has been selected, all remaining workers are terminated and their result and monitor messages are removed. The predicate does not enumerate later solutions on backtracking, and an empty goal list fails.

The two-argument form continues after an individual solver fails and stops on the first exception. The three-argument form accepts:

- `on_fail(stop|continue)`
`continue` removes the failed solver and keeps waiting while alternatives remain; `stop` terminates all workers and fails immediately. Default is `continue`.
- `on_error(stop|continue)`
`continue` removes the solver that raised an exception and keeps waiting while alternatives remain; `stop` terminates all workers and rethrows the exception. Default is `stop`.

If all alternatives are removed under a `continue` policy, the predicate fails. Goals should not depend on side effects produced by sibling solvers.

A non-list `Goals` or `Options` raises `type_error(list, Term)`. An action other than `stop` or `continue` raises `type_error(oneof([stop,continue])`,

Action). An unknown option raises `domain_error(first_solution_option, Option)`.

A.7 Predicates for remote Prolog-style calls

Predicate: `rpc/2-3`

ISOBASE

```
rpc(+URI, +Goal) is nondet.
rpc(+URI, +Goal, +Options) is nondet.
```

The URI argument alone selects the node on which Goal is executed. In particular, `localhost` means the node on which `rpc/2-3` is running. Source-loading options such as `load_uri/1` only supply source text for the temporary private database used by the call; they do not change the target selected by URI. Thus source and execution target may intentionally differ, for example:

```
?- rpc('http://n2.org', Goal, [
    load_uri('http://n3.org')
]).
```

This executes Goal on `n2.org` using source fetched from `n3.org`.

Semantically, `rpc/2-3` executes a copy of Goal on the remote node identified by URI and then unifies the local goal with the remote copy on backtracking. The current implementation uses the stateless `/call` endpoint. The following options are valid:

- `limit(+Integer)`
Restricts the number of solutions retrieved per roundtrip.
- `once(+Boolean)`
When `true`, the returned slice may not be the only solutions that Goal has, but the answer term may still indicate that it is.
- `timeout(+Number)`
Requests a timeout in seconds for goal execution on the remote node. The effective timeout is the minimum of the requested value and the timeout configured by the node owner.
- `http_timeout(+Number)`
Sets the client-side HTTP transport timeout in seconds for each request.
- `load_text(+AtomOrString)`
Loads source text into the underlying actor's private database before calling Goal.
- `load_list(+ListOfClauses)`
Loads a list of clauses into the underlying actor's private database before calling Goal.

- `load.uri(+URI)`
Loads source text from URI into the underlying actor's private database before calling Goal.
- `load.predicates(+ListOfPredicateIndicators)`
Loads the local predicates denoted by the supplied predicate indicators into the underlying actor's private database before calling Goal.

`http.timeout/1` controls only the client-side HTTP transport timeout. It does not change the remote execution timeout.

Predicate: `promise/3-4`

ISOBASE

`promise(+URI, +Goal, -Ref)` is det.
`promise(+URI, +Goal, -Ref, +Options)` is det.

Makes an asynchronous RPC call to node URI with Goal. As with `rpc/2-3`, the first argument selects the node on which the goal is executed. In particular, `localhost` denotes the node on which `promise/3-4` is running. Source-loading options materialise private source text for the promised call; they do not change its execution node. The following options are supported:

- `template(+Template)`
Template is a term sharing variables with the goal. By default, the template is identical to the goal.
- `offset(+Integer)`
Collect only the slice of solutions starting from Integer. Default is 0.
- `limit(+Integer)`
Restrict the number of returned solutions to Integer.
- `load.text(+AtomOrString)`
Load source text into the promised call's temporary private database.
- `load.list(+ListOfClauses)`
Convert the clauses to source text and load them into the promised call's temporary private database.
- `load.uri(+SourceURI)`
Fetch source text from SourceURI and load it into the promised call's temporary private database. SourceURI identifies only the source location; it does not change the execution node selected by the first argument.
- `load.predicates(+ListOfPredicateIndicators)`
Copy the caller's indicated predicates into the promised call's temporary private database.

The reference returned in Ref can later be used by `yield/2-3` to collect the answer.

Predicate: `yield/2-3`

ISOBASE

`yield(+Ref, ?Answer)` is det.
`yield(+Ref, ?Answer, +Options)` is det.

Returns the promised answer from a previous call to `promise/3-4`. This predicate must be called by the same process from which the previous call to `promise/3-4` was made. Valid options:

- `timeout(+Number)`
If nothing appears in the current mailbox within `Number` seconds, the goal supplied in the `on_timeout` option is called. Default is no timeout.
- `on_timeout(+Goal)`
If the timeout occurs, `Goal` is called.

A.8 ISO predicates and related constructs

The Web Prolog supports the following ISO predicates and related constructs from the appendix; directives and DCG rules are source-level items, and `read/1` and the write predicates are provided through the session I/O layer rather than ambient ISO stream I/O.

Control and meta-call

`!/0`, `(,)/2`, `(;)/2`, `(->)/2`, `(\+)/1`, `call/1--8`, `catch/3`, `fail/0`, `false/0`, `repeat/0`, `throw/1`, `true/0`

Directives and DCGs

`-->/2`, `discontiguous/1`, `dynamic/1`, `multifile/1`, `phrase/2--3`

Term relations, comparison, and construction

`(=)/2`, `(=..)/2`, `(==)/2`, `(@<)/2`, `(@=<)/2`, `(@>)/2`, `(@>=)/2`, `(\=)/2`, `(\==)/2`, `arg/3`, `compare/3`, `copy_term/2`, `functor/3`, `subsumes_term/2`, `term_variables/2`, `unify_with_occurs_check/2`

Arithmetic predicates

`(<)/2`, `(=:)/2`, `(=<)/2`, `(=\=)/2`, `(>)/2`, `(>=)/2`, `is/2`

Arithmetic evaluable functors

All arithmetic evaluable functors listed in the appendix are supported in arithmetic expressions.

Type testing

`acyclic_term/1`, `atom/1`, `atomic/1`, `callable/1`, `compound/1`, `float/1`, `ground/1`, `integer/1`, `nonvar/1`, `number/1`, `var/1`

Database and all-solutions

abolish/1, asserta/1, assertz/1, bagof/3, clause/2, findall/3,
retract/1, retractall/1, setof/3

Atomic term processing

atom_chars/2, atom_codes/2, atom_concat/3, atom_length/2,
char_code/2, number_chars/2, number_codes/2, sub_atom/5

Session I/O

nl/0, read/1, write/1, write_canonical/1, write_term/2, writeq/1

A.9 Node profile APIs

The Web Prolog node profiles expose progressively richer APIs, corresponding to the interaction models introduced in Section 3.5. These APIs are best understood not primarily as transport choices, but as *interaction contracts* between clients and nodes. The four standard profiles are:

Profile	Transport model	Interaction style
RELATION	Stateless HTTP	relational querying
ISOBASE	Stateless HTTP	stateless Prolog execution and RPC
ISOTOPE	Semi-stateful HTTP	toplevel session interaction
ACTOR	Stateful WebSocket	actor messaging and events

The profiles nest: each retains the surfaces of the ones below it. In the current implementation they are realised by three route families – `/call`, `/toplevel_*`, and `/ws` – so an ACTOR node still exposes `/call` and `/toplevel_*`, and an ISOTOPE node still exposes `/call`. The four subsections below state what each profile adds over its predecessor; the precise feature breakdown for all four is given once, in the summary matrix of Section A.9.5. In a concrete deployment, a node owner may narrow any profile’s surface further by capability policy.

A.9.1 RELATION node API

RELATION is the minimal profile. It exposes a stateless relational querying interface over HTTP: clients may query the relations the node advertises in its shared database, but cannot submit arbitrary goals, load source, update a database, message actors, or perform I/O. There is no session anchor and no server push.

A.9.2 ISOBASE node API

ISOBASE extends RELATION with general stateless Prolog execution over the same HTTP interface, together with synchronous and asynchronous remote calls: `rpc/2-3`, `promise/3-4`, and `yield/2-3`, plus source loading via `load_text` and `load_uri`. It remains stateless – no session-local database, no dynamic updates, no actor messaging, no I/O.

A.9.3 ISOTOPE node API

ISOTOPE exposes a shell-like interaction model backed by a persistent toplevel actor. Transport is still HTTP, but requests are anchored in a long-lived toplevel identified by its pid, so a client gains a session-local database, incremental clause loading, dynamic predicates, and session I/O. Server push is not available; the client observes results by a long-polling variant. Actor messaging within submitted code is still out of profile.

A.9.4 ACTOR node API

ACTOR is the richest profile. It exposes the full stateful interaction model over a persistent WebSocket connection: spawning, messaging (`!/2`), `receive`, monitoring, linking, and server-pushed events, in addition to everything the lower profiles provide. In the current implementation an ACTOR node also retains the lower HTTP surfaces (`/call` and `/toplevel_*`). The full WebSocket contract is specified in Section A.10.

An ACTOR implementation also provides the owner-only administrative predicates `register_service/2` and `unregister_service/1`. They publish and withdraw stable names in the node's public service namespace. They are part of ACTOR conformance but are not exposed to ordinary clients; clients may only send to published service addresses for which the node's authorisation policy grants access.

A.9.5 Summary comparison of node profiles

Feature	RELATION	ISOBASE	ISOTOPE	ACTOR
Stateless /call surface	✓	✓	✓	✓
Semi-stateful /toplevel_* surface	×	×	✓	✓
Stateful /ws surface	×	×	×	✓
Arbitrary Prolog goals	×	✓	✓	✓
Asynchronous RPC	×	✓	✓	✓
I/O predicates	×	×	✓	✓
Session-local database	×	×	✓	✓
Dynamic predicates	×	×	✓	✓
Actor messaging	×	×	×	✓
Owner service publication	×	×	×	✓
Server push	×	×	×	✓
Persistent session anchor	×	×	✓	✓

A.10 The stateful WebSocket API

The stateful API is exposed at /ws. The client opens a persistent WebSocket connection and exchanges JSON frames with the node. Each client frame is a JSON object with a mandatory `command` field; each server frame is a JSON object with a mandatory `type` field. Unlike the HTTP APIs there is no polling step: outputs, prompts, solution slices, termination notifications, and trace messages are pushed asynchronously to the client.

The connection provides three groups of commands: *toplevel session* commands for REPL-style interaction, *bare actor* commands for spawning and messaging concurrent actors, and a *trace* command for controlling session-level execution tracing.

A.10.1 Connection lifecycle

On upgrade, the server authenticates the principal, creates a per-connection message queue, and spawns a relay thread that forwards actor events to the WebSocket as JSON frames. On close, all actors owned by the connection are killed and the queue is destroyed. Client-owned pids are scoped to the connection: a client may manipulate only actors or sessions owned by that connection, except that `send` also permits owner-published service names (and, for a principal holding `internal_transport` capability, any actor).

A.10.2 Toplevel session commands

Parameter	Type	Description	Default
options	list/string	Spawn options passed to the toplevel actor.	[]
load_text	string	Source text loaded into the session before use.	empty string
trace	boolean	Enable statechart/session-level trace output.	false

Table A.1 Parameters for the `toplevel_spawn` command.

Response: `spawned` with the session pid, or `error` if `load_text` fails to parse.

Parameter	Type	Description	Default
pid	pid	The toplevel session to call.	<i>required</i>
goal	string	The goal to call (Prolog term as text).	<i>required</i>
template	string	Output template; named query variables are returned as a JSON dict.	Same as goal
offset	int ≥ 0	Offset in the virtual list of solutions.	0
limit	int > 0	Max solutions to return.	Effectively unlimited
once	boolean	Restrict to one solution slice.	false
load_text	string	Additional source text loaded before the call. Reloaded only if changed since the previous call.	empty string
format	atom	Response format (<code>json</code> or <code>prolog</code>).	<code>json</code>

Table A.2 Parameters for the `toplevel_call` command.

Asynchronous responses: `success`, `failure`, `error`, `output`, `prompt`, `timeout`, or `abort`.

Table A.3 summarises the remaining toplevel commands, which take fewer parameters.

A.10.3 Bare actor commands

Response: `spawned` with the actor pid.

The `send` command is fire-and-forget: no response is returned to the client. The target may be a connection-owned actor, a published named service, or – if the principal holds `internal_transport` capability – any actor.

Command	Parameters	Response types
<code>toplevel_next</code>	<code>pid</code> (required), <code>limit</code> (opt.)	<code>success</code> , <code>failure</code> , <code>error</code> , <code>output</code> , <code>prompt</code> , <code>timeout</code> , <code>abort</code>
<code>toplevel_stop</code>	<code>pid</code> (required)	<code>stop</code>
<code>toplevel_abort</code>	<code>pid</code> (required)	<code>abort</code>
<code>toplevel_halt</code>	<code>pid</code> (required)	<code>down</code>
<code>toplevel_respond</code>	<code>pid</code> (required), <code>input</code> (required)	<code>responded</code>
<code>set_trace</code>	<code>pid</code> (required), <code>enabled</code> (required, boolean)	<code>responded</code>

Table A.3 Remaining toplevel session commands.

Parameter	Type	Description	Default
<code>goal</code>	string	The goal the actor executes (Prolog term as text).	<i>required</i>
<code>options</code>	list/string	Spawn options (<code>load_text</code> , <code>load_uri</code> , etc.).	<code>[]</code>

Table A.4 Parameters for the `spawn` command.

Command	Parameters	Response types
<code>send</code>	<code>pid</code> (required), <code>message</code> (required)	<i>none</i> (fire-and-forget)
<code>exit</code>	<code>pid</code> (required), <code>reason</code> (opt., default <code>kill</code>)	<code>down</code> (asynchronous)

Table A.5 Actor messaging commands.

A.10.4 Server-push event types

The server sends events to the client asynchronously via the relay thread. Table A.6 lists all possible JSON event types.

A.10.5 Summary of WebSocket commands

A.11 The semi-stateful HTTP API

The semi-stateful API is built on long-lived toplevel actors plus per-session queues. It exposes the same toplevel commands as the WebSocket API – `toplevel_spawn`, `toplevel_call`, `toplevel_next`, `toplevel_respond`, `toplevel_stop`, `toplevel_abort` – and the command parameters are those given for the WebSocket API in Section A.10 (Tables A.1 and A.2). Only the differences introduced by the HTTP transport are described here.

The substantive differences are:

JSON type	Fields	Trigger
spawned	pid	Actor or session created
success	pid, data, more	Goal produces solution(s)
failure	pid	Goal has no (more) solutions
error	pid, data	Execution error
output	pid, data	I/O output (writeln, format, etc.)
prompt	pid, data	Actor awaits input
timeout	pid	Execution exceeded time limit
stop	pid	Toplevel cleanly stopped
abort	pid	Toplevel execution aborted
down	pid, reason	Actor terminated
error (no pid)	data	Protocol-level error (bad frame, rate limit, auth)

Table A.6 Server-push event types on the WebSocket connection.

Command	Group	Ownership	Response
toplevel_spawn	session	—	spawned
toplevel_call	session	owned session	async events
toplevel_next	session	owned session	async events
toplevel_stop	session	owned session	stop
toplevel_abort	session	owned session	abort
toplevel_halt	session	owned session	down
toplevel_respond	session	owned session	responded
set_trace	session	owned session	responded
spawn	actor	—	spawned
send	actor	owned or published	none
exit	actor	owned actor	down

Table A.7 Summary of all WebSocket commands.

- **No server push.** Results are observed by long-polling rather than pushed; the additional endpoint `/toplevel_poll` (taking `pid`, optional `format`, and optional `timeout`) retrieves the next pending event for a session.
- **Session forced.** The `/toplevel_spawn` endpoint always forces `session(true)`, regardless of any `session/1` value in `options`.
- **Output redirection.** The endpoint injects a small prelude that redirects `writeln/1` to `actor:output/1`.
- **Owner-capped timeout.** A `timeout` parameter on `/toplevel_call` and `/toplevel_next` requests a wait timeout in seconds; the effective value is the minimum of the request and the node owner's timeout cap.

Table A.8 lists the complete set of endpoints and their current responses in the semi-stateful API.

API endpoint	Response types
/toplevel.spawn	spawned or error
/toplevel.call	success, failure, error, output, prompt, timeout, or abort
/toplevel.next	success, failure, error, output, prompt, timeout, or abort
/toplevel.respond	responded or error
/toplevel.poll	success, failure, error, output, prompt, timeout, or abort
/toplevel.stop	stop
/toplevel.abort	abort

Table A.8 The complete set of endpoints and responses in the semi-stateful API.

A.12 The stateless HTTP API

The stateless API is exposed at `/call`.

Parameter	Type	Description	Default Value
goal	callable	The goal to be called.	None
template	term	The template to be used. For <code>format=json</code> , named query variables are returned as a JSON dict template.	Same as goal
offset	int ≥ 0	The offset in the virtual list of solutions.	0
limit	int > 0	Max number of solutions to be returned.	Effectively no limit
load_text	string	Source text to be injected into the temporary actor context.	empty string
timeout	number	Requested timeout for remote goal execution, capped by the node owner's timeout setting.	Owner timeout setting
once	boolean-ish atom	Restrict execution to one slice even if more solutions may exist.	false
format	atom	The format of answer responses.	json

Table A.9 HTTP API parameters for the `/call` endpoint.

The only required parameter is `goal`.

Although the API is stateless from the client's perspective, the server may cache a live `toplevel` actor internally to continue paged solution retrieval without recomputing earlier work.

Appendix B

Glossary

This appendix collects working definitions for core terms used across the book and the current node implementation. The bias is conceptual first and operational second: each entry states what the term means in the book and, when useful, how the present implementation realises it.

Actor. An actor is a long-lived concurrent process with a pid, a mailbox, and an explicit message interface. In the ACTOR profile, actors are the basic unit of concurrency. Toplevels, statechart actors, and node-resident services are all specialised actor forms.

Agent. An agent is a persistent interactive process that participates in ongoing interaction with its environment. In this book, a Prolog agent is defined by its interface and behaviour rather than by its implementation language: it talks Prolog terms to other agents by receiving goals or messages and returning answers, messages, or effects in the same term-based form.

Actors, toplevels, and nodes can all count as agents, but they play different roles. Actors are loci of concurrent computation, toplevels expose a conversational interface, and nodes host knowledge, expose web APIs, and enforce policy.

Capability. A capability is authority to do or affect something. In the book, the term is often used in the capability-security sense: possession of an unforgeable or hard-to-guess handle, such as a pid or a reply channel, confers some authority. In the current node implementation, the auth layer also uses named policy capabilities such as `execute`, `admin`, and `internal_transport`, attached to a principal. These are different mechanisms, but they answer the same question: what authority is being granted?

Containment. Containment is the structural security discipline that keeps client-created computation bounded in lifetime and effect. A contained computation may still do real work, but it should not outlive its session, escape its authority bounds, or leave behind uncontrolled background activity.

In the current node, containment is reflected in the default rule that client-spawned actors are linked to the client or session that created them. Long-lived detached services are owner-managed exceptions, not the default.

Federation. Federation is the decision to standardise what systems share at the boundary rather than to force them to become internally identical. In the Trinity, this means shared web-facing interaction surfaces and execution profiles, while leaving room for different Prolog systems to diverge internally.

Inexpressibility. Inexpressibility is the structural security property that certain dangerous operations cannot be expressed at all in client code. Rather than relying only on runtime refusal, the exposed language fragment excludes operations such as direct file I/O, raw operating-system access, or other ambient-authority mechanisms that would be unsafe to hand to arbitrary clients.

In the book, this is the sandbox side of the security story. In the current node, profile checks and sandbox policy are the implementation mechanisms that approximate this boundary.

Instant portability. Instant portability is the limiting case of portability in which relocation requires no adaptation step. A program can be downloaded from one node and run locally, or shipped to another node, immediately because everything it depends on is either bundled with it or guaranteed by the target profile.

Interaction surface. An interaction surface is the externally visible contract through which systems meet: routes, message formats, APIs, protocol rules, and profile promises. The book uses this term to emphasise that interoperability depends on shared boundaries, not on shared internals.

Interoperability. Interoperability is the ability of independently implemented systems to work together correctly through shared protocols, representations, and profile contracts. In this project, it is a stronger notion than mere syntactic compatibility: two nodes interoperate when a client can rely on the same observable behaviour across them at the advertised profile boundary.

Invisibility. Invisibility is the structural security property that a client can interact only with actors or services whose handles or names have been intentionally revealed. It is the name-discipline side of security: what cannot be named cannot be addressed.

In the book, opaque pids and owner-curated service publication are the key mechanisms. In the current node, this also means that published services are not the same thing as the ordinary client name registry.

Node. A node is a network-addressable host of execution, knowledge, and policy. A node exposes Web Prolog APIs, advertises a profile, enforces authentication and authorisation policy, and may host both shared predicates and node-resident actors.

Owner. An owner is the authority responsible for configuring a node's public surface and long-lived resources. The owner role includes installing shared predicates, publishing or withdrawing services, changing node settings and policy, and taking responsibility for durable service lifecycle concerns such as supervision and recovery.

In the current implementation, owner authority is realised through administrative policy rather than through ordinary client capabilities. It is best understood as a role in the security model, not merely as "the client who happens to connect first."

Pid. A pid is an opaque process identifier for an actor. It is a routable handle, not a descriptive name, and in security terms it often functions as a capability. Clients should treat a pid as an implementation handle that may be passed around or used for communication, but not as something that should be globally enumerable.

Portability. Portability is the extent to which code, programs, or interaction patterns can move between nodes or implementations without semantic breakage or manual rewriting. In the book, portability is pursued by standardising profiles and their mandatory capabilities rather than by demanding total internal uniformity across Prolog systems.

Principal. A principal is the identity on whose behalf a request is made. It is not the same thing as a user, session, connection, or pid. A principal may be an anonymous caller, a human user, a service account, or a trusted internal transport identity.

In the current node implementation, principals are resolved by the auth layer and mapped to policy capabilities. Authorisation answers what a principal may do; it does not by itself determine which pid or session is involved.

Private database. A private database is the actor-local body of clauses and facts associated with an actor. It forms part of that process's private workspace rather than the node's shared knowledge, and it is not directly accessible to other actors. In practical terms, the private database can be populated at spawn time by `load_*` options, it may shadow definitions from the shared database, and updates to it affect only that actor.

Profile. A profile is a deliberately scoped contract describing which language features, APIs, and interaction patterns are available. Profiles let Web Prolog specify a shared operational subset without turning Web Prolog into a separate fork of Prolog.

In the current node implementation, the configured node profile is node-scoped, not principal-scoped. It governs which routes and goal forms are in profile, and public execution checks enforce that boundary.

Profile hierarchy (RELATION, ISOBASE, ISOTOPE, ACTOR). The profile hierarchy is the nested sequence of profiles by which a system declares how much of Web Prolog it supports, each profile extending the one before it. **RELATION** is the minimal profile, exposing relational querying only. **ISOBASE** adds a pure ISO-style toplevel without mutable state or Erlang-style concurrency, and so suits strictly declarative systems. **ISOTOPE** adds the further machinery needed for a standard interactive Prolog environment. **ACTOR**, the richest profile, adds the Erlang-style concurrency and distribution predicates on which long-lived agents and node-resident services depend. Because the profiles nest, a system need only conform to the simplest profile that covers its capabilities, which is how the hierarchy admits even systems lacking multi-threading.

PTCP. PTCP is the Prolog Toplevel Communication Protocol. It is the explicit protocol governing conversation between a client and a toplevel actor: how goals are submitted, how answers are streamed incrementally, how output and prompts may occur, and how abort, stop, and exit are handled.

Its importance in the book is that it turns informal REPL behaviour into a first-class behavioural contract. In implementation terms, PTCP underlies the observable behaviour of toplevel actors across the stateless, semi-stateful, and stateful APIs.

Public surface. A node's public surface is the set of predicates, services, routes, and policies that it intentionally exposes to clients. It is smaller than the total internal runtime and should be treated as an explicit design choice rather than as an accident of what happens to be running.

Registered name. A registered name is a symbolic name bound to an actor. Ordinary registered names are convenience names for actor lookup and messaging; they are not necessarily public services.

Under the current regime, ordinary client registration and published service registration are separate. Client-side `register/2`, `whereis/2`, and `unregister/1` operate within a client-visible naming scope, while published services use an owner-controlled service registry.

Reply channel. A reply channel is a communication endpoint created for the purpose of receiving one reply or a narrowly bounded class of replies. In capability terms, it should carry reply authority only, not general authority to invoke the client.

The book treats reply channels as a way to avoid leaking a client's full pid to an untrusted peer. The general principle is that communication authority should be as narrow and short-lived as the protocol step that requires it.

rpc/2-3. `rpc/2-3` is the remote procedure call interface by which a client submits a goal to a remote node and receives its answers, as if calling a predicate across the network. It is the sequential, query-oriented face of the Prolog Web: a stateless or semi-stateful request that returns answers on backtracking, in contrast to the long-lived message exchange of the actor model. In implementation terms it is built on toplevel actors and governed by PTCP, and it is the mechanism through which facilities such as service discovery are exposed.

Run-to-completion. Run-to-completion is the execution discipline of statechart actors in which processing a single external event may trigger a finite cascade of internal transitions and actions before the next external message is consumed. It is what makes a statechart actor's control flow explicit and inspectable: between external events the machine always settles in a stable configuration, so its observable state is well defined at every message boundary.

Server. A server, in the sense used by the book's generic-behaviour chapters, is a stateful actor that maintains private state and serves clients through a stable request-response message protocol. The generic server behaviour factors out the machinery of message routing, state threading, and reply handling so that the application-specific logic can be supplied as a callback predicate.

This is an Erlang-style behaviour: it exposes a protocol, not a predicate-level logical interface, and it does not bind the caller's variables directly.

Service. A service is a durable, shared conversational resource intentionally offered by a node for repeated use by multiple clients. In the Prolog Web, services may

be predicate-based or actor-based, but node-resident actor services are especially important because they provide shared state or shared coordination over time.

A published actor service is not just any actor with a convenient name. It is something the node owner explicitly publishes under a stable service name, and it is this owner-controlled publication – rather than ordinary client registration – that distinguishes a service from a merely named actor. The two following entries describe the act of publishing such a service and the means by which clients find it.

Service discovery. Service discovery is the process by which clients learn which published services a node makes available. It resolves published service names without revealing everything that happens to be running, and so is an owner-curated lookup rather than a form of process introspection. In the current book example, it is realised declaratively through relations such as `service/2` queryable via `rpc/2-3`.

Service publication. Service publication is the act by which a node owner places a name into the node's public service namespace, making the service part of the node's advertised public surface. It is managed through the owner-controlled service registry, and is therefore distinct from the ordinary client-side `register/2` used for client-managed actors (see Service and Registered name).

Session. A session is a bounded continuing interaction context in which state can persist across multiple protocol steps. The book contrasts session-oriented interaction with purely stateless request-response exchange.

In the current node, HTTP ISOTOPE sessions and WebSocket ACTOR sessions are both session forms, but they have different timing and failure boundaries.

Shared database. The shared database is the node-owned body of predicates and facts made available as shared knowledge. It is loaded once at node startup into a shared runtime module and then imported by actor modules or used by node-level query execution.

Clients may query against shared knowledge and combine it with private code, but the shared database is not the same thing as a client's private mutable workspace.

Statechart actor. A statechart actor is a Web Prolog actor whose control logic is given by a statechart document. It behaves like an ordinary actor with a mailbox, but incoming messages are interpreted as events that drive a structured state machine rather than an ad hoc `receive/1-2` loop.

The distinctive semantic idea is run-to-completion: processing one external event may trigger a finite cascade of internal work before the next external message is consumed. This makes control explicit and inspectable while keeping the process inside the ordinary actor model.

Supervisor. A supervisor is an actor whose main responsibility is to start, monitor, and manage child actors, especially in the face of failure. In the book's Erlang-style account, a supervisor embodies the “let it crash” discipline: workers are allowed to fail, and the supervisor applies the configured recovery policy.

Supervisors matter most for durable node-resident services, where restart and cleanup are part of the node owner's operational responsibility rather than something each worker should reimplement ad hoc.

Toplevel actor. A toplevel actor is an actor whose behaviour is governed by an explicit conversational protocol for submitting goals, receiving answers incrementally, handling output, and managing abort or stop. It turns the ordinary Prolog toplevel into a first-class actor process.

In implementation terms, toplevel actors underpin `rpc/2-3`, the semi-stateful HTTP API, and much of the browser-facing shell machinery.

Appendix C

How to implement a Prolog node

Vision without execution is just hallucination.

Thomas Alva Edison

We would have loved to be able to present a stable, speedy and secure implementation of a Prolog node, ready to be deployed to help building the Prolog Web. However, there exists no such implementation at this point in time. There are some proof-of-concept implementations, but they are neither stable nor speedy, nor secure. How can we build one that is? And how can we build *more* than one, so that we can make sure that interoperability across different implementations works as intended?

We also say something about browser-based implementations of Web Prolog and briefly discuss the prospects for developing a virtual machine with properties similar to Erlang's BEAM VM.¹

C.1 Wrapping a node around an existing Prolog system

Make it work, then make it beautiful, then if you really, really have to, make it fast. 90% of the time, if you make it beautiful, it will already be fast. So really, just make it beautiful!

Joe Armstrong

It is likely that the first implementations of Prolog nodes would be Prolog systems providing as libraries whatever is required to comply with Web Prolog requirements. This is how our proof-of-concept implementations were built.

Some really excellent Prolog systems exist out there, so if you are a Prolog implementer, one obvious advantage with this approach is that most of the necessary

¹ [https://en.wikipedia.org/wiki/BEAM_\(Erlang_virtual_machine\)](https://en.wikipedia.org/wiki/BEAM_(Erlang_virtual_machine))

work has already been done. The amount of additional work required to implement a node depends on which system it is built on top of.

In this section, we look at different ways to wrap a node around a system that supports the ISO Prolog working draft for threads.² We are aiming for an almost complete ISOBASE node, and an ACTOR node, albeit less complete.

Using SWI-Prolog, we show a way to implement the stateless HTTP API. We do not focus solely on semantics, but on performance too. In particular, we devise a way to optimize the API by avoiding the unnecessary recomputation of solutions that a naive implementation would have to do. Furthermore, we implement a version of `rpc/2-3` on top of the stateless HTTP API.

We implement specifications for how we believe predicates such as `spawn/1-3`, `!/2` and `receive/1-2` should work. On top of actors, we implement the behavior of Prolog toplevels. These implementations focus on semantics rather than performance.

We are seeking, if not beauty, then at least as much clarity and simplicity as possible. Our implementations are only partial, but we also indicate what else would be needed to complete them.

C.1.1 Implementing an ISOBASE node

A Prolog ISOBASE node is equipped with a stateless HTTP API. Managing this API is actually the only task its node controller is responsible for. It means that we can make good use of a library for building web servers. Here is how a web server may be written in SWI-Prolog using `library(http/http_server)`:

```
:- use_module(library(http/http_server)).

:- http_handler(root(call), node_controller_isobase, []).

node_controller_isobase(Request) :-
    http_parameters(Request, [
        goal(GoalAtom, [atom]),
        template(TemplateAtom, [default(GoalAtom)]),
        offset(Offset, [integer, default(0)]),
        limit(Limit, [integer, default(10 000 000 000)]),
        format(Format, [atom, default(json)])
    ]),
    atomic_list_concat([GoalAtom,+,TemplateAtom], QTAtom),
    read_term_from_atom(QTAtom, Goal+Template, []),
    compute_answer(Goal, Template, Offset, Limit, Answer),
    respond_with_answer(Format, Answer).
```

² <http://logtalk.org/plstd/threads.pdf>


```
node(Port) :-
    http_server(http_dispatch, [port(Port)]).
```

The call to `compute_answer/5` is responsible for the real work here. It takes a goal, a template, an offset and a limit, and computes an answer term serving as a response to the request which can be sent back to the client formatted as Prolog or JSON. There is more than one way to implement this predicate. Let us first look at a simple (but from a performance point of view naive) way of doing it.

SWI-Prolog offers a library predicate `findnsols/4` which provides a useful foundation for our implementation. It is somewhat similar to the standard `findall/3`, but expects an integer `Limit` in its first argument and will generate at most that many solutions. It is also nondeterministic, so on backtracking it will do it again. We borrow an example of its use from the SWI-Prolog manual:³

```
?- findnsols(5, I, between(1, 12, I), L).
L = [1, 2, 3, 4, 5] ;
L = [6, 7, 8, 9, 10] ;
L = [11, 12].
?- 
```

Another SWI-Prolog library predicate `offset/2` will also prove useful.⁴ Its purpose is to *skip* the first n solutions to a goal, i.e. the first n solutions are computed, but not collected. Here is an example of its use:

```
?- offset(10, between(1, 12, I)).
I = 11 ;
I = 12.
?- 
```

Combining `findnsols/4` with `offset/2` allows us to implement a predicate `slice/5` capable of computing a *slice* of solutions to a goal:

```
slice(Goal, Template, Offset, Limit, Slice) :-
    findnsols(Limit, Template, offset(Offset, Goal), Slice).
```

However, we are looking for *answers*, rather than just slices of solutions. By wrapping a call to `slice/5` in a call to `call_cleanup/2` wrapped by a call to `catch/3` we arrive at a predicate `answer/5` capable of producing the four different forms of answer terms that we need:

```
answer(Goal, Template, Offset, Limit, Answer) :-
    catch(
        call_cleanup(slice(Goal, Template, Offset, Limit, Slice),
            Det = true),
        Error, true),
```

³ https://www.swi-prolog.org/pldoc/doc_for?object=findnsols/4

⁴ https://www.swi-prolog.org/pldoc/doc_for?object=offset/2

```

(   Slice == []
->  Answer = failure
;   nonvar(Error)
->  Answer = error(Error)
;   var(Det)
->  Answer = success(Slice, true)
;   Det = true
->  Answer = success(Slice, false)
).
```

This predicate will turn out to be useful in more than one way. In this context it will be used for the implementation of `compute_answer/5`. In this role we want `compute_answer/5` to be deterministic, so since the call to `answer/5` is non-deterministic we need to wrap it in a call to `once/1` like so:

```

compute_answer(Goal, Template, Offset, Limit, Answer) :-
    once(answer(Goal, Template, Offset, Limit, Answer)).
```

The implementation of our simple but naive stateless HTTP API is almost complete, and assuming we also have a suitable implementation of `respond_with_answer/2`, we can now start running a node:

```

?- node(3010).
% Started server at http://localhost:3010/
true.
?- 
```

At this point we may want to take the node's stateless HTTP API for a trial run by entering the following URI in a web browser:

```
http://localhost:3010/call?goal=member(X,[a,b])&format=prolog
```

In the browser's window, we should then see the following:

```
success([member(a,[a,b]),member(b,[a,b])],false)
```

By appending `&template=X&offset=0&limit=1` to the URI we should get

```
success([a],true)
```

and by incrementing the `offset` parameter by 1 we should see

```
success([b],false)
```

Note that it is important that we do not expose the node to the whole world at this point, as it is not secure.

C.1.2 Implementing `rpc/2-3` on top of the stateless HTTP API

As soon as we have an implementation of the stateless HTTP API, we can easily, by means of two other libraries provided by SWI-Prolog,⁵ implement `rpc/2-3` on top of it. Here is the source code:

```
:- use_module(library(http/http_open)).
:- use_module(library(url)).

rpc(URI, Goal) :-
    rpc(URI, Goal, []).

rpc(URI, Goal, Options) :-
    parse_url(URI, Ps),
    term_variables(Goal, Vars),
    Template =.. [v|Vars],
    format(atom(GA), "~p", [Goal]),
    format(atom(TA), "~p", [Template]),
    option(limit(L), Options, 10 000 000 000),
    rpc_7(Template, 0, L, GA, TA, Ps, Options).

rpc_7(Template, 0, L, GA, TA, Ps, Os) :-
    parse_url(ExpandedURI, [
        path('/call'),
        search([goal=GA, template=TA, offset=0,
            limit=L, format=prolog])
    | Ps
    ]),
    setup_call_cleanup(
        http_open(ExpandedURI, Stream, Os),
        read(Stream, Answer),
        close(Stream)),
    rpc_8(Answer, Template, 0, L, GA, TA, Ps, Os).

rpc_8(success(Slice, true), Template, 0, L, GA, TA, Ps, Os) :- !,
    ( member(Template, Slice)
    ; New0 is 0 + L,
      rpc_7(Template, New0, L, GA, TA, Ps, Os)
    ).

rpc_8(success(Slice, false), Template, _, _, _, _, _, _) :-
    member(Template, Slice).

rpc_8(failure, _, _, _, _, _, _, _) :- fail.
rpc_8(error(E), _, _, _, _, _, _, _) :- throw(E).
```

⁵ See <https://www.swi-prolog.org/pldoc/man?section=httpopen> and <https://www.swi-prolog.org/pldoc/man?section=url>

The idea behind this code is to use `http_open/3` in a loop in order to make one or more requests for consecutive slices of solutions to the goal in the first argument using the stateless HTTP API. The URI of each request takes the form

`BaseURI/call?goal=G&template=T&offset=O&limit=L&format=prolog`

where `O` is initially `0` and is incremented by `L` between requests.

The most interesting parts of the implementation are the use of the disjunction in the body of the first `rpc/7` clause and the use of `member/2` in the first and second clauses. They are responsible for turning the responses to the deterministic requests made by `http_open/3` into the nondeterministic behavior we want `rpc/2-3` to show.

Let us test our implementation by running an example from Chapter 4, showing how `rpc/2-3` can be used:

```
?- [user].
|: human(plato).
|: human(aristotle).
|: ^D% user://1 compiled 0.00 sec, 2 clauses
true.
?- rpc('http://localhost:3010', human(Who)).
Who = plato ;
Who = aristotle.
?-
```

Note that although the query has two solutions, only one network roundtrip is made, triggered by the following HTTP request:

```
GET http://localhost:3010/call?goal=human(Who)&format=prolog
```

The response contains the following answer term:

```
success([human(plato),human(aristotle)],false)
```

The above code is just a sketch that leaves out some of the details that are necessary for a fully working node. In particular, it does not implement `respond_with_answer/2` and it does not handle syntax errors in queries. None of this would be difficult to add, and with such additions, this section together with the previous one implements the stateless API of an ISOBASE node, as well as the `rpc/2-3` predicate.

C.1.3 Fixing a problem due to spurious recomputation

The above implementation of the HTTP API suffers from a performance problem. The problem is easy to spot when timing a goal simulating a situation where a first solutions takes a long time to compute while a second solution takes almost no time at all – a goal such as the disjunction (`sleep(1), X=foo ; X=bar`) for example. Here is how this looks in a system such as SWI-Prolog:

```
?- time((sleep(1), X=foo ; X=bar)).
% 1 inferences, 0.000 CPU in 1.005 seconds
X = foo ;
% 7 inferences, 0.000 CPU in 0.000 seconds
X = bar.
?-
```

As expected, solving the first disjunct took one second, while the second disjunct took almost no time at all. However, when calling this goal using `rpc/3` with the `limit` options set to 1, we see the following:

```
?- _URI = 'http://localhost:3010',
    time(rpc(_URI, (sleep(1), X=foo ; X=bar), [limit(1)])).
% 1,984 inferences, 0.001 CPU in 1.006 seconds
X = foo ;
% 1,804 inferences, 0.001 CPU in 1.009 seconds
X = bar.
?-
```

The cause of this problem lies not in the implementation of `rpc/2-3`, but in the HTTP API, and more precisely in the way `compute_answer/5` works. Consider the following call, where the third argument (for the offset) is 1:

```
?- _Goal = (sleep(1), X=foo ; X=bar),
    time(compute_answer(_Goal, X, 1, 1, Answer)).
% 30 inferences, 0.000 CPU in 1.005 seconds
Answer = success([bar], false).
?-
```

In general, computing the first slice (i.e. the one starting at offset 0) is as fast as it can be, but computing the second slice involves the recomputation of the first slice and, more generally, computing the *n*th slice involves the recomputation of all preceding slices, the results of which are then just thrown away. This, of course, is a waste of resources and puts an unnecessary burden on the node.

This is not as bad as it looks. Most uses of `rpc/2-3` will compute all solutions at once and thus make only one network roundtrip.

```
?- time(rpc($_URI, (sleep(1), X=foo ; X=bar))).
% 2,011 inferences, 0.001 CPU in 1.007 seconds
X = foo ;
% 5 inferences, 0.000 CPU in 0.000 seconds
X = bar.
?-
```

As can be seen in the following example run, it is only when more than one network roundtrip may have to be made, so that the `limit` option must be employed, that the problem surfaces:

```
?- _Goal = (sleep(1), X=foo ; X=bar),
    time(compute_answer(_Goal, X, 0, 2, Answer)).
% 29 inferences, 0.000 CPU in 1.002 seconds
Answer = success([foo, bar], false).
?-
```

Still, to achieve a less wasteful and more efficient stateless querying even when more than one network roundtrip must be made, recomputation of the kind described in the previous section should be avoided. In this section we lay out an approach where the node controller (subject to a setting) may *cache* the state of the toplevel process that produced the *n*th slice of solutions to a query, so that the work spent on producing it will not have to be repeated. This can still be done without requiring that the node controller remembers *which* client made the request for the previous slices of solutions.

The method can be seen as a kind of *pooling* of toplevel processes, but while pooling usually involves a pool of merely initialized but idle processes which stand ready to be given work, this method involves a pool where each member has already done some real work. In other words, the idea here is not to cache *already computed* solutions but rather to cache the *potential* for new solutions in the form of processes that are idle, but have “more to give” if put to work.⁶

A consequence of this approach is that it allows the computation of the full set of solutions to a query to be distributed over more than one toplevel process. We can avoid spawning a new process for each incoming request, but instead, when available, select a member from a pool of suspended processes which, since it has already performed some of the work, needs to do as little as possible in order to compute the requested solutions. Using this approach, it is likely (although not guaranteed) that the work that generated the *n*th slice of solutions does not have to be repeated if a request for the next slice is made.

One way to realize this is to make the node controller responsible for the maintenance of a cache consisting of entries pointing to members of the pool of suspended processes. Such a cache has a very straightforward implementation in Prolog thanks to its dynamic database. The signature of a cache entry can be given as follows:

`cache(+Gid, +N, -Pid)` is nondet.

Here, *Gid* is an identifier representing a goal *G* and a template *T*. *N* is an integer > 0 , and *Pid* is the pid of an already spawned process which, after having computed *N* solutions to *G* and returned them to the client, is now suspended but can be activated again at any point. A cache is simply a dynamic predicate comprising an ordered sequence of `cache/3` clauses. The cache will be searched from the top, stopping when the first match is found. Updates will be added to the bottom.⁷

The cache forms a queue-like data structure and can be seen as a kind of priority queue. When a request comes in which specifies a goal, a template, and an offset *i*, 1,

⁶ Credits for this idea goes to Jan Wielemaker. The implementation is our's.

⁷ Note that the implementation of the cache as a Prolog predicate is not mandated. A node would be free to implement it in a way that suits the host platform best.

Check the principle of locality in

<https://chatgpt.com/share/68d7f62b-68d7-4b2b-bc6d-24dc-800f-a795-94d2958c9671>

the cache is scanned from the beginning of the queue, the first matching entry is dequeued, and the corresponding process is employed. If no matching entry is found, a new process is spawned. Newly created as well as updated cache entries are added to the end of the queue.

The maximum size of the cache for a particular node can be specified by its owner by means of a setting. What is a reasonable size depends on the host platform of the node, and in particular on the cost of keeping suspended toplevel actors around.

Here is an implementation of two predicates for managing the cache:

```
:- dynamic cache/3.

cache_retract(Gid, N, Pid) :-
    once(retract(cache(Gid, N, Pid))).

cache_update(Gid, N, Pid) :-
    assertz(cache(Gid, N, Pid)),
    setting(cache_size, Size),
    predicate_property(cache(_,_,_),
        number_of_clauses(N)),
    N > Size -> cache_retract(_,_,_) ; true.
```

To ensure efficient cache lookup, the goal identifier Gid is a hash value computed from a grounded copy of the goal. In SWI-Prolog, `goal_id/2` may be implemented as follows:

```
goal_id(GoalTemplate, Gid) :-
    copy_term(GoalTemplate, Gid0),
    numbervars(Gid0, 0, _),
    term_hash(Gid0, Gid).
```

Equipped with the above utility predicates, `compute_answer/5` can be implemented like so:

```
compute_answer(Goal, Template, Offset, Limit, Answer) :-
    goal_id(Goal-Template, Gid),
    (   cache_retract(Gid, Offset, Pid)
->   thread_self(Self),
        toplevel_next(Pid, [
            limit(Limit),
            target(Self)
        ])
    ;   toplevel_spawn(Pid, [session(false)]),
        toplevel_call(Pid, Goal, [
            template(Template),
            offset(Offset),
            limit(Limit)
        ])
    )
```

```

),
setting(timeout, Timeout),
receive({
    success(Pid, Slice, true) ->
        Index is Offset + Limit,
        cache_update(Gid, Index, Pid),
        Answer = success(Slice, true);
    success(Pid, Slice, false) ->
        Answer = success(Slice, false);
    failure(Pid) ->
        Answer = failure;
    error(Pid, Error) ->
        Answer = error(Error)
}, [
    timeout(Timeout),
    on_timeout((Answer = error(timeout),
        exit(Pid, kill)))
]).

```

Given a goal and a template, a goal identifier `Gid` is computed. Since more than one client may request the same slice of solutions, the `Gid` is not unique. Based on the `gid` and the value of the `offset` parameter, an attempt to look up a cache entry pointing to a suitable toplevel process will be made. If this succeeds, `toplevel_next/2` will be called, which will compute an answer holding a slice of solutions no longer than the value of the `limit` parameter specifies. If it fails, a new toplevel will be spawned using `toplevel_spawn/3`, and `toplevel_call/3` will be called, which will compute the answer instead. *Note.* In this implementation, `toplevel_call/3` reads only the options `template/1`, `offset/1`, `limit/1`, `once/1`, and `target/1`. Other options in the list are silently ignored. This keeps wrappers simple, but misspelled option names will also be ignored.

The answer term resulting from this is sent to the thread in which the request handler is running and can be caught by `receive/2`. Note that if the reception of the term takes too long, it will result in a timeout error.

```

?- _Goal = (sleep(1), X=foo ; X=bar),
   time(compute_answer(_Goal, X, 0, 1, Answer)).
% 30 inferences, 0.000 CPU in 1.005 seconds
Answer = success([foo], true).
?-
...
?- _Goal = (sleep(1), X=foo ; X=bar),
   time(compute_answer(_Goal, X, 1, 1, Answer)).
% 30 inferences, 0.000 CPU in 0.005 seconds
Answer = success([bar], false).
?-

```


How can we extend the implementation of the ISOBASE node so that it can serve also as an ISOTOPE node? As evident from the diagram in Figure 3.8, it needs support for the `load_text` parameter. Its value must be sent along when calling `toplevel_spawn/2`, which will inject the code in the private database of the `toplevel`. Moreover, the goal identifier must be based on *both* the goal, the template and this value. Code for handling all of this would be easy to add.

C.1.4 Implementing the Erlang-style concurrency predicates

This section implement specifications for how we believe predicates such as `spawn/1-3`, `exit/1-2`, `!/2` and `receive/1-2` might work. To keep things as succinct as possible we do not add code checking the instantiation of arguments. (However, some such tests are present in the proof-of-concept mini implementation.)

Today widely available Prolog systems can be differentiated whether they are multi-threaded or not. In a multi-threaded Prolog system we can create multiple threads that run concurrently over the same knowledge base. From Table 2 in *Fifty Years of Prolog and Beyond* we learn that out of the Prolog systems listed above, five implement multi-threading support. According to this table, these are Ciao, ECLiPSe, SWI-Prolog, tuProlog and XSB. However, we that that Trealla Prolog should also be added to the list, and thus we have six systems with multi-threading support.

There is a draft standard for multi-threading support in Prolog, specified in a document that begins like so:

ISO/IEC DTR 13211-5:2007 Prolog multi-threading support [...] is an optional part of the International Standard for Prolog, ISO/IEC 13211. [...] Multi-thread predicates are based on the semantics of POSIX threads. They have been implemented in some Prolog systems. As such, they are deemed a worthy extension to the ISO/IEC 13211 Prolog standard.⁸

Except for Ciao Prolog, which takes a different approach to multi-threading, the six systems listed above all implement the draft standard.

In order to support the Erlang-style concurrency predicates offered by the ACTOR profile of Web Prolog the five predicates on the left can be implemented by means of the seven predicates from the draft standard on the right:

<code>spawn/3</code>	<code>thread_create/3</code>
<code>self/1</code>	<code>thread_self/1</code>
<code>!/2, send/2</code>	<code>thread_send_message/2</code>
<code>receive/1-2</code>	<code>thread_get_message/3</code>
<code>exit/2</code>	<code>thread_signal/2</code>
	<code>thread_detach/1</code>
	<code>thread_property/2</code>

⁸ <https://logtalk.org/plstd/threads.pdf>

The drafts standard specifies more than a dozen more predicates, such as predicates for creating message queues and managing mutexes. We do not need those.

Here is a first sketch of an implementation of `spawn/1-3`:

Change `start/4`
into `initialize/4`,
Change `stop/2`
into `terminate/2`,
change `down/3` to
`terminate/3`, see to
it that eason 'kill'
is converted into
killed, by changing
the implementation
if `down_reason/2`?

```
:- op(800, xfx, !).
:- op(1000, xfy, when).

:- dynamic link/2.

spawn(Goal) :-
    spawn(Goal, _Pid).

spawn(Goal, Pid) :-
    spawn(Goal, Pid, []).

spawn(Goal, Pid, Options) :-
    thread_self(Self),
    make_pid(Pid),
    thread_create(start(Self, Pid, Goal, Options), Pid, [
        alias(Pid),
        at_exit(stop(Pid, Self))
    ]),
    thread_get_message(initialized(Pid)).

make_pid(Pid) :-
    random_between(100000000, 99999990, Num),
    atom_number(Pid, Num).

:- thread_local parent/1.

start(Parent, Pid, Goal, Options) :-
    assertz(parent(Parent)),
    option(link(Link), Options, true),
    (   Link == true
    -> assertz(link(Parent, Pid))
    ;   true
    ),
    option(monitor(Monitor), Options, false),
    (   Monitor == true
    -> assertz(monitor(Parent, Pid))
    ;   true
    ),
    thread_send_message(Parent, initialized(Pid)),
    call(Goal).
```

```

stop(Pid, Parent) :-
    thread_detach(Pid),
    retractall(link(Parent, Pid)),
    retractall(registered(_Name, Pid)),
    forall(retract(link(Pid, ChildPid)),
           exit(ChildPid, kill)),
    down_reason(Pid, Reason),
    forall(retract(monitor(Other, Pid)),
           Other ! down(Pid, Reason)).

down_reason(Pid, Reason) :-
    retract(exit_reason(Pid, Reason)),
    !.
down_reason(Pid, Reason) :-
    thread_property(Pid, status(Reason)).

```

A thread implements an actor. The thread comes with its own message queue, which will serve as the actor's mailbox. The thread identifier works like a pid.

A number of thread-related predicates are called that finds the identity of the soon-to-become parent, creates a thread that, just before terminating, calls `down/3`, which takes care of what must be done in the last moment before the actor terminates – the termination of any children that it may have spawned during its life cycle (in case `link` is set to `true`), and the sending of a `down` message to the parent (if `monitor` is set to `true`).

The above implementation of `spawn/1-3` calls two predicates – `exit/2` and `!/2` – that must be implemented. In addition, `exit/1` must be implemented, and this can be done as follows:

```

:- dynamic exit_reason/2.

exit(Reason) :-
    self(Self),
    asserta(exit_reason(Self, Reason)),
    abort.

```

For the implementation of `exit/2`, ISO/IEC DTR 13211-5:2007 specifies a predicate `thread_signal/2` to make a thread execute some goal as an interrupt. Signaling may be used to cancel no-longer-needed threads. This means that `exit/2` may be implemented like so:

```

exit(Pid, Reason) :-
    catch(thread_signal(Pid, exit(Reason)),
          error(existence_error(_, _), _),
          true).

```

All of the above must be replaced - especially `down/2` must become `down/3`

Note that `thread_signal/2` throws an error if the thread ID in the first argument points to a thread that does not exist. Since `exit/2` must succeed also in this case, we have wrapped the call to `thread_signal/2` in a call to `catch/3`.

For the implementation of `!/2`, ISO/IEC DTR 13211-5:2007 offers a predicate `thread_send_message/2` which is somewhat similar to Erlang's send primitive. It allows any term to be sent to any thread. Just like in Erlang, the term is copied to the receiving process and variable bindings are thus lost. However, `thread_send_message/2` throws an error if the thread ID in the first argument points to a thread that does not exist. Again, since `!/2`, just like in Erlang, should succeed also in this case, we wrap the call in `catch/3` like so:

```
Pid ! Message :-
    send(Pid, Message).

send(Name, Message) :-
    registered(Name, Pid),
    !,
    send(Pid, Message).
send(Pid, Message) :-
    catch(thread_send_message(Pid, Message),
          error(existence_error(_,_), _),
          true).
```

In effect, this makes any attempt to send a message to a non-existing actor a no-op.

The implementation of the receive operation is somewhat more involved. Relying on `thread_get_message/3`, what might be regarded as a reference implementation of `receive/1-2` looks like this:

```
:- thread_local deferred/1.

receive(Clauses) :-
    receive(Clauses, []).

receive(Clauses, Options) :-
    thread_self(Mailbox),
    (   clause(deferred(Msg), true, Ref),
        select_body(Clauses, Msg, Body)
    ->  erase(Ref),
        call(Body)
    ;   receive(Mailbox, Clauses, Options)
    ).

receive(Mailbox, Clauses, Options) :-
    (   thread_get_message(Mailbox, Msg, Options)
    ->  (   select_body(Clauses, Msg, Body)
        ->  call(Body)
```

```

        ;   assertz(deferred(Msg)),
            receive(Mailbox, Clauses, Options)
        )
    ;   option(on_timeout(Body), Options, true),
        call(Body)
    ).

select_body(_M:{Clauses}, Message, Body) :-
    select_body_aux(Clauses, Message, Body).

select_body_aux((Clause ; Clauses), Message, Body) :-
    (   select_body_aux(Clause, Message, Body)
    ;   select_body_aux(Clauses, Message, Body)
    ).

select_body_aux((Head -> Body), Message, Body) :-
    (   subsumes_term(if(Pattern, Guard), Head)
    ->  if(Pattern, Guard) = Head,
        subsumes_term(Pattern, Message),
        Pattern = Message,
        catch(once(Guard), _, fail)
    ;   subsumes_term(Head, Message),
        Head = Message
    ).

```

C.1.5 Implementing output/1, input/2 and respond/2

The predicates `output/1-2`, `input/2-3` and `respond/2` are implemented on top of combinations of the `!/2` and `receive/2-3` primitives.

Here is the suggested implementation of `output/1-2`:

```

output(Term) :-
    output(Term, []).

output(Term, Options) :-
    self(Self),
    parent(Parent),
    option(target(Target), Options, Parent),
    Target ! output(Self, Term).

```

The implementation of `input/2-3` is slightly more complicated:

```

input(Prompt, Input) :-
    input(Prompt, Input, []).

```

```
input(Prompt, Input, Options) :-
    self(Self),
    parent(Parent),
    option(target(Target), Options, Parent),
    Target ! prompt(Self, Prompt),
    receive({
        '$input'(Target, Input) ->
            true
    }).
```

The predicate `respond/2` is used to respond to a prompt:

```
respond(Pid, Term) :-
    self(Self),
    Pid ! '$input'(Self, Term).
```

Since messages are wrapped in a binary term with the pid of the sender in its first argument, the target will always know if the message came from the right process.

C.1.6 What is missing from the sketches?

The predicates implemented so far are sufficient for running the majority of the example programs given in Chapter 2 and Chapter 3 of this book. Of course, this is just a start, and to be able to run *all* programs, and in particular the ones in Chapter 4, more is needed. Notably, the current implementation sketch does not support

- network-transparent concurrency and distribution,
- the implementation of an actors's private database, and
- security.

As for network transparency, the scenarios in Chapter 4 show in great detail how the stateful distribution layer might work. Recall that to spawn an actor on a remote node, the node option must be passed to `spawn/3` with a URI pointing to the node:

```
?- spawn(foo, Pid, [
    node('http://n7.org')
]).
Pid = 34925412@'http://n7.org'.
?-
```

Note that once this works for `spawn/3`, it will work for `toplevel_spawn/2` too.

Exiting *remote* processes must also be implemented so that it can be used in the following way:

```
?- exit(34925412@'http://n7.org', normal).
true.
?-
```

Our implementation of the send operator will only work for the simplest of cases of local messaging, but a complete implementation of an actor node must also allow sending to remote processes, like so:

```
?- 34925412@'http://n7.org' ! bar.
true.
?-
```

Once this works for `!/2`, it will also make `toplevel_call/2-3`, `toplevel_next/1-2` and related predicates work.

Note that the stateful distribution layer depends on WebSockets and that, as far as we know, at this point in time SWI-Prolog is the only Prolog system that offers a WebSocket library.

Source code injection such as in the following example must also be supported by an ACTOR node:

```
?- spawn(baz, Pid, [
    load_text('p(a). p(b).')
]).
Pid = 71123976@'http://n1.org'.
?-
```

Injected source code must end up in the spawned actor's private Prolog database and thus we need a viable approach to the implementation of this database and the isolation it requires. Isolation can be based on `thread_local/1` or the use of temporary modules. (Temporary modules are used by `library(pengines)`.)

If source code injection works for `spawn/3`, it will work for `toplevel_spawn/2` and `rpc/3` as well.

On the subject of security, a very important requirement relates to *sandboxing*. The approach taken by `library(sandbox)` in SWISH is not satisfactory.

Implementation on top of engines

Here is how the SWI-Prolog manual introduces engines, an idea that originated by Paul Tarau.

Engines are closely related to threads. An engine is a Prolog virtual machine that has its own stacks and (virtual) machine state. Unlike normal Prolog threads though, they are not associated with an operating system thread. Instead, you ask an engine for a next answer (`engine_next/2`). Asking an engine for the next answer attaches the engine to the calling operating system thread and cause it to run until the engine calls `engine_yield/1` or its associated goal completes with an answer, failure or an exception. After the engine yields or completes, it is detached from the operating system thread and the answer term is made available to the calling thread. Communicating with an engine is similar to communicating with a Prolog system though the terminal. In this sense engines are related to Pengines as provided by `library(pengines)`, but where Pengines aim primarily at accessing Prolog engines through the network, engines are in-process entities.

Some Prolog systems implement concurrency but do not use ISO/IEC DTR 13211–5:2007. SICStus Prolog and Ciao Prolog are cases in point, but there are others. Presumably, at least some of the Web Prolog profiles can still be implemented using alternative constructs provided by these dialects.⁹

C.2 Sandboxing, isolation, and capability control

A production-quality Prolog node requires more than correct semantics and acceptable performance. It must also enforce a security boundary strong enough to allow the execution of untrusted client code. From an implementation point of view, this raises a practical question: where should that boundary be drawn, and by what mechanisms should it be enforced?

At least four concerns must be kept apart. First, there is *expressibility*: which language constructs and built-ins are available to client code. Second, there is *authority*: which actors, services, and external resources a computation may affect. Third, there is *isolation*: whether the runtime itself can escape into the host operating system or interfere with other computations. Fourth, there is *resource consumption*: how much CPU time, memory, process space, mailbox space, or network activity a session may consume. These concerns are related, but they are not identical, and no single mechanism addresses them all.

C.2.1 Whitelist and blacklist strategies

A first implementation choice concerns the form of the language-level sandbox. One strategy is *whitelisting*: client code is allowed to call only predicates that have been explicitly declared safe. Another is *blacklisting*: client code is allowed to use the exposed language fragment except for specifically forbidden predicates or capabilities.

For a reflective Prolog system, whitelisting is the safer default. The reason is not merely that some predicates are dangerous, but that authority may often be reconstructed indirectly through meta-calling, predicate inspection, dynamic database operations, foreign predicates, or other reflective mechanisms. In such an environment, whitelisting has a clear advantage: newly added predicates are denied by default until they have been reviewed.

Blacklisting becomes more plausible only when the language fragment exposed to clients is already narrow and capability-oriented. This is one reason why Web Prolog is a better candidate for blacklisting than a full Prolog system. If client code is already denied direct access to file I/O, operating-system interaction, unrestricted reflection, foreign predicates, and persistent code modification, then the remaining

⁹ (Clark et al., 2001), section 7 compare communication in Qu-Prolog with that of SICStus-MT, BinProlog, CIAO, Erlang, Mozart-Oz and April.

surface is small enough that one may hope to enumerate the forbidden capabilities rather than all the permitted predicates. Even then, however, blacklisting should be treated as an engineering convenience rather than as a foundation for trust.

C.2.2 Language sandboxing versus host isolation

Language-level sandboxing is only one layer of defence. Even a carefully restricted Prolog fragment still runs inside some host environment, and that host may itself provide dangerous authority. For this reason, a node implementation should distinguish between *language sandboxing* and *host isolation*.

Language sandboxing decides what client code can express in Web Prolog. Host isolation decides what the runtime process itself is able to do. A system that relies only on sandboxed predicates, but runs with unrestricted filesystem and network access in the host process, still places great trust in the correctness of the sandbox implementation. Conversely, a strongly isolated host environment can remove whole classes of risk even if the Prolog-level sandbox is imperfect.

This suggests a layered implementation strategy. The inner layer is the Web Prolog language fragment exposed to clients. Around that sits a capability policy governing which exported services, actors, and owner-defined predicates may be reached. Outside that sits the deployment environment, which should itself restrict what the runtime may access.

C.2.3 WASM and WASI

A particularly interesting host environment for Web Prolog is WebAssembly (WASM). A WASM runtime starts with substantially less ambient authority than a native process. In the browser it inherits the browser's security model; in a standalone embedding it can be run inside a tightly controlled execution environment.

The case becomes even stronger with WASI. In a WASI-style setting, access to external resources is explicitly granted rather than silently inherited. For a Web Prolog implementation, this means that many operations that would otherwise need to be denied at the Prolog level may simply be absent from the host environment. If the runtime is not given filesystem handles, unrestricted sockets, or privileged host callbacks, then no client program can reach them, regardless of whether the language-level sandbox is phrased as a whitelist or a blacklist.

This does not solve everything. WASM and WASI do not by themselves control Prolog-level reflection, and they do not remove the need for quotas and timeouts. But they can substantially reduce the amount of trust that must be placed in the Prolog sandbox alone.

C.2.4 Capability control inside the node

Even inside a host-isolated runtime, a Prolog node still needs a policy for authority within the Prolog Web. An implementation must decide which operations are available to ordinary clients and which are reserved for node owners. This includes, at minimum, the distinction between session-local computation and durable node-level authority.

Examples of owner-only capabilities include publishing services, registering public names, installing persistent code, modifying shared node state, or detaching computations so that they survive the session that created them. Client capabilities, by contrast, should normally be limited to session-scoped actor creation, messaging, remote procedure calls to exported services, and other operations whose effects are contained by the lifetime discipline described earlier in the book.

Implementation-level security is therefore not merely a matter of preventing operating-system escape. It is equally a matter of ensuring that the node's own public surface is small, explicit, and role-sensitive.

C.2.5 Resource limits are not sandboxing

It is important to distinguish *sandboxing* from *resource governance*. Timeouts, actor limits, mailbox bounds, RPC limits, and cache bounds are essential to a production deployment, but they serve a different purpose. A sandbox answers the question *what may this program do?* Resource governance answers the question *how much may it do?*

This distinction matters in implementation work. A program may remain entirely within the sandbox and still cause denial-of-service by spawning large numbers of actors, generating very large terms, forcing unbounded result streams, or repeatedly issuing remote calls. Conversely, a program may use very few resources and yet still violate the security boundary if it reaches an authority it should not possess. A complete node therefore requires both capability control and operational limits.

C.2.6 A realistic implementation strategy

For near-term implementations, the most realistic strategy is therefore a layered one. The exposed Web Prolog fragment should be restricted by a language-level sandbox. The node should enforce a capability policy distinguishing client authority from owner authority. Sessions should be subject to operational limits. And, where possible, the runtime should be deployed inside a host environment that reduces ambient authority, such as a browser sandbox, a Wasm runtime, a WASI-based embedding, or an operating-system-level isolation boundary.

Under such a strategy, whitelisting and blacklisting become local design choices inside a broader architecture of defence. For implementations built directly on top of existing Prolog systems, whitelisting is likely to remain the safer option. For restricted Web Prolog runtimes deployed inside capability-oriented hosts such as Wasm or WASI, blacklisting may become more plausible because so much dangerous authority has already been removed at the outer layers.

C.3 Alternative implementation approaches

Some (most?) Prolog systems could probably be deployed on the Web using OS based sandboxing where every client talks to a private copy. There are several language neutral environments that would let us do this. At least some profiles of Web Prolog could probably be implemented using this approach.

C.3.1 Implementation for browsers

There seems to be at least three approaches one might take in order to implement a Web Prolog system (minus features that makes it into a node) that runs in a browser:¹⁰

1. **JavaScript implementation:** Prolog can be interpreted or compiled into JavaScript, allowing it to run directly in a web browser. JavaScript engines are highly optimized and available in all modern browsers, providing a convenient platform. However, the performance and capabilities might be limited compared to native Prolog environments. This approach is taken by Tau Prolog.¹¹
2. **WebAssembly (WASM):** WebAssembly offers a way to run code written in multiple languages at near-native speed in web browsers. Some Prolog system can be compiled to WebAssembly, enabling it to run efficiently in the browser. This approach could potentially offer a better performance than a pure JavaScript implementation. This approach is taken by SWI-Prolog, Ciao Prolog, Trealla Prolog and Scryer Prolog.
3. **Browser extensions or plug-ins:** Developing a browser extension or plug-in could be another way to integrate Prolog into a web environment. This would

¹⁰ There is actually a fourth approach: to try to convince browser providers (e.g. Mozilla) that they should support Prolog. Interestingly, already back in 1999, Mozilla ran an umbrella project for experimental work investigating the integration of logic and inference capabilities into the RDF based information management system in the Mozilla application environment. Based on SWI-Prolog, Geoff Chappel implemented an early prototype Prolog plugin for the Mozilla/Netscape browser. (See <https://www-archive.mozilla.org/rdf/doc/inference.html>) Presumably, the same approach can be used to support Prolog in Firefox. The problem with this approach is that it would likely be very hard to convince other browser vendors to follow suit.

¹¹ <http://tau-prolog.org/>

allow more direct interaction with the browser's capabilities but might limit the audience due to the need for installing the extension.

—
As suggested in ??, the most promising approach to enable the running of Web Prolog in browsers seems to be using WebAssembly.

—
Flach and Wielemaker: "There are two ways to achieve that. One is to run Prolog in a WebWorker (roughly, a browser task without a window) and make it communicate with the browser task associated to a normal browser window. The other is to yield from the Prolog virtual machine and resume Prolog when the data become available. This leads to at least two interface designs: – with a WebWorker we get interaction with a Prolog process that is not very different from Prolog running on a remote server."

—
Problems: No concurrency.

—
Trealla Prolog:

<https://news.ycombinator.com/item?id=35624654>

<https://github.com/trealla-prolog/trealla>

C.3.2 Implementing a browser node

The remark above – that running Prolog in a Web Worker yields “interaction with a Prolog process that is not very different from Prolog running on a remote server” – is more prescient than it may first appear. It describes, almost exactly, the architecture we arrived at when building a browser-resident node on top of SWI-Prolog’s WebAssembly port. The terse worry that follows it in our notes, *no concurrency*, turns out to be the very constraint that forces the design. This section explains why, and sketches the architecture that results.¹²

The substrate: a single-threaded VM with cooperative engines

The SWI-Prolog WASM port exposes a JavaScript call-in interface rather than a message-passing one. A host page holds a Prolog object and calls *into* it: calling `Prolog.query/1-3` returns a JavaScript iterator over solutions, calling `Prolog.forEach/2-4` runs a goal asynchronously so that the browser remains responsive, and the WASM build supplies *engines* – additional virtual machine contexts, each with its own stacks, that can enumerate the solutions of several open

¹² The implementation details below were inspected against SWI-Prolog 10.1.8, the version deployed in our demonstrator. The WASM port is still evolving, and some of these details are version-dependent.

queries and so act as coroutines.¹³ SWI's framing is thus: *here is an engine; call into it*. The framing we want for the Prolog Web is the opposite: *here is a node; populate it with actors*, addressed uniformly by `spawn/1-3`, `!/2` and `receive/1-2`, with the same message vocabulary whether an actor is local or on a remote node. The engine is the mechanism; the actor is the abstraction we layer on top of it.

The decisive question is therefore what kind of concurrency the substrate actually provides, because that determines whether the actor abstraction can be honoured. The answer is unambiguous, and it is the reason for everything that follows. The distributed WASM build is compiled single-threaded: it does not use Emscripten's POSIX threads, shared linear memory, or atomics.¹⁴ Engines, consequently, provide *cooperative* concurrency only. They share one VM and one global Prolog state, and exactly one engine executes WASM instructions at any instant; switching engines creates neither an operating-system thread nor a Web Worker. Engines give coroutines and interleaving, never parallelism and never process-level isolation.

Pre-emption, such as it is within a single VM, is delivered by an explicit *heartbeat* mechanism rather than by any operating-system scheduler. When a goal is run under `forEach/2-4` with a heartbeat budget, the VM's inference counter periodically forces the engine to yield: the C runtime uses `PL_yield_address()` so that the solution loop returns control to JavaScript, which later resumes the query.¹⁵ This is a genuinely useful property, and it qualifies the bare worry in our notes: a *pure-Prolog* actor run under `forEach` is interrupted automatically, even if its own code never reaches a yield point, so it does not silently monopolise the VM. The qualification matters because it narrows the danger. The heartbeat, together with the related yield-and-resume mechanism the engine uses to wait for asynchronous input, handles all *cooperative* suspension within a single VM. What it cannot do is *involuntarily* wrest control from a computation that never yields: a synchronous `query` or `call`, a long-running foreign predicate, or synchronous JavaScript can still seize the VM and refuse to release it, and for these, within a single VM, there is no remedy at all. We return to this distinction below, since it is precisely what determines where a Worker boundary becomes necessary.

Why the node becomes worker-per-actor

This is the crux. The ACTOR profile requires two runtime properties that the abstract actor model leaves to the implementation, and that systems such as the BEAM supply: that one misbehaving actor cannot starve its peers, and that a runaway actor can be terminated. Cooperative engines in a single VM can supply neither in general. They cannot pre-empt a non-yielding foreign call; and although an engine can be *closed*

¹³ <https://www.swi-prolog.org/pldoc/man?section=wasm-calling>

¹⁴ The Emscripten port configuration sets multi-threading off; the build relies on no `SharedArrayBuffer` and no shared WASM memory.

¹⁵ The heartbeat flag sets an inference budget – by default of the order of ten thousand inferences, checked in small groups – after which the VM yields to the browser event loop. It is an inference budget, not a wall-clock guarantee.

in the ordinary course of events, it cannot be forcibly interrupted while it is blocking the Worker, because to close it JavaScript must first regain control, which a wedged engine never returns. Nor do engines provide process-level isolation: while each has its own execution stacks, they share the VM-global structures – predicates, modules, the atom table, the dynamic database – of the one VM that hosts them. To obtain the two properties that make an actor an actor – independent progress and hard termination – one must step outside the VM. In the browser, the only available unit of genuine, browser-scheduled, separately-terminable execution is the Web Worker.

The node we built therefore runs *every local Prolog actor in its own Web Worker*, each a complete and independent SWI-Prolog WASM instance, and keeps a JavaScript *node controller* on the main thread. The controller owns no Prolog state of its own, but it is far from idle: it owns pid allocation, the actor and name registries, monitors, delayed sends, the lifecycle of the Workers, the WebSocket connections to remote nodes and the HTTP RPC to stateless ones, and the terminal and the page – and thus the window and document that only the main thread can reach. It is, in short, where everything that is not Prolog inference lives. What runs inside a Worker depends on the kind of actor: an ordinary actor created with `spawn/1-3` simply executes the goal it was given, which may implement any protocol at all, whereas a *toplevel* actor runs the mailbox-driven `ptcp/3` loop that interprets the `toplevel_call/toplevel_next` control messages. The *toplevel* that answers queries typed at the terminal is one of the latter: semantically it is a `ptcp/3` *toplevel* actor like any other, occupying its own Worker and addressed by a pid.

The shell's *toplevel* is not, however, created or governed quite like the rest, and it is worth being exact about how it differs. It is special only *operationally*, not at the message level. The controller does not spawn it through `toplevel_spawn/1-2`; it allocates the pid itself and starts `ptcp/3` directly in a distinguished *shell* role, so that it can attach the terminal to it and manage its lifetime by hand. Its Abort button is a genuine hard reset: the controller terminates the shell's Worker and recreates it under the same pid, and – by design, so that the terminal's session survives an abort – the shell emits no `down/3` and the actors it had spawned are left running. An ordinary *toplevel*, by contrast, is spawned through the normal path and follows the normal lifecycle. So the shell's *toplevel* and a count server speak the same messages and run in the same kind of Worker; what sets the shell apart is entirely the controller's special handling of its creation, its terminal attachment and its reset, not anything visible in the actor protocol itself.

Each Worker carries a complete and private VM: its own linear memory, stacks, dynamic database, atoms, modules and virtual filesystem. Nothing of the Prolog state is shared between actors. (The browser may cache the fetched and compiled WASM artifacts and share *those* across Workers; what is private per actor is the instantiated VM – its linear memory, Prolog state and virtual filesystem – not necessarily the compiled module bytes.) A tight, non-yielding goal in one actor blocks only its own Worker, and the controller can stop a wedged actor by the blunt but reliable expedient of terminating its Worker – as the shell's Abort button does, and as `exit/2` does when directed at another actor. This hard termination should be distinguished from `toplevel_abort/1`, which merely posts an abort *message* to the target and

therefore cannot dislodge an actor that has stopped returning to its event loop; for that, terminating the Worker is the only remedy. This is the local counterpart of ending a remote session by closing its WebSocket – though the analogy is not an identity: closing a connection releases the local session, but whether the remote actor itself is terminated depends on the lifecycle policy of the node that owns it, a matter of the containment discipline discussed earlier in this book rather than an automatic consequence of the close.

It is worth pausing on what this costs and what it buys, because the trade is a clean one. Because actors inhabit different VMs with different heaps, they cannot share a blackboard or a dynamic predicate; everything they share, they share by sending. The platform does not, strictly, *forbid* all sharing – Workers can in principle communicate through a `SharedArrayBuffer`, through browser storage, or via a server – but it shares no *Prolog VM state* between them, and our implementation deliberately declines the out-of-band channels that would reintroduce it. The actor model’s defining discipline – no shared mutable state, communication by message only – thus ceases to be a convention we must police and becomes, for Prolog state, a property of the Worker boundary. Where the BEAM *encourages* share-nothing, the browser substrate makes it the path of least resistance; the same discipline the rest of this book argues for on semantic grounds is, here, simply the one the substrate hands us.

The corresponding cost is borne at actor creation. Each Worker must instantiate a fresh VM and grow its own memory, and with a single-file WASM bundle every Worker re-runs the full bootstrap and re-mounts the embedded data. Worker start-up latency and per-Worker memory are therefore the first limits on how many actors a tab can sustain. In our experiments a browser node hosts of the order of tens of actors comfortably; it will plainly not host the millions of lightweight processes a BEAM node can, and we should not pretend otherwise. Density is not the only limit, however: for workloads that are chatty or that move large payloads, message throughput can dominate well before memory does. Where actor density does become the binding constraint, the mitigations are familiar but each carries a cost of its own. A pool of pre-started Workers onto which actors are multiplexed reduces start-up latency, but it reintroduces sharing between actors and so gives up precisely the one-VM-per-actor isolation this design exists to defend. Splitting the single-file bundle can improve resource caching across Workers, but it does not by itself cause Workers to share compiled modules or linear memory; VM instantiation and linear memory remain irreducibly per-Worker.

Messages are terms on the wire

Because actors do not share a VM, messages between them cross a boundary – the JavaScript-to-Worker boundary within the tab, or a WebSocket to another node – and the transport we use is deliberately the simplest one that is faithful to the semantics. An outgoing message is rendered to text with `term_string/2`, posted across the boundary as a string, and parsed back into a term with `term_string/2`

on arrival. The cost of a message is thus linear in the size of the *printed* term – dominated by writing, reading and allocation – and there is no cheaper local path: the current bridge uses the same textual representation even for a self-send, which crosses no Worker boundary but is serialised and re-parsed all the same. The transport is faithful, but only for the subset of terms that survive a round trip through their textual form: ordinary compound terms, lists and atoms travel transparently, whereas attributed variables, blobs, streams, and terms whose reading depends on a differing operator environment do not, and must be excluded or specially encoded. Within that subset the scheme has a compensating virtue that bears directly on the argument of this book. The local message and the remote message now have essentially the same shape on the wire: a printed Prolog term. The isomorphism between talking to a local Worker actor and talking to one across a WebSocket is not merely interactional but, very nearly, representational – modulo the distribution envelope that the remote case adds for framing and routing. The placement of an actor – this Worker, another Worker, or another machine – is invisible above the `!/2` and `receive/1-2` line, which is precisely the location transparency the ACTOR profile demands. The transparency is in the *addressing*, not in the form of the address: a local pid is displayed as a plain integer, whereas a distributed pid carries its node, as in `1234567890@'https://n4.example.org'` – but once an actor holds a pid, sending to it is the same operation whichever shape it has.

The practical rule that follows is the one the actor model has always recommended, here given a concrete cost basis: *exchange results, not steps*. An actor that sends a single message carrying a computed answer pays the serialisation cost once; an actor that chatters its way to that answer through a thousand small messages pays it a thousand times. Fine-grained message protocols are the anti-pattern; a coarse-grained one is not free, but it amortises the per-message overhead over the computation it summarises. We pass no transferable buffers and use no shared memory; importing such a buffer back into an ordinary Prolog term would still require copying, so the simplicity of the textual transport is, for now, the right default. One operational corollary should be recorded: because an incoming message is parsed before it can be inspected, a node that accepts terms from untrusted remote peers must bound message size and guard against parsing-time resource exhaustion, lest a single large or pathological term become a denial-of-service vector. Message-size limits and parse-time quotas therefore belong with the other operational limits discussed in the security section.

The DOM stays on the main thread

One consequence of placing every actor in a Worker deserves explicit mention, because it overturns an apparent advantage of the call-in approach. SWI's `library(dom)` and the Tau-compatible `library(dialect/tau/dom)` allow Prolog to manipulate the page directly, binding events and constructing DOM nodes

from Prolog terms.¹⁶ These libraries reach `window` and `document` through the local JavaScript realm – and a Web Worker has neither. A Worker-resident actor therefore cannot drive the DOM directly, whatever interface style wraps it. The constraint is not a property of the actor abstraction but of the Worker placement: it is main-thread placement that DOM access requires, and the isolation and forcible termination of actors that put them in Workers, and a single SWI-WASM VM cannot be in both places at once. In our design no Prolog actor is on the main thread at all; the only main-thread component is the JavaScript controller, and so direct DOM access from Prolog is, by construction, unavailable to every actor alike.

The actor model supplies its own answer to this, and a characteristic one. The DOM is a stateful, single-threaded, location-bound resource – exactly the shape of thing the model says one should wrap in a process and address by message. Since the only thing on the main thread is the JavaScript controller, that controller is where DOM access lives: an actor that wishes to affect the page sends a display message to the controller, and the controller, running in the realm that owns `window` and `document`, performs the manipulation and posts back any browser events as ordinary messages. The natural way to present this to actor code is the indirection the model would prescribe in any case: a distinguished main-thread *view actor* – the controller wearing an actor’s face – to which Worker-resident actors send display commands and from which they receive events. In the current demonstrator the controller already plays a limited version of this role, rendering the shell’s terminal output and delivering typed input; generalising it to an arbitrary view actor is a matter of giving it a continuously running consumer for display messages from any application interface, which the present design leaves as future work.

The general case: a user interface talking to an actor

It helps, at this point, to widen the view beyond the shell. The terminal is a special case of a more general and more common situation: a JavaScript user interface in conversation with an actor. In the remote scenarios of Chapter 4 a browser talks over a WebSocket to a toplevel actor on a node, but it might equally talk to an application actor of another kind – a social-robot dialogue actor, a game-state actor, a chat session – running its own protocol rather than a toplevel’s. The toplevel shell is merely the instance of this pattern in which the actor happens to be a Prolog read–eval–print loop. Seen this way, the question is not “where does the shell live?” but “where does the actor that a local interface talks to live?”, and the answer the actor model wants is clear: in its own Worker, isolated, mailbox-driven, addressed by a pid – exactly like the count server, and exactly like a remote actor.

This is why the design carries no special main-thread *engine* for the shell. The shell’s toplevel is a Worker-resident actor like the rest; what the main thread holds is the controller, which is presentation and plumbing, not logic. The general application has the same shape: a presentation layer on the main thread, owning the DOM, and

¹⁶ <https://www.swi-prolog.org/pldoc/man?section=dom>

an application actor in a Worker, owning the logic, the two conversing by message. Drawn this way, a browser node is as clean as a remote one: the user interface stands where the robot's face stands in the remote figure, and the application actor stands where the robot's `social_robot` actor stands on the node – only now the node is the tab. The local application actor and a remote one are interchangeable in use, addressed through the same `ptcp/3` discipline and the same message vocabulary; only the form of the `pid` differs, a bare integer locally and a node-qualified term remotely.

We should be candid that our current demonstrator does not yet support the general case in full: the shell is at present the only local interface wired up to a local actor, and the view actor described above remains to be built. But the architecture extends to it naturally, precisely because the shell was never a special case to be generalised away from; it is already an instance of the general pattern, and the discussion of frameworks that follows shows why that pattern is the one worth building toward.

Why frameworks make the Worker/message model the default

The separation just described is not merely available; for a large class of applications it is strongly favoured, because the dominant front-end frameworks are built around it. A developer reaching for React or Vue works under a clear principle: the framework owns the part of the DOM it renders, and application logic should not mutate that part behind its back. One changes *state*, and the framework reconciles the view; reaching in to mutate framework-owned nodes from outside the component tree – which is what `library(dom)` would do – fights the reconciler and is the standard anti-pattern. (Frameworks do leave deliberate escape hatches – refs, and imperative third-party widgets mounted into a node the framework is told to leave alone – so the prohibition is on mutating *reconciled* DOM, not on imperative DOM as such.) For the main body of such an application, then, direct DOM access from Prolog is not an asset but a hazard: a React developer will not drive the reconciled view by calling `append_html/2` from Prolog, because that is to fight React.

This makes the Worker/message model not merely natural but the obvious fit. Prolog is then free to be what the actor model wants everywhere: pure application logic, Worker-resident, isolated, addressed by messages, holding state and rules and dialogue, never touching the DOM. It feeds the framework by message and receives user-interface events by message. And the integration shape this requires is one the frameworks already provide for any asynchronous source – a WebSocket, a store, a subscription. Subscribing a component to a local Worker actor is the same operation as subscribing it to a remote one over a socket; a hook such as

```
function useActor(pid) {
  const [state, setState] = useState(null);
  useEffect(() => subscribe(pid, setState), [pid]);
  return state;
}
```

cannot tell, and need not tell, whether `pid` names an actor in a Worker in the same tab or one on a node across the network. The location transparency the ACTOR profile provides thus reaches all the way into the component tree: the framework consumes a local actor and a remote actor by identical means.

This is why the Worker/message model is the one the rest of this book assumes, and the one we treat as the default for a browser node. It is not, however, the only mode our demonstrator offers. A Prolog engine can instead be run *on the main thread*, selectable as a setting, where it has direct access to `window` and `document` and can drive the page through `library(dom)` without the controller's mediation. This main-thread mode coexists with the Worker/message model rather than replacing it: it is intended for the one case in which a piece of Prolog code is also its own renderer – a self-contained interactive page that is simultaneously its own user interface. What it gives up is local, not global. The main-thread engine itself forfeits Worker isolation and hard termination, and the DOM code it runs is inherently bound to that one main-thread realm; but actors it spawns still run in their own Workers, and messaging to local and remote actors remains exactly as it is in the default model. So the trade is confined to the self-rendering engine: a single node may run such an engine for its presentation surface while its application logic runs, as always, in isolated Worker actors, fully location-transparent.

This choice has an advocate worth naming. The author of the SWI-WASM port favours keeping Prolog on the main thread and exploiting the DOM and HTML5 directly, with yielding from the virtual machine to handle asynchrony; he identifies the two options exactly – a Worker addressed by messages, as Ciao does, or yielding from the VM – and prefers the latter for the richness of interaction it allows.¹⁷ For a self-rendering page this is an excellent choice, and the main-thread mode is its natural realisation. But it is a specialisation, not a rival architecture. Everywhere else, which is to say for application logic, the direct-DOM capability the yield route is built around is precisely what a framework forbids, while the message-passing actor it sets aside is precisely what a framework is built to consume. The default is therefore the Worker/message model, and the main-thread mode is reserved for the narrow case that genuinely needs it.

Summary of the architecture

The architecture follows, then, from a single principle. Every actor – application logic and the shell's toplevel alike – lives in its own Worker, isolated and addressable by message; the main thread holds no Prolog at all, only the JavaScript controller that owns the terminal and the page and routes messages. (The lone exception, the

¹⁷ See the SWI-Prolog Discourse thread *WASM wait for user input*, <https://swi-prolog.discourse.group/t/wasm-wait-for-user-input/8787>. The two-option framing, and the observation that the worker route makes direct DOM access harder, are Wielemaker's. The same yield machinery underlies both the heartbeat and a recent facility, depending on the `engine` option to `forEach` in releases from 9.3.20, by which a main-thread goal can create a Promise, `await/2` its resolution, and resume on user input.

selectable main-thread mode for a self-rendering page, was described above and does not alter this picture.) The browser node is thus best described in a single sentence: *a JavaScript node controller on the main thread, owning pid allocation, registries, lifecycle, networking, terminal and page; one independent Worker-resident SWI-Prolog WASM instance per local actor – ordinary actors running the goal passed to spawn/1–3, toplevel actors running the ptcp/3 protocol, and the shell a ptcp/3 toplevel whose creation, terminal attachment and hard reset the controller manages specially; textual term_string/2 message transport between actors; and WebSocket connections to reach actors on remote nodes.* The call-in interface that SWI exposes is not discarded – it remains the mechanism by which each Worker’s VM is driven from the JavaScript inside that Worker – but it is wrapped, actor by actor, in the mailbox-and-message envelope that the ACTOR profile requires. The result honours the abstraction the rest of this book has assumed, at a cost in actor density that the browser, rather than our design, imposes. It is, in the end, the architecture Flach and Wielemaker’s first option foresaw: a Prolog process in a Worker, not very different from Prolog running on a remote server – and that resemblance, far from being a limitation, is the property we were seeking all along.

We should be candid, finally, about what this design is and is not. It is, we believe, the best fit we found for the current substrate: it extracts independent browser scheduling of non-cooperative computation, fault isolation and forcible termination from a SWI-Prolog WASM port that is compiled single-threaded and offers none of these within a single VM. It is also, we would argue, the implementation best *aligned* with the thesis of this book, irrespective of substrate, because the very mechanism that makes it work – one VM per actor, messages as printed terms – is what renders a local Worker actor and a remote one nearly indistinguishable, and so delivers the location transparency the ACTOR profile demands almost as a by-product. What it is not is the implementation we expect to be *optimal* in the longer run. Two futures would supersede it. The first is a multi-threaded SWI-Prolog WASM build: single-threading is not a limitation of WebAssembly in principle, although the current SWI-WASM port does not provide a supported threaded build, and a port using Emscripten’s threads and shared memory could make it possible to map actors onto true SWI threads within one VM – though only the raw threading would come for free; scheduling, mailbox isolation, failure containment and termination semantics would still have to be implemented on top, much as they are here. The second is the more distant prospect discussed in Section C.3.4 – a dedicated, BEAM-like virtual machine for Web Prolog, which would furnish lightweight, pre-emptible, isolated processes natively rather than reconstructing them from Web Workers. Both of those could substantially reduce the per-actor start-up and memory costs that bound actor density here; but both would also have to work harder to preserve the local/remote isomorphism that our design obtains by deliberately imposing a wire representation on local messages, because in each the cheap in-VM path and the costly network path diverge. The architecture presented here is thus best understood as a faithful realisation of the actor model on the browser substrate available at the time of writing, and as a demonstration that the model is implementable in the browser at all – not as the last word on how it should be done.

C.3.3 Implementations on top of other programming systems

Dedicated Prolog systems such as SWI-Prolog or SICStus Prolog are usually written in fairly low-level languages such as C, C++ or (more recently) Rust, but one surprisingly often also comes across Prolog implementations in more high-level languages such as Scala, Haskell, Erlang, Lisp or Scheme.

Such implementations may be nothing more than the result of programming exercises – perhaps by a student or a hobbyist wishing to implement a couple of somewhat challenging algorithms such as unification and backtracking. But they may also be the results of a real need felt by a programmer confronted with a problem – the need for a “rule engine” or “logic engine.” Then, rather than move an application to Prolog, the programmer decides to implement Prolog in the language of his application. Such implementations are often slow and incomplete, but there are exceptions. Indeed, it can be very useful to embed Prolog-style unification and backtracking in a more traditional imperative or functional programming language. Some people even do it for fun.

Interestingly, there is already Erlog¹⁸ – an unusually complete Prolog implementation in Erlang written by Robert Virding (one of the people behind Erlang, who seems to belong to the do-i-for-fun category of programmers).

An implementation of a Web Prolog node in Erlang would be interesting as it would most probably have a performance profile very different from the implementation in SWI-Prolog. Prolog is likely to be a lot slower in Erlang than in the implementation in SWI-Prolog, whereas the super-fast lightweight processes in Erlang have other advantages, making it scale better to very many simultaneous users on a network. Besides, Erlang is particularly famous for extremely efficient implementations of web-related technologies such as web servers (e.g. Yaws and Cowboy) and this could also be a distinctive advantage of an Erlang implementation.

Then there is Akka + (say) tuProlog.

C.3.4 Prospects for a dedicated Web Prolog VM

The Erlang virtual machine is known as the BEAM.¹⁹ The BEAM VM is the virtual machine at the core of the Erlang Open Telecom Platform (OTP), designed to efficiently execute Erlang code. The BEAM VM is renowned for its support of highly concurrent, distributed, and fault-tolerant applications, qualities that stem directly from the design principles of the Erlang language itself. One of the defining characteristics of the BEAM VM is its lightweight process model. Unlike operating system processes or threads, BEAM processes are managed within the VM, allowing for the creation of millions of concurrent processes. These processes are isolated, com-

¹⁸ <https://github.com/rvirding/erlog>

¹⁹ <http://blog.erlang.org/beam-compiler-history>

Compare performance of Erlang vs Web Prolog on the process ring.

municate via message passing, and do not share memory, which mitigates common concurrency issues such as race conditions and deadlocks.

The BEAM VM is not limited to running Erlang code. It also supports other languages that compile to its bytecode, such as Elixir, a modern, functional programming language that is fully interoperable with Erlang, leveraging the same VM for concurrent, distributed, and fault-tolerant applications.

In a paper presented at an Erlang conference (Lager, 2019), the following statement was made:

The holy grail for a Web Prolog runtime system is a compiler targeting a virtual machine with BEAM-like properties, capable of producing code which when run will create processes as small and efficient as Erlang processes, yet with the useful capabilities that Prolog offers. *We do not dare to guess whether building such a virtual machine is feasible.*

When Richard O’Keefe, with a lot of experience with both Prolog and Erlang, kindly agreed to read the paper, he made the following comment on the passage:

I do! It is! Let’s face it, processes in Logix were *tiny* compared with Erlang ones. And the BEAM was a reaction to JAM, which was inspired by the WAM, and Aquarius Prolog used the BAM which had some similarities to BEAM.

Designing and implementing a BEAM-like VM for Web Prolog is likely to be really hard, but with time it might prove to be the optimal way forward. One must not forget, however, that a node built on top of a current mature Prolog system might be able to offer a client a lot more than an efficient but bare-bones implementation which compiles source code into a BEAM-like VM.

C.3.5 Less capable profiles need less work

Unfortunately, implementing a Prolog Web node capable of running the full set of example programs given in the previous chapters is not exactly an easy task, and this is what is required of an ACTOR node. Such difficulties have motivated us to define three other profiles which are less powerful and easier to implement than the ACTOR profile. Hopefully, this may encourage developers of Prolog systems to implement nodes capable of contributing to the Prolog Web.

An ISOBASE node can, since it does not have to load data or programs dynamically, pull a few tricks in order to optimise query processing, tricks that may be slow to play, but which can be performed offline if dynamic loading is not permitted. (Compiling Web Prolog code into C, for example.) In contrast, a node that implements the ACTOR or ISOTOPE profile, and thus offers code injection capabilities, must do the updating quickly and must probably avoid some of the slower ways of optimizing code.

Without doubt, a node that conforms to the RELATION profile would be the easiest to implement, especially since it does not have to implement any ISO Prolog built-in predicates or control structures. One can imagine writing a RELATION node in (say) Python which offers developers just a single owner-defined predicate. If it

supports querying over the stateless HTTP API in the manner described elsewhere in this book, and is able to return answers in the form of either Prolog or JSON, then it it does conform to the **RELATION** profile of Web Prolog, yet would be easy to implement. Note that by our own definition, since it is a process on the Prolog Web that talks Prolog, such a node must also be categorized as a Prolog agent, despite not being written in Prolog.

References

- Andersen C, Swift T (2023) The janus system: A bridge to new prolog applications. In: Prolog: The Next 50 Years, Springer, Lecture Notes in Computer Science, vol 13900, pp 93–104
- Arias J, Carro M, Salazar E, Marple K, Gupta G (2018) Constraint answer set programming without grounding. *Theory and Practice of Logic Programming* 18(3–4):337–354, DOI 10.1017/S1471068418000285
- Armstrong J (2003a) Concurrency oriented programming in erlang. Invited talk, FFG
- Armstrong J (2003b) Making reliable distributed systems in the presence of software errors. PhD thesis, Royal Institute of Technology, Stockholm
- Barnett J, Akolkar R, Auburn R, Bodell M, Burnett D, Carter J, McGlashan S, Lager T, Helbing M, Hosn R, Reifenrath K, Rosenthal N, Roxendal J (2015) State chart xml (scxml) state machine notation for control abstraction. w3c recommendation
- Berners-Lee T, Hendler J, Lassila O (2001) The semantic web. *Scientific American* 284(5):34–43, URL <http://www.sciam.com/article.cfm?articleID=00048144-10D2-1C70-84A9809EC588EF21>
- Brusk J, Lager T (2008) Developing natural language enabled games in SCXML. *Journal of Advanced Computational Intelligence and Intelligent Informatics (JACIII)* 12(2):156–163, DOI 10.20965/jaciii.2008.p0156, URL <https://doi.org/10.20965/jaciii.2008.p0156>
- Ceri S, Gottlob G, Tanca L (1989) What you always wanted to know about Datalog (and never dared to ask). *IEEE Transactions on Knowledge and Data Engineering* 1(1):146–166, DOI 10.1109/69.43410
- Cicirelli F, Furfaro A, Giordano A, Nigro L (2009) Statechart-based actors for modelling and distributed simulation of complex multi-agent systems. In: European Conference on Modelling and Simulation, ECMS 2009, Madrid, Spain, June 9–12, 2009, pp 233–239, DOI 10.7148/2009-0233-0239, URL <https://doi.org/10.7148/2009-0233-0239>
- Clark K, Robinson PJ, Hagen R (2001) Multi-threading and message communication in qu-Prolog. *Theory and Practice of Logic Programming* 1:283–301

- Clark KL (1978) Negation as failure. In: Gallaire H, Minker J (eds) *Logic and Data Bases*, Springer, pp 293–322
- Clark KL, Gregory S (1986) *PARLOG: Parallel Programming in Logic*. ACM Press
- Costantini S, Tocchio A (2004) The DALI logic programming agent-oriented language. In: Lifschitz V, Niemelä I (eds) *Logics in Artificial Intelligence, Lecture Notes in Artificial Intelligence*, vol 3229, Springer, pp 685–688, DOI 10.1007/978-3-540-30227-8_57
- Cropper A, Dumančić S, Evans R, Muggleton SH (2022) Inductive logic programming at 30. *Machine Learning* 111(1):147–172, DOI 10.1007/s10994-021-06089-1, URL <https://doi.org/10.1007/s10994-021-06089-1>
- De Raedt L, Kimmig A, Toivonen H (2007) ProbLog: A probabilistic Prolog and its application in link discovery. In: Veloso MM (ed) *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI)*, pp 2462–2467
- Frühwirth T (2009) *Constraint Handling Rules*. Cambridge University Press
- Gelernter D (1985) Generative communication in Linda. *ACM Transactions on Programming Languages and Systems* 7(1):80–112, DOI 10.1145/2363.2433
- Gelfond M, Lifschitz V (1988) The stable model semantics for logic programming. In: Kowalski RA, Bowen KA (eds) *Proceedings of the Fifth International Conference and Symposium on Logic Programming*, MIT Press, pp 1070–1080
- Green TJ, Huang SS, Loo BT, Zhou W (2013) Datalog and recursive query processing. *Foundations and Trends in Databases* 5(2):105–195, DOI 10.1561/19000000017
- Hachey G, Gasevic D (2012) Semantic web user interfaces: A systematic mapping study and review
- Harel D (1987) Statecharts: A visual formalism for complex systems. *Sci Comput Program* 8(3):231–274, DOI 10.1016/0167-6423(87)90035-9, URL [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9)
- Harel D, Naamad A (1996) The statemate semantics of statecharts. *ACM Trans Softw Eng Methodol* 5:293–333, DOI 10.1145/235321.235322
- Hébert F (2013) *Learn You Some Erlang for Great Good!: A Beginner's Guide*. No Starch Press, San Francisco, CA, USA
- Hendler JA (2001) Agents and the semantic web. *IEEE Intelligent Systems* 16(2):30–37, DOI 10.1109/5254.920597
- Hendler JA (2007) Where are all the intelligent agents? *IEEE Intelligent Systems* 22(3):2–3, DOI 10.1109/MIS.2007.62
- Horrocks I (1999) *Constructing the user interface with statecharts*. Addison-Wesley Longman Publishing Co., Inc.
- Horrocks I, Parsia B, Patel-Schneider P, Hendler J (2005) Semantic web architecture: Stack or two towers? In: Fages F, Soliman S (eds) *Principles and Practice of Semantic Web Reasoning: Third International Workshop, PPSWR 2005, Dagstuhl Castle, Germany, September 11–16, 2005, Proceedings*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp 37–41

- Jaffar J, Lassez JL (1987) Constraint logic programming. Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL) pp 111–119, DOI 10.1145/41625.41635
- Junger D, Lager T, Roxendal J (2012) Sxaml for building conversational agents in the dialog web lab. In: Proceedings of the Swedish Language Technology Conference (SLTC) 2012
- Kakas AC, Kowalski RA, Toni F (1992) Abductive logic programming. *Journal of Logic and Computation* 2(6):719–770, DOI 10.1093/logcom/2.6.719
- Kifer M, de Bruijn J, Boley H, Fensel D (2005) A realistic architecture for the semantic web. In: Adi A, Stoutenburg S, Tabet S (eds) *Rules and Rule Markup Languages for the Semantic Web: First International Conference, RuleML 2005*, Galway, Ireland, November 10–12, 2005. Proceedings, Springer Berlin Heidelberg, Berlin, Heidelberg, pp 17–29
- Körner P, Leuschel M, Barbosa J, Santos Costa V, Dahl V, Hermenegildo MV, Morales JF, Wielemaker J, Diaz D, Abreu S, Ciatto G (2022) Fifty years of Prolog and beyond. *Theory and Practice of Logic Programming* 22(6):776–858, DOI 10.1017/S1471068422000102
- Kowalski R (1974) Predicate logic as programming language. *Information Processing 74* pp 569–574
- Kowalski R, Dato A (2022) Logical english meets legal english for swaps and derivatives. *Artificial Intelligence and Law* 30(2):163–197, DOI 10.1007/s10506-021-09295-3, URL <https://doi.org/10.1007/s10506-021-09295-3>
- Kowalski R, Dávila J, Sartor G, Calejo M (2023a) Logical english for law and education. In: Warren DS, Dahl V, Eiter T, Hermenegildo M, Kowalski R, Rossi F (eds) *Prolog: The Next 50 Years, Lecture Notes in Computer Science*, vol 13900, Springer, pp 287–299, DOI 10.1007/978-3-031-35254-6_24
- Kowalski R, Sadri F, Calejo M, Dávila J (2023b) Combining logic programming and imperative programming in lps. In: Warren DS, Dahl V, Eiter T, Hermenegildo M, Kowalski R, Rossi F (eds) *Prolog: The Next 50 Years, Lecture Notes in Computer Science*, vol 13900, Springer, pp 210–223, DOI 10.1007/978-3-031-35254-6_19
- Kowalski RA (1979) Algorithm = logic + control. *Commun ACM* 22(7):424–436, DOI 10.1145/359131.359136, URL <https://doi.org/10.1145/359131.359136>
- Kowalski RA, Sadri F (2015) Reactive computing as model generation. *New Generation Computing* 33(1):33–67, DOI 10.1007/s00354-015-0103-z
- Lager T (2019) Intro to web prolog for erlangers. In: *Proceeding of the 18th ACM SIGPLAN International Workshop on Erlang*, ACM, pp 18–29, DOI 10.1145/3331542.3342569
- Lager T, Myrendal J (2012) Exploring the web of coined catchy phrases. In: *Proceedings of WWW2012: Web Science Track*
- Loke SW (2006) Declarative programming of integrated peer-to-peer and web based systems: the case of prolog. *Journal of Systems and Software* 79(4):523–536, URL <http://dx.doi.org/10.1016/j.jss.2005.04.005>

- Lombardi A (2015) *WebSocket: Lightweight Client-server Communications*. O'Reilly Media, Inc.
- Manhaeve R, Dumančić S, Kimmig A, Demeester T, De Raedt L (2021) Neural probabilistic logic programming in DeepProbLog. *Artificial Intelligence* 298:103504, DOI 10.1016/j.artint.2021.103504, URL <https://www.sciencedirect.com/science/article/pii/S0004370221000552>
- Merritt D (1989) *Building Expert Systems in Prolog*. Springer-Verlag
- Muggleton S, De Raedt L (1994) Inductive logic programming: Theory and methods. *The Journal of Logic Programming* 19–20(Supplement 1):629–679, DOI 10.1016/0743-1066(94)90035-3
- Nash C, Loukides M (2015) *Everything Is Distributed*. O'Reilly Media, URL <https://www.oreilly.com/radar/everything-is-distributed/>, o'Reilly Radar report
- Ousterhout JK (1998) Scripting: higher level programming for the 21st century. *Computer* 31(3):23–30
- Radomski S, Schnelle-Walka D, Radeck-Arneth S (2013) A prolog datamodel for state chart xml. In: *Proceedings of the SIGDIAL 2013 Conference, Association for Computational Linguistics*, pp 127–131, URL <http://aclweb.org/anthology/W13-4019>
- Ranta A (2011) *Grammatical Framework: Programming with Multilingual Grammars*. CSLI Publications, Stanford, ISBN-10: 1-57586-626-9 (Paper), 1-57586-627-7 (Cloth)
- Riguzzi F (2018) *Foundations of Probabilistic Logic Programming: Languages, Semantics, Inference and Learning*. River Publishers, Gistrup, Denmark, DOI 10.1201/9781003338192
- Robinson JA (1965) A machine-oriented logic based on the resolution principle. *Journal of the ACM* 12(1):23–41, DOI 10.1145/321250.321253
- Roy PV, Haridi S (2004) *Concepts, Techniques, and Models of Computer Programming*. The MIT Press, Cambridge, Massachusetts
- Roy PV, Haridi S, Brand P, Smolka G, Mehl M, Scheidhauer R (1997) Mobile objects in Distributed Oz. *ACM Transactions on Programming Languages and Systems* 19(5):804–851, DOI 10.1145/265943.265972
- Roy PV, Brand P, Duchier D, Haridi S, Henz M, Schulte C (2003) Logic programming in the context of multiparadigm programming: The Oz experience. *Theory and Practice of Logic Programming* 3(6):715–763, DOI 10.1017/S1471068403001741
- Russell SJ, Norvig P (2016) *Artificial intelligence: a modern approach*. Malaysia; Pearson Education Limited,
- Samek M (2002) *Practical statecharts in C/C++: Quantum programming for embedded systems*. CRC Press
- Shapiro EY (1987) *Concurrent Prolog: Collected Papers*. MIT Press
- Skantze G, Moubayed SA (2012) Iristk: a statechart-based toolkit for multi-party face-to-face interaction. In: *International Conference on Multimodal Interaction, ICMI '12, Santa Monica, CA, USA, October 22–26, 2012*, pp 69–76, DOI 10.1145/2388676.2388698, URL <https://doi.org/10.1145/2388676.2388698>

- Smith RW, Biermann AW, Hipp DR (1995) An architecture for voice dialog systems based on prolog-style theorem proving. *Computational Linguistics* 21(3):281–318, URL <http://www.aclweb.org/anthology/J95-3001>
- Smolka G (1995) The Oz programming model. In: van Leeuwen J (ed) *Computer Science Today: Recent Trends and Developments*, Lecture Notes in Computer Science, vol 1000, Springer, Berlin, pp 324–343, DOI 10.1007/BFb0015252
- van Steen M, Tanenbaum AS (2023) *Distributed Systems*, 4th edn. distributed-systems.net, free digital edition available from distributed-systems.net
- Sterling L, Shapiro EY (1994) *The Art of Prolog: Advanced Programming Techniques*, 2nd edn. The MIT Press, Cambridge, MA
- Svensson H, Fredlund LÅ, Earle CB (2010) A unified semantics for future erlang. In: Fritchie SL, Sagonas KF (eds) *Proceedings of the 9th ACM SIGPLAN Workshop on Erlang*, Baltimore, Maryland, USA, September 30, 2010, ACM, pp 23–32, URL <http://dl.acm.org/citation.cfm?id=1863509>
- Tarau P (2000) Fluents: A refactoring of prolog for uniform reflection and inter-operation with external objects. In: Lloyd JW, et al. (eds) *Computational Logic - CL 2000*, First International Conference, London, UK, 24–28 July, 2000, Proceedings, Springer, Lecture Notes in Computer Science, vol 1861, pp 1225–1239, DOI 10.1007/3-540-44957-4_82, URL https://doi.org/10.1007/3-540-44957-4_82
- Ueda K (1986) Guarded horn clauses. In: *Proceedings of the 1986 International Conference on Logic Programming*, Springer, pp 168–179
- Van Gelder A, Ross KA, Schlipf JS (1991) The well-founded semantics for general logic programs. *Journal of the ACM* 38(3):620–650, DOI 10.1145/116825.116838
- Van Roy P (2009) Programming paradigms for dummies: What every programmer should know. *New computational paradigms for computer music* 104
- Verborgh R, Vander Sande M (2020) The Semantic Web identity crisis: in search of the trivialities that never were. *Semantic Web Journal* 11(1):19–27, DOI 10.3233/SW-190372, URL <https://ruben.verborgh.org/articles/the-semantic-web-identity-crisis/>
- Virding R, Wikström C, Williams M, Armstrong J (1996) *Concurrent programming in ERLANG* (2nd ed.). Prentice Hall International (UK) Ltd., GBR
- Warren DS, Dahl V, Eiter T, Hermenegildo MV, Kowalski R, Rossi F (eds) (2023) *Prolog: The Next 50 Years*, Lecture Notes in Computer Science, vol 13900. Springer, Cham, DOI 10.1007/978-3-031-35254-6
- Wielemaker J (2023) Janus: A Bi-directional Interface Between Prolog and Python. URL <https://www.swi-prolog.org/pldoc/package/janus.html>, *swi-Prolog documentation*
- Wielemaker J, Beek W, Hildebrand M, van Ossenbruggen J (2016) Cliopatria: A SWI-Prolog infrastructure for the semantic web. *Semantic Web* 7(5):529–541
- Wooldridge M, Jennings NR (1995) Intelligent agents: Theory and practice. *The Knowledge Engineering Review* 10(2):115–152, DOI 10.1017/S0269888900008122, URL <https://doi.org/10.1017/S0269888900008122>

- Yang Z, Ishay A, Lee J (2020) NeurASP: Embracing neural networks into answer set programming. In: Bessiere C (ed) Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-20), International Joint Conferences on Artificial Intelligence Organization, pp 1755–1762, DOI 10.24963/ijcai.2020/243, URL <https://doi.org/10.24963/ijcai.2020/243>